LÖVE WIKI · PART 3 OF 9

Types (1/2)

640 pages

wiki pages 11–650

Contents of this part

(File):close
(File):eof
(File):flush
(File):getBuffer
(File):getFilename
(File):getMode
(File):getSize
(File):isEOF
(File):isOpen
(File):lines
(File):open
(File):read
(File):seek
(File):setBuffer
(File):tell
(File):write
(Image):getData
(Image):getDimensions
(Image):getFilter
(Image):getFlags
(Image):getHeight
(Image):getWidth
(Image):getWrap
(Image):isCompressed
(Image):isFormatLinear
(Image):refresh
(Image):replacePixels
(Image):setFilter
(Image):setWrap
AeroGUI
AlignMode
AnAL
Animation
ArcType
AreaSpreadDistribution
Astray
Astropatrolonium
Atom
AttributeDataType
AutoBenchmark
Autumn
beholder.lua
BezierCurve
BezierCurve:evaluate
BezierCurve:getControlPoint
BezierCurve:getControlPointCount
BezierCurve:getDegree
BezierCurve:getDerivative
BezierCurve:getSegment
BezierCurve:insertControlPoint
BezierCurve:removeControlPoint
BezierCurve:render
BezierCurve:renderSegment
BezierCurve:rotate
BezierCurve:scale
BezierCurve:setControlPoint
BezierCurve:translate
BlendAlphaMode
Blender
BlendMode
Blob
Body
Body:applyAngularImpulse
Body:applyForce
Body:applyImpulse
Body:applyLinearImpulse
Body:applyTorque
Body:destroy
Body:getAllowSleeping
Body:getAngle
Body:getAngularDamping
Body:getAngularVelocity
Body:getContactList
Body:getContacts
Body:getFixtureList
Body:getFixtures
Body:getGravityScale
Body:getInertia
Body:getJointList
Body:getJoints
Body:getLinearDamping
Body:getLinearVelocity
Body:getLinearVelocityFromLocalPoint
Body:getLinearVelocityFromWorldPoint
Body:getLocalCenter
Body:getLocalPoint
Body:getLocalPoints
Body:getLocalVector
Body:getMass
Body:getMassData
Body:getPosition
Body:getShape
Body:getShapes
Body:getTransform
Body:getType
Body:getUserData
Body:getWorld
Body:getWorldCenter
Body:getWorldPoint
Body:getWorldPoints
Body:getWorldVector
Body:getX
Body:getY
Body:hasCustomMassData
Body:isActive
Body:isAwake
Body:isBullet
Body:isDestroyed
Body:isDynamic
Body:isFixedRotation
Body:isFrozen
Body:isSleeping
Body:isSleepingAllowed
Body:isStatic
Body:isTouching
Body:putToSleep
Body:resetMassData
Body:setActive
Body:setAllowSleeping
Body:setAngle
Body:setAngularDamping
Body:setAngularVelocity
Body:setAwake
Body:setBullet
Body:setFixedRotation
Body:setGravityScale
Body:setInertia
Body:setLinearDamping
Body:setLinearVelocity
Body:setMass
Body:setMassData
Body:setMassFromShapes
Body:setPosition
Body:setSleepingAllowed
Body:setTransform
Body:setType
Body:setUserData
Body:setX
Body:setY
Body:wakeUp
BodyType
BookOfBartbes
BoundingBox.lua
Bubbles
BufferDataFormat
BufferDataUsage
BufferMode
bump.lua
BuryMyHeart
ByteData
ByteData:setString
CameraMgr
Canvas
Canvas:clear
Canvas:generateMipmaps
Canvas:getDimensions
Canvas:getFilter
Canvas:getFormat
Canvas:getFSAA
Canvas:getHeight
Canvas:getImageData
Canvas:getMipmapMode
Canvas:getMSAA
Canvas:getPixel
Canvas:getWidth
Canvas:getWrap
Canvas:new ImageData
Canvas:renderTo
Canvas:setFilter
Canvas:setWrap
CanvasFormat
CarWar
Categorie:Tutoriels
ChainShape
ChainShape:getChildEdge
ChainShape:getNextVertex
ChainShape:getPoint
ChainShape:getPoints
ChainShape:getPreviousVertex
ChainShape:getVertexCount
ChainShape:setNextVertex
ChainShape:setPreviousVertex
ChainShape:setPrevVertex
Changelog
Channel
Channel:clear
Channel:demand
Channel:getCount
Channel:hasRead
Channel:peek
Channel:performAtomic
Channel:pop
Channel:push
Channel:supply
Check
CircleShape
CircleShape:getLocalCenter
CircleShape:getPoint
CircleShape:getRadius
CircleShape:getWorldCenter
CircleShape:setPoint
CircleShape:setRadius
Clamping
Class
ClientObject
ClientObject:connect
ClientObject:disableBroadcast
ClientObject:disconnect
ClientObject:doPing
ClientObject:enableBroadcast
ClientObject:Init
ClientObject:receive
ClientObject:send
ClientObject:setCallback
ClientObject:setHandshake
ClientObject:setPing
ClientObject:update
cock.addJoystick
cock.addOption
cock.bind
cock.cancelCapture
cock.convertAxis
cock.convertDelta
cock.convertDevice
cock.convertInverse
cock.convertJoystick
cock.convertJoystickHat
cock.convertJoystickHatMode
cock.convertKey
cock.delete
cock.deleteJoystick

cock.deleteOption
cock.explodeCapturedData
cock.find
cock.getBinded
cock.getCapture
cock.getEmptyOption
cock.getJoystickDeadzone
cock.getJoystickHatMode
cock.getJoysticksList
cock.getMouseFactor
cock.getMouseOffset
cock.new
cock.reloadJoysticks
cock.remapJoystickHat
cock.setCallbacks
cock.setCapture
cock.setControls
cock.setDefault
cock.setDefaultXBox360
cock.setJoystickDeadzone
cock.setJoystickHatMode
cock.setMouseFactor
cock.setMouseOffset
cock.unbind
cock.update
cock.updateAll
CodeBreak
Collisions
ColorFactory
ColorMode
CommonPath
CompareMode
Component
CompressedData
CompressedData:getFormat
CompressedDataFormat
CompressedImageData
CompressedImageData:getDimensions
CompressedImageData:getFormat
CompressedImageData:getHeight
CompressedImageData:getMipmapCount
CompressedImageData:getWidth
CompressedImageFormat
Contact
Contact:getChildren
Contact:getFixtures
Contact:getFriction
Contact:getNormal
Contact:getPosition
Contact:getPositions
Contact:getRestitution
Contact:getSeparation
Contact:getShapes
Contact:getVelocity
Contact:isEnabled
Contact:isTouching
Contact:resetFriction
Contact:resetRestitution
Contact:setEnabled
Contact:setFriction
Contact:setRestitution
ContainerType
coroutine.resume
cron.lua
CTRL
CullMode
Cupid
Cursor
Cursor:getType
CursorType
Data
Data:clone
Data:getFFIPointer
Data:getPointer
Data:getSize
Data:getString
Debug
Decoder
Decoder:clone
Decoder:decode
Decoder:getBitDepth
Decoder:getBits
Decoder:getChannelCount
Decoder:getChannels
Decoder:getDuration
Decoder:getSampleRate
Decoder:seek
DeviceType
DisplayOrientation
DistanceBasedCollision
DistanceJoint
DistanceJoint:getDamping
DistanceJoint:getDampingRatio
DistanceJoint:getFrequency
DistanceJoint:getLength
DistanceJoint:setDamping
DistanceJoint:setDampingRatio
DistanceJoint:setFrequency
DistanceJoint:setLength
DistanceModel
Donut
Dox
Draw able
Draw Mode
DroppedFile
EdgeShape
EdgeShape:getNextVertex
EdgeShape:getPoints
EdgeShape:getPreviousVertex
EdgeShape:setNextVertex
EdgeShape:setPreviousVertex
EffectType
EffectWaveform
ELScheduler
EncodeFormat
enet.event
enet.host
enet.host create
enet.host:bandw idth limit
enet.host:broadcast
enet.host:channel limit
enet.host:check events
enet.host:compress w ith range coder
enet.host:connect
enet.host:destroy
enet.host:flush
enet.host:get peer
enet.host:get socket address
enet.host:peer count
enet.host:service
enet.host:service time
enet.host:total received data
enet.host:total sent data
enet.linked version
enet.peer
enet.peer:connect id
enet.peer:disconnect
enet.peer:disconnect later
enet.peer:disconnect now
enet.peer:index
enet.peer:last round trip time
enet.peer:ping
enet.peer:ping interval
enet.peer:receive
enet.peer:reset
enet.peer:round trip time
enet.peer:send
enet.peer:state
enet.peer:throttle configure
enet.peer:timeout
Event
examples.love
FactoryFunctions
FAQ
File
FileData
FileData:getExtension
FileData:getFilename
FileDecoder
FileDialogType
FileMode
FileType
FilterMode
FilterType
Fixture
Fixture:destroy
Fixture:getBody
Fixture:getBoundingBox
Fixture:getCategory
Fixture:getDensity
Fixture:getFilterData
Fixture:getFriction
Fixture:getGroupIndex
Fixture:getMask
Fixture:getMassData
Fixture:getRestitution
Fixture:getShape
Fixture:getUserData
Fixture:isDestroyed
Fixture:isSensor
Fixture:rayCast
Fixture:setCategory
Fixture:setDensity
Fixture:setFilterData
Fixture:setFriction
Fixture:setGroupIndex
Fixture:setMask
Fixture:setRestitution
Fixture:setSensor
Fixture:setUserData
Fixture:testPoint
Flux
Font
Font:getAscent
Font:getBaseline
Font:getDescent
Font:getDPIScale
Font:getFilter
Font:getHeight
Font:getKerning
Font:getLineHeight
Font:getWidth
Font:getWrap
Font:hasGlyphs
Font:setFallbacks
Font:setFilter
Font:setLineHeight
FontData
FontForge
FPSGraph
FractalNoise
Framebuffer
Framebuffer:getImageData
Framebuffer:getWrap
Framebuffer:renderTo
Framebuffer:setWrap
Framew ork
FrictionJoint
FrictionJoint:getMaxForce
FrictionJoint:getMaxTorque
FrictionJoint:setMaxForce
FrictionJoint:setMaxTorque
FullscreenType
GamepadAxis
GamepadButton
GamepadType
Gangrene
Geany
GearJoint
GearJoint:getJoints
GearJoint:getRatio
GearJoint:setRatio
GIMP
Git
GlyphData
GlyphData:getAdvance
GlyphData:getBearing
GlyphData:getBoundingBox
GlyphData:getDimensions
GlyphData:getFormat

GlyphData:getGlyph
GlyphData:getGlyphString
GlyphData:getHeight
GlyphData:getWidth
Goo
Gradients
GraphicsBuffer
GraphicsBuffer:clear
GraphicsBuffer:getElementCount
GraphicsBuffer:getElementStride
GraphicsBuffer:getSize
GraphicsFeature
GraphicsGale
GraphicsLimit
Graphoon
Grease
Gw ee
HashFunction
HC
HintingMode
HooECS
HS2
HUDebug
HUMP
Image
ImageData
ImageData:encode
ImageData:getDimensions
ImageData:getFormat
ImageData:getHeight
ImageData:getPixel
ImageData:getString
ImageData:getWidth
ImageData:mapPixel
ImageData:paste
ImageData:setPixel
ImageEncodeFormat
ImageFlag
ImageFontFormat
IndexDataType
Input
InputField
inspect.lua
InvaderGame
Isome
JesseH
Joint
Joint:destroy
Joint:getAnchors
Joint:getBodies
Joint:getCollideConnected
Joint:getReactionForce
Joint:getReactionTorque
Joint:getType
Joint:getUserData
Joint:isDestroyed
Joint:setCollideConnected
Joint:setUserData
JointType
Joystick
Joystick:getAxes
Joystick:getAxis
Joystick:getAxisCount
Joystick:getButtonCount
Joystick:getDeviceInfo
Joystick:getGamepadAxis
Joystick:getGamepadMapping
Joystick:getGamepadMappingString
Joystick:getGamepadType
Joystick:getGUID
Joystick:getHat
Joystick:getHatCount
Joystick:getID
Joystick:getJoystickType
Joystick:getName
Joystick:getPlayerIndex
Joystick:getSensorData
Joystick:getVibration
Joystick:hasSensor
Joystick:isConnected
Joystick:isDow n
Joystick:isGamepad
Joystick:isGamepadDow n
Joystick:isSensorEnabled
Joystick:isVibrationSupported
Joystick:setPlayerIndex
Joystick:setSensorEnabled
Joystick:setVibration
JoystickHat
JoystickInputType
JoystickType
Jumper
Jupiter
KeyColor
KeyConstant
Knightvania
Komoplode
Kuey
Kurosuke
LalaLove
Lexicon
LibCompress
Library
LifeOnLove
Lily
LineJoin
LineStippleSnippet
LineStyle
ListBox
Live2LOVE
LoadMode
lol.lua
Lope2D
love.(underscore)os
Love3D
LOVE:About
LOVE:Autoconfirmed users
LOVE:Copyrights
LOVE:Users
LoveAStar
Lovebird
LoveFS
LOVELi
Lover
LoverNet
LoveSpin
LoveToAll
Lovetoys
LoveUI
LoveUpdate
LuaC
LuaNameGen
Luaoop
LuaPill
LuaTable
LUBE.bin:pack
LUBE.bin:setseperators
LUBE.bin:unpack
LUBE.client
LUBE.server
LUIGI
Lume
lure.article.renderpreview
lure.dom.attribute
lure.dom.cdata
lure.dom.comment
lure.dom.document
lure.dom.DOMParser
lure.dom.element
lure.dom.HTMLCssRule
lure.dom.HTMLScript
lure.dom.HTMLStyle
lure.dom.HTMLStylesheet
lure.dom.namednodemap
lure.dom.node
lure.dom.nodelist
lure.dom.style
lure.dom.text
lure.dom.w indow
lure.dom.XMLHttpRequest
lure.load
lure.tuts.parsepreview
lure.tuts.xmlhttprequest
Lynput
main.lua
Mappy
Math
MatrixLayout
memoize.lua

(File):close

(File):close · revision 15133 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Closes a File.

Function

Synopsis

```
success = File:close( )
```

Arguments

None.

Returns

`boolean success`
: Whether closing was successful.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):eof

(File):eof · revision 15803 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.10.0

It has been renamed to File:isEOF.

If the end-of-file has been reached

Function

Synopsis

```
eof = File:eof( )
```

Arguments

None.

Returns

`boolean eof`

Whether EOF has been reached

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

(File):flush

(File):flush · revision 11011 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Flushes any buffered written data in the file to the disk.

Function

Synopsis

```
success, err = File:flush( )
```

Arguments

None.

Returns

`boolean success`

Whether the file successfully flushed any buffered data to the disk.

`string err (nil)`

The error string, if an error occurred and the file could not be flushed.

See Also

- File
- File:write
- File:setBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

(File):getBuffer

(File):getBuffer · revision 15148 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the buffer mode of a file.

Function

Synopsis

```
mode, size = File:getBuffer( )
```

Arguments

None.

Returns

`BufferMode mode`
: The current buffer mode of the file.

`number size`
: The maximum size in bytes of the file's buffer.

See Also

- File
- File:setBuffer
- File:write

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):getFilename

(File):getFilename · revision 15868 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the filename that the File object was created with. If the file object originated from the love.filedropped callback, the filename will be the full platform-dependent file path.

Function

Synopsis

```
filename = File:getFilename( )
```

Arguments

None.

Returns

`string filename`

The filename of the File.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):getMode

(File):getMode · revision 10878 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the FileMode the file has been opened with.

Function

Synopsis

```
mode = File:getMode( )
```

Arguments

None.

Returns

`FileMode mode`

The mode this file has been opened with.

See Also

- File
- FileMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):getSize

(File):getSize · revision 22409 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns the file size.

Function

Synopsis

```
size = File:getSize( )
```

Arguments

None.

Returns

`number size`
: The file size in bytes.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):isEOF

(File):isEOF · revision 15026 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

It has been renamed from File:eof.

Gets whether end-of-file has been reached.

Function

Synopsis

```
eof = File:isEOF( )
```

Arguments

None.

Returns

`boolean eof`

Whether EOF has been reached.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

(File):isOpen

(File):isOpen · revision 10879 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the file is open.

Function

Synopsis

```
open = File:isOpen( )
```

Arguments

None.

Returns

`boolean open`

True if the file is currently open, false otherwise.

See Also

- File
- File:open

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):lines

(File):lines · revision 29429 · last edited 2025-01-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Iterate over all the lines in a file.

Function

Synopsis

```
iterator = File:lines( )
```

Arguments

None.

Returns

`function iterator`
: The iterator (can be used in for loops).

Notes

If the EOF (end of file) is reached during iteration, the File will be automatically closed.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):open

(File):open · revision 14540 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Open the file for write, read or append.

Function

Synopsis

```
ok, err = File:open( mode )
```

Arguments

`FileMode mode`
: The mode to open the file in.

Returns

`boolean ok`
: True on success, false otherwise.

`string err`
: The error string if an error occurred.

Notes

If you are getting the error message "Could not set write directory", try setting the save directory. This is done either with love.filesystem.setIdentity or by setting the identity field in love.conf (only available with love 0.7 or higher).

See Also

- File
- FileMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):read

(File):read · revision 22105 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Read a number of bytes from a file.

Function

Synopsis

```
contents, size = File:read( bytes )
```

Arguments

number bytes (all)
: The number of bytes to read.

Returns

string contents
: The contents of the read bytes.

number size
: How many bytes have been read.

Function

Available since LÖVE 11.0
This variant is not supported in earlier versions.

Reads the contents of a file into either a string or a FileData object.

Synopsis

```
contents, size = File:read( container, bytes )
```

Arguments

ContainerType container
: What type to return the file's contents as.

number bytes (all)
: The number of bytes to read.

Returns

value contents
: FileData or string containing the read bytes.

number size
: How many bytes have been read.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):seek

(File):seek · revision 25518 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Seek to a position in a file

Function

Synopsis

```
success = File:seek( pos )
```

Arguments

`number pos`
: The position to seek to, relative to start of file.

Returns

`boolean success`
: Whether the operation was successful

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):setBuffer

(File):setBuffer · revision 11644 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the buffer mode for a file opened for writing or appending. Files with buffering enabled will not write data to the disk until the buffer size limit is reached, depending on the buffer mode.

File:flush will force any buffered data to be written to the disk.

Function

Synopsis

```
success, errorstr = File:setBuffer( mode, size )
```

Arguments

`BufferMode mode`
: The buffer mode to use.

`number size (0)`
: The maximum size in bytes of the file's buffer.

Returns

`boolean success`
: Whether the buffer mode was successfully set.

`string errorstr (nil)`
: The error string, if the buffer mode could not be set and an error occurred.

See Also

- File
- File:getBuffer
- File:write
- File:flush

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):tell

(File):tell · revision 19890 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the position in the file.

Function

Synopsis

```
pos = File:tell( )
```

Arguments

None.

Returns

`number pos`
: The current position.

See Also

- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(File):write

(File):write · revision 21851 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Write data to a file.

Function

Synopsis

```
success, err = File:write( data, size )
```

Arguments

`string data`
: The string data to write.

`number size (all)`
: How many bytes to write.

Returns

`boolean success`
: Whether the operation was successful.

`string err`
: The error string if an error occurred.

Function

Synopsis

```
success, err = File:write( data, size )
```

Arguments

`Data data`
: The Data object to write.

`number size (all)`
: How many bytes to write.

Returns

`boolean success`
: Whether the operation was successful.

`string errorstr`
: The error string if an error occurred.

Notes

Writing to multiple lines: In Windows, some text editors (e.g. Notepad before Windows 10 1809) only treat CRLF ("\r\n") as a new line.

```
--example
f = love.filesystem.newFile("note.txt")
f:open("w")
```

```
for i = 1, 10 do
    f:write("This is line "..i.."!\r\n")
end
f:close()
```

See Also

- File
- File:flush
- File:setBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getData

(Image):getData · revision 28247 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 11.0

Create an ImageData from a file, create the Image from that, and keep a reference to the ImageData, instead of using this function.

Gets the original ImageData or CompressedData used to create the Image.

All Images keep a reference to the Data that was used to create the Image. The Data is used to refresh the Image when love.window.setMode or Image:refresh is called.

Function

Synopsis

```
data = Image:getData( )
```

Arguments

None.

Returns

`ImageData data`

The original ImageData used to create the Image, if the image is not compressed.

Function

Synopsis

```
data = Image:getData( )
```

Arguments

None.

Returns

`CompressedData data`

The original CompressedData used to create the Image, if the image is compressed.

Examples

Edit the Image's ImageData and refresh the Image using the edited ImageData.

```
function love.load()
    image = love.graphics.newImage("pig.png")
end

function love.draw()
    love.graphics.draw(image)
end

function love.keypressed(key)
    -- If the image is compressed, it will return CompressedData which doesn't have a mapPixel method.
```

```
    -- Currently only dds files can become compressed images.
    if key == "e" and not image:isCompressed() then
        local data = image:getData()
        data:mapPixel(function(x, y, r, g, b, a) return r/2, g/2, b/2, a/2 end)
        image:refresh()
    end
end
```

But you can get the ImageData if you draw it as canvas (probably too slow):

```
local function newImageData (image)
    local canvas = love.graphics.newCanvas (image:getDimensions())
    love.graphics.setCanvas (canvas)
        love.graphics.setColor (1,1,1)
        love.graphics.draw (image)
    love.graphics.setCanvas ()
    return canvas:newImageData()
end
```

See Also

- Image
- Image:refresh
- Image:isCompressed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getDimensions

(Image):getDimensions · revision 20179 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the Image.

Function

Synopsis

```
width, height = Image:getDimensions( )
```

Arguments

None.

Returns

`number width`
: The width of the Image, in pixels.

`number height`
: The height of the Image, in pixels.

See Also

- Image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getFilter

(Image):getFilter · revision 20190 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Gets the filter mode for an image.

Function

Synopsis

```
min, mag = Image:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when minifying the image.

`FilterMode mag`
: Filter mode used when magnifying the image.

Function

Available since LÖVE 0.9.0
This variant is not supported in earlier versions.

Synopsis

```
min, mag, anisotropy = Image:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when minifying the image.

`FilterMode mag`
: Filter mode used when magnifying the image.

`number anisotropy`
: Maximum amount of anisotropic filtering used.

See Also

- Image
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getFlags

(Image):getFlags · revision 26134 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 11.0

This function is not supported in that and later versions.

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the flags used when the image was created.

Function

Synopsis

```
flags = Image:getFlags( )
```

Arguments

None.

Returns

`table flags`

A table with ImageFlag keys.

See Also

- Image
- love.graphics.newImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getHeight

(Image):getHeight · revision 26887 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the height of the Image.

Function

Synopsis

```
height = Image:getHeight( )
```

Arguments

None.

Returns

number height
: The height of the Image, in pixels.

See Also

- Image
- (Image):getWidth

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getWidth

(Image):getWidth · revision 26886 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the width of the Image.

Function

Synopsis

```
width = Image:getWidth( )
```

Arguments

None.

Returns

number width
: The width of the Image, in pixels.

See Also

- Image
- (Image):getHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):getWrap

(Image):getWrap · revision 20191 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the wrapping properties of an Image.

This function returns the currently set horizontal and vertical wrapping modes for the image.

Function

Synopsis

```
horiz, vert = Image:getWrap( )
```

Arguments

None.

Returns

`WrapMode horiz`
: Horizontal wrapping mode of the image.

`WrapMode vert`
: Vertical wrapping mode of the image.

See Also

- Image
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):isCompressed

(Image):isCompressed · revision 11051 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Image was created from CompressedData.

Compressed images take up less space in VRAM, and drawing a compressed image will generally be more efficient than drawing one created from raw pixel data.

Function

Synopsis

```
compressed = Image:isCompressed( )
```

Arguments

None.

Returns

`boolean compressed`

Whether the Image is stored as a compressed texture on the GPU.

See Also

- Image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):isFormatLinear

(Image):isFormatLinear · revision 26136 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether the Image was created with the linear (non-gamma corrected) flag set to true.

This method always returns false when gamma-correct rendering is not enabled.

Function

Synopsis

```
linear = Image:isFormatLinear( )
```

Arguments

None.

Returns

`boolean linear`

Whether the Image's internal pixel format is linear (not gamma corrected), when gamma-correct rendering is enabled.

See Also

- Image
- love.graphics.newImage

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):refresh

(Image):refresh · revision 21580 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 11.0
It has been replaced by Image:replacePixels.

Reloads the Image's contents from the ImageData or CompressedData used to create the image.

Function

Synopsis

```
Image:refresh( )
```

Arguments

None.

Returns

Nothing.

Function

Available since LÖVE 0.10.0 and removed in LÖVE 11.0
It has been replaced by Image:replacePixels.

Synopsis

```
Image:refresh( x, y, width, height )
```

Arguments

number x
: The x-axis of the top-left corner of the area within the Image to reload.

number y
: The y-axis of the top-left corner of the area within the Image to reload.

number width
: The width of the area within the Image to reload.

number height
: The height of the area within the Image to reload.

Returns

Nothing.

Examples

```
function love.load()
    imagedata = love.image.newImageData("pig.png")
    image = love.graphics.newImage(imagedata)
end
```

```
function love.draw()
    love.graphics.draw(image)
end

function love.keypressed(key)
    if key == "e" then
        -- Modify the original ImageData and apply the changes to the Image.
        imagedata:mapPixel(function(x, y, r, g, b, a) return r/2, g/2, b/2, a/2 end)
        image:refresh()
    end
end
```

See Also

- Image
- Image:getData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):replacePixels

(Image):replacePixels · revision 29164 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 11.0

This function replaces Image:refresh.

Replace the contents of an Image.

Function

Synopsis

```
Image:replacePixels( data, slice, mipmap, x, y, reloadmipmaps )
```

Arguments

`ImageData data`
: The new ImageData to replace the contents with.

`number slice (nil)`
: Which cubemap face, array index, or volume layer to replace, if applicable; the value is ignored otherwise.

`number mipmap (1)`
: The mimap level to replace, if the Image has mipmaps.

`number x (0)`
: The x-offset in pixels from the top-left of the image to replace. The given ImageData's width plus this value must not be greater than the pixel width of the Image's specified mipmap level.

`number y (0)`
: The y-offset in pixels from the top-left of the image to replace. The given ImageData's height plus this value must not be greater than the pixel height of the Image's specified mipmap level.

`boolean reloadmipmaps (true/false)`
: Whether to generate new mipmaps after replacing the Image's pixels. True by default if the Image was created with automatically generated mipmaps, false by default otherwise.

Returns

Nothing.

Examples

```
function love.load()
    imagedata = love.image.newImageData("pig.png")
    image = love.graphics.newImage(imagedata)
end

function love.draw()
    love.graphics.draw(image)
end

function love.keypressed(key)
    if key == "e" then
        -- Modify the original ImageData and apply the changes to the Image.
        imagedata:mapPixel(function(x, y, r, g, b, a) return r/2, g/2, b/2, a/2 end)
        image:replacePixels(imagedata)
```

```
    end
end
```

See Also

- Image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):setFilter

(Image):setFilter · revision 20192 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the filter mode for an image.

Function

Synopsis

```
Image:setFilter( min, mag )
```

Arguments

`FilterMode min`
: How to scale an image down.

`FilterMode mag`
: How to scale an image up.

Returns

Nothing.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
Image:setFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: How to scale an image down.

`FilterMode mag`
: How to scale an image up.

`number anisotropy (1)`
: Maximum amount of anisotropic filtering used.

Returns

Nothing.

See Also

- Image
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

(Image):setWrap

(Image):setWrap · revision 20193 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the wrapping properties of an Image.

This function sets the way an Image is repeated when it is drawn with a Quad that is larger than the image's extent. An image may be clamped or set to repeat in both horizontal and vertical directions. Clamped images appear only once, but repeated ones repeat as many times as there is room in the Quad.

N.B. If you use a Quad that is larger than the image extent and do not use repeated tiling, there may be an unwanted visual effect of the image stretching all the way to fill the Quad. If this is the case, setting `Image:setWrap("repeat", "repeat")` for all the images to be repeated, and using Quads of appropriate size will result in the best visual appearance.

Function

Synopsis

```
Image:setWrap( horiz, vert )
```

Arguments

`WrapMode horiz`
: Horizontal wrapping mode of the image.

`WrapMode vert`
: Vertical wrapping mode of the image.

Returns

Nothing.

See Also

- Image
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

AeroGUI

AeroGUI · revision 24516 · last edited 2023-07-05

On this page

- Still under development. Coming soon.
- Enumerators
  - Example
- Functions (love.gui)
  - Example
  - Example
  - Example
  - Example
- Objects
  - Screen
- Other Languages

Still under development. Coming soon.

AeroGUI is a Love2D GUI library developed by User:Aeromcfly.

Enumerators

Under table GuiEnum

GuiEnum

- AlignX
  - Left
  - Right
  - Center
- AlignY
  - Top
  - Bottom
  - Center

Example

```
Object.AlignX = GuiEnum.AlignX.Left
Object.AlignY = GuiEnum.AlignY.Center

-- Can also be written by the values of the enums directly:

Object.AlignX = "left"
Object.AlignY = "center"
```

Functions (love.gui)

love.gui.draw()

Updates and draws GUI objects. Essential for GUI system to work.
Returns nil

Example

```
function love.draw()
   -- All code here
   love.gui.draw()
end
```

love.gui.create(string Class [, GuiObject Screen])

Creates a new GuiObject
Class: The object's class (e.i. "Screen")
Screen: An existing Screen object

Returns GuiObject

Example

```
local Obj = love.gui.create("Frame")
```

love.gui.get(name)

Returns GuiObject with named 'name' if it exists

Example

```
local Obj = love.gui.get("MyObject")
if (Obj) then
-- ...
end
```

love.gui.clear()

Deletes all GuiObjects
Returns nil

Example

```
love.gui.clear()
```

Objects

Locked properties are read-only. Trying to set a new value to them will raise an error.

Screen

Screen objects act as rendering places for GUI objects. Screens are not actually seen. Any object whose 'Screen' property is set to a Screen object will scale itself based on the Screen's position and size.

Property	Locked	Type	Description	Default
Class	TRUE	string	Class name	Class name
Name	FALSE	string	Object name	(class)_(index)
Visible	FALSE	boolean	Whether or not the object is drawn on screen	true
Position	FALSE	UDim2	Position of the object drawn	UDim2.new()
Size	FALSE	UDim2	Size of the object drawn	UDim2.new(1,0,1,0)
AbsolutePosition	TRUE	Vector2	Actual position of object drawn	Vector2.new()
AbsoluteSize	TRUE	Vector2	Actual size of object drawn	Vector2.new()
AlignX	FALSE	GuiEnum AlignX	Horizontal alignment of object	GuiEnum.AlignX.Left
AlignY	FALSE	GuiEnum AlignY	Vertical alignment of object	GuiEnum.AlignY.Top
Screen	FALSE	GuiObject Screen	The Screen this object renders to (Optional for Screen objects)	screen (argument)
ZIndex	FALSE	float	Draw layer of object (draw operations simply sorted by this number)	0
ZIndexLast	TRUE	float	Used to check for changes in ZIndex	0

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

AlignMode

AlignMode · revision 23111 · last edited 2023-07-05

Text alignment.

Constants

center
: Align text center.

left
: Align text left.

right
: Align text right.

Available since LÖVE 0.9.0

justify
: Align text both left and right.

See Also

- love.graphics
- love.graphics.printf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

AnAL

AnAL · revision 20204 · last edited 2021-03-14


AnAL was a compatibility library to restore 0.5.0's animation in 0.6.0, it is not actively developed and it is recommended to look at other libraries for new projects

Animation

Animation · revision 25316 · last edited 2023-07-05

Animation Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
Flux	Fast, lightweight tweening library		Any	
tween	Small sets of functions for performing tweening in Lua		Any	
tween.lua	Simple tweening lib for Lua.		Any	
Live2LOVE	LÖVE library to show Live2D Cubism 2 models	No	11.0	
anim8	Animation Library for LÖVE.		0.9.x	
Löve Bone	A robust object-oriented 2D Skeletal Animation framework for LÖVE.		0.9.1	

ArcType

ArcType · revision 16200 · last edited 2026-04-19

Available since LÖVE 0.10.1

This enum is not supported in earlier versions.

Different types of arcs that can be drawn.

Constants

pie

The arc is drawn like a slice of pie, with the arc circle connected to the center at its end-points.

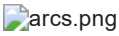
open

The arc circle's two end-points are unconnected when the arc is drawn as a line. Behaves like the "closed" arc type when the arc is drawn in filled mode.

closed

The arc circle's two end-points are connected to each other.

Notes



See Also

- love.graphics
- love.graphics.arc

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

AreaSpreadDistribution

AreaSpreadDistribution · revision 27493 · last edited 2023-07-05

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

Types of particle area spread distribution.

Constants

uniform
: Uniform distribution.

normal
: Normal (gaussian) distribution.

Available since LÖVE 0.10.2

ellipse
: Uniform distribution in an ellipse.

Available since LÖVE 11.0

borderellipse
: Distribution in an ellipse with particles spawning at the edges of the ellipse.

borderrectangle
: Distribution in a rectangle with particles spawning at the edges of the rectangle.

none
: No distribution - area spread is disabled.

See Also

- ParticleSystem
- ParticleSystem:setEmissionArea
- ParticleSystem:getEmissionArea

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Astray

Astray · revision 24541 · last edited 2023-07-05

Astray is a lua based maze, room and dungeon generation library for dungeon crawlers and rougelike video games.

Grab it on Github:

https://github.com/SiENcE/astray ↗

Love Forum Thread:

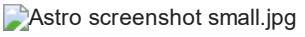
http://love2d.org/forums/viewtopic.php?t=76929

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Astropatrolonium

Astropatrolonium · revision 27976 · last edited 2022-08-17




```
Spacemen destroy... mega asteroids
Certain time and place
Floatin' out to space
Time to release the hatch!
```

This is basically an Asteroids (https://en.w ikipedia.org/w iki/Asteroids_(video_game)) remake.

LÖVE Version: 11.3

Multiplayer is possible with five players on one keyboard or with gamepads.

Timelapse video of coding the first prototype (http://w w w.youtube.com/w atch?v=aTF9eZmlgJM)

Loveclub forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=1716)(with old downloads)

download newest .love or .exe from itch.io (https://gutholz.itch.io/astropatrolonium)

Atom

Atom · revision 28324 · last edited 2023-04-28

On this page

- Packages for Love2D
  - Snippets
  - Launching your project
  - Hyperclick
  - Love IDE
- See Also

Atom is a free and open-source text and source code editor created by GitHub for OS X, Linux, and Windows. It can be downloaded here, (https://atom.io/) from the official website.

Packages for Love2D

Snippets

There are 2 different packages that can snippets for Love, love-atom (https://atom.io/packages/love-atom), and autocomplete-love (https://atom.io/packages/autocomplete-love). They both accomplish the same thing, although love-atom (https://atom.io/packages/love-atom) appears to be up to date.

Launching your project

The love-launcher (https://atom.io/packages/love-launcher) package can open your project by pressing "Alt-L". Before this works, you need to input the location of the love executable. The love-ide (https://atom.io/packages/love-ide) bundle of packages has this feature as well.

Hyperclick

The hyperclick-love (https://atom.io/packages/hyperclick-love) package makes it so you can click on certain pieces of love code and open the corresponding love wiki page.

Love IDE

One solution to many problems is the love-ide (https://atom.io/packages/love-ide) package, which installs many other packages to add all of the features mentioned above, and more.

See Also

- Official Atom Website (https://atom.io/)
- Atom Lua Package (https://atom.io/packages/language-lua)

AttributeDataType

AttributeDataType · revision 29006 · last edited 2026-04-10

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

Data types used in a Mesh's vertex format.

Constants

byte
: An 8-bit normalized (from 0 to 1) value.

unorm16
: A 16-bit normalized (from 0 to 1) value.

float
: A floating point number.

See Also

- love.graphics
- Mesh:getVertexFormat
- love.graphics.newMesh
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

AutoBenchmark

AutoBenchmark · revision 2893 · last edited 2021-03-14

AutoBenchmark

This project was originally designed with the idea to test system performance in terms of Lua and more specifically the LOVE Engine/Framework; central to the design ideal was that it should be simple to use! And what could be simpler than starting the program and letting it do its thing?

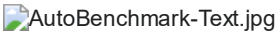
Tests

- High Intensity Text Display
- High Intensity Animation Display
- Empty Space Test (Control Phase)
- High Intensity Image Display
- High Intensity Primitives Display
- High Intensity Mandelbrot Fractal Generation

(Listed in order)

Screenshots

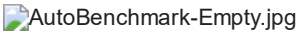
Text



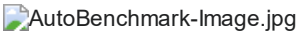
Animation



Empty Space



Image



Primitives



Mandelbrot



Results Report



Releases

Version 0.90 (http://love2d.org/forums/viewtopic.php?f=5&t=1308#p12781)

Credits

Original Design by CyaNox (http://love2d.org/forum/memberlist.php?mode=viewprofile&u=960) (Mark Sanders)

Automated and Reporting added by schme16 (http://love2d.org/forum/memberlist.php?mode=viewprofile&u=184) (Shane Gadsby)

Autumn

Autumn · revision 13874 · last edited 2021-03-14

Project page and download: http://www.bitoutsidethebox.com/autumn (http://w w w.bitoutsidethebox.com/autumn)

Watch the official trailer (https://w w w.youtube.com/w atch?v=MSfvHPovO_c)

More than a game, Autumn is a relaxing place for spiritual learning and growth with a soothing atmosphere.

In each game you play as a Spirit which depends on the natural world for its progress in life. As you help the world around you to thrive, you receive vital energy(Prana) in return. During the Autumn season, while the trees are shedding their leaves, orbs of Prana are released into the air, which you can absorb. Your own Prana can be utilized to help and change the world around you and to obtain spiritual teachings.

More than levels, puzzles or in-game progressions, Autumn's focus is on the growth of the player. Not the one in the game, but the one in the chair.

There are no time limits, no enemies to defeat, no goals to reach… except the ones you set for yourself. Relax, learn, grow… and enjoy Autumn.

beholder.lua

beholder.lua · revision 7822 · last edited 2023-07-05

Original forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=3868).

Latest version:

https://github.com/kikito/beholder.lua ↗

beholder.lua simplifies the management of event-based information in Lua.

Please refer to the forum post above or the latest version link for examples of use.

BezierCurve

BezierCurve · revision 27226 · last edited 2023-07-05

On this page

- Constructors
- Functions
- Supertypes
- Examples
- Draw a Bézier curve
- See Also
- Other Languages

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

A Bézier curve object that can evaluate and render Bézier curves of arbitrary degree. For more information on Bézier curves check this great article on Wikipedia (https://en.wikipedia.org/wiki/B%C3%A9zier_curve).

Constructors

love.math.newBezierCurve	Creates a new BezierCurve object.	Added since 0.9.0

Functions

BezierCurve:evaluate	Evaluate Bézier curve at parameter t.	Added since 0.9.0
BezierCurve:getControlPoint	Get coordinates of the i-th control point.	Added since 0.9.0
BezierCurve:getControlPointCount	Get the number of control points in the Bézier curve.	Added since 0.9.0
BezierCurve:getDegree	Get degree of the Bézier curve.	Added since 0.9.0
BezierCurve:getDerivative	Get derivate of the Bézier curve.	Added since 0.9.0
BezierCurve:getSegment	Gets a BezierCurve that corresponds to the specified segment of this BezierCurve.	Added since 0.10.0
BezierCurve:insertControlPoint	Insert control point after the i-th control point.	Added since 0.9.0
BezierCurve:removeControlPoint	Removes the specified control point.	Added since 0.10.0
BezierCurve:render	Get a list of points on the curve.	Added since 0.9.0
BezierCurve:renderSegment	Get a list of points on a specific part of the curve.	Added since 0.10.0
BezierCurve:rotate	Rotate the Bézier curve.	Added since 0.9.0
BezierCurve:scale	Scale the Bézier curve.	Added since 0.9.0
BezierCurve:setControlPoint	Set coordinates of the i-th control point.	Added since 0.9.0
BezierCurve:translate	Move the Bézier curve.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object

Examples

Draw a Bézier curve

```
local curve = love.math.newBezierCurve({25,25, 25,125, 75,25, 125,25})
function love.draw()
    love.graphics.line(curve:render())
end
```

See Also

- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:evaluate

BezierCurve:evaluate · revision 27964 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Make a circle follow a curve
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Evaluate Bézier curve at parameter t. The parameter must be between 0 and 1 (inclusive) and there must be at least two control points for the curve.

This function can be used to move objects along paths or tween parameters. However it should not be used to render the curve - see BezierCurve:render for that purpose.

Function

Synopsis

```
x, y = BezierCurve:evaluate( t )
```

Arguments

number t
: Where to evaluate the curve. Must be between 0 and 1.

Returns

number x
: x coordinate of the curve at parameter t.

number y
: y coordinate of the curve at parameter t.

Examples

Make a circle follow a curve

```
local controlPoints = {125,125, 125,225, 175,125, 225,125}
local curve         = love.math.newBezierCurve(controlPoints)

function love.draw()
    local time     = love.timer.getTime()
    local loopTime = 4
    local t        = (time / loopTime) % 1
    local x, y     = curve:evaluate(t)

    love.graphics.circle("fill", x, y, 8)
    love.graphics.line(curve:render())
end
```

See Also

- BezierCurve
- BezierCurve:render
- love.math
- Bezier Curves, how to trace a line? - LÖVE forum (https://love2d.org/forums/viewtopic.php?f=4&t=83825)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:getControlPoint

BezierCurve:getControlPoint · revision 27228 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
  - Get position of all control points
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Get coordinates of the i-th control point. Indices start with 1.

Function

Synopsis

```
x, y = BezierCurve:getControlPoint(i)
```

Arguments

number i
: Index of the control point.

Returns

number x
: Position of the control point along the x axis.

number y
: Position of the control point along the y axis.

Example

Get position of all control points

```
local count    = curve:getControlPointCount()
local vertices = {}

for i = 1, count do
    local x, y = curve:getControlPoint(i)
    table.insert(vertices, x)
    table.insert(vertices, y)
end
```

See Also

- BezierCurve
- BezierCurve:getDegree
- BezierCurve:setControlPoint
- BezierCurve:insertControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

BezierCurve:getControlPointCount

BezierCurve:getControlPointCount · revision 11867 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Get the number of control points in the Bézier curve.

Function

Synopsis

```
count = BezierCurve:getControlPointCount( )
```

Arguments

None.

Returns

`number count`

The number of control points.

See Also

- BezierCurve
- BezierCurve:getControlPoint
- BezierCurve:setControlPoint
- BezierCurve:insertControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:getDegree

BezierCurve:getDegree · revision 11147 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Get degree of the Bézier curve. The degree is equal to number-of-control-points - 1.

Function

Synopsis

```
degree = BezierCurve:getDegree( )
```

Arguments

None.

Returns

`number degree`

Degree of the Bézier curve.

See Also

- BezierCurve
- BezierCurve:getControlPoint
- BezierCurve:setControlPoint
- BezierCurve:insertControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:getDerivative

BezierCurve:getDerivative · revision 25521 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
  - Position a sprite along a Bézier-curve.
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Get the derivative of the Bézier curve.

This function can be used to rotate sprites moving along a curve in the direction of the movement and compute the direction perpendicular to the curve at some parameter t.

Function

Synopsis

```
derivative = BezierCurve:getDerivative()
```

Arguments

None.

Returns

`BezierCurve derivative`

The derivative curve.

Example

Position a sprite along a Bézier-curve.

```
curve      = love.math.newBezierCurve({25,25,75,50,125,25})
derivative = curve:getDerivative()
sprite     = love.graphics.newImage('sprite.png') -- to demonstrate orientation; assuming the sprite's front is upwards.

local t = 0.0 -- range: [0,1]
function love.update(dt)
    t = (t + dt / 10) % 1.0
end

function love.draw()
    local ex,ey = curve:evaluate(t)
    local dx,dy = derivative:evaluate(t)
    -- If the sprite were facing to the right, the +math.pi/2 part would not be needed.
    love.graphics.draw(sprite, ex, ey, math.atan2(dy,dx)+math.pi/2, 1, 1, sprite:getWidth()/2, sprite:getHeight()/2)
end
```

See Also

- BezierCurve
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

BezierCurve:getSegment

BezierCurve:getSegment · revision 15430 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets a BezierCurve that corresponds to the specified segment of this BezierCurve.

Function

Synopsis

```
curve = BezierCurve:getSegment( startpoint, endpoint )
```

Arguments

`number startpoint`
: The starting point along the curve. Must be between 0 and 1.

`number endpoint`
: The end of the segment. Must be between 0 and 1.

Returns

`BezierCurve curve`
: A BezierCurve that corresponds to the specified segment.

See Also

- BezierCurve
- BezierCurve:renderSegment
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:insertControlPoint

BezierCurve:insertControlPoint · revision 19893 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Insert control point as the new i-th control point. Existing control points from i onwards are pushed back by 1. Indices start with 1. Negative indices wrap around: -1 is the last control point, -2 the one before the last, etc.

Function

Synopsis

```
BezierCurve:insertControlPoint( x, y, i )
```

Arguments

`number x`
: Position of the control point along the x axis.

`number y`
: Position of the control point along the y axis.

`number i (-1)`
: Index of the control point.

Returns

Nothing.

See Also

- BezierCurve
- BezierCurve:getDegree
- BezierCurve:getControlPoint
- BezierCurve:setControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:removeControlPoint

BezierCurve:removeControlPoint · revision 17908 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Removes the specified control point.

Function

Synopsis

```
BezierCurve:removeControlPoint( index )
```

Arguments

`number index`
: The index of the control point to remove.

Returns

Nothing.

See Also

- BezierCurve
- BezierCurve:getDegree
- BezierCurve:setControlPoint
- BezierCurve:getControlPoint
- BezierCurve:insertControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:render

BezierCurve:render · revision 27343 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
  - Draw a bezier curve
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Get a list of coordinates to be used with love.graphics.line.

This function samples the Bézier curve using recursive subdivision. You can control the recursion depth using the depth parameter.

If you are just interested to know the position on the curve given a parameter, use BezierCurve:evaluate.

Function

Synopsis

```
coordinates = BezierCurve:render(depth)
```

Arguments

number depth (5)
: Number of recursive subdivision steps.

Returns

table coordinates
: List of x,y-coordinate pairs of points on the curve.

Example

Draw a bezier curve

```
curve = love.math.newBezierCurve({25,25,75,50,125,25})
function love.draw()
    love.graphics.line(curve:render())
end
```

See Also

- BezierCurve
- BezierCurve:renderSegment
- BezierCurve:evaluate
- love.math
- love.graphics.polygon

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:renderSegment

BezierCurve:renderSegment · revision 27988 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
  - Draw a segment of a bezier curve
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Get a list of coordinates on a specific part of the curve, to be used with love.graphics.line.

This function samples the Bézier curve using recursive subdivision. You can control the recursion depth using the depth parameter.

If you just need to know the position on the curve given a parameter, use BezierCurve:evaluate.

Function

Synopsis

```
coordinates = BezierCurve:renderSegment( startpoint, endpoint, depth )
```

Arguments

number startpoint
: The starting point along the curve. Must be between 0 and 1.

number endpoint
: The end of the segment to render. Must be between 0 and 1.

number depth (5)
: Number of recursive subdivision steps.

Returns

table coordinates
: List of x,y-coordinate pairs of points on the specified part of the curve.

Example

Draw a segment of a bezier curve

```
curve = love.math.newBezierCurve({25,25,75,50,125,25})
function love.draw()
    love.graphics.line(curve:renderSegment(0, .75))
end
```

See Also

- BezierCurve
- BezierCurve:render
- BezierCurve:evaluate
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

BezierCurve:rotate

BezierCurve:rotate · revision 11360 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Rotate the Bézier curve by an angle.

Function

Synopsis

```
BezierCurve:rotate(angle, ox, oy)
```

Arguments

`number angle`
: Rotation angle in radians.

`number ox (0)`
: X coordinate of the rotation center.

`number oy (0)`
: Y coordinate of the rotation center.

Returns

Nothing.

See Also

- BezierCurve:translate
- BezierCurve:scale
- BezierCurve
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:scale

BezierCurve:scale · revision 11359 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Scale the Bézier curve by a factor.

Function

Synopsis

```
BezierCurve:scale(s, ox, oy)
```

Arguments

`number s`
: Scale factor.

`number ox (0)`
: X coordinate of the scaling center.

`number oy (0)`
: Y coordinate of the scaling center.

Returns

Nothing.

See Also

- BezierCurve:translate
- BezierCurve:rotate
- BezierCurve
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:setControlPoint

BezierCurve:setControlPoint · revision 17912 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Set coordinates of the i-th control point. Indices start with 1.

Function

Synopsis

```
BezierCurve:setControlPoint(i, x, y)
```

Arguments

`number i`
: Index of the control point.

`number x`
: Position of the control point along the x axis.

`number y`
: Position of the control point along the y axis.

Returns

Nothing.

See Also

- BezierCurve
- BezierCurve:getDegree
- BezierCurve:getControlPoint
- BezierCurve:insertControlPoint
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BezierCurve:translate

BezierCurve:translate · revision 11358 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Move the Bézier curve by an offset.

Function

Synopsis

```
BezierCurve:translate(dx, dy)
```

Arguments

`number dx`
: Offset along the x axis.

`number dy`
: Offset along the y axis.

Returns

Nothing.

See Also

- BezierCurve:rotate
- BezierCurve:scale
- BezierCurve
- love.math

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BlendAlphaMode

BlendAlphaMode · revision 16032 · last edited 2026-04-19

Available since LÖVE 0.10.0
This enum is not supported in earlier versions.

Different ways alpha affects color blending. See BlendMode and the BlendMode Formulas for additional notes.

Constants

alphamultiply
: The RGB values of what's drawn are multiplied by the alpha values of those colors during blending. This is the default alpha mode.

premultiplied
: The RGB values of what's drawn are **not** multiplied by the alpha values of those colors during blending. For most blend modes to work correctly with this alpha mode, the colors of a drawn object need to have had their RGB values multiplied by their alpha values at some point previously ("premultiplied alpha").

Notes

The "premultiplied" constant should generally be used when drawing a Canvas to the screen, because the RGB values of the Canvas' texture had previously been multiplied by its alpha values when drawing content to the Canvas itself.

The "alphamultiply" constant does not affect the "multiply" BlendMode. Similarly, the "screen" BlendMode's math is only correct if the "premultiplied" alpha mode is used and the alpha of drawn objects has already been multiplied with its RGB values previously (possibly inside a shader).

Several articles have been written about *premultiplied alpha* and when to use it:

- http://www.realtimerendering.com/blog/gpus-prefer-premultiplication/ ↗
- https://developer.nvidia.com/content/alpha-blending-pre-or-not-pre ↗
- http://blogs.msdn.com/b/shawnhar/archive/2009/11/06/premultiplied-alpha.aspx ↗
- http://blogs.msdn.com/b/shawnhar/archive/2009/11/07/premultiplied-alpha-and-image-composition.aspx ↗

See Also

- love.graphics
- BlendMode
- BlendMode Formulas
- love.graphics.setBlendMode
- love.graphics.getBlendMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Blender

Blender · revision 17281 · last edited 2021-03-14

Blender is the free open source 3D content creation suite, available for all major operating systems under the GNU General Public License.

Blender website (http://w w w.blender.org/)

BlendMode

BlendMode · revision 24731 · last edited 2023-07-05

Available since LÖVE 0.2.0
This enum is not supported in earlier versions.

Different ways to do color blending. See BlendAlphaMode and the BlendMode Formulas for additional notes.

Constants

alpha
Alpha blending (normal). The alpha of what's drawn determines its opacity.

Available since LÖVE 0.9.0

replace
The colors of what's drawn completely replace what was on the screen, with no additional blending. The BlendAlphaMode specified in love.graphics.setBlendMode still affects what happens.

Available since LÖVE 0.9.1

screen
'Screen' blending.

Available since LÖVE 0.10.0

add
The pixel colors of what's drawn are added to the pixel colors already on the screen. The alpha of the screen is not modified.
subtract
The pixel colors of what's drawn are subtracted from the pixel colors already on the screen. The alpha of the screen is not modified.
multiply
The pixel colors of what's drawn are multiplied with the pixel colors already on the screen (darkening them). The alpha of drawn objects is multiplied with the alpha of the screen rather than determining how much the colors on the screen are affected, even when the "alphamultiply" BlendAlphaMode is used.

Available since LÖVE 0.10.1

lighten
The pixel colors of what's drawn are compared to the existing pixel colors, and the larger of the two values for each color component is used. Only works when the "premultiplied" BlendAlphaMode is used in love.graphics.setBlendMode.
darken
The pixel colors of what's drawn are compared to the existing pixel colors, and the smaller of the two values for each color component is used. Only works when the "premultiplied" BlendAlphaMode is used in love.graphics.setBlendMode.

Removed in LÖVE 0.10.0

additive
Additive blend mode.

Available since LÖVE 0.7.0 and removed in LÖVE 0.10.0

subtractive
: Subtractive blend mode.

multiplicative
: Multiply blend mode.

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

premultiplied
: Premultiplied alpha blend mode.

See Also

- love.graphics
- BlendMode Formulas
- love.graphics.setBlendMode
- love.graphics.getBlendMode
- BlendAlphaMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Blob

Blob · revision 28038 · last edited 2023-07-05

Blob.lua is a LuaJIT library that performs serialization into a compact binary format. It can be used to parse arbitrary binary data, or to serialize data for efficient storage or transmission.

(Internet Archive) GitHub repositiory (https://web.archive.org/web/20210125002105/https://github.com/megagrump/blob) (Download (https://web.archive.org/web/20210125002138/https://codeload.github.com/megagrump/blob/tar.gz/2.0.2))

LÖVE forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=84626&p=216827)

Ohter Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body

Body · revision 24714 · last edited 2024-12-17

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Bodies are objects with velocity and position.

Constructors

love.physics.newBody	Creates a new body.

Functions

Body:applyAngularImpulse	Applies an angular impulse to a body.	Added since 0.8.0	
Body:applyForce	Apply force to a Body.		
Body:applyImpulse	Applies an impulse to a body.		Removed in 0.8.0
Body:applyLinearImpulse	Applies an impulse to a body.	Added since 0.8.0	
Body:applyTorque	Apply torque to a body.		
Body:destroy	Explicitly destroys the Body.		
Body:getAllowSleeping	Return whether a body is allowed to sleep.	Added since 0.7.0	Removed in 0.8.0
Body:getAngle	Get the angle of the body.		
Body:getAngularDamping	Gets the Angular damping of the Body.		
Body:getAngularVelocity	Get the angular velocity of the Body.		
Body:getContactList	Gets a list of all Contacts attached to the Body.	Added since 0.9.2	Deprecated in 11.0
Body:getContacts	Gets a list of all Contacts attached to the Body.	Added since 11.0	
Body:getFixtureList	Returns a table with all fixtures.	Added since 0.8.0	Deprecated in 11.0
Body:getFixtures	Returns a table with all fixtures.	Added since 11.0	
Body:getGravityScale	Returns the gravity scale factor.	Added since 0.8.0	
Body:getInertia	Gets the rotational inertia of the body.		
Body:getJointList	Returns a table containing the Joints attached to this Body.	Added since 0.9.2	Deprecated in 11.0
Body:getJoints	Returns a table containing the Joints attached to this Body.	Added since 11.0	
Body:getLinearDamping	Gets the linear damping of the Body.		
Body:getLinearVelocity	Gets the linear velocity of the Body from its center of mass.		
Body:getLinearVelocityFromLocalPoint	Get the linear velocity of a point on the body.		
Body:getLinearVelocityFromWorldPoint	Get the linear velocity of a point on the body.		
Body:getLocalCenter	Get the center of mass position in local coordinates.		
Body:getLocalPoint	Transform a point from world coordinates to local coordinates.		
Body:getLocalPoints	Transforms multiple points from world coordinates to local coordinates.	Added since 11.4	
Body:getLocalVector	Transform a vector from world coordinates to local coordinates.		
Body:getMass	Get the mass of the body.		
Body:getMassData	Returns the mass, its center, and the rotational inertia.	Added since 0.8.0	
Body:getPosition	Get the position of the body.		
Body:getType	Returns the type of the body.	Added since 0.8.0	
Body:getUserData	Returns the Lua value associated with this Body.	Added since 0.9.1	
Body:getWorld	Gets the World the body lives in.	Added since 0.9.2	
Body:getWorldCenter	Get the center of mass position in world coordinates.		

Body:getWorldPoint	Transform a point from local coordinates to world coordinates.		
Body:getWorldPoints	Transforms multiple points from local coordinates to world coordinates.	Added since 0.8.0	
Body:getWorldVector	Transform a vector from local coordinates to world coordinates.		
Body:getX	Get the x position of the body in world coordinates.		
Body:getY	Get the y position of the body in world coordinates.		
Body:isActive	Returns whether the body is actively used in the simulation.	Added since 0.8.0	
Body:isAwake	Returns the sleep status of the body.	Added since 0.8.0	
Body:isBullet	Get the bullet status of a body.		
Body:isDestroyed	Gets whether the Body is destroyed.	Added since 0.9.2	
Body:isDynamic	Get the dynamic status of the body.		Removed in 0.8.0
Body:isFixedRotation	Returns whether the body rotation is locked.	Added since 0.8.0	
Body:isFrozen	Get the frozen status of the body.		Removed in 0.8.0
Body:isSleeping	Get the sleeping status of a body.		Removed in 0.8.0
Body:isSleepingAllowed	Returns the sleeping behaviour of the body.	Added since 0.8.0	
Body:isStatic	Get the static status of the body.		Removed in 0.8.0
Body:isTouching	Gets whether the Body is touching the given other Body.	Added since 11.0	
Body:putToSleep	Put the body to sleep.		Removed in 0.8.0
Body:resetMassData	Resets the mass of the body.	Added since 0.8.0	
Body:setActive	Sets whether the body is active in the world.	Added since 0.8.0	
Body:setAllowSleeping	Set the sleep behaviour of a body.		Removed in 0.8.0
Body:setAngle	Set the angle of the body.		
Body:setAngularDamping	Sets the angular damping of a Body		
Body:setAngularVelocity	Sets the angular velocity of a Body.		
Body:setAwake	Wakes the body up or puts it to sleep.	Added since 0.8.0	
Body:setBullet	Set the bullet status of a body.		
Body:setFixedRotation	Set whether a body has fixed rotation.		
Body:setGravityScale	Sets a new gravity scale factor for the body.	Added since 0.8.0	
Body:setInertia	Set the inertia of a body.		
Body:setLinearDamping	Sets the linear damping of a Body.		
Body:setLinearVelocity	Sets a new linear velocity for the Body.		
Body:setMass	Sets the mass properties directly.		
Body:setMassData	Overrides the calculated mass data.	Added since 0.8.0	
Body:setMassFromShapes	Sets mass properties from attatched shapes.		Removed in 0.8.0
Body:setPosition	Set the position of the body.		
Body:setSleepingAllowed	Sets the sleeping behaviour of the body.	Added since 0.8.0	
Body:setType	Sets a new body type.	Added since 0.8.0	
Body:setUserData	Associates a Lua value with the Body.	Added since 0.9.1	
Body:setX	Set the x position of the body.		
Body:setY	Set the y position of the body.		
Body:wakeUp	Wake up a sleeping body.		Removed in 0.8.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		

Enums

BodyType	The types of a Body.

Supertypes

- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:applyAngularImpulse

Body:applyAngularImpulse · revision 28968 · last edited 2024-01-12

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Applies an angular impulse to a body. This makes a single, instantaneous addition to the body momentum. Applying an angular impulse large enough will rotate the body.

A body with with a larger mass will react less. The reaction does **not** depend on the timestep, and is equivalent to applying a force continuously for 1 second. Impulses are best used to give a single push to a body. For a continuous push to a body it is better to use Body:applyTorque.

Function

Synopsis

```
Body:applyAngularImpulse( impulse )
```

Arguments

`number impulse`
: The impulse in kilogram-square meter per second.

Returns

Nothing.

See Also

- Body
- Body:applyLinearImpulse
- Body:applyTorque
- Body:getAngle
- Body:getAngularVelocity
- Body:getInertia

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:applyForce

Body:applyForce · revision 24131 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Apply force to a Body.

A force pushes a body in a direction. A body with with a larger mass will react less. The reaction also depends on how long a force is applied: since the force acts continuously over the entire timestep, a short timestep will only push the body for a short time. Thus forces are best used for many timesteps to give a continuous push to a body (like gravity). For a single push that is independent of timestep, it is better to use Body:applyLinearImpulse.

If the position to apply the force is not given, it will act on the center of mass of the body. The part of the force not directed towards the center of mass will cause the body to spin (and depends on the rotational inertia).

Note that the force components and position must be given in world coordinates.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
Body:applyForce( fx, fy )
```

Arguments

`number fx`
: The x component of force to apply to the center of mass.

`number fy`
: The y component of force to apply to the center of mass.

Returns

Nothing.

Function

Synopsis

```
Body:applyForce( fx, fy, x, y )
```

Arguments

`number fx`
: The x component of force to apply.

`number fy`
: The y component of force to apply.

`number x`
: The x position to apply the force.

`number y`

The y position to apply the force.

Returns

Nothing.

See Also

- Body
- Body:applyTorque
- Body:applyLinearImpulse

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:applyImpulse

Body:applyImpulse · revision 8375 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

It has been renamed to Body:applyLinearImpulse.

Applies an impulse to a body. This makes a single, instantaneous addition to the body momentum.

An impulse pushes a body in a direction. A body with with a larger mass will react less. The reaction does **not** depend on the timestep, and is equivalent to applying a force continuously for 1 second. Impulses are best used to give a single push to a body. For a continuous push to a body it is better to use Body:applyForce.

If the position to apply the impulse is not given, it will act on the center of mass of the body. The part of the impulse not directed towards the center of mass will cause the body to spin (and depends on the rotational inertia).

Note that the impulse components and position must be given in world coordinates.

Function

Synopsis

```
Body:applyImpulse( ix, iy, x, y )
```

Arguments

`number ix`
: The x component of the impulse.

`number iy`
: The y component of the impulse.

`number x`
: The x position to apply the impulse.

`number y`
: The y position to apply the impulse.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:applyLinearImpulse

Body:applyLinearImpulse · revision 30209 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Applies an impulse to a body. This makes a single, instantaneous addition to the body momentum.

An impulse pushes a body in a direction. A body with a larger mass will react less. The reaction does **not** depend on the timestep, and is equivalent to applying a force continuously for 1 second. Impulses are best used to give a single push to a body. For a continuous push to a body it is better to use Body:applyForce.

If the position to apply the impulse is not given, it will act on the center of mass of the body. The part of the impulse not directed towards the center of mass will cause the body to spin (and depends on the rotational inertia).

Note that the impulse components and position must be given in world coordinates.

Function

Synopsis

```
Body:applyLinearImpulse( ix, iy )
```

Arguments

number ix
: The x component of the impulse applied to the center of mass.

number iy
: The y component of the impulse applied to the center of mass.

Returns

Nothing.

Function

Synopsis

```
Body:applyLinearImpulse( ix, iy, x, y )
```

Arguments

number ix
: The x component of the impulse.

number iy
: The y component of the impulse.

number x
: The x position to apply the impulse.

number y

The y position to apply the impulse.

Returns

Nothing.

See Also

- Body
- Body:applyAngularImpulse
- Body:getLinearVelocity
- Body:setLinearVelocity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:applyTorque

Body:applyTorque · revision 24134 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Apply torque to a body.

Torque is like a force that will change the angular velocity (spin) of a body. The effect will depend on the rotational inertia a body has.

Function

Synopsis

```
Body:applyTorque( torque )
```

Arguments

`number torque`
: The torque to apply.

Returns

Nothing.

See Also

- Body
- Body:applyForce
- Body:applyAngularImpulse
- Body:getAngle
- Body:getAngularVelocity
- Body:setAngularVelocity
- Body:getInertia

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:destroy

Body:destroy · revision 8377 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Explicitly destroys the Body and all fixtures and joints attached to it.

An error will occur if you attempt to use the object after calling this function. In 0.7.2, when you don't have time to wait for garbage collection, this function may be used to free the object immediately.

Function

Synopsis

```
Body:destroy( )
```

Arguments

None.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getAllowSleeping

Body:getAllowSleeping · revision 17994 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

This method is not supported in earlier or later versions.

Return whether a body is allowed to sleep.

A sleeping body is much more efficient to simulate than when awake.

Function

Synopsis

```
status = Body:getAllowSleeping( )
```

Arguments

None.

Returns

`boolean status`

True when the body is allowed to sleep.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getAngle

Body:getAngle · revision 24150 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the angle of the body.

The angle is measured in radians (http://en.wikipedia.org/wiki/Radian). If you need to transform it to degrees, use math.deg (http://www.lua.org/manual/5.1/manual.html#pdf-math.deg).

A value of 0 radians will mean "looking to the right". Although radians increase counter-clockwise, the y axis points down so it becomes *clockwise* from our point of view.

Function

Synopsis

```
angle = Body:getAngle( )
```

Arguments

None.

Returns

`number angle`
: The angle in radians.

See Also

- Body
- Body:setAngle
- Body:getAngularVelocity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getAngularDamping

Body:getAngularDamping · revision 24137 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the Angular damping of the Body

The angular damping is the *rate of decrease of the angular velocity over time*: A spinning body with no damping and no external forces will continue spinning indefinitely. A spinning body with damping will gradually stop spinning.

Damping is not the same as friction - they can be modelled together. However, only damping is provided by Box2D (and LOVE).

Damping parameters should be between 0 and infinity, with 0 meaning no damping, and infinity meaning full damping. Normally you will use a damping value between 0 and 0.1.

Function

Synopsis

```
damping = Body:getAngularDamping( )
```

Arguments

None.

Returns

number damping
: The value of the angular damping.

See Also

- Body
- Body:setAngularDamping
- Body:getLinearDamping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getAngularVelocity

Body:getAngularVelocity · revision 30215 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the angular velocity of the Body.

The angular velocity is the *rate of change of angle over time*.

It is changed in World:update by applying torques, off centre forces/impulses, and angular damping. It can be set directly with Body:setAngularVelocity.

If you need the *rate of change of position over time*, use Body:getLinearVelocity.

Function

Synopsis

```
w = Body:getAngularVelocity( )
```

Arguments

None.

Returns

number w
: The angular velocity in radians/second.

See Also

- Body
- Body:setAngularVelocity
- Body:getAngle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getContactList

Body:getContactList · revision 24059 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

It has been renamed to Body:getContacts.

Available since LÖVE 0.9.2

This method is not supported in earlier versions.

Gets a list of all Contacts attached to the Body.


The list can change multiple times during a World:update call, so some collisions may be missed if the World collide callbacks are not used at all.

Function

Synopsis

```
contacts = Body:getContactList( )
```

Arguments

None.

Returns

`table contacts`

A list with all contacts associated with the Body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getContacts

Body:getContacts · revision 29560 · last edited 2025-03-29

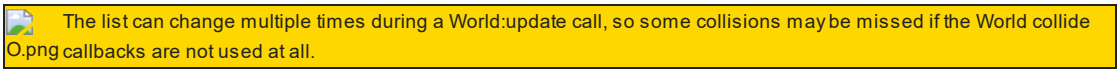
On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from Body:getContactList.

Gets a list of all Contacts attached to the Body.



Function

Synopsis

```
contacts = Body:getContacts( )
```

Arguments

None.

Returns

`table contacts`

A list with all contacts associated with the Body.

See Also

- Body
- Contact
- World:getContacts
- World:setCallbacks

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getFixtureList

Body:getFixtureList · revision 24060 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

It has been renamed to Body:getFixtures.

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns a table with all fixtures.

Function

Synopsis

```
fixtures = Body:getFixtureList( )
```

Arguments

None.

Returns

table **fixtures**

A sequence with all fixtures.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getFixtures

Body:getFixtures · revision 24136 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from Body:getFixtureList.

Returns a table with all fixtures.

Function

Synopsis

```
fixtures = Body:getFixtures( )
```

Arguments

None.

Returns

`table fixtures`

A sequence with all fixtures.

See Also

- Body
- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getGravityScale

Body:getGravityScale · revision 24139 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the gravity scale factor.

Function

Synopsis

```
scale = Body:getGravityScale( )
```

Arguments

None.

Returns

`number scale`

The gravity scale factor.

See Also

- Body
- Body:setGravityScale
- World:setGravity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Body:getInertia

Body:getInertia · revision 24140 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the rotational inertia of the body.

The rotational inertia is how hard is it to make the body spin.

Function

Synopsis

```
inertia = Body:getInertia( )
```

Arguments

None.

Returns

`number inertia`
: The rotational inertial of the body.

See Also

- Body
- Body:setInertia
- Body:getMassData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getJointList

Body:getJointList · revision 24061 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

It has been renamed to Body:getJoints.

Available since LÖVE 0.9.2

This method is not supported in earlier versions.

Returns a table containing the Joints attached to this Body.

Function

Synopsis

```
joints = Body:getJointList( )
```

Arguments

None.

Returns

table joints

A sequence with the Joints attached to the Body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getJoints

Body:getJoints · revision 24064 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from Body:getJointList.

Returns a table containing the Joints attached to this Body.

Function

Synopsis

```
joints = Body:getJoints( )
```

Arguments

None.

Returns

`table joints`

A sequence with the Joints attached to the Body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLinearDamping

Body:getLinearDamping · revision 14983 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the linear damping of the Body.

The linear damping is the *rate of decrease of the linear velocity over time*. A moving body with no damping and no external forces will continue moving indefinitely, as is the case in space. A moving body with damping will gradually stop moving.

Damping is not the same as friction - they can be modelled together.

Function

Synopsis

```
damping = Body:getLinearDamping( )
```

Arguments

None.

Returns

`number damping`
: The value of the linear damping.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLinearVelocity

Body:getLinearVelocity · revision 6248 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Gets the linear velocity of the Body from its center of mass.

The linear velocity is the *rate of change of position over time*.

If you need the *rate of change of angle over time*, use Body:getAngularVelocity. If you need to get the linear velocity of a point different from the center of mass:

- Body:getLinearVelocityFromLocalPoint allows you to specify the point in local coordinates.
- Body:getLinearVelocityFromWorldPoint allows you to specify the point in world coordinates.

See page 136 of "Essential Mathematics for Games and Interactive Applications" (http://books.google.com/books?id=g0-G2Dbk5vEC&lpg=PA250&ots=H6T6pJsOqu&dq=%22Essential%20Mathematics%20for%20Games%20and%20Interactive%20Applications%20local%20space&pg=PA136#v=onepage&q&f=false) for definitions of local and world coordinates.

Function

Synopsis

```
x, y = Body:getLinearVelocity( )
```

Arguments

None.

Returns

number **x**
: The x-component of the velocity vector

number **y**
: The y-component of the velocity vector

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLinearVelocityFromLocalPoint

Body:getLinearVelocityFromLocalPoint · revision 5988 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the linear velocity of a point on the body.

The linear velocity for a point on the body is the velocity of the body center of mass plus the velocity at that point from the body spinning.

The point on the body must given in local coordinates. Use Body:getLinearVelocityFromWorldPoint to specify this with world coordinates.

Function

Synopsis

```
vx, vy = Body:getLinearVelocityFromLocalPoint( x, y )
```

Arguments

number x
: The x position to measure velocity.

number y
: The y position to measure velocity.

Returns

number vx
: The x component of velocity at point (x,y).

number vy
: The y component of velocity at point (x,y).

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLinearVelocityFromWorldPoint

Body:getLinearVelocityFromWorldPoint · revision 5989 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Get the linear velocity of a point on the body.

The linear velocity for a point on the body is the velocity of the body center of mass plus the velocity at that point from the body spinning.

The point on the body must given in world coordinates. Use Body:getLinearVelocityFromLocalPoint to specify this with local coordinates.

Function

Synopsis

```
vx, vy = Body:getLinearVelocityFromWorldPoint( x, y )
```

Arguments

number x
: The x position to measure velocity.

number y
: The y position to measure velocity.

Returns

number vx
: The x component of velocity at point (x,y).

number vy
: The y component of velocity at point (x,y).

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLocalCenter

Body:getLocalCenter · revision 5990 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the center of mass position in local coordinates.

Use Body:getWorldCenter to get the center of mass in world coordinates.

Function

Synopsis

```
x, y = Body:getLocalCenter( )
```

Arguments

None.

Returns

number x
: The x coordinate of the center of mass.

number y
: The y coordinate of the center of mass.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLocalPoint

Body:getLocalPoint · revision 25859 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Transform a point from world coordinates to local coordinates.

Function

Synopsis

```
localX, localY = Body:getLocalPoint( worldX, worldY )
```

Arguments

number worldX
: The x position in world coordinates.

number worldY
: The y position in world coordinates.

Returns

number localX
: The x position in local coordinates.

number localY
: The y position in local coordinates.

See Also

- Body
- Body:getWorldPoint
- Body:getWorldPoints

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLocalPoints

Body:getLocalPoints · revision 27413 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.4

This method is not supported in earlier versions.

Transforms multiple points from world coordinates to local coordinates.

Function

Synopsis

```
x1, y1, x2, y2, ... = Body:getLocalPoints( x1, y1, x2, y2, ... )
```

Arguments

number x1
: The x position of the first point.

number y1
: The y position of the first point.

number x2
: The x position of the second point.

number y2
: The y position of the second point.

number ...
: You can continue passing x and y position of the points.

Returns

number x1
: The transformed x position of the first point.

number y1
: The transformed y position of the first point.

number x2
: The transformed x position of the second point.

number y2
: The transformed y position of the second point.

number ...
: Additional transformed x and y position of the points.

See Also

- Body
- Body:getWorldPoints
- Body:getWorldPoint
- Body:getLocalPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getLocalVector

Body:getLocalVector · revision 16122 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Transform a vector from world coordinates to local coordinates.

Function

Synopsis

```
localX, localY = Body:getLocalVector( worldX, worldY )
```

Arguments

number worldX
: The vector x component in world coordinates.

number worldY
: The vector y component in world coordinates.

Returns

number localX
: The vector x component in local coordinates.

number localY
: The vector y component in local coordinates.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getMass

Body:getMass · revision 14984 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the mass of the body.

Static bodies always have a mass of 0.

Function

Synopsis

```
mass = Body:getMass( )
```

Arguments

None.

Returns

number mass
: The mass of the body (in kilograms).

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getMassData

Body:getMassData · revision 8638 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the mass, its center, and the rotational inertia.

Function

Synopsis

```
x, y, mass, inertia = Body:getMassData( )
```

Arguments

None.

Returns

number x
: The x position of the center of mass.

number y
: The y position of the center of mass.

number mass
: The mass of the body.

number inertia
: The rotational inertia.

See Also

- Body
- Body:setMassData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getPosition

Body:getPosition · revision 5994 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the position of the body.

Note that this may not be the center of mass of the body.

Function

Synopsis

```
x, y = Body:getPosition( )
```

Arguments

None.

Returns

number x
: The x position.

number y
: The y position.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getShape

Body:getShape · revision 29137 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets the first Shape attached to the Body.

Function

Synopsis

```
shape = Body:getShape( )
```

Arguments

None.

Returns

Shape **shape**

The first Shape attached to the Body.

See Also

- Body
- Shape
- Body:getShapes

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Body:getShapes

Body:getShapes · revision 29138 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has replaced Body:getFixtures.

Returns a table with all Shapes attached to the Body.

Function

Synopsis

```
shapes = Body:getShapes( )
```

Arguments

None.

Returns

`table shapes`

A sequence with all shapes.

See Also

- Body
- Shape
- Body:getShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Body:getTransform

Body:getTransform · revision 23369 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.2

This function is not supported in earlier versions.

Get the position and angle of the body.

Note that the position may not be the center of mass of the body. An angle of 0 radians will mean "looking to the right". Although radians increase counter-clockwise, the y axis points down so it becomes clockwise from our point of view.

Function

Synopsis

```
x, y, angle = Body:getTransform( )
```

Arguments

None.

Returns

number x
: The x component of the position.

number y
: The y component of the position.

number angle
: The angle in radians.

See Also

- Body
- Body:setTransform

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getType

Body:getType · revision 8639 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the type of the body.

Function

Synopsis

```
type = Body:getType( )
```

Arguments

None.

Returns

`BodyType type`

The body type.

See Also

- Body
- Body:setType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getUserData

Body:getUserData · revision 19352 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This method is not supported in earlier versions.

Returns the Lua value associated with this Body.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
value = Body:getUserData( )
```

Arguments

None.

Returns

`any value`
: The Lua value associated with the Body.

See Also

- Body
- Body:setUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getWorld

Body:getWorld · revision 13701 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This method is not supported in earlier versions.

Gets the World the body lives in.

Function

Synopsis

```
world = Body:getWorld( )
```

Arguments

None.

Returns

`World world`

The world the body lives in.

See Also

- Body
- World

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getWorldCenter

Body:getWorldCenter · revision 5995 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the center of mass position in world coordinates.

Use Body:getLocalCenter to get the center of mass in local coordinates.

Function

Synopsis

```
x, y = Body:getWorldCenter( )
```

Arguments

None.

Returns

number x
: The x coordinate of the center of mass.

number y
: The y coordinate of the center of mass.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getWorldPoint

Body:getWorldPoint · revision 25857 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Transform a point from local coordinates to world coordinates.

Function

Synopsis

```
worldX, worldY = Body:getWorldPoint( localX, localY )
```

Arguments

number localX
: The x position in local coordinates.

number localY
: The y position in local coordinates.

Returns

number worldX
: The x position in world coordinates.

number worldY
: The y position in world coordinates.

See Also

- Body
- Body:getWorldPoints
- Body:getLocalPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getWorldPoints

Body:getWorldPoints · revision 26817 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Transforms multiple points from local coordinates to world coordinates.

Function

Synopsis

```
x1, y1, x2, y2, ... = Body:getWorldPoints( x1, y1, x2, y2, ... )
```

Arguments

number x1
: The x position of the first point.

number y1
: The y position of the first point.

number x2
: The x position of the second point.

number y2
: The y position of the second point.

Returns

number x1
: The transformed x position of the first point.

number y1
: The transformed y position of the first point.

number x2
: The transformed x position of the second point.

number y2
: The transformed y position of the second point.

See Also

- Body
- Body:getWorldPoint
- Body:getLocalPoint
- Body:getLocalPoints

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getWorldVector

Body:getWorldVector · revision 16124 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Transform a vector from local coordinates to world coordinates.

Function

Synopsis

```
worldX, worldY = Body:getWorldVector( localX, localY )
```

Arguments

number localX
: The vector x component in local coordinates.

number localY
: The vector y component in local coordinates.

Returns

number worldX
: The vector x component in world coordinates.

number worldY
: The vector y component in world coordinates.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getX

Body:getX · revision 5998 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the x position of the body in world coordinates.

Function

Synopsis

```
x = Body:getX( )
```

Arguments

None.

Returns

`number x`
: The x position in world coordinates.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:getY

Body:getY · revision 5999 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the y position of the body in world coordinates.

Function

Synopsis

```
y = Body:getY( )
```

Arguments

None.

Returns

number y
: The y position in world coordinates.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:hasCustomMassData

Body:hasCustomMassData · revision 29267 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets whether the Body's mass data has been set to custom values via Body:setMass, Body:setInertia, or Body:setMassData.

When a Body does not have customized mass data, it is automatically calculated based on attached Shapes and their densities.

Function

Synopsis

```
custommass = Body:hasCustomMassData( )
```

Arguments

None.

Returns

`boolean custommass`

True if the body has custom mass data, false if not.

See Also

- Body
- Body:resetMassData
- Body:setMass
- Body:setInertia
- Body:setMassData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isActive

Body:isActive · revision 8641 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns whether the body is actively used in the simulation.

Function

Synopsis

```
status = Body:isActive( )
```

Arguments

None.

Returns

`boolean status`

True if the body is active or false if not.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isAwake

Body:isAwake · revision 8642 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the sleep status of the body.

Function

Synopsis

```
status = Body:isAwake( )
```

Arguments

None.

Returns

`boolean status`

True if the body is awake or false if not.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isBullet

Body:isBullet · revision 6000 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the bullet status of a body.

There are two methods to check for body collisions:

- at their location when the world is updated (default)
- using continuous collision detection (CCD)

The default method is efficient, but a body moving very quickly may sometimes jump over another body without producing a collision. A body that is set as a bullet will use CCD. This is less efficient, but is guaranteed not to jump when moving quickly.

Note that static bodies (with zero mass) always use CCD, so your walls will not let a fast moving body pass through even if it is not a bullet.

Function

Synopsis

```
status = Body:isBullet( )
```

Arguments

None.

Returns

`boolean status`
: The bullet status of the body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isDestroyed

Body:isDestroyed · revision 14637 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether the Body is destroyed. Destroyed bodies cannot be used.

Function

Synopsis

```
destroyed = Body:isDestroyed( )
```

Arguments

None.

Returns

`boolean destroyed`

Whether the Body is destroyed.

See Also

- Body
- Body:destroy

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isDynamic

Body:isDynamic · revision 15825 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.8.0
It has been replaced by Body:getType.

Get the dynamic status of the body.

A static body has no mass and a constant position. It will not react to collisions. Often used for walls.

A dynamic body has mass and can move. It will react to collisions when the world is updated.

Function

Synopsis

```
status = Body:isDynamic( )
```

Arguments

None.

Returns

`boolean status`
The dynamic status of the body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isFixedRotation

Body:isFixedRotation · revision 29112 · last edited 2024-02-17

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns whether the body rotation is locked.

Function

Synopsis

```
fixed = Body:isFixedRotation( )
```

Arguments

None.

Returns

`boolean fixed`

True if the body's rotation is locked or false if not.

See Also

- Body
- Body:setFixedRotation

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isFrozen

Body:isFrozen · revision 8775 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the frozen status of the body.

A body becomes frozen when it goes outside the world boundary. A frozen body is no longer changed by World:update.

Function

Synopsis

```
status = Body:isFrozen( )
```

Arguments

None.

Returns

`boolean status`

The frozen status of the body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isSleeping

Body:isSleeping · revision 8776 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the sleeping status of a body.

A sleeping body is much more efficient to simulate than when awake.

If sleeping is allowed, a body that has come to rest will sleep.

Function

Synopsis

```
status = Body:isSleeping( )
```

Arguments

None.

Returns

`boolean status`

The sleeping status of the body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isSleepingAllowed

Body:isSleepingAllowed · revision 8644 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the sleeping behaviour of the body.

Function

Synopsis

```
allowed = Body:isSleepingAllowed( )
```

Arguments

None.

Returns

`boolean allowed`

True if the body is allowed to sleep or false if not.

See Also

- Body
- Body:setSleepingAllowed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isStatic

Body:isStatic · revision 8777 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the static status of the body.

A static body has no mass and a constant position. It will not react to collisions. Often used for walls.

A dynamic body has mass and can move. It will react to collisions when the world is updated.

Function

Synopsis

```
status = Body:isStatic( )
```

Arguments

None.

Returns

`boolean status`

The static status of the body.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:isTouching

Body:isTouching · revision 24083 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether the Body is touching the given other Body.


Using this in love.update may miss collisions, since World:update can begin and end a collision within a single call. The collision callbacks are recommended instead, when feasible.

Function

Synopsis

```
touching = Body:isTouching( otherbody )
```

Arguments

`Body otherbody`
: The other body to check.

Returns

`boolean touching`
: True if this body is touching the other body, false otherwise.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:putToSleep

Body:putToSleep · revision 8778 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Put the body to sleep.

A sleeping body is much more efficient to simulate than when awake.

The body will wake up if another body collides with it, if a joint or contact attached to it is destroyed, or if Body:wakeUp is called.

Function

Synopsis

```
Body:putToSleep( )
```

Arguments

None.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:resetMassData

Body:resetMassData · revision 8645 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Resets the mass of the body by recalculating it from the mass properties of the fixtures.

Function

Synopsis

```
Body:resetMassData( )
```

Arguments

None.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setActive

Body:setActive · revision 8646 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets whether the body is active in the world.

An inactive body does not take part in the simulation. It will not move or cause any collisions.

Function

Synopsis

```
Body:setActive( active )
```

Arguments

`boolean active`
: If the body is active or not.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setAllowSleeping

Body:setAllowSleeping · revision 15828 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Removed in LÖVE 0.8.0

It has been replaced by Body:setSleepingAllowed.

Set the sleep behaviour of a body.

A sleeping body is much more efficient to simulate than when awake.

If sleeping is allowed, a body that has come to rest will sleep.

Function

Synopsis

```
Body:setAllowSleeping( permission )
```

Arguments

`boolean permission`
: Permission for the body to sleep.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setAngle

Body:setAngle · revision 30214 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the angle of the body.

The angle is measured in radians (http://en.wikipedia.org/wiki/Radian). If you need to transform it from degrees, use math.rad (http://www.lua.org/manual/5.1/manual.html#pdf-math.rad).

A value of 0 radians will mean "looking to the right". Although radians increase counter-clockwise, the y axis points down so it becomes *clockwise* from our point of view.

It is possible to cause a collision with another body by changing its angle.

Function

Synopsis

```
Body:setAngle( angle )
```

Arguments

`number angle`
: The angle in radians.

Returns

Nothing.

See Also

- Body
- Body:getAngle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setAngularDamping

Body:setAngularDamping · revision 6008 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sets the angular damping of a Body

See Body:getAngularDamping for a definition of angular damping.

Angular damping can take any value from 0 to infinity. It is recommended to stay between 0 and 0.1, though. Other values will look unrealistic.

Function

Synopsis

```
Body:setAngularDamping( damping )
```

Arguments

number damping
: The new angular damping.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setAngularVelocity

Body:setAngularVelocity · revision 6009 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the angular velocity of a Body.

The angular velocity is the *rate of change of angle over time*.

This function will not accumulate anything; any impulses previously applied since the last call to World:update will be lost.

Function

Synopsis

```
Body:setAngularVelocity( w )
```

Arguments

number w
: The new angular velocity, in radians per second

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setAwake

Body:setAwake · revision 8647 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Wakes the body up or puts it to sleep.

Function

Synopsis

```
Body:setAwake( awake )
```

Arguments

`boolean awake`
: The body sleep status.

Returns

Nothing.

See Also

- Body
- Body:isAwake

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setBullet

Body:setBullet · revision 24129 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the bullet status of a body.

There are two methods to check for body collisions:

- at their location when the world is updated (default)
- using continuous collision detection (CCD)

The default method is efficient, but a body moving very quickly may sometimes jump over another body without producing a collision. A body that is set as a bullet will use CCD. This is less efficient, but is guaranteed not to jump when moving quickly.

Note that static bodies (with zero mass) always use CCD, so your walls will not let a fast moving body pass through even if it is not a bullet.

Function

Synopsis

```
Body:setBullet( status )
```

Arguments

`boolean status`
: The bullet status of the body.

Returns

Nothing.

See Also

- Body
- Body:isBullet

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setFixedRotation

Body:setFixedRotation · revision 22586 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set whether a body has fixed rotation.

Bodies with fixed rotation don't vary the speed at which they rotate. Calling this function causes the mass to be reset.


LÖVE 0.6.2 does not contain the fix for a bug with this function when it is called with a false argument.

Function

Synopsis

```
Body:setFixedRotation( isFixed )
```

Arguments

`boolean isFixed`
: Whether the body should have fixed rotation.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setGravityScale

Body:setGravityScale · revision 8648 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new gravity scale factor for the body.

Function

Synopsis

```
Body:setGravityScale( scale )
```

Arguments

`number scale`
: The new gravity scale factor.

Returns

Nothing.

See Also

- Body
- Body:getGravityScale

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setInertia

Body:setInertia · revision 20416 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the inertia of a body.

Function

Synopsis

```
Body:setInertia( inertia )
```

Arguments

`number inertia`
: The new moment of inertia, in kilograms * pixel squared.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setLinearDamping

Body:setLinearDamping · revision 28303 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Sets the linear damping of a Body. The linear damping is the rate of decrease of the linear velocity over time. A moving body with no damping and no external forces will continue moving indefinitely, as is the case in space. A moving body with damping will gradually stop moving.

Linear damping can take any value from 0 to infinity. It is recommended to stay between 0 and 1, though. Other values will make the objects look "floaty"(if gravity is enabled).

Function

Synopsis

```
Body:setLinearDamping( ld )
```

Arguments

number ld
: The new linear damping

Returns

Nothing.

See Also

- Body
- Body:getLinearDamping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setLinearVelocity

Body:setLinearVelocity · revision 6014 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets a new linear velocity for the Body.

This function will not accumulate anything; any impulses previously applied since the last call to World:update will be lost.

Function

Synopsis

```
Body:setLinearVelocity( x, y )
```

Arguments

number x
: The x-component of the velocity vector.

number y
: The y-component of the velocity vector.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setMass

Body:setMass · revision 17958 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Sets a new body mass.

Function

Available since LÖVE 0.8.0
This variant is not supported in earlier versions.

Synopsis

```
body:setMass( mass )
```

Arguments

number mass
: The mass, in kilograms.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Sets the mass properties directly.

If you're not sure what all this stuff means, you can use Body:setMassFromShapes after adding shapes instead.

The first two parameters will be the local coordinates of the Body's center of mass (http://en.w ikipedia.org/w iki/Center_of_mass).

The third parameter is the mass, in kilograms.

The last parameter is the rotational inertia (http://en.w ikipedia.org/w iki/Moment_of_inertia).

Synopsis

```
body:setMass( x, y, m, i )
```

Arguments

number x
: The x-component of the center of mass in local coordinates.

number y
: The y-component of the center of mass in local coordinates.

number m
: The mass, in kilograms.

number i

The rotational inertia, in kilograms per squared meter.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setMassData

Body:setMassData · revision 8649 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Overrides the calculated mass data.

Function

Synopsis

```
Body:setMassData( x, y, mass, inertia )
```

Arguments

`number x`
: The x position of the center of mass.

`number y`
: The y position of the center of mass.

`number mass`
: The mass of the body.

`number inertia`
: The rotational inertia.

Returns

Nothing.

See Also

- Body
- Body:getMassData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setMassFromShapes

Body:setMassFromShapes · revision 8443 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Sets mass properties from attatched shapes.

If you feel that finding the correct mass properties is tricky, then this function may be able to help you. After creating the needed shapes on the Body, a call to this function will set the mass properties based on those shapes. Remember to call this function after adding the shapes.

Setting the mass properties this way always results in a realistic (or at least good-looking) simulation, so using it is highly recommended.

Function

Synopsis

```
Body:setMassFromShapes( )
```

Arguments

None.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setPosition

Body:setPosition · revision 13070 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Set the position of the body.

Note that this may not be the center of mass of the body.

This function cannot wake up the body.

Function

Synopsis

```
Body:setPosition( x, y )
```

Arguments

number x
: The x position.

number y
: The y position.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setSleepingAllowed

Body:setSleepingAllowed · revision 15057 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the sleeping behaviour of the body. Should sleeping be allowed, a body at rest will automatically sleep. A sleeping body is not simulated unless it collided with an awake body. Be wary that one can end up with a situation like a floating sleeping body if the floor was removed.

Function

Synopsis

```
Body:setSleepingAllowed( allowed )
```

Arguments

`boolean allowed`

True if the body is allowed to sleep or false if not.

Returns

Nothing.

See Also

- Body
- Body:isSleepingAllowed

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setTransform

Body:setTransform · revision 21878 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.2

This function is not supported in earlier versions.

Set the position and angle of the body.

Note that the position may not be the center of mass of the body. An angle of 0 radians will mean "looking to the right". Although radians increase counter-clockwise, the y axis points down so it becomes clockwise from our point of view.

This function cannot wake up the body.

Function

Synopsis

```
Body:setTransform( x, y, angle )
```

Arguments

`number x`
: The x component of the position.

`number y`
: The y component of the position.

`number angle`
: The angle in radians.

Returns

Nothing.

See Also

- Body
- Body:getTransform
- Body:setPosition
- Body:setAngle

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setType

Body:setType · revision 18004 · last edited 2025-11-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets a new body type.

Function

Synopsis

```
Body:setType( type )
```

Arguments

`BodyType type`
: The new type.

Returns

Nothing.

See Also

- Body
- Body:getType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setUserData

Body:setUserData · revision 19351 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This method is not supported in earlier versions.

Associates a Lua value with the Body.

To delete the reference, explicitly pass nil.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
Body:setUserData( value )
```

Arguments

`any value`
: The Lua value to associate with the Body.

Returns

Nothing.

See Also

- Body
- Body:getUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setX

Body:setX · revision 13068 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the x position of the body.

This function cannot wake up the body.

Function

Synopsis

```
Body:setX( x )
```

Arguments

`number x`
: The x position.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:setY

Body:setY · revision 13069 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Set the y position of the body.

This function cannot wake up the body.

Function

Synopsis

```
Body:setY( y )
```

Arguments

number y
: The y position.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Body:wakeUp

Body:wakeUp · revision 8782 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Wake a sleeping body up.

A sleeping body is much more efficient to simulate than when awake.

A sleeping body will also wake up if another body collides with it or if a joint or contact attached to it is destroyed.

Function

Synopsis

```
Body:wakeUp( )
```

Arguments

None.

Returns

Nothing.

See Also

- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BodyType

BodyType · revision 18836 · last edited 2025-11-22

The types of a Body.

Constants

static
: Static bodies do not move.

dynamic
: Dynamic bodies collide with all bodies.

kinematic
: Kinematic bodies only collide with dynamic bodies.

See Also

- Body
- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BookOfBartbes

BookOfBartbes · revision 13864 · last edited 2021-03-14

The Gospel of Bartbes has been taken down.

BoundingBox.lua

BoundingBox.lua · revision 28424 · last edited 2023-07-05

This script is used for axis-aligned bounding box (AABB) collision detection. It will check whether two rectangular objects overlap. It returns true if they do, and false otherwise.

This form of collision detection is not suited for games that have lots of circles, non-rectangular shapes, or rotated rectangles, since space not filled by those shapes may still be part of the collision rectangle, or alternatively there may be parts that lie outside of it.

However, this should work fine for games such as Space Invaders clones.

```
-- Collision detection function;
-- Returns true if two boxes overlap, false if they don't;
-- x1,y1 are the top-left coords of the first box, while w1,h1 are its width and height;
-- x2,y2,w2 & h2 are the same, but for the second box.
function CheckCollision(x1,y1,w1,h1, x2,y2,w2,h2)
  return x1 < x2+w2 and
         x2 < x1+w1 and
         y1 < y2+h2 and
         y2 < y1+h1
end
```

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Bubbles

Bubbles · revision 2836 · last edited 2021-03-14

Bubbles is a Jellyfish-controlling, baby-saving game of underwater adventure developed by Roybie.

Created for the **Experimental Gameplay Project** (http://experimentalgameplay.com/)



The download, more details and instructions can be found **here** (http://yaygames.yiay.org/index.php/2010/07/junejuly-egp-casual-addiction-2/).

BufferDataFormat

BufferDataFormat · revision 29646 · last edited 2026-08-08

Available since LÖVE 12.0

This enum is not supported in earlier versions.

Data formats of individual array elements in GraphicsBuffers and Meshes.

Constants

Data Format string	Description	Vertex buffers	Index buffers	Texel buffers	Shader Storage buffers	Indirect Arguments buffers
float	Single 32 bit floating point number.	Yes		Yes	Yes	
floatvec2	2-component vector of 32 bit floats.	Yes		Yes	Yes	
floatvec3	3-component vector of 32 bit floats.	Yes		Yes	Yes	
floatvec4	4-component vector of 32 bit floats.	Yes		Yes	Yes	
floatmat2x2	Matrix of 32 bit floats with 2 rows and 2 columns.				Yes	
floatmat2x3	Matrix of 32 bit floats with 2 rows and 3 columns.				Yes	
floatmat2x4	Matrix of 32 bit floats with 2 rows and 4 columns.				Yes	
floatmat3x2	Matrix of 32 bit floats with 3 rows and 2 columns.				Yes	
floatmat3x3	Matrix of 32 bit floats with 3 rows and 3 columns.				Yes	
floatmat3x4	Matrix of 32 bit floats with 3 rows and 4 columns.				Yes	
floatmat4x2	Matrix of 32 bit floats with 4 rows and 2 columns.				Yes	
floatmat4x3	Matrix of 32 bit floats with 4 rows and 3 columns.				Yes	
floatmat4x4	Matrix of 32 bit floats with 4 rows and 4 columns.				Yes	
int32	Single 32 bit signed integer.	Yes		Yes	Yes	Yes
int32vec2	2-component vector of 32 bit signed integers.	Yes		Yes	Yes	
int32vec3	3-component vector of 32 bit signed integers.	Yes		Yes	Yes	
int32vec4	4-component vector of 32 bit signed integers.	Yes		Yes	Yes	
uint32	Single 32 bit unsigned integer.	Yes	Yes	Yes	Yes	Yes
uint32vec2	2-component vector of 32 bit unsigned integers.	Yes		Yes	Yes	
uint32vec3	3-component vector of 32 bit unsigned integers.	Yes		Yes	Yes	
uint32vec4	4-component vector of 32 bit unsigned integers.	Yes		Yes	Yes	
snorm8vec4	4-component vector of 8 bit signed fixed-point normalized numbers.	Yes				
unorm8vec4	4-component vector of 8 bit unsigned fixed-point normalized numbers. Useful for colors.	Yes		Yes		
int8vec4	4-component vector of 8 bit signed integers.	Yes		Yes		

uint8vec4	4-component vector of 8 bit unsigned integers.	Yes		Yes		
snorm16vec2	2-component vector of 16 bit signed fixed-point normalized numbers.	Yes				
snorm16vec4	4-component vector of 16 bit signed fixed-point normalized numbers.	Yes				
unorm16vec2	2-component vector of 16 bit unsigned fixed-point normalized numbers.	Yes		Yes		
unorm16vec4	4-component vector of 16 bit unsigned fixed-point normalized numbers.	Yes		Yes		
int16vec2	2-component vector of 16 bit signed integers.	Yes		Yes		
int16vec4	4-component vector of signed 16 bit integers.	Yes		Yes		
uint16	Single 16 bit unsigned integer.	Yes	Yes	Yes		
uint16vec2	4-component vector of 16 bit unsigned integers.	Yes		Yes		
uint16vec4	4-component vector of 16 bit unsigned integers.	Yes		Yes		

See Also

- love.graphics
- GraphicsBuffer
- love.graphics.newBuffer
- love.graphics.newMesh
- GraphicsBuffer:getFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BufferDataUsage

BufferDataUsage · revision 29266 · last edited 2025-10-23

Available since LÖVE 0.8.0

This enum is not supported in earlier versions.

Usage hints for SpriteBatches, Meshes, and GraphicsBuffers to optimize data storage and access.

Constants

dynamic

The object's data will change fairly frequently during its lifetime.

static

The object will not be modified frequently or at all.

stream

The object data will always change every frame.

See Also

- love.graphics
- love.graphics.newMesh
- love.graphics.newSpriteBatch
- SpriteBatch
- Mesh
- GraphicsBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

BufferMode

BufferMode · revision 11000 · last edited 2023-07-05

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Buffer modes for File objects.

Constants

none

No buffering. The result of write and append operations appears immediately.

line

Line buffering. Write and append operations are buffered until a newline is output or the buffer size limit is reached.

full

Full buffering. Write and append operations are always buffered until the buffer size limit is reached.

See Also

- File
- File:setBuffer
- File:getBuffer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

bump.lua

bump.lua · revision 16323 · last edited 2023-07-05

bump.lua is a very barebones collision detection library. It only manages axis aligned bounding boxes, but has a very nice interface for doing so.

Please refer to the forum post below or the source code link for examples of use.

Links

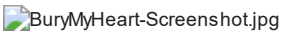
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=9489)
- Source code (https://github.com/kikito/bump.lua)

BuryMyHeart

BuryMyHeart · revision 2542 · last edited 2021-03-14

Bury My Heart at Towlr Mountain

BuryMyHeart is a certain kind of game, known as a Towlr. Among several requirements, Towlr must not give the player any indication of what goals the player is trying to achieve. In other words, the purpose of a Towlr is to figure out the goals of the game (and then complete the game, of course). Read the full Towlr manifesto to see all the requirements.





Download BuryMyHeart here (http://williambowers.net/games/burymyheart/burymyheart.love), or play the flash port on BuryMyHeart's homepage (http://williambowers.net/games/burymyheart/).

ByteData

ByteData · revision 25541 · last edited 2025-09-30

Available since LÖVE 11.0

This type is not supported in earlier versions.

Data object containing arbitrary bytes in an contiguous memory.

There are currently no LÖVE functions provided for manipulating the contents of a ByteData, but Data:getFFIPointer can be used with LuaJIT's FFI to access and write to the contents directly. Used primarily for creating ImageData, Image, also love.filesystem.mount etc.

Constructors

love.data.newByteData	Creates a new Data object containing arbitrary bytes.	Added since 11.0
love.graphics.readbackBuffer	Generates or updates ByteData from the contents of the given GraphicsBuffer.	Added since 12.0

Functions

ByteData:setString	Replaces all or part of the ByteData's memory with the contents of a string.	Added since 12.0
Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object
- Data

See Also

- love.data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ByteData:setString

ByteData:setString · revision 29143 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Replaces all or part of the ByteData's memory with the contents of a string.

Function

Synopsis

```
ByteData:setString( data, offset )
```

Arguments

`string data`
: The bytes to copy to the Data object.

`number offset (0)`
: An optional byte offset into the Data's memory to copy to.

Returns

Nothing.

See Also

- ByteData
- Data:getString

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CameraMgr

CameraMgr · revision 25505 · last edited 2023-07-05

CameraMgr is a highly dynamic and robust love2d camera library in around 200 lines of code.

Designed to be easy to modify, and easy to use.

Learn more and get it at Gitlab.com (https://gitlab.com/V3X3D/love-libs/-/tree/master/CameraMgr).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas

Canvas · revision 28244 · last edited 2023-07-05

On this page

- Constructors
- Functions
- Enums
- Supertypes
- Examples
- Drawing something to the Canvas and then drawing the Canvas to the screen
- Save canvas as picture
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from Framebuffer.

A Canvas is used for off-screen rendering. Think of it as an invisible screen that you can draw to, but that will not be visible until you draw it to the actual visible screen. It is also known as "render to texture".

By drawing things that do not change position often (such as background items) to the Canvas, and then drawing the entire Canvas instead of each item, you can reduce the number of draw operations performed each frame.

In versions prior to 0.10.0, not all graphics cards that LÖVE supported could use Canvases. love.graphics.isSupported("canvas") could be used to check for support at runtime.



When drawing content to a Canvas using regular alpha blending, the alpha values of the content get multiplied with its RGB values.

Therefore the Canvas' pixel colors will have *premultiplied alpha* once it has been drawn to, so when drawing the Canvas to the screen or to another Canvas you must use premultiplied alpha blending – love.graphics.setBlendMode("alpha", "premultiplied").

Constructors

love.graphics.newCanvas	Creates a new Canvas.	Added since 0.8.0

Functions

Canvas:clear	Clears the contents of a Canvas to a specific color.	Added since 0.8.0	Removed in 0.10.0
Canvas:generateMipmaps	Generates mipmaps for the Canvas, based on the contents of the highest-resolution mipmap level.	Added since 11.0	
Canvas:getFSAA	Gets the number of FSAA samples used when drawing to the Canvas.	Added since 0.9.1	Removed in 0.10.0
Canvas:getFormat	Gets the texture format of the Canvas.	Added since 0.9.1	Removed in 11.0
Canvas:getImageData	Generates ImageData from the contents of the Canvas.	Added since 0.8.0	Removed in 0.10.0
Canvas:getMSAA	Gets the number of MSAA samples used when drawing to the Canvas.	Added since 0.9.2	
Canvas:getMipmapMode	Gets the MipmapMode this Canvas was created with.	Added since 11.0	
Canvas:getPixel	Gets the pixel at the specified position in a Canvas.	Added since 0.9.0	Removed in 0.10.0
Canvas:newImageData	Generates ImageData from the contents of the Canvas.	Added since 0.10.0	
Canvas:renderTo	Render to a Canvas using a function.	Added since 0.8.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Texture:getDPIScale	Gets the DPI scale factor of the Texture.	Added since 11.0	

Texture:getDepth	Gets the depth of a Volume Texture.	Added since 11.0
Texture:getDepthSampleMode	Gets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0
Texture:getDimensions	Gets the width and height of the Texture.	Added since 0.9.0
Texture:getFilter	Gets the filter mode of the Texture.	
Texture:getFormat	Gets the pixel format of the Texture.	Added since 11.0
Texture:getHeight	Gets the height of the Texture.	
Texture:getLayerCount	Gets the number of layers / slices in an Array Texture.	Added since 11.0
Texture:getMipmapCount	Gets the number of mipmaps contained in the Texture.	Added since 11.0
Texture:getMipmapFilter	Gets the mipmap filter mode for a Texture.	Added since 0.9.0
Texture:getPixelDimensions	Gets the width and height in pixels of the Texture.	Added since 11.0
Texture:getPixelHeight	Gets the height in pixels of the Texture.	Added since 11.0
Texture:getPixelWidth	Gets the width in pixels of the Texture.	Added since 11.0
Texture:getTextureType	Gets the type of the Texture.	Added since 11.0
Texture:getWidth	Gets the width of the Texture.	
Texture:getWrap	Gets the wrapping properties of a Texture.	
Texture:isReadable	Gets whether the Texture can be drawn and sent to a Shader.	Added since 11.0
Texture:setDepthSampleMode	Sets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0
Texture:setFilter	Sets the filter mode of the Texture.	
Texture:setMipmapFilter	Sets the mipmap filter mode for a Texture.	Added since 0.9.0
Texture:setWrap	Sets the wrapping properties of a Texture.	

Enums

MipmapMode	Controls whether a Canvas has mipmaps, and its behaviour when it does. Added since 11.0

Supertypes

- Texture
- Drawable
- Object

Examples

Drawing something to the Canvas and then draw the Canvas to the screen

```
function love.load()
    canvas = love.graphics.newCanvas(800, 600)

    -- Rectangle is drawn to the canvas with the regular/default alpha blend mode ("alphamultiply").
    love.graphics.setCanvas(canvas)
        love.graphics.clear(0, 0, 0, 0)
        love.graphics.setBlendMode("alpha")
        love.graphics.setColor(1, 0, 0, .5)
        love.graphics.rectangle("fill", 0,0, 100,100)
    love.graphics.setCanvas()
end

function love.draw()
    -- The colors for the rectangle on the canvas have already been alpha blended.
```

```
    -- Use the "premultiplied" alpha blend mode when drawing the canvas to the screen for proper color blending.
    -- (Also set the color to white so the canvas itself doesn't get tinted.)
    love.graphics.setBlendMode("alpha", "premultiplied")
    love.graphics.setColor(1, 1, 1, 1)
    love.graphics.draw(canvas, 0,0)
    -- Observe the difference if the canvas is drawn with the regular alpha blend mode instead.
    love.graphics.setBlendMode("alpha")
    love.graphics.draw(canvas, 100,0)

    -- Rectangle is drawn directly to the screen with the regular alpha blend mode.
    love.graphics.setBlendMode("alpha")
    love.graphics.setColor(1, 0, 0, .5)
    love.graphics.rectangle("fill", 200,0, 100,100)

    -- (Helper texts.)
    love.graphics.setColor(1, 1, 1, 1)
    love.graphics.printf("canvas", 0,0, 100)
    love.graphics.printf("incorrect\n(looks too dark)", 100,0, 100)
    love.graphics.printf("rectangle\n(what canvas should look like)", 200,0, 100)
end
```

Save canvas as picture

```
    canvas:newImageData():encode("png","filename.png")
```

See Also

- love.graphics
- love.graphics.setCanvas
- love.graphics.setBlendMode
- love.graphics.isSupported
- love.graphics.getDimensions
- love.graphics.draw
- ImageData:encode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:clear

Canvas:clear · revision 27175 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Clear canvas before drawing
  - How to clear canvas
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

It has been replaced by love.graphics.clear.

Clears the contents of a Canvas to a specific color.

Calling this function directly after the Canvas becomes active (via love.graphics.setCanvas or Canvas:renderTo) is more efficient than calling it when the Canvas isn't active, especially on mobile devices.

love.graphics.setScissor will restrict the area of the Canvas that this function affects.

Function

Clear the canvas to transparent black: (0, 0, 0, 0).

Synopsis

```
Canvas:clear( )
```

Arguments

None.

Returns

Nothing.

Function

Clear the canvas to a specific color.

Synopsis

```
Canvas:clear( red, green, blue, alpha )
```

Arguments

number red
: Red component of the clear color (0-255).

number green
: Green component of the clear color (0-255).

number blue
: Blue component of the clear color (0-255).

number alpha (255)

Alpha component of the clear color (0-255).

Returns

Nothing.

Function

Synopsis

```
Canvas:clear( rgba )
```

Arguments

table rgba
A sequence with the red, green, blue and alpha values as numbers (alpha may be ommitted).

Returns

Nothing.

Examples

Clear canvas before drawing

If the c-key is pressed the canvas will be cleared before drawing a new line on the screen.

```
local canvas = love.graphics.newCanvas()
local clear
function love.update()
    -- Use an anonymous function to draw lines on our canvas.
    canvas:renderTo(function()
        if clear then canvas:clear() end   -- Clear the canvas before drawing lines.
        love.graphics.setColor(love.math.random(255), 0, 0)
        love.graphics.line(0, 0, love.math.random(0, love.graphics.getWidth()), love.math.random(0, love.graphics.getHeight()))
    end)
end

function love.draw()
    love.graphics.setColor(255, 255, 255)
    love.graphics.draw(canvas)
end

function love.keypressed(key)
    if key == "c" then clear = not clear end
end
```

How to clear canvas

```
love.graphics.setCanvas(canvas) -- set to canvas
love.graphics.clear(0,0,0,0) -- clear the canvas
love.graphics.setCanvas() -- back to standard render output
```

See Also

- Canvas
- love.graphics.clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:generateMipmaps

Canvas:generateMipmaps · revision 22744 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Generates mipmaps for the Canvas, based on the contents of the highest-resolution mipmap level.

The Canvas must be created with mipmaps set to a MipmapMode other than "none" for this function to work. It should only be called while the Canvas is not the active render target.

If the mipmap mode is set to "auto", this function is automatically called inside love.graphics.setCanvas when switching from this Canvas to another Canvas or to the main screen.

Function

Synopsis

```
Canvas:generateMipmaps( )
```

Arguments

None.

Returns

Nothing.

See Also

- Canvas
- love.graphics.newCanvas
- Canvas:getMipmapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getDimensions

Canvas:getDimensions · revision 20178 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the Canvas.

Function

Synopsis

```
width, height = Canvas:getDimensions( )
```

Arguments

None.

Returns

number width
: The width of the Canvas, in pixels.

number height
: The height of the Canvas, in pixels.

See Also

- Canvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getFilter

Canvas:getFilter · revision 20196 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the filter mode of the Canvas.

Function

Synopsis

```
min, mag = Canvas:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when minifying the canvas.

`FilterMode mag`
: Filter mode used when magnifying the canvas.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
min, mag, anisotropy = Canvas:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when minifying the canvas.

`FilterMode mag`
: Filter mode used when magnifying the canvas.

`number anisotropy`
: Maximum amount of anisotropic filtering used.

See Also

- Canvas

- Canvas:setFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getFormat

Canvas:getFormat · revision 31044 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1 and removed in LÖVE 11.0

It has been replaced by Texture:getFormat.

Gets the texture format of the Canvas.

Function

Synopsis

```
format = Canvas:getFormat( )
```

Arguments

None.

Returns

`CanvasFormat format`

The format of the Canvas.

See Also

- Canvas
- love.graphics.newCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Canvas:getFSAA

Canvas:getFSAA · revision 31042 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

It has been renamed to Canvas:getMSAA.

Gets the number of antialiasing samples used when drawing to the Canvas.

This may be different than the number used as an argument to love.graphics.newCanvas if the system running LÖVE doesn't support that number.

Function

Synopsis

```
samples = Canvas:getFSAA( )
```

Arguments

None.

Returns

`number samples`

The number of antialiasing samples used by the canvas when drawing to it.

See Also

- Canvas
- love.graphics.newCanvas
- love.graphics.getSystemLimit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getHeight

Canvas:getHeight · revision 20176 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the height of the Canvas.

Function

Synopsis

```
height = Canvas:getHeight( )
```

Arguments

None.

Returns

number height
: The height of the Canvas, in pixels.

See Also

- Canvas
- Canvas:getWidth
- Canvas:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getImageData

Canvas:getImageData · revision 15915 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

It has been renamed to Canvas:newImageData.

Generates ImageData from the contents of the Canvas. Think of it as taking a screenshot of the hidden screen that is the Canvas.

Function

Synopsis

```
data = Canvas:getImageData( )
```

Arguments

None.

Returns

`ImageData data`

The new ImageData made from the Canvas' image.

See Also

- Canvas
- love.graphics.newScreenshot

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Canvas:getMipmapMode

Canvas:getMipmapMode · revision 22742 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the MipmapMode this Canvas was created with.

Function

Synopsis

```
mode = Canvas:getMipmapMode( )
```

Arguments

None.

Returns

`MipmapMode mode`

The mipmap mode this Canvas was created with.

See Also

- Canvas
- love.graphics.newCanvas
- Canvas:generateMipmaps

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getMSAA

Canvas:getMSAA · revision 27703 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

It has been renamed from Canvas:getFSAA.

Gets the number of multisample antialiasing (MSAA) samples used when drawing to the Canvas.

This may be different than the number used as an argument to love.graphics.newCanvas if the system running LÖVE doesn't support that number.

Function

Synopsis

```
samples = Canvas:getMSAA( )
```

Arguments

None.

Returns

`number samples`

The number of multisample antialiasing samples used by the canvas when drawing to it.

See Also

- Canvas
- love.graphics.newCanvas
- love.graphics.getSystemLimits

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getPixel

Canvas:getPixel · revision 15918 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

Use Canvas:newImageData instead.

Gets the pixel at the specified position from a Canvas.

Valid x and y values start at 0 and go up to canvas width and height minus 1.



This function can be very slow: it can cause the CPU to wait for the GPU to finish all the work currently in its queue, which can be a whole frame's worth (or more.)

Function

Synopsis

```
r, g, b, a = Canvas:getPixel( x, y )
```

Arguments

number x
: The position of the pixel on the x-axis.

number y
: The position of the pixel on the y-axis.

Returns

number r
: The red component (0-255).

number g
: The green component (0-255).

number b
: The blue component (0-255).

number a
: The alpha component (0-255).

See Also

- Canvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getWidth

Canvas:getWidth · revision 20177 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the width of the Canvas.

Function

Synopsis

```
width = Canvas:getWidth( )
```

Arguments

None.

Returns

`number width`
: The width of the Canvas, in pixels.

See Also

- Canvas
- Canvas:getHeight
- Canvas:getDimensions

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:getWrap

Canvas:getWrap · revision 20197 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from Framebuffer:getWrap.

Gets the wrapping properties of a Canvas.

This function returns the currently set horizontal and vertical wrapping modes for the Canvas.

Function

Synopsis

```
horiz, vert = Canvas:getWrap( )
```

Arguments

None.

Returns

`WrapMode horiz`
: Horizontal wrapping mode of the Canvas.

`WrapMode vert`
: Vertical wrapping mode of the Canvas.

See Also

- Canvas
- Canvas:setWrap
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:newImageData

Canvas:newImageData · revision 25485 · last edited 2026-04-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

It has been renamed from Canvas:getImageData.

Generates ImageData from the contents of the Canvas.



This function can be slow if it is called repeatedly, such as from love.update or love.draw. If you need to use a specific resource often, create it once and store it somewhere it can be reused!

Function

Synopsis

```
data = Canvas:newImageData( )
```

Arguments

None.

Returns

`ImageData data`

The new ImageData made from the Canvas' contents.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
data = Canvas:newImageData( slice, mipmap, x, y, width, height )
```

Arguments

`number slice`

The cubemap face index, array index, or depth layer for cubemap, array, or volume type Canvases, respectively. This argument is ignored for regular 2D canvases.

`number mipmap (1)`

The mipmap index to use, for Canvases with mipmaps.

`number x`

The x-axis of the top-left corner (in pixels) of the area within the Canvas to capture.

`number y`

The y-axis of the top-left corner (in pixels) of the area within the Canvas to capture.

`number width`

The width in pixels of the area within the Canvas to capture.

`number height`
: The height in pixels of the area within the Canvas to capture.

Returns

`ImageData data`
: The new ImageData made from the Canvas' contents.

Function

Removed in LÖVE 11.0
This variant is not supported in that and later versions.

Synopsis

```
data = Canvas:newImageData( x, y, width, height )
```

Arguments

`number x`
: The x-axis of the top-left corner (in pixels) of the area within the Canvas to capture.

`number y`
: The y-axis of the top-left corner (in pixels) of the area within the Canvas to capture.

`number width`
: The width in pixels of the area within the Canvas to capture.

`number height`
: The height in pixels of the area within the Canvas to capture.

Returns

`ImageData data`
: The new ImageData made from the Canvas' contents.

See Also

- Canvas
- love.graphics.captureScreenshot
- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:renderTo

Canvas:renderTo · revision 29166 · last edited 2024-04-30

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Using an anonymous function for drawing to a Canvas
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from Framebuffer:renderTo.

Render to the Canvas using a function.

This is a shortcut to love.graphics.setCanvas:

```
canvas:renderTo( func )
```

is the same as

```
local prevcanvas = love.graphics.getCanvas()
love.graphics.setCanvas( canvas )
func()
love.graphics.setCanvas( prevcanvas )
```

Function

Synopsis

```
Canvas:renderTo( func, ... )
```

Arguments

function func

A function performing drawing operations.

Available since LÖVE 11.5

any ...

Additional arguments to call the function with.

Returns

Nothing.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
Canvas:renderTo( index, func, ... )
```

Arguments

number index

An index to a layer (for array textures and volume textures) or an index to a cubemap face (for cubemap textures).

function func

A function performing drawing operations.

Available since LÖVE 11.5

`any ...`
Additional arguments to call the function with.

Returns

Nothing.

Examples

Using an anonymous function for drawing to a Canvas

This example randomly draws a bunch of red lines from the top left corner of the screen to the bottom.

```
local canvas = love.graphics.newCanvas()
function love.update()
    canvas:renderTo(function()
        love.graphics.setColor(love.math.random(), 0, 0);
        love.graphics.line(0, 0, love.math.random(0, love.graphics.getWidth()), love.math.random(0, love.graphics.getHeight()));
    end);
end

function love.draw()
    love.graphics.setColor(1, 1, 1);
    love.graphics.draw(canvas);
end
```

See Also

- Canvas
- love.graphics.setCanvas

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:setFilter

Canvas:setFilter · revision 20198 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the filter mode of the Canvas.

Function

Synopsis

```
Canvas:setFilter( min, mag )
```

Arguments

`FilterMode min`
: Filter mode to use when minifying the canvas.

`FilterMode mag`
: Filter mode to use when magnifying the canvas.

Returns

Nothing.

Function

Available since LÖVE 0.9.0

This variant is not supported in earlier versions.

Synopsis

```
Canvas:setFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: Filter mode to use when minifying the canvas.

`FilterMode mag`
: Filter mode to use when magnifying the canvas.

`number anisotropy (1)`
: Maximum amount of anisotropic filtering to use.

Returns

Nothing.

See Also

- Canvas

- Canvas:getFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Canvas:setWrap

Canvas:setWrap · revision 20199 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

It has been renamed from Framebuffer:setWrap.

Sets the wrapping properties of a Canvas.

This function sets the way the edges of a Canvas are treated if it is scaled or rotated. If the WrapMode is set to 'clamp', the edge will not be interpolated. If set to 'repeat', the edge will be interpolated with the pixels on the opposing side of the framebuffer.

Function

Synopsis

```
Canvas:setWrap( horiz, vert )
```

Arguments

`WrapMode horiz`
: Horizontal wrapping mode of the Canvas.

`WrapMode vert`
: Vertical wrapping mode of the Canvas.

Returns

Nothing.

See Also

- Canvas
- Canvas:getWrap
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CanvasFormat

CanvasFormat · revision 23343 · last edited 2026-04-19

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Deprecated in LÖVE 11.0

It has been superseded by PixelFormat..

Canvas formats.

Constants

normal

The default Canvas format - usually an alias for the `rgba8` format, or the `srgb` format if gamma-correct rendering is enabled in LÖVE 0.10.0 and newer.

hdr

A format suitable for high dynamic range content - an alias for the `rgba16f` format, normally.

Available since LÖVE 0.9.2

rgba8

8 bits per channel (32 bpp) RGBA. Color channel values range from 0-255 (0-1 in shaders).

rgba4

4 bits per channel (16 bpp) RGBA.

rgb5a1

RGB with 5 bits each, and a 1-bit alpha channel (16 bpp).

rgb565

RGB with 5, 6, and 5 bits each, respectively (16 bpp). There is no alpha channel in this format.

rgb10a2

RGB with 10 bits per channel, and a 2-bit alpha channel (32 bpp).

rgba16f

Floating point RGBA with 16 bits per channel (64 bpp). Color values can range from [-65504, +65504].

rgba32f

Floating point RGBA with 32 bits per channel (128 bpp).

rg11b10f

Floating point RGB with 11 bits in the red and green channels, and 10 bits in the blue channel (32 bpp). There is no alpha channel. Color values can range from [0, +65024].

srgb

The same as `rgba8`, but the Canvas is *interpreted* as being in the sRGB color space. Everything drawn to the Canvas will be converted from linear RGB to sRGB. When the Canvas is drawn (or used in a shader), it will be decoded from sRGB to linear RGB. This reduces color banding when doing gamma-correct rendering, since sRGB encoding has more precision than linear RGB for darker colors.

Available since LÖVE 0.10.0

r8

Single-channel (red component) format (8 bpp).

rg8

Two channels (red and green components) with 8 bits per channel (16 bpp).

r16f

Floating point single-channel format (16 bpp). Color values can range from [-65504, +65504].

rg16f

Floating point two-channel format with 16 bits per channel (32 bpp). Color values can range from [-65504, +65504].

r32f

Floating point single-channel format (32 bpp).

rg32f

Floating point two-channel format with 32 bits per channel (64 bpp).

Notes

The 16 bpp RGB and RGBA formats use half as much VRAM as the 32 bpp RGBA formats, but they have significantly lower quality.

The HDR / floating point formats are most useful when combined with pixel shaders. Effects such as tonemapped HDR with bloom can be accomplished, or the canvas can be used to store arbitrary non-color data such as positions which can then be used in a custom shader.

The sRGB format should only be used when doing gamma-correct rendering, which is an advanced topic and it's easy to get color-spaces mixed up. If you're not sure whether you need this, you might want to avoid it. Read more about gamma-correct rendering here (http://http.developer.nvidia.com/GPUGems3/gpugems3_ch24.html), here (http://filmicgames.com/archives/299), and here (http://renderwonk.com/blog/index.php/archive/adventures-with-gamma-correct-rendering/).

Not all systems support every format. Use love.graphics.getCanvasFormats to check before creating the Canvas.

In general, `rgba8`, `rgba4`, and `rgb5a1` are supported everywhere. `rgb10a2` is also supported everywhere on desktop platforms.

The regular single- and two-channel formats (`r8` and `rg8`), as well as the `srgb` format, are supported on desktop graphics cards capable of OpenGL 3+ / DirectX 10+ (nvidia GeForce 8000 series and up, ATI/AMD HD 2000 series and up, and the Intel HD 2000 and up), and mobile GPUs capable of OpenGL ES 3.

The floating-point formats (`rgba16f`, `rgba32f`, `rg11b10f`, `r16f`, `rg16f`, `r32f`, and `rg32f`) are supported on OpenGL 3-capable desktop graphics cards, and some OpenGL ES 3 mobile GPUs. The 32-bit-per-channel floating point formats are rarely supported on mobile devices.

`rgb565` is often only supported on OpenGL ES / mobile devices, or with very new desktop OpenGL drivers.

See Also

- love.graphics
- love.graphics.getCanvasFormats
- love.graphics.newCanvas
- Canvas:getFormat

Other Languages

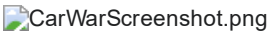
Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CarWar

CarWar · revision 20776 · last edited 2021-03-14

Hi there! This is CarWar, a game about cars with guns! There is a skin system, chat and more! Check it out! If you have any problems, contact me at Telegram: @DimaTillyaev (You can download it here: https://love2d.org/forums/viewtopic.php?f=14&t=84594)

Screenshots:



Categorie:Tutoriels

Categorie:Tutoriels · revision 3147 · last edited 2023-07-05

Tous les Tutoriels

GLES Shader Testing in Desktop	
In-Memory Audio Streaming	In-memory Source "stream". Stores the encoded audio data and decodes it from RAM.
examples.love	A set of simple samples for LÖVE.

ChainShape

ChainShape · revision 10221 · last edited 2023-07-05

Available since LÖVE 0.8.0

This shape is not supported in earlier versions.

A ChainShape consists of multiple line segments. It can be used to create the boundaries of your terrain. The shape does not have volume and can only collide with PolygonShape and CircleShape.

Unlike the PolygonShape, the ChainShape does not have a vertices limit or has to form a convex shape, but self intersections are not supported.

Constructors

love.physics.newChainShape	Creates a new ChainShape.	Added since 0.8.0

Functions

ChainShape:getChildEdge	Returns a child of the shape as an EdgeShape.	Added since 0.8.0	
ChainShape:getNextVertex	Gets the vertex that establishes a connection to the next shape.	Added since 0.10.2	
ChainShape:getPoint	Returns a point of the shape.	Added since 0.8.0	
ChainShape:getPoints	Returns all points of the shape.	Added since 0.8.0	
ChainShape:getPreviousVertex	Gets the vertex that establishes a connection to the previous shape.	Added since 0.10.2	
ChainShape:getVertexCount	Returns the number of vertices the shape has.	Added since 0.8.0	
ChainShape:setNextVertex	Sets a vertex that establishes a connection to the next shape.	Added since 0.8.0	
ChainShape:setPrevVertex	Sets a vertex that establishes a connection to the previous shape.	Added since 0.8.0	Removed in 0.9.0
ChainShape:setPreviousVertex	Sets a vertex that establishes a connection to the previous shape.	Added since 0.9.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Shape:computeAABB	Returns the points of the bounding box for the transformed shape.	Added since 0.8.0	
Shape:computeMass	Computes the mass properties for the shape.	Added since 0.8.0	
Shape:destroy	Explicitly destroys the Shape.		Removed in 0.8.0
Shape:getBody	Get the body the shape is attached to.	Added since 0.7.0	Removed in 0.8.0
Shape:getBoundingBox	Gets the bounding box of the shape.		Removed in 0.8.0
Shape:getCategory	Gets the categories this shape is a member of.		Removed in 0.8.0
Shape:getCategoryBits	Gets the categories as a 16-bit integer.		Removed in 0.8.0
Shape:getChildCount	Returns the number of children the shape has.	Added since 0.8.0	
Shape:getData	Get the data set with setData.		Removed in 0.8.0
Shape:getDensity	Gets the density of the Shape.		Removed in 0.8.0
Shape:getFilterData	Gets the filter data of the Shape.		Removed in 0.8.0
Shape:getFriction	Gets the friction of this shape.		Removed in 0.8.0
Shape:getMask	Gets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:getRadius	Gets the radius of the shape.		
Shape:getRestitution	Gets the restitution of this shape.		Removed in 0.8.0
Shape:getType	Gets a string representing the Shape.		
Shape:isSensor	Checks whether a Shape is a sensor or not.		Removed in 0.8.0
Shape:rayCast	Casts a ray against the shape.	Added since 0.8.0	
Shape:setCategory	Sets the categories this shape is a member of.		Removed in 0.8.0
Shape:setData	Set data to be passed to the collision callback.		Removed in 0.8.0
Shape:setDensity	Sets the density of a Shape.		Removed in 0.8.0
Shape:setFilterData	Sets the filter data for a Shape.		Removed in 0.8.0
Shape:setFriction	Sets the friction of the shape.		Removed in 0.8.0
Shape:setMask	Sets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:setRestitution	Sets the restitution of the shape.		Removed in 0.8.0
Shape:setSensor	Sets whether this shape should act as a sensor.		Removed in 0.8.0
Shape:testPoint	Checks whether a point lies inside the shape.		

Shape:testSegment	Checks whether a line segment intersects a shape.	Removed in 0.8.0

Supertypes

- Shape
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getChildEdge

ChainShape:getChildEdge · revision 12314 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns a child of the shape as an EdgeShape.

Function

Synopsis

```
shape = ChainShape:getChildEdge( index )
```

Arguments

`number index`
: The index of the child.

Returns

`EdgeShape shape`
: The child as an EdgeShape.

See Also

- ChainShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getNextVertex

ChainShape:getNextVertex · revision 18104 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the vertex that establishes a connection to the next shape.

Setting next and previous ChainShape vertices can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
x, y = ChainShape:getNextVertex( )
```

Arguments

None.

Returns

`number x (nil)`
: The x-component of the vertex, or nil if ChainShape:setNextVertex hasn't been called.

`number y (nil)`
: The y-component of the vertex, or nil if ChainShape:setNextVertex hasn't been called.

See Also

- ChainShape
- ChainShape:setNextVertex
- ChainShape:getPreviousVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getPoint

ChainShape:getPoint · revision 10574 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns a point of the shape.

Function

Synopsis

```
x, y = ChainShape:getPoint( index )
```

Arguments

number index
: The index of the point to return.

Returns

number x
: The x-coordinate of the point.

number y
: The y-coordinate of the point.

See Also

- ChainShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getPoints

ChainShape:getPoints · revision 23930 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns all points of the shape.


A bug in 0.8.0 causes memory corruption if this function gets called on a ChainShape with more than 8 points.

Function

Synopsis

```
x1, y1, x2, y2, ... = ChainShape:getPoints( )
```

Arguments

None.

Returns

number x1
The x-coordinate of the first point.

number y1
The y-coordinate of the first point.

number x2
The x-coordinate of the second point.

number y2
The y-coordinate of the second point.

See Also

- ChainShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getPreviousVertex

ChainShape:getPreviousVertex · revision 18103 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the vertex that establishes a connection to the previous shape.

Setting next and previous ChainShape vertices can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
x, y = ChainShape:getPreviousVertex( )
```

Arguments

None.

Returns

number x (nil)
: The x-component of the vertex, or nil if ChainShape:setPreviousVertex hasn't been called.

number y (nil)
: The y-component of the vertex, or nil if ChainShape:setPreviousVertex hasn't been called.

See Also

- ChainShape
- ChainShape:setPreviousVertex
- ChainShape:getNextVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:getVertexCount

ChainShape:getVertexCount · revision 10572 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the number of vertices the shape has.

Function

Synopsis

```
count = ChainShape:getVertexCount( )
```

Arguments

None.

Returns

`number count`

The number of vertices.

See Also

- ChainShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:setNextVertex

ChainShape:setNextVertex · revision 18102 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets a vertex that establishes a connection to the next shape.

This can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
ChainShape:setNextVertex( x, y )
```

Arguments

number `x`
: The x-component of the vertex.

number `y`
: The y-component of the vertex.

Returns

Nothing.

See Also

- ChainShape
- ChainShape:setPreviousVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:setPreviousVertex

ChainShape:setPreviousVertex · revision 11836 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from ChainShape:setPrevVertex.

Sets a vertex that establishes a connection to the previous shape.

This can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
ChainShape:setPreviousVertex( x, y )
```

Arguments

number x
: The x-component of the vertex.

number y
: The y-component of the vertex.

Returns

Nothing.

See Also

- ChainShape
- ChainShape:setNextVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ChainShape:setPrevVertex

ChainShape:setPrevVertex · revision 11829 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Removed in LÖVE 0.9.0

It has been renamed to ChainShape:setPreviousVertex.

Sets a vertex that establishes a connection to the previous shape.

This can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
ChainShape:setPrevVertex( x, y )
```

Arguments

number x
: The x-component of the vertex.

number y
: The y-component of the vertex.

Returns

Nothing.

See Also

- ChainShape
- ChainShape:setNextVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Changelog

Changelog · revision 9551 · last edited 2021-03-14

A changelog should probably be here, if you are looking for the Changelog wiki items, see below:

Changelog for the wiki pages

Changelog for the wiki pages use Changelog/header and Changelog/item

An example:

Version	Description
0.0.1	function removed
0.0.3	function readded
0.8.3	*sx* and *sy* arguments added

Channel

Channel · revision 15423 · last edited 2025-09-30

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

An object which can be used to send and receive data between different threads.

Constructors

love.thread.getChannel	Creates or retrieves a named thread channel.	Added since 0.9.0
love.thread.newChannel	Creates a new unnamed thread channel.	Added since 0.9.0

Functions

Channel:clear	Clears all the messages in the Channel queue.	Added since 0.9.0
Channel:demand	Wait for and retrieve the value of a Channel message.	Added since 0.9.0
Channel:getCount	Retrieves the number of messages in the Channel queue.	Added since 0.9.0
Channel:hasRead	Gets whether a pushed value has been popped or otherwise removed from the Channel.	Added since 11.0
Channel:peek	Receive a message from a thread Channel, but leave it in the queue.	Added since 0.9.0
Channel:performAtomic	Executes the specified function atomically with respect to this Channel.	Added since 0.10.0
Channel:pop	Retrieve the value of a Channel message.	Added since 0.9.0
Channel:push	Send a message to a thread Channel.	Added since 0.9.0
Channel:supply	Send a message to a thread Channel and wait for a thread to accept it.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object

See Also

- love.thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:clear

Channel:clear · revision 23063 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Clears all the messages in the Channel queue.


This function doesn't free up the memory used by a Channel, it needs to be garbage collected to release the memory.

Function

Synopsis

```
Channel:clear( )
```

Arguments

None.

Returns

Nothing.

See Also

- Channel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:demand

Channel:demand · revision 26773 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Retrieves the value of a Channel message and removes it from the message queue.

It waits until a message is in the queue then returns the message value.

Function

Synopsis

```
value = Channel:demand( )
```

Arguments

None.

Returns

`Variant value`

The contents of the message.

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
value = Channel:demand( timeout )
```

Arguments

`number timeout`

The maximum amount of time to wait, in seconds. Given as a decimal, accurate to the millisecond.

Returns

`Variant value`

The contents of the message or nil if the timeout expired.

See Also

- Channel
- Channel:supply

- Channel:pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:getCount

Channel:getCount · revision 19917 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Retrieves the number of messages in the thread Channel queue.

Function

Synopsis

```
value = Channel:getCount( )
```

Arguments

None.

Returns

`number count`

The number of messages in the queue.

See Also

- Channel
- Channel:pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:hasRead

Channel:hasRead · revision 22642 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets whether a pushed value has been popped or otherwise removed from the Channel.

Function

Synopsis

```
hasread = Channel:hasRead( id )
```

Arguments

`number id`
: An id value previously returned by Channel:push.

Returns

`boolean hasread`
: Whether the value represented by the id has been removed from the Channel via Channel:pop, Channel:demand, or Channel:clear.

See Also

- Channel
- Channel:push
- Channel:pop
- Channel:demand
- Channel:clear

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:peek

Channel:peek · revision 19345 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Retrieves the value of a Channel message, but leaves it in the queue.

It returns nil if there's no message in the queue.

Function

Synopsis

```
value = Channel:peek( )
```

Arguments

None.

Returns

`Variant value`

The contents of the message.

See Also

- Channel
- Channel:pop

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:performAtomic

Channel:performAtomic · revision 15433 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Re-set a Channel's contents to only have a single value
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Executes the specified function atomically with respect to this Channel.

Calling multiple methods in a row on the same Channel is often useful. However if multiple Threads are calling this Channel's methods at the same time, the different calls on each Thread might end up interleaved (e.g. one or more of the second thread's calls may happen in between the first thread's calls.)

This method avoids that issue by making sure the Thread calling the method has exclusive access to the Channel until the specified function has returned.


Long-running or otherwise expensive code should be avoided in the function given to this method.

Function

Synopsis

```
ret1, ... = Channel:performAtomic( func, arg1, ... )
```

Arguments

function func
: The function to call, the form of `function(channel, arg1, arg2, ...) end`. The Channel is passed as the first argument to the function when it is called.

any arg1
: Additional arguments that the given function will receive when it is called.

any ...
: Additional arguments that the given function will receive when it is called.

Returns

any ret1
: The first return value of the given function (if any.)

any ...
: Any other return values.

Examples

Re-set a Channel's contents to only have a single value

```
local function setChannel(channel, value)
    channel:clear()
    channel:push(value)
end
```

```
local c = love.thread.getChannel("MyChannel")
c:performAtomic(setChannel, "hello world")
```

See Also

- Channel
- Thread

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:pop

Channel:pop · revision 21122 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Retrieves the value of a Channel message and removes it from the message queue.

It returns nil if there are no messages in the queue.



This function doesn't free up the memory used by a Channel, it needs to be garbage collected to release the memory.

Function

Synopsis

```
value = Channel:pop( )
```

Arguments

None.

Returns

`Variant value`

The contents of the message.

See Also

- Channel
- Channel:push
- Channel:demand

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:push

Channel:push · revision 21482 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Send a message to the thread Channel.

See Variant for the list of supported types.

Function

Synopsis

```
id = Channel:push( value )
```

Arguments

`Variant value`

The contents of the message.

Returns

Available since LÖVE 11.0

`number id`

Identifier which can be supplied to Channel:hasRead

See Also

- Channel
- Channel:pop
- Channel:supply

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Channel:supply

Channel:supply · revision 26771 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Send a message to the thread Channel and wait for a thread to accept it.

See Variant for the list of supported types.

Function

Synopsis

```
success = Channel:supply( value )
```

Arguments

`Variant value`
: The contents of the message.

Returns

Available since LÖVE 11.0

`boolean success`
: Whether the message was successfully supplied (always `true`).

Function

Available since LÖVE 11.0

This variant is not supported in earlier versions.

Synopsis

```
success = Channel:supply( value, timeout )
```

Arguments

`Variant value`
: The contents of the message.

`number timeout`
: The maximum amount of time to wait, in seconds.

Returns

`boolean success`

Whether the message was successfully supplied before the timeout expired.

See Also

- Channel
- Channel:demand
- Channel:push

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Check

Check · revision 19053 · last edited 2021-03-14

This function checks the given arguments and calls the error-function if something is wrong. Finally it makes debugging easier since you'll get useful error-messages when calling a selfmade function with wrong or malformed arguments.

Syntax

```
void Check( string funcname, var types1, var arg1, string argname1, [ ... ] )
```

Required Arguments

- **funcname**: Name of the function in which the Check-function is called.
- **types1**: Type(s) that arg1 should/may be of. It's a string if only one type is possible, otherwise it can be a table with strings. Possible types are "nil", "boolean", "number", "string", "function", "table", "thread" and "userdata".
- **arg1**: Value of the argument that should be checked.
- **argname1**: Name of the argument whose value is arg1.

Optional Arguments

- **...**: You may pass as many triples as you want to.

Source

```
function Check(funcname, ...)
    local arg = {...}

    if (type(funcname) ~= "string") then
        error("Argument type mismatch at 'Check' ('funcname'). Expected 'string', got '"..type(funcname).."'.", 2)
    end
    if (#arg % 3 > 0) then
        error("Argument number mismatch at 'Check'. Expected #arg % 3 to be 0, but it is "..(#arg % 3)..".", 2)
    end

    for i=1, #arg-2, 3 do
        if (type(arg[i]) ~= "string" and type(arg[i]) ~= "table") then
            error("Argument type mismatch at 'Check' (arg #"..i..."). Expected 'string' or 'table', got '"..type(arg[i]).."'.", 2)
        elseif (type(arg[i+2]) ~= "string") then
            error("Argument type mismatch at 'Check' (arg #"..(i+2)..."). Expected 'string', got '"..type(arg[i+2]).."'.", 2)
        end

        if (type(arg[i]) == "table") then
            local aType = type(arg[i+1])
            for _, pType in next, arg[i] do
                if (aType == pType) then
                    aType = nil
                    break
                end
            end
            if (aType) then
                error("Argument type mismatch at '"..funcname.."' ('"..arg[i+2].."'). Expected '"..table.concat(arg[i], "' or '").."', got
'"..aType.."'.", 3)
            end
        elseif (type(arg[i+1]) ~= arg[i]) then
            error("Argument type mismatch at '"..funcname.."' ('"..arg[i+2].."'). Expected '"..arg[i].."', got '"..type(arg[i+1]).."'.", 3)
        end
    end
end
```

Example

This example shows the Check-function's usage in case of a more or less useless division function.

```lua
function divide(dividend, divisor, round)
    -- check the passed arguments
    Check("divide", "number", dividend, "dividend", "number", divisor, "divisor", {"nil","boolean"}, round, "round")

    -- error if divisor is 0
    if (divisor == 0) then error("Someone tried to divide by 0. YOU MUST NOT DO THAT!!!") end

    -- calculate the result
    local quotient = dividend / divisor
    -- round the result if round is true
    if (round) then quotient = math.floor(quotient + .5)

    -- return the result
    return quotient
end
```

CircleShape

CircleShape · revision 10222 · last edited 2023-07-05

Circle extends Shape and adds a radius and a local position.

Constructors

love.physics.newCircleShape	Creates a new CircleShape.

Functions

CircleShape:getLocalCenter	Get the center of the circle in local coordinates.		Removed in 0.8.0
CircleShape:getPoint	Gets the center point of the circle shape.	Added since 0.9.1	
CircleShape:getRadius	Gets the radius of the circle shape.		
CircleShape:getWorldCenter	Get the center of the circle in world coordinates.		Removed in 0.8.0
CircleShape:setPoint	Sets the center point of the circle shape.	Added since 0.9.1	
CircleShape:setRadius	Sets the radius of the circle.	Added since 0.8.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Shape:computeAABB	Returns the points of the bounding box for the transformed shape.	Added since 0.8.0	
Shape:computeMass	Computes the mass properties for the shape.	Added since 0.8.0	
Shape:destroy	Explicitly destroys the Shape.		Removed in 0.8.0
Shape:getBody	Get the body the shape is attached to.	Added since 0.7.0	Removed in 0.8.0
Shape:getBoundingBox	Gets the bounding box of the shape.		Removed in 0.8.0
Shape:getCategory	Gets the categories this shape is a member of.		Removed in 0.8.0
Shape:getCategoryBits	Gets the categories as a 16-bit integer.		Removed in 0.8.0
Shape:getChildCount	Returns the number of children the shape has.	Added since 0.8.0	
Shape:getData	Get the data set with setData.		Removed in 0.8.0
Shape:getDensity	Gets the density of the Shape.		Removed in 0.8.0
Shape:getFilterData	Gets the filter data of the Shape.		Removed in 0.8.0
Shape:getFriction	Gets the friction of this shape.		Removed in 0.8.0
Shape:getMask	Gets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:getRadius	Gets the radius of the shape.		
Shape:getRestitution	Gets the restitution of this shape.		Removed in 0.8.0
Shape:getType	Gets a string representing the Shape.		
Shape:isSensor	Checks whether a Shape is a sensor or not.		Removed in 0.8.0
Shape:rayCast	Casts a ray against the shape.	Added since 0.8.0	
Shape:setCategory	Sets the categories this shape is a member of.		Removed in 0.8.0
Shape:setData	Set data to be passed to the collision callback.		Removed in 0.8.0
Shape:setDensity	Sets the density of a Shape.		Removed in 0.8.0
Shape:setFilterData	Sets the filter data for a Shape.		Removed in 0.8.0
Shape:setFriction	Sets the friction of the shape.		Removed in 0.8.0
Shape:setMask	Sets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:setRestitution	Sets the restitution of the shape.		Removed in 0.8.0
Shape:setSensor	Sets whether this shape should act as a sensor.		Removed in 0.8.0
Shape:testPoint	Checks whether a point lies inside the shape.		
Shape:testSegment	Checks whether a line segment intersects a shape.		Removed in 0.8.0

Supertypes

- Shape
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CircleShape:getLocalCenter

CircleShape:getLocalCenter · revision 8783 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the center of the circle in local coordinates.

Function

Synopsis

```
lx, ly = CircleShape:getLocalCenter( )
```

Arguments

None.

Returns

number lx

The x position of the circle in local coordinates.

number ly

The y position of the circle in local coordinates.

See Also

- CircleShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

CircleShape:getPoint

CircleShape:getPoint · revision 12935 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Gets the center point of the circle shape.

Function

Synopsis

```
x, y = CircleShape:getPoint( )
```

Arguments

None.

Returns

number x
: The x-component of the center point of the circle.

number y
: The y-component of the center point of the circle.

See Also

- CircleShape
- CircleShape:setPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CircleShape:getRadius

CircleShape:getRadius · revision 6024 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the radius of the circle shape.

Function

Synopsis

```
radius = CircleShape:getRadius( )
```

Arguments

None.

Returns

`number radius`
: The radius of the circle

See Also

- CircleShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CircleShape:getWorldCenter

CircleShape:getWorldCenter · revision 8784 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Others Languages

Removed in LÖVE 0.8.0

This method is not supported in that and later versions.

Get the center of the circle in world coordinates.

Function

Synopsis

```
wx, wy = CircleShape:getWorldCenter( )
```

Arguments

None.

Returns

number **wx**
: The x position of the circle in world coordinates.

number **wy**
: The y position of the circle in world coordinates.

See Also

- CircleShape

Others Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CircleShape:setPoint

CircleShape:setPoint · revision 20052 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.1

This function is not supported in earlier versions.

Sets the location of the center of the circle shape.

Function

Synopsis

```
CircleShape:setPoint( x, y )
```

Arguments

`number x`
: The x-component of the new center point of the circle.

`number y`
: The y-component of the new center point of the circle.

Returns

Nothing.

See Also

- CircleShape
- CircleShape:getPoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CircleShape:setRadius

CircleShape:setRadius · revision 8785 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the radius of the circle.

Function

Synopsis

```
CircleShape:setRadius( radius )
```

Arguments

`number radius`
: The radius of the circle

Returns

Nothing.

See Also

- CircleShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Clamping

Clamping · revision 19054 · last edited 2021-03-14

Clamps a value to a certain range. Use this function to make sure the value doesn't get bigger than the given maximum value and smaller than the minimum value.

```
---
-- Clamps a value to a certain range.
-- @param min - The minimum value.
-- @param val - The value to clamp.
-- @param max - The maximum value.
--
function clamp(min, val, max)
    return math.max(min, math.min(val, max));
end
```

Class

Class · revision 25317 · last edited 2023-07-05

Class Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
30log	A small and effective object-orientation framework in 30 lines		Any	
Graphoon	A force directed graph algorithm written in Lua.		Any	
Luaoop	OOP (Object Oriented Programming) class-based library		Any	
MiddleClass	Object-orientation for Lua		Any	
Simple Educative Class Library	A class implementation		Any	
clasp	Tiny Lua class library		Any	
lol.lua	Object prototyping library for Lua.		Any	
subclass	A simple OOP library for Lua.		11.5	11.5
32 lines of goodness	32 lines of goodness is a small OO library that uses a domain specific language to make a neat syntax so OO is easy.		0.7.2	

ClientObject

ClientObject · revision 3659 · last edited 2023-07-05

A Client object

Functions

ClientObject:Init	Initializes the client object.
ClientObject:connect	Connect to a server.
ClientObject:disableBroadcast	Disables connecting to the broadcast address.
ClientObject:disconnect	Disconnects the client from server.
ClientObject:doPing	Commence a ping.
ClientObject:enableBroadcast	Enable connecting to the broadcast address.
ClientObject:receive	Receive data from the server.
ClientObject:send	Send data to a server.
ClientObject:setCallback	Sets the callbacks for the client.
ClientObject:setHandshake	Sets a handshake for the client.
ClientObject:setPing	Sets the ping for the client.
ClientObject:update	Update the client.

See Also

- LUBE
- LUBE.client

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:connect

ClientObject:connect · revision 9971 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Connect to a server.

Function

Synopsis

```
clientobject:connect(host, port, dns)
```

Arguments

`string host`
: The host to connect to.

`number port`
: The port to connect to on the host.

`string dns`
: Specifies if the host is a name or an IP (If its a hostname, Then do a DNS lookup). Default is IP.

Returns

Assertion upon failure, or true upon success. self.host becomes the host name. self.port becomes the port name. self.connected becomes a boolean set to true.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:disableBroadcast

ClientObject:disableBroadcast · revision 3711 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Warning
- Returns
- See Also
- Other Languages

Disables connecting to the broadcast address.

Function

Synopsis

```
clientobject:disableBroadcast()
```

Warning

Has direct effect, old connecton terminates uncleanly.

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:disconnect

ClientObject:disconnect · revision 3712 · last edited 2023-07-05

Disconnects the client from server.

Function

Synopsis

```
clientobject:disconnect()
```

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:doPing

ClientObject:doPing · revision 3713 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Commence a ping (According to settings), Does a ping whenever needed.

Function

Synopsis

```
clientobject:doPing(dt)
```

Arguments

`number dt`
: Delta time, as passed to update

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:enableBroadcast

ClientObject:enableBroadcast · revision 3714 · last edited 2023-07-05

Enable connecting to the broadcast address.

Function

Synopsis

```
clientobject:enableBroadcast()
```

Returns

Nothing.

See Also

- ClientObject
- ClientObject:disableBroadcast

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:Init

ClientObject:Init · revision 3717 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Initializes the client object.

Function

Synopsis

```
clientobject:Init( sockettype )
```

Arguments

`string sockettype`
: "tcp", "udp" or user-defined.

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:receive

ClientObject:receive · revision 3718 · last edited 2023-07-05

Receive data from the server.

Function

Synopsis

```
clientobject:receive()
```

Returns

data from the server.

See Also

- ClientObject
- ClientObject:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:send

ClientObject:send · revision 5501 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Send data to a server.

Function

Synopsis

```
clientobject:send( data )
```

Arguments

`string data`
: The updatable data to send to the server. If you want to update a table, then use lube.bin:pack() before sending.

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:setCallback

ClientObject:setCallback · revision 5503 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the callbacks for the client. Unlike the server object, the client only has the receive data callback.

Function

Synopsis

```
client:setCallbacks(receive)
```

Arguments

`function receive`
: Callback called on incoming data(filtered): function(data).

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:setHandshake

ClientObject:setHandshake · revision 5502 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets a handshake for the client. Should be the same as the server.

Function

Synopsis

```
client:setHandshake(handshake)
```

Arguments

`string handshake`
: A unique string, share by client and server to identify connecting and disconnecting client(s).

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:setPing

ClientObject:setPing · revision 5506 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the ping for the client.

Function

Synopsis

```
client:setPing(enabled, time, msg)
```

Arguments

`boolean enabled`
: Is enabled (True/False).

`number time`
: How many seconds between pings.

`string msg`
: Ping package content.

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ClientObject:update

ClientObject:update · revision 5507 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Update the client. Checks for ping and data the server is sending to the client.

Function

Synopsis

```
client:update(dt)
```

Arguments

`number dt`
: Delta-time.

Returns

Nothing.

See Also

- ClientObject

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

cock.addJoystick

cock.addJoystick · revision 11575 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Adds new joystick to the object by given joystick number.

Function

Synopsis

```
joystick = cock.addJoystick ( self, number )
```

Arguments

`table self`
: An object to use.

`number number`
: Optional. LÖVE joystick number to hook up. Will use 1 if not provided.

Returns

`number joystick`
: Internal joystick number.

Remarks

If function fails to find specified joystick, it will create *a dummy joystick*. It will represent virtual non-functional joystick. It will accept all joystick operaitons, but will not carry out any of them. Dummy joysticks do not yield any output. Function cock.reloadJoysticks will happily override dummy joysticks with real ones.

See also

- cock.deleteJoystick
- cock.setJoystickDeadzone
- cock.reloadJoysticks
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.addOption

cock.addOption · revision 11576 · last edited 2023-07-05

Creates new input option to bind contols into in current layout.

Function

Synopsis

```
option = cock.addOption ( self, option )
```

Arguments

table self
: An object to use.

string option
: Optional. Will create new option with specified name. Will generate random name if not provided.

Returns

string option
: Name of the new option.

See also

- cock.deleteOption
- cock.getEmptyOption
- cock.setDefault
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.bind

cock.bind · revision 11578 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Function
- Synopsis
- Arguments
- Remarks
- See also

Binds controls to the current layout.

Function

Synopsis

```
cock.bind ( self, map, option, device, key, inverse, delta, value, joystick )
```

Arguments

`table self`
: An object to use.

`string map`
: A controls input map to bind controls.

`string option`
: Optional. An option to which bind the controls. Will revert to defaultOption if not provided.

`string device`
: Input device to use.

`string key`
: Reading of an input device to bind.

`string inverse`
: Optional. Inversion mode. Will use "positive cutoff" if not provided.

`string delta`
: Optional. Delta mode. Will use "bypass" if nor provided.

`number value`
: Optional. Raw value of the input reading during capture. Will use 0 if not provided.

`string joystick`
: Optional. Internal alias of joystick to use. Will use 1 if not provided.

Function

Synopsis

```
cock.bind ( self, longdata )
```

Arguments

`table self`
: An object to use.

`string longdata`

A string coming from cock.controlcaptured event.

Remarks

Alternatively, instead of using literal values for device, key, inverse, delta and joystick arguments, you can pass numerical values. Although, just using magic numbers to set values by hand in your code will only make it messier, and given that internal conversion is an optimized operation and you wouldn't call this function hundreds of times every frame, that would make no performance difference.

See also

- cock.addOption
- cock.unbind
- cock.getBinded
- Common Organization of Controls Kit Callbacks
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.cancelCapture

cock.cancelCapture · revision 11604 · last edited 2023-07-05

Cancels "input capture" mode.

Function

Synopsis

```
cock.cancelCapture ( )
```

Remarks

This function will reset mouse position back where it was before cock.setCapture.

See also

- cock.setCapture
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertAxis

cock.convertAxis · revision 11633 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical axis names and vice versa.

Function

Synopsis

```
converted = cock.convertAxis ( input, forceliteral )
```

Arguments

`number input`
: Axis, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted axis.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- cock.convertKey
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertDelta

cock.convertDelta · revision 11637 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical delta mode names and vice versa.

Function

Synopsis

```
converted = cock.convertDelta ( input, forceliteral )
```

Arguments

`number input`
: Input delta mode, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted delta mode.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertDevice

cock.convertDevice · revision 11635 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical device names and vice versa.

Function

Synopsis

```
converted = cock.convertDevice ( input, forceliteral )
```

Arguments

`number input`
: Input device, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted input device.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertInverse

cock.convertInverse · revision 11629 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical inverse mode names and vice versa.

Function

Synopsis

```
converted = cock.convertInverse ( input, forceliteral )
```

Arguments

`number input`
: Input inverse mode, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted inverse mode.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertJoystick

cock.convertJoystick · revision 11630 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical internal joystick names and vice versa.

Function

Synopsis

```
converted = cock.convertJoystick ( self, input, forceliteral )
```

Arguments

`table self`
: An object to use.

`number input`
: Internal joystick name, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted internal joystick name.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- cock.addJoystick
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertJoystickHat

cock.convertJoystickHat · revision 11631 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical joystick hat names and vice versa.

Function

Synopsis

```
converted = cock.convertJoystickHat ( input, forceliteral )
```

Arguments

`number input`
: Joystick hat name, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted joystick hat.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

Unlike cock.convertDevice, this function aims specifically at joystick hats. It converts hat name to *hat number* rather than device number. E.g. "joystick hat 3" converted to "3" and vice versa, and not to "8" as with convertDevice function.

See also

- cock.remapJoystickHat
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertJoystickHatMode

cock.convertJoystickHatMode · revision 11632 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts literal to numerical joystick hat mode names and vice versa.

Function

Synopsis

```
converted = cock.convertJoystickHatMode ( input, forceliteral )
```

Arguments

`number input`
: Hat mode, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted joystick hat mode.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- cock.setJoystickHatMode
- Common Organization of Controls Kit Hat modes
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.convertKey

cock.convertKey · revision 11634 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Remarks
- See also

Conditionally converts literal to numerical input key names and vice versa. Essentially, performs cock.convertAxis if device was axis-like, otherwise returns unchanged input.

Function

Synopsis

```
converted = cock.convertKey ( device, input, forceliteral )
```

Arguments

`number device`
: Input device, numerical or literal.

`number input`
: Input key, numerical or literal.

`boolean forceliteral`
: Optional. Forces literal ("true") or numerical ("false") output. Will perform conversion regardless if not provided.

Returns

`string converted`
: Converted input key.

Remarks

Function will make no change if it was supplied with literal input and forced to output literal value. Same for numerical values.

See also

- cock.convertDevice
- cock.convertAxis
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.delete

cock.delete · revision 11579 · last edited 2023-07-05

Deletes control object. Simple nilling the variable that contains this object isn't enough to delete it, you need to call this function as well.

Function

Synopsis

```
cock.delete ( control )
```

Arguments

table control
: An object to delete.

string control
: ID of an object to delete.

See also

- cock.new
- cock.find
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.deleteJoystick

cock.deleteJoystick · revision 11580 · last edited 2023-07-05

Deletes specified joystick from the object.

Function

Synopsis

```
cock.deleteJoystick ( self, joystick )
```

Arguments

table self
: An object to use.

number joystick
: Internal joystick number.

string joystick
: Internal joystick name.

See also

- cock.addJoystick
- cock.setJoystickDeadzone
- cock.reloadJoysticks
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.deleteOption

cock.deleteOption · revision 11581 · last edited 2023-07-05

Deletes specified option from current layout.

Function

Synopsis

```
cock.deleteOption ( self, option )
```

Arguments

table self
: An object to use.

string option
: Option name to delete.

See also

- cock.addOption
- cock.getEmptyOption
- cock.setDefault
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.explodeCapturedData

cock.explodeCapturedData · revision 11610 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- Remarks
- See also

Converts "longdata" string passed to the controlcaptured callback into meaningful data.

Function

Synopsis

```
map, option, device, key, inverse, value, joystick = cock.explodeCapturedData ( longdata )
```

Arguments

`string longdata`
: Data from controlcaptured callback.

Returns

`string map`
: Map requested for capturing.

`string option`
: Option requested for capturing.

`number device`
: Captured device number.

`number key`
: Captured key number. May be a string depending on the device.

`number inverse`
: Captued inversion mode. Axis-like devices tilted to opposite direction yield negative modes.

`number value`
: Exact value of the input readings in the moment of capture.

`number joystick`
: Internal joystick number. Will be 1 for all non-joystick devices.

Remarks

The function doesn't return delta mode because it's impossible to capture it. You can always assume that delta mode is "bypass", or override it whereever that's appropriate.

See also

- cock.setCapture
- Common Organization of Controls Kit Callbacks
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.find

cock.find · revision 11582 · last edited 2023-07-05

Finds control object by ID.

Function

Synopsis

```
control = cock.find ( id )
```

Arguments

string id
: An identifier to find object by.

Returns

table control
: Found control object (nil on failure).

See also

- cock.new
- cock.delete
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getBinded

cock.getBinded · revision 11609 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See also

Returns input devices binded to the map on the corresponding option.

Function

Synopsis

```
device, key, inverse, delta, joystick = cock.getBinded ( self, map, option )
```

Arguments

table self
: An object to use.

string map
: Control map to look up.

string option
: Optional. Map option to look up bindings in. Will revert to defaultOption if not provided.

Returns

number device
: Input device number.

number key
: Input key value. May be a string, depending on the input device.

number inverse
: Inverse mode number.

number delta
: Delta mode number.

number joystick
: Internal joystick number. Will be 1 for non-joystick devices.

Function

Synopsis

```
table = cock.getBinded ( self, map, all )
```

Arguments

table self
: An object to use.

string map
: Control map to look up.

boolean all
: Value "true" will return a table with bindings on all options for the map.

Returns

table table
: List of all binded controls for the map.

Table is generated as following:

```
table = {
    option = { "primary", "secondary", "etc."... },
    device = { 1, 2, 3... },
       key = { "x", 1, 5... },
   inverse = { 2, 1, -2... },
     delta = { 0, 1, -1... },
  joystick = { 1, 1, 3 }
}
```

First sub-table contains option names in arbitrary order. Values in all other tables correspond to the options in the first table.

See also

- cock.bind
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Modes
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getCapture

cock.getCapture · revision 11603 · last edited 2023-07-05

Returns input capture mode state.

Function

Synopsis

```
control = cock.getCapture ( )
```

Returns

table control
: Object that currently captures the input, nil if no capture is going on.

See also

- cock.setCapture
- cock.cancelCapture
- cock.find
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getEmptyOption

cock.getEmptyOption · revision 11583 · last edited 2023-07-05

Finds next (in arbitrary order) unused input option in current layout.

Synopsis

Function

```
option = cock.getEmptyOption ( self, map )
```

Arguments

table self
: An object to use.

string map
: A control mapping to find unused options for.

Returns

string option
: Found unused option (nil on failure).

See also

- cock.addOption
- cock.deleteOption
- cock.setDefault
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getJoystickDeadzone

cock.getJoystickDeadzone · revision 11584 · last edited 2023-07-05

Gets joystick deadzone.

Function

Synopsis

```
threshold = cock.getJoystickDeadzone ( self, joystick, axis )
```

Arguments

table self
: An object to use.

number joystick
: Optional. Internal joystick number. Will use 1 if not provided.

string joystick
: Optional. Internal joystick name.

number axis
: Optional. LÖVE joystick axis. Will use 1 if not provided.

string axis
: Optional. Axis name.

Returns

number threshold
: Deadzone threshold.

See also

- cock.setJoystickDeadzone
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getJoystickHatMode

cock.getJoystickHatMode · revision 11585 · last edited 2023-07-05

Gets joystick hat mode.

Function

Synopsis

```
mode = cock.getJoystickHatMode ( self, joystick, hat )
```

Arguments

table self
: An object to use.

number joystick
: Optional. Internal joystick number. Will use 1 if not provided.

string joystick
: Optional. Internal joystick name.

number hat
: Optional. LÖVE joystick hat. Will use 1 if not provided.

string hat
: Optional. Hat name.

Returns

number mode
: Hat mode number.

See also

- cock.setJoystickHatMode
- cock.setJoystickDeadzone
- cock.remapJoystickHat
- Common Organization of Controls Kit Hat modes
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getJoysticksList

cock.getJoysticksList · revision 11611 · last edited 2023-07-05

Returns a table with joystick names converted to internal format.

Function

Synopsis

```
table = cock.getJoysticksList ( )
```

Returns

table table
: A table where each table item corresponds to it's internal joystick name.

See also

- cock.addJoystick
- cock.reloadJoysticks
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getMouseFactor

cock.getMouseFactor · revision 11586 · last edited 2023-07-05

Gets horizontal and vertical mouse factor. You can access these values directly, see etc.mouse.* fields.

Function

Synopsis

```
x, y = cock.getMouseFactor ( self )
```

Arguments

table self
: An object to use.

Returns

number x
: Scaling factor by X axis.

number y
: Scaling factor by Y axis.

See also

- cock.setMouseFactor
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.getMouseOffset

cock.getMouseOffset · revision 11587 · last edited 2023-07-05

Gets horizontal and vertical mouse offset. You can access these values directly, see etc.mouse.* fields.

Function

Synopsis

```
x, y = cock.getMouseOffset ( self )
```

Arguments

table self
: An object to use.

Returns

number x
: X-coordinate used as center.

number y
: Y-coordinate used as center.

See also

- cock.setMouseOffset
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.new

cock.new · revision 11642 · last edited 2023-07-05

Creates a new control object.

Function

Synopsis

```
control = cock.new ( id, data )
```

Arguments

string id
: Optional. An identifier to use for this object. Generated automatically if not provided.

table data
: Optional. Specifically formated setup data to build layouts from. Will not instantly create layout if not provided.

Returns

table control
: Newly created control object.

See also

- cock.delete
- cock.find
- Common Organization of Controls Kit Setup data
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.reloadJoysticks

cock.reloadJoysticks · revision 11589 · last edited 2023-07-05

Attempts to pick up previously used joysticks.

Function

Synopsis

```
cock.reloadJoysticks ( self )
```

Arguments

table self
: An object to use.

Remarks

If function fails to find a joystick, it will keep a reference to it for further attempts. If layout only had one joystick utilized, and at least one joystick is present, then the function will temporarily use available joystick in place of previously used. The reason why it only will do so for only one joystick is because if controls were spread to more than one joystick, re-picking them would mess up the layout, forcing player to re-define all controls, which makes re-picking joysticks meaningless to begin with. Dummy joysticks will be replaced with actual ones as soon as it possible. Re-defining controls for new joystick used temporarily removes previous joystick record, since it's no longer needed by definition, and it can't be picked up.

See also

- cock.addJoystick
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.remapJoystickHat

cock.remapJoystickHat · revision 11590 · last edited 2023-07-05

Maps exactly 4 joystick button to work as a joystick hat. It results in virtual hat that would act exactly like regular hat.

Function

Synopsis

```
cock.remapJoystickHat ( self, joystick, hat, up, down, left, right )
```

Arguments

table self
: An object to use.

number joystick
: Optional. Internal joystick number. Will use 1 if not provided.

string joystick
: Optional. Internal joystick name.

number hat
: Optional. LÖVE hat number. Will use 1 if not provided.

string hat
: Optional. Hat name.

number up
: Optional. Button to map to dpad Up direction. Will use last value if not provided.

number down
: Optional. Button to map to dpad Down direction. Will use last value if not provided.

number left
: Optional. Button to map to dpad Left direction. Will use last value if not provided.

number right
: Optional. Button to map to dpad Right direction. Will use last value if not provided.

Remarks

Omitting all directions un-remaps the hat. You can map buttons one by one, but incomplete mapping would result in partial functionality (some directions won't work).

See also

- cock.addJoystick
- cock.setJoystickHatMode
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setCallbacks

cock.setCallbacks · revision 11574 · last edited 2023-07-05

On this page

- Funciton
  - Synopsis
  - Arguments
- Function
  - Synopsis
  - Arguments
- See also

Enables or disables specified callbacks. All callbacks are disabled by default.

Funciton

Synopsis

```
cock.setCallbacks ( self, pressed, released, peaked, zeroed, changed )
```

Arguments

`table self`
: An object to use.

`boolean pressed`
: Optional. Corresponds to controlpressed callback. Will use old value if not provided.

`boolean released`
: Optional. Corresponds to controlreleased callback. Will use old value if not provided.

`boolean peaked`
: Optional. Corresponds to controlpeaked callback. Will use old value if not provided.

`boolean zeroed`
: Optional. Corresponds to controlzeroed callback. Will use old value if not provided.

`boolean changed`
: Optional. Corresponds to controlchanged callback. Will use old value if not provided.

Function

Synopsis

```
cock.setCallbacks ( self, callback, value )
```

Arguments

`table self`
: An object to use.

`string callback`
: Callback name.

`boolean value`
: Desired state.

See also

- cock.setCapture
- Common Organization of Controls Kit Callbacks
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setCapture

cock.setCapture · revision 11602 · last edited 2023-07-05

Assumes "input capture" mode.

Function

Synopsis

```
cock.setCapture ( self, map, option, eventlock, updatelock, nomouse, callback )
```

Arguments

`table self`
: An object to use.

`string map`
: Control map to capture controls into.

`string option`
: Optional. Map option to capture controls to. Will revert to defaultOption if not provided.

`boolean eventlock`
: Optional. Specifies whether or not lock game's input event handlers during capture mode. Will use "false" if not provided.

`boolean updatelock`
: Optional. Specifies whether or not lock game's update function during capture mode. Will use "false" if not provided.

`boolean nomouse`
: Optional. Specifies whether or not capture mouse motion. Will use "false" if not provided.

`boolean callback`
: Optional. Specifies whether or not call controlcaptured callback on input capture. Will use "true" if not provided.

Remarks

Locking input handlers may be necessary to avoid undesired activity during capturing the input. This includes both LÖVE and COCK input handler callbacks. Locking update function serves same purpose, but due to processing data in it, locking it is generally not desired. If you use cock.current.* fields to process user input in update function, then you are encouraged to lock it. Alternatively, make sure no such processing happens when the library is in "input capture" mode. Locking option for love.draw and other functions is not provided because it is assumed that activity in corresponding functions won't interfere with input capture.

Because mouse have normally specifically defined unique function in the game, there's an option to ignore mouse motion during input capture to ensure that none of such can be captured. If this is enabled, the libary will temporary set the mouse to the screen center, and will reset it back once input is captured. It is up to the programmer to hide and show mouse cursor before and after mouse position shifting respectively.

Not calling controlcaptured callback is equivalent to instantly binding captured input to specified map and option. Requesting to call it while having no controlcaptured callback defined is effectively equivalent to the same. Doing so if the callback is defined results in plain call to the callback with "longdata" arranged argument with no additional side-effects going along. User must manually call cock.bind to use obtained data.

NOTE: Assuming input capture mode without cancelling previous input capture mode of any other object will instantly bluescreen the game (stack overflow). This is due to internal structure. If you want capture input to several objects simultaneously, you define a controlcaptured callback that would bind obtained data across several objects, and only assume input mode once.

See also

- cock.getCapture
- cock.cancelCapture
- cock.bind
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setControls

cock.setControls · revision 11591 · last edited 2023-07-05

Creates control layouts from specifically formatted data table.

Function

Synopsis

```
cock.setControls ( self, data )
```

Arguments

table self
: Control object to use.

table data
: Specifically formated setup data to build layouts from.

See also

- cock.setDefault
- Common Organization of Controls Kit Setup data
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setDefault

cock.setDefault · revision 11592 · last edited 2023-07-05

Restores specified default layout, permanently erasing previously defined bindings.

Function

Synopsis

```
cock.setDefault ( self, default )
```

Arguments

table self
: An object to use.

string default
: Optional. One of the defaults created earlier. Will revert to control.defaultLayout field if not provided.

See also

- cock.setControls
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setDefaultXBox360

cock.setDefaultXBox360 · revision 11593 · last edited 2023-07-05

Creates a layout for XBox360 controller.

NOTE: this feature is not ready to use. It partially works with Windows (both triggers are mapped to the same axis), and being unusable under Linux and Mac OS (joystick hat is not functional and axis are messed up). So far you shouldn't use it. Proper implementation of this function is scheduled.

Function

Synopsis

```
joystick = cock.setDefaultXBox360 ( self, override, joystick )
```

Arguments

`table self`
: An object to use.

`string override`
: Optional. Will override standard layout name. Will use "XBox360" if not provided.

`number joystick`
: Optional. Will link specified joystick to the layout. Will use joystick #1 if not provided.

Returns

`number joystick`
: Internal joystick number. May differ from specified.

See also

- cock.addJoystick
- cock.setDefault
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setJoystickDeadzone

cock.setJoystickDeadzone · revision 11595 · last edited 2023-07-05

Sets joystick deadzones per axis.

Function

Synopsis

```
cock.setJoystickDeadzone ( self, joystick, axis, threshold )
```

Arguments

table self
: An object to use.

number joystick
: Optional. Internal joystick number. Will use 1 if not provided.

string joystick
: Internal joystick name.

number axis
: LÖVE joystick axis.

string axis
: Axis name.

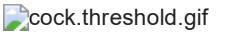
boolean axis
: Value "true" will affect all axis.

number threshold
: Deadzone threshold.

Remarks

Axis value "true" will set given threshold to all axis. Threshold value must be less than 1.

Setting a threshold results in axis readings range being "shrinked" towards the end. E.g., setting a threshold to 0.2 will only trigger joystick motion if reading surpass the threshold, but at that point it will return 0, gradually catching up with actual readings while going up to 1. This maintains entire axis range while getting rid of jittering, like when thumbstick doesn't goes to the dead center and jitters a little bit. Here's a figure displaying this:


cock.threshold.gif

See also

- cock.getJoystickDeadzone
- cock.setJoystickHatMode
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Input keys
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setJoystickHatMode

cock.setJoystickHatMode · revision 18454 · last edited 2023-07-05

Sets joystick hat mode per hat.

Synopsis

Function

```
cock.setJoystickHatMode ( self, joystick, hat, mode )
```

Arguments

table self
: An object to use.

number joystick
: Optional. Internal joystick number. Will use 1 if not provided.

string joystick
: Optional. Internal joystick name.

number hat
: Optional. LÖVE joystick hat. Will use 1 if not provided.

string hat
: Optional. Hat name.

number mode
: Optional. Hat mode number. Will use 1 if not provided.

string mode
: Optional. Hat mode name.

See also

- cock.getJoystickHatMode
- cock.setJoystickDeadzone
- cock.remapJoystickHat
- Common Organization of Controls Kit Hat modes
- Common Organization of Controls Kit Input devices
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setMouseFactor

cock.setMouseFactor · revision 11597 · last edited 2023-07-05

Sets horizontal and vertical mouse factor. You can access these values directly, see etc.mouse.* fields.

Function

Synopsis

```
cock.setMouseFactor ( self, x, y )
```

Arguments

table self
: An object to use.

number x
: Optional. Scaling factor by X axis. Will use old value if not provided.

number y
: Optional. Scaling factor by Y axis. Will use old value if not provided.

Remarks

Mouse readings will be multiplied by provided values. This is essentially a mouse sensevity.

See also

- cock.setMouseOffset
- cock.getMouseFactor
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.setMouseOffset

cock.setMouseOffset · revision 11598 · last edited 2023-07-05

Sets horizontal and vertical mouse offset. You can access these values directly, see etc.mouse.* fields.

Function

Synopsis

```
cock.setMouseOffset ( self, x, y )
```

Arguments

table self
: An object to use.

number x
: Optional. X-coordinate to use as center. Will use old value if not provided.

number y
: Optional. Y-coordinate to use as center. Will use old value if not provided.

Remarks

Values will be treaten as the "screen center", e.g. if mouse goes to the left or the the above of it then mouse position will read negative value.

See also

- cock.getMouseOffset
- cock.setMouseFactor
- Common Organization of Controls Kit Fields
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.unbind

cock.unbind · revision 11599 · last edited 2023-07-05

Unbinds specified map and option from current layout.

Function

Synopsis

```
cock.unbind ( self, map, option )
```

Arguments

table self
: An object to use.

string map
: A control map name to unbind.

string option
: Optional. Option to unbind map from. Will revert to defaultOption if not provided.

See also

- cock.bind
- cock.addOption
- cock.getEmptyOption
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.update

cock.update · revision 11607 · last edited 2023-07-05

Updates control states of an object.

Function

Synopsis

```
cock.update ( self )
```

Arguments

table self
: An object to update.

See also

- cock.updateAll
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

cock.updateAll

cock.updateAll · revision 11605 · last edited 2023-07-05

Updates control states of all objects. Essentially, iterates over all objects in a loop and calls cock.update on each object.

Function

Synopsis

```
cock.updateAll ( )
```

See also

- cock.update
- Common Organization of Controls Kit Manual
- Common Organization of Controls Kit

CodeBreak

CodeBreak · revision 3007 · last edited 2021-03-14

CodeBreak

This is a small game/timekiller that will let you try to break a code to unlock the lock. There is a small hint system, but you can ignore it and do it the hard way ;)

This is the first version and could use input how to improve it.



Downloads

LOVE file (+ readme and screenshot) (http://dl.dropbox.com/u/6074045/CodeBreak_love.zip)

Self contained game (+ readme and screenshot / for Windows) (http://dl.dropbox.com/u/6074045/CodeBreak_exe.zip)

CodeBreak

Collisions

Collisions · revision 29606 · last edited 2025-04-28

Collisions is a relaxing puzzle game based on timing, logic and anticipation. Explore a world of surreal machines and uncover the story behind the mysterious object looming above the horizon.



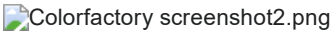
Features

- Over 60 physics-based puzzles
- 20 secret solutions
- Mini-games
- Relaxing soundtrack
- Engaging hints expressed in proverb form
- Subtle, non-verbal story

Play Collisions (https://2dengine.com/game/collisions?ref=love2d.org)

ColorFactory

ColorFactory · revision 10698 · last edited 2021-03-14



Download https://github.com/downloads/perky/Color-factory/colorfactory-0.13.love ↗

Maybe this game is made in LÖVE 0.7.0

Some function are renamed.

Thread:receive ---> Thread:get

Thread:send ---> Thread:set

and cote have a extra mistake.

\states\menu.lua line 80 "font_secretcode_12"should be"font_secretcode_16"

Other,I don't know how to start it.

ZDX朱世杰 10:43, 11 June 2013 (CEST)

ColorMode

ColorMode · revision 17382 · last edited 2026-03-24

Removed in LÖVE 0.9.0
This enum is not supported in that and later versions.

Controls whether images will be affected by the current color.

Constants

modulate
: Images (etc) will be affected by the current color.

replace
: Replace color mode. Images (etc) will not be affected by current color.

combine (since 0.8.0)
: Colorize images (etc) with the current color. While 'modulate' works like a filter that absorbs colors that differ from the current color (possibly to a point where a color channel is entirely absorbed), 'combine' tints the image in a way that preserves a hint of the original colors.

See Also

- love.graphics
- love.graphics.setColorMode
- love.graphics.getColorMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CommonPath

CommonPath · revision 30597 · last edited 2026-02-04

Available since LÖVE 12.0

This enum is not supported in earlier versions.

Enumeration representing a type of commonly-used directory, used in love.filesystem.mountCommonPath, love.filesystem.getFullCommonPath.

Constants

appsavedir

Application save data directory. Generally equivalent to userappdata/love/game_identity

appdocuments

Application documents directory. Generally equivalent to userdocuments/love/game_identity

userhome

OS Home directory for the OS user.

userappdata

OS AppData directory for the OS user.

userdesktop

OS Desktop directory for the OS user. Does not exist on Android.

userdocuments

OS Documents directory for the OS user.

See Also

- love.filesystem
- love.filesystem.mountCommonPath
- love.filesystem.getFullCommonPath

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompareMode

CompareMode · revision 23450 · last edited 2026-04-19

Available since LÖVE 0.10.0
This enum is not supported in earlier versions.

Different types of per-pixel stencil test and depth test comparisons. The pixels of an object will be drawn if the comparison succeeds, for each pixel that the object touches.

Constants

equal

- stencil tests: the stencil value of the pixel must be equal to the supplied value.
- depth tests: the depth value of the drawn object at that pixel must be equal to the existing depth value of that pixel.

notequal

- stencil tests: the stencil value of the pixel must **not** be equal to the supplied value.
- depth tests: the depth value of the drawn object at that pixel must **not** be equal to the existing depth value of that pixel.

less

- stencil tests: the stencil value of the pixel must be less than the supplied value.
- depth tests: the depth value of the drawn object at that pixel must be less than the existing depth value of that pixel.

lequal

- stencil tests: the stencil value of the pixel must be less than or equal to the supplied value.
- depth tests: the depth value of the drawn object at that pixel must be less than or equal to the existing depth value of that pixel.

gequal

- stencil tests: the stencil value of the pixel must be greater than or equal to the supplied value.
- depth tests: the depth value of the drawn object at that pixel must be greater than or equal to the existing depth value of that pixel.

greater

- stencil tests: the stencil value of the pixel must be greater than the supplied value.
- depth tests: the depth value of the drawn object at that pixel must be greater than the existing depth value of that pixel.

never
: Objects will never be drawn.

always
: Objects will always be drawn. Effectively disables the depth or stencil test.

See Also

- love.graphics
- love.graphics.setStencilTest
- love.graphics.stencil
- love.graphics.setDepthMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Component

Component · revision 26724 · last edited 2023-07-05

DEAD LINK

This is just a library that eases communication between game components,

by requesting references to them through initializer functions.

https://github.com/matiasah/component ↗

Other languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedData

CompressedData · revision 25488 · last edited 2026-04-10

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Available since LÖVE 0.10.0

In versions prior to 0.10.0 the CompressedData name was used for CompressedImageData objects.

Represents byte data compressed using a specific algorithm.

love.data.decompress can be used to de-compress the data (or love.math.decompress in 0.10.2 or earlier).

Constructors

love.data.compress	Compresses a string or data using a specific compression algorithm.	Added since 11.0
love.math.compress	Compresses a string or data using a specific compression algorithm.	Added since 0.10.0 Deprecated in 11.0

Functions

CompressedData:getFormat	Gets the compression format of the CompressedData.	Added since 0.10.0
Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

CompressedDataFormat	Compressed data formats.	Added since 0.10.0

Supertypes

- Data
- Object

See Also

- love.data
- love.data.decompress
- love.math (deprecated since 11.0)
- love.math.decompress (deprecated since 11.0)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedData:getFormat

CompressedData:getFormat · revision 15392 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the compression format of the CompressedData.

Function

Synopsis

```
format = CompressedData:getFormat( )
```

Arguments

None.

Returns

`CompressedDataFormat format`

The format of the CompressedData.

See Also

- CompressedData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedDataFormat

CompressedDataFormat · revision 25489 · last edited 2026-04-10

Available since LÖVE 0.10.0
This enum is not supported in earlier versions.

Compressed data formats.

Constants

lz4
: The LZ4 compression format. Compresses and decompresses very quickly, but the compression ratio is not the best. LZ4-HC is used when compression level 9 is specified. Some benchmarks are available here (https://github.com/Cyan4973/lz4#user-content-benchmarks).

zlib
: The zlib format is DEFLATE-compressed data with a small bit of header data. Compresses relatively slowly and decompresses moderately quickly, and has a decent compression ratio.

gzip
: The gzip format is DEFLATE-compressed data with a slightly larger header than zlib. Since it uses DEFLATE it has the same compression characteristics as the zlib format.

Available since LÖVE 11.0

deflate
: Raw DEFLATE-compressed data (no header).

See Also

- love.data
- love.data.compress
- love.math.compress (deprecated since 11.0)
- CompressedData
- CompressedData:getFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedImageData

CompressedImageData · revision 15376 · last edited 2026-04-22

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Available since LÖVE 0.9.0

In versions prior to 0.10.0 it was named CompressedData, however that name is used for something else now.

Represents compressed image data designed to stay compressed in RAM.

CompressedImageData encompasses standard compressed texture formats such as DXT1, DXT5, and BC5 / 3Dc (http://renderingpipeline.com/2012/07/texture-compression/).

You can't draw CompressedImageData directly to the screen. See Image for that.

Constructors

love.image.newCompressedData	Create a new CompressedImageData object from a compressed image file.	Added since 0.9.0

Functions

CompressedImageData:getDimensions	Gets the width and height of the CompressedImageData.	Added since 0.9.0
CompressedImageData:getFormat	Gets the format of the CompressedImageData.	Added since 0.9.0
CompressedImageData:getHeight	Gets the height of the CompressedImageData.	Added since 0.9.0
CompressedImageData:getMipmapCount	Gets the number of mipmap levels in the CompressedImageData.	Added since 0.9.0
CompressedImageData:getWidth	Gets the width of the CompressedImageData.	Added since 0.9.0
Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

CompressedImageFormat	Compressed image data formats.	Added since 0.9.0

Supertypes

- Data
- Object

See Also

- love.image
- love.image.newCompressedData
- love.image.isCompressed
- Image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

CompressedImageData:getDimensions

CompressedImageData:getDimensions · revision 15404 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the CompressedImageData.

Function

Synopsis

```
width, height = CompressedImageData:getDimensions( )
```

Arguments

None.

Returns

number width
: The width of the CompressedImageData.

number height
: The height of the CompressedImageData.

Function

Synopsis

```
width, height = CompressedImageData:getDimensions( level )
```

Arguments

number level
: A mipmap level. Must be in the range of [1, CompressedImageData:getMipmapCount()].

Returns

number width
: The width of a specific mipmap level of the CompressedImageData.

number height
: The height of a specific mipmap level of the CompressedImageData.

See Also

- CompressedImageData
- CompressedImageData:getMipmapCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

CompressedImageData:getFormat

CompressedImageData:getFormat · revision 15391 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the format of the CompressedImageData.

Function

Synopsis

```
format = CompressedImageData:getFormat( )
```

Arguments

None.

Returns

`CompressedImageFormat format`

The format of the CompressedImageData.

See Also

- CompressedImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedImageData:getHeight

CompressedImageData:getHeight · revision 15383 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the height of the CompressedImageData.

Function

Synopsis

```
height = CompressedImageData:getHeight( )
```

Arguments

None.

Returns

`number height`
The height of the CompressedImageData.

Function

Synopsis

```
height = CompressedImageData:getHeight( level )
```

Arguments

`number level`
A mipmap level. Must be in the range of [1, CompressedImageData:getMipmapCount()].

Returns

`number height`
The height of a specific mipmap level of the CompressedImageData.

See Also

- CompressedImageData
- CompressedImageData:getWidth
- CompressedImageData:getMipmapCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedImageData:getMipmapCount

CompressedImageData:getMipmapCount · revision 15385 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the number of mipmap (http://en.w ikipedia.org/w iki/Mipmap) levels in the CompressedImageData. The base mipmap level (original image) is included in the count.

Function

Synopsis

```
mipmaps = CompressedImageData:getMipmapCount( )
```

Arguments

None.

Returns

`number mipmaps`

The number of mipmap levels stored in the CompressedImageData.

Notes

Mipmap filtering cannot be activated for an Image created from a CompressedImageData which does not have enough mipmap levels to go down to 1x1. For example, a 256x256 image created from a CompressedImageData should have 8 mipmap levels or Image:setMipmapFilter will error. Most tools which can create compressed textures are able to automatically generate mipmaps for them in the same file.

See Also

- CompressedImageData
- Image:setMipmapFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedImageData:getWidth

CompressedImageData:getWidth · revision 15387 · last edited 2026-04-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width of the CompressedImageData.

Function

Synopsis

```
width = CompressedImageData:getWidth( )
```

Arguments

None.

Returns

`number width`
The width of the CompressedImageData.

Function

Synopsis

```
width = CompressedImageData:getWidth( level )
```

Arguments

`number level`
A mipmap level. Must be in the range of [1, CompressedImageData:getMipmapCount()].

Returns

`number width`
The width of a specific mipmap level of the CompressedImageData.

See Also

- CompressedImageData
- CompressedImageData:getHeight
- CompressedImageData:getMipmapCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CompressedImageFormat

CompressedImageFormat · revision 25491 · last edited 2026-04-22

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Deprecated in LÖVE 11.0

It has been superseded by PixelFormat..

Compressed image data formats. Here (http://renderingpipeline.com/2012/07/texture-compression/) and here (http://www.reedbeta.com/blog/2012/02/12/understanding-bcn-texture-compression-formats/) are a couple overviews of many of the formats.

Unlike traditional PNG or jpeg, these formats stay compressed in RAM and in the graphics card's VRAM. This is good for saving memory space as well as improving performance, since the graphics card will be able to keep more of the image's pixels in its fast-access cache when drawing it.


In LÖVE versions prior to 0.10.0, these constants are all lower-case.

Constants

DXT1

The DXT1 format. RGB data at 4 bits per pixel (compared to 32 bits for ImageData and regular Images.) Suitable for fully opaque images on desktop systems.

DXT3

The DXT3 format. RGBA data at 8 bits per pixel. Smooth variations in opacity do not mix well with this format.

DXT5

The DXT5 format. RGBA data at 8 bits per pixel. Recommended for images with varying opacity on desktop systems.

BC4

The BC4 format (also known as 3Dc+ or ATI1.) Stores just the red channel, at 4 bits per pixel.

BC4s

The signed variant of the BC4 format. Same as above but pixel values in the texture are in the range of [-1, 1] instead of [0, 1] in shaders.

BC5

The BC5 format (also known as 3Dc or ATI2.) Stores red and green channels at 8 bits per pixel.

BC5s

The signed variant of the BC5 format.

Available since LÖVE 0.9.2

BC6h

The BC6H format. Stores half-precision (http://en.wikipedia.org/wiki/Half-precision_floating-point_format) floating-point RGB data in the range of [0, 65504] at 8 bits per pixel. Suitable for HDR images on desktop systems.

BC6hs

The signed variant of the BC6H format. Stores RGB data in the range of [-65504, +65504].

BC7

The BC7 format (also known as BPTC.) Stores RGB or RGBA data at 8 bits per pixel.

Available since LÖVE 0.10.0

ETC1

The ETC1 format. RGB data at 4 bits per pixel. Suitable for fully opaque images on older Android devices.

ETC2rgb

The RGB variant of the ETC2 format. RGB data at 4 bits per pixel. Suitable for fully opaque images on newer mobile devices.

ETC2rgba

The RGBA variant of the ETC2 format. RGBA data at 8 bits per pixel. Recommended for images with varying opacity on newer mobile devices.

ETC2rgba1
: The RGBA variant of the ETC2 format where pixels are either fully transparent or fully opaque. RGBA data at 4 bits per pixel.

EACr
: The single-channel variant of the EAC format. Stores just the red channel, at 4 bits per pixel.

EACrs
: The signed single-channel variant of the EAC format. Same as above but pixel values in the texture are in the range of [-1, 1] instead of [0, 1] in shaders.

EACrg
: The two-channel variant of the EAC format. Stores red and green channels at 8 bits per pixel.

EACrgs
: The signed two-channel variant of the EAC format.

PVR1rgb2
: The 2 bit per pixel RGB variant of the PVRTC1 format. Stores RGB data at 2 bits per pixel. Textures compressed with PVRTC1 formats must be square and power-of-two sized.

PVR1rgb4
: The 4 bit per pixel RGB variant of the PVRTC1 format. Stores RGB data at 4 bits per pixel.

PVR1rgba2
: The 2 bit per pixel RGBA variant of the PVRTC1 format.

PVR1rgba4
: The 4 bit per pixel RGBA variant of the PVRTC1 format.

ASTC4x4
: The 4x4 pixels per block variant of the ASTC format. RGBA data at 8 bits per pixel.

ASTC5x4
: The 5x4 pixels per block variant of the ASTC format. RGBA data at 6.4 bits per pixel.

ASTC5x5
: The 5x5 pixels per block variant of the ASTC format. RGBA data at 5.12 bits per pixel.

ASTC6x5
: The 6x5 pixels per block variant of the ASTC format. RGBA data at 4.27 bits per pixel.

ASTC6x6
: The 6x6 pixels per block variant of the ASTC format. RGBA data at 3.56 bits per pixel.

ASTC8x5
: The 8x5 pixels per block variant of the ASTC format. RGBA data at 3.2 bits per pixel.

ASTC8x6
: The 8x6 pixels per block variant of the ASTC format. RGBA data at 2.67 bits per pixel.

ASTC8x8
: The 8x8 pixels per block variant of the ASTC format. RGBA data at 2 bits per pixel.

ASTC10x5
: The 10x5 pixels per block variant of the ASTC format. RGBA data at 2.56 bits per pixel.

ASTC10x6
: The 10x6 pixels per block variant of the ASTC format. RGBA data at 2.13 bits per pixel.

ASTC10x8
: The 10x8 pixels per block variant of the ASTC format. RGBA data at 1.6 bits per pixel.

ASTC10x10
: The 10x10 pixels per block variant of the ASTC format. RGBA data at 1.28 bits per pixel.

ASTC12x10
: The 12x10 pixels per block variant of the ASTC format. RGBA data at 1.07 bits per pixel.

ASTC12x12
: The 12x12 pixels per block variant of the ASTC format. RGBA data at 0.89 bits per pixel.

Notes

Not all formats are supported in love.graphics Images on all systems, although the DXT formats have close to 100% support on desktop operating systems.

The BC4 and BC5 formats are supported on systems with DirectX 10 / OpenGL 3-capable desktop hardware and drivers. The BC6H and BC7 formats are only supported on desktop systems with DirectX 11 / OpenGL 4-capable hardware and very recent drivers. macOS does not support BC6H or BC7 at all currently.

ETC1 is supported by Android devices, as well as newer (OpenGL ES 3-capable) iOS devices.

The PVR1 formats are supported by iOS devices, as well as Android devices with PowerVR GPUs.

The ETC2 and EAC formats are supported by newer (OpenGL ES 3-capable) iOS and Android devices.

ASTC is only supported by very new mobile devices (e.g. the iPhone 6), and the latest Skylake (and newer) integrated Intel GPUs. It has a variety of variants to allow for picking the most compressed possible one that doesn't have any noticeable compression artifacts, for a given texture.

Use love.graphics.getCompressedImageFormats to check for support:

```
local supportedformats = love.graphics.getCompressedImageFormats()

if not supportedformats["DXT5"] then
    -- Can't load CompressedImageData with the DXT5 format into images!
    -- On some Linux systems with Mesa drivers, the user will need to install a "libtxc-dxtn" package because the DXT (aka S3TC) formats
used to be patented.
    -- Support for DXT formats on all other desktop drivers is pretty much guaranteed.
end

if not supportedformats["BC5"] then
    -- Can't load CompressedImageData with the BC5 format into images!
    -- The user likely doesn't have a video card capable of using that format.
end
```

See Also

- love.image
- CompressedImageData
- CompressedImageData:getFormat
- love.graphics.getCompressedImageFormats

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact

Contact · revision 27511 · last edited 2023-07-05

Contacts are objects created to manage collisions in worlds.

The lifetimes of Contacts are short. When you receive them in callbacks, they may be destroyed immediately after the callback returns. Cache their values instead of storing Contacts directly.

Functions

Contact:getChildren	Gets the child indices of the shapes of the two colliding fixtures.	Added since 0.9.0	
Contact:getFixtures	Gets the two Fixtures that hold the shapes that are in contact.	Added since 0.9.2	
Contact:getFriction	Get the friction between two shapes that are in contact.		
Contact:getNormal	Get the normal vector between two shapes that are in contact.		
Contact:getPosition	Get the location of the contact point between two shapes.		Removed in 0.8.0
Contact:getPositions	Returns the contact points of the two colliding fixtures.	Added since 0.8.0	
Contact:getRestitution	Get the restitution between two shapes that are in contact.		
Contact:getSeparation	Get the separation between two shapes that are in contact.		Removed in 0.8.0
Contact:getVelocity	Get the linear impact velocity of a contact.		Removed in 0.8.0
Contact:isEnabled	Returns whether the contact is enabled.	Added since 0.8.0	
Contact:isTouching	Returns whether the two colliding fixtures are touching each other.	Added since 0.8.0	
Contact:resetFriction	Resets the contact friction to the mixture value of both fixtures.	Added since 0.8.0	
Contact:resetRestitution	Resets the contact restitution to the mixture value of both fixtures.	Added since 0.8.0	
Contact:setEnabled	Enables or disables the contact.	Added since 0.8.0	
Contact:setFriction	Sets the contact friction.	Added since 0.8.0	
Contact:setRestitution	Sets the contact restitution.	Added since 0.8.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
World:setCallbacks	Sets functions to be called when shapes collide.	Added since 0.8.0	

Supertypes

- Object

See Also

- World:setCallbacks
- World:getContacts
- love.physics

Examples

```
-- Demonstration of correct use of Contacts to avoid "Attempt to use destroyed contact."
-- Most relevant parts are physics.collisionOnEnter, love.update, love.draw.

local screen = {
    x = 500,
    y = 500,
}
local color = {
    blue = {0.23, 0.25, 0.59, 1},
    purple = {0.25, 0.09, 0.28, 1},
    white = {0.89, 0.91, 0.90, 1},
}

local physics = {
    normals = {},
}

-- Source: https://www.love2d.org/forums/viewtopic.php?p=17018&sid=5c8509cf6b97e3ce5aab1bc49e85acc5#p17018
```

```lua
local function draw_arrow(x1, y1, x2, y2)
    local head_length = 10
    local head_angle = math.pi * 0.2
    love.graphics.line(x1, y1, x2, y2)
    local a = math.atan2(y1 - y2, x1 - x2)
    love.graphics.polygon('fill', x2, y2, x2 + head_length * math.cos(a + head_angle), y2 + head_length * math.sin(a + head_angle), x2 +
head_length * math.cos(a - head_angle), y2 + head_length * math.sin(a - head_angle))
end


function physics.collisionOnEnter(fixture_a, fixture_b, contact)
    local dx,dy = contact:getNormal()
    dx = dx * 30
    dy = dy * 30
    local point = {contact:getPositions()}
    for i=1,#point,2 do
        local x,y = point[i], point[i+1]
        -- Cache the values inside the contacts because they're not guaranteed
        -- to be valid later in the frame.
        table.insert(physics.normals, {x,y, x+dx, y+dy})
    end

    -- do not use contact after this function returns
end

function love.load()
    love.physics.setMeter(64)
    local can_bodies_sleep = true
    physics.world = love.physics.newWorld(0, 9.8, can_bodies_sleep)
    physics.world:setCallbacks(physics.collisionOnEnter)

    physics.geo = {}
    for y=0,1 do
        local side = {}
        side.body = love.physics.newBody(physics.world, screen.x/2, screen.y * y)
        side.shape = love.physics.newRectangleShape(screen.x, 50)
        side.fixture = love.physics.newFixture(side.body, side.shape)
        table.insert(physics.geo, side)
    end
    for x=0,1 do
        local side = {}
        side.body = love.physics.newBody(physics.world, screen.x * x, screen.y/2)
        side.shape = love.physics.newRectangleShape(50, screen.y)
        side.fixture = love.physics.newFixture(side.body, side.shape)
        table.insert(physics.geo, side)
    end

    physics.bouncers = {}
    for y=50,screen.y-50,50 do
        for x=50,screen.x-100,50 do
            local ball = {}
            ball.body = love.physics.newBody(physics.world, x, y, "dynamic")
            ball.shape = love.physics.newCircleShape(20)
            ball.fixture = love.physics.newFixture(ball.body, ball.shape, 1)
            ball.fixture:setRestitution(0.9)
            ball.body:setMass(50)
            ball.body:setLinearVelocity(100, 10)
            table.insert(physics.bouncers, ball)
        end
    end

    love.graphics.setBackgroundColor(color.blue)
    love.window.setMode(screen.x, screen.y)
end

function love.update(dt)
    -- Updating the world will trigger collisionOnEnter
    physics.world:update(dt)

    if love.keyboard.isDown("space") then
        for _,ball in ipairs(physics.bouncers) do
            ball.body:applyLinearImpulse(0, 1000)
        end
```

```
    elseif love.keyboard.isDown("escape") then
        love.event.quit()
    end
end

function love.draw()
    love.graphics.setColor(color.purple)
    for i,side in ipairs(physics.geo) do
        love.graphics.polygon("fill", side.body:getWorldPoints(side.shape:getPoints()))
    end
    for i,ball in ipairs(physics.bouncers) do
        love.graphics.circle("fill", ball.body:getX(), ball.body:getY(), ball.shape:getRadius())
    end

    love.graphics.setColor(color.white)
    love.graphics.printf("smoosh balls with space", 5,5, 200, "left")
    love.graphics.printf(string.format("%i contacts", #physics.normals), screen.x-100,5, 100, "right")
    love.graphics.printf("arrows show contact normals", 5,screen.y-20, 200, "left")

    -- Use our cached normals
    for key,args in pairs(physics.normals) do
        local x1, y1, x2, y2 = unpack(args)
        draw_arrow(x1, y1, x2, y2)
        physics.normals[key] = nil
    end
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getChildren

Contact:getChildren · revision 29098 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the child indices of the shapes of the two colliding fixtures. For ChainShapes, an index of 1 is the first edge in the chain.

Used together with Fixture:rayCast or ChainShape:getChildEdge.

Function

Synopsis

```
indexA, indexB = Contact:getChildren( )
```

Arguments

None.

Returns

`number indexA`
: The child index of the first fixture's shape.

`number indexB`
: The child index of the second fixture's shape.

See Also

- Contact
- Fixture:rayCast
- ChainShape:getChildEdge
- World:getBodies

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getFixtures

Contact:getFixtures · revision 13759 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the two Fixtures that hold the shapes that are in contact.

Function

Synopsis

```
fixtureA, fixtureB = Contact:getFixtures( )
```

Arguments

None.

Returns

`Fixture fixtureA`
: The first Fixture.

`Fixture fixtureB`
: The second Fixture.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getFriction

Contact:getFriction · revision 6053 · last edited 2025-11-21

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the friction between two shapes that are in contact.

Function

Synopsis

```
friction = Contact:getFriction( )
```

Arguments

None.

Returns

`number friction`
The friction of the contact.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getNormal

Contact:getNormal · revision 6054 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the normal vector between two shapes that are in contact.

This function returns the coordinates of a unit vector that points from the first shape to the second.

Function

Synopsis

```
nx, ny = Contact:getNormal( )
```

Arguments

None.

Returns

number nx
: The x component of the normal vector.

number ny
: The y component of the normal vector.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getPosition

Contact:getPosition · revision 10599 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Get the location of the contact point between two shapes.

Function

Synopsis

```
cx, cy = Contact:getPosition( )
```

Arguments

None.

Returns

`number cx`
: The x coordinate of the contact point.

`number cy`
: The y coordinate of the contact point.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getPositions

Contact:getPositions · revision 29490 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns the contact points of the two colliding fixtures. There can be one or two points.

If one of the colliding fixtures is a sensor, this will return nil.

Function

Synopsis

```
x1, y1, x2, y2 = Contact:getPositions( )
```

Arguments

None.

Returns

number x1
: The x coordinate of the first contact point.

number y1
: The y coordinate of the first contact point.

number x2
: The x coordinate of the second contact point.

number y2
: The y coordinate of the second contact point.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getRestitution

Contact:getRestitution · revision 6056 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the restitution between two shapes that are in contact.

Function

Synopsis

```
restitution = Contact:getRestitution( )
```

Arguments

None.

Returns

`number restitution`
: The restitution between the two shapes.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getSeparation

Contact:getSeparation · revision 10601 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Get the separation between two shapes that are in contact.

The return value of this function is always zero or negative, with a negative value indicating overlap between the two shapes.

Function

Synopsis

```
distance = Contact:getSeparation( )
```

Arguments

None.

Returns

`number distance`

The separation between the two shapes.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getShapes

Contact:getShapes · revision 29135 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

It has replaced Contact:getFixtures.

Gets the two Shapes that are in contact.

Function

Synopsis

```
shapeA, shapeB = Contact:getShapes( )
```

Arguments

None.

Returns

Shape shapeA

The first Shape.

Shape shapeB

The second Shape.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:getVelocity

Contact:getVelocity · revision 10602 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0

This function is not supported in that and later versions.

Get the linear impact velocity of a contact.

Function

Synopsis

```
vx, vy = Contact:getVelocity( )
```

Arguments

None.

Returns

number vx
: The x component of the velocity vector.

number vy
: The y component of the velocity vector.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:isEnabled

Contact:isEnabled · revision 19665 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns whether the contact is enabled. The collision will be ignored if a contact gets disabled in the preSolve callback.

Function

Synopsis

```
enabled = Contact:isEnabled( )
```

Arguments

None.

Returns

`boolean enabled`

True if enabled, false otherwise.

See Also

- Contact
- Contact:setEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:isTouching

Contact:isTouching · revision 10604 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Returns whether the two colliding fixtures are touching each other.

Function

Synopsis

```
touching = Contact:isTouching( )
```

Arguments

None.

Returns

`boolean touching`

True if they touch or false if not.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:resetFriction

Contact:resetFriction · revision 10605 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Resets the contact friction to the mixture value of both fixtures.

Function

Synopsis

```
Contact:resetFriction( )
```

Arguments

None.

Returns

Nothing.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:resetRestitution

Contact:resetRestitution · revision 10606 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Resets the contact restitution to the mixture value of both fixtures.

Function

Synopsis

```
Contact:resetRestitution( )
```

Arguments

None.

Returns

Nothing.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:setEnabled

Contact:setEnabled · revision 10607 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Enables or disables the contact.

Function

Synopsis

```
Contact:setEnabled( enabled )
```

Arguments

`boolean enabled`

True to enable or false to disable.

Returns

Nothing.

See Also

- Contact
- Contact:isEnabled

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:setFriction

Contact:setFriction · revision 30237 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the contact friction.

Function

Synopsis

```
Contact:setFriction( friction )
```

Arguments

`number friction`
: The contact friction.

Returns

Nothing.

See Also

- Contact
- Contact:getFriction

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Contact:setRestitution

Contact:setRestitution · revision 10609 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the contact restitution.

Function

Synopsis

```
Contact:setRestitution( restitution )
```

Arguments

`number restitution`
: The contact restitution.

Returns

Nothing.

See Also

- Contact

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ContainerType

ContainerType · revision 21483 · last edited 2023-07-05

Available since LÖVE 11.0
This enum is not supported in earlier versions.

Return type of various data-returning functions.

Constants

data
: Return type is ByteData.

string
: Return type is string.

Notes

Use **data** return type if you want to pass a reference to the data to another thread or LÖVE function, and use **string** return type if you want to use it immediately, like sending it to network.

See Also

- love.data
- love.data.compress
- love.data.decompress
- love.data.encode
- love.data.decode
- love.data.pack
- love.filesystem.read
- File:read

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

coroutine.resume

coroutine.resume · revision 19084 · last edited 2021-03-14

Usually, errors that occur inside a coroutine do not get reported, which can lead to all sorts of coding catastrophes. To fix, include the following code before any use of coroutine.resume

Source

```
_coroutine_resume = coroutine.resume
function coroutine.resume(...)
    local state,result = _coroutine_resume(...)
    if not state then
        error( tostring(result), 2 )    -- Output error message
    end
    return state,result
end
```

cron.lua

cron.lua · revision 24463 · last edited 2023-07-05

Original forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2882).

Latest version:

https://github.com/kikito/cron.lua ↗

cron.lua are a group of functions that make it easier to program things so they happen in the future - either once or periodically.

Please refer to the forum post above or the latest version link for examples of use.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CTRL

CTRL · revision 20591 · last edited 2023-07-05

On this page

- Basic Use
  - Example Code
  - List of Valid Inputs
- Advanced
  - Automatic Input Grab
  - Custom Bindings
  - Gamepad Mapping
  - Custom Input Method
  - Device Mapping
- Reference Manual
- Links
- Other Languages

CTRL - Control & Trigger Relays Library - LÖVE INPUT

Simple for basic usage and at the same time robust and advanced for extended use, this library provides all necessary facilities to handle user input, and with enough skill, can be extended to handle even the input devices not native supported by the framework.

Basic Use

To get started, an instance of CTRL class needs to be created (so if for whatever reason you need multiple different CTRL instances - you can do that), and event handler callback functions need to be defined. They largely work the same way as normal LÖVE input callbacks, except CTRL strips input events of all information that's not relevant, and refines the rest to the most usable state. Then, some input bindings need to be made. And finally, CTRL instance needs to be hooked up to LÖVE input events.

Example Code

```
local ctrl = require ( 'ctrl' ) ( )
function ctrl:inputpressed ( name, value ) print ( "pressed", name, value ) end
function ctrl:inputreleased ( name, value ) print ( "released", name, value ) end
function ctrl:inputmoved ( name, value ) print ( "moved", name, value ) end
ctrl:bind ( "fire", { "keyboard", "space" } )
ctrl:bind ( "fire", { "joystick", "default", "button", 1 } )
ctrl:bind ( "fire", { "mouse", "left" } )
ctrl:hookup ( )
```

List of Valid Inputs

- `"keyboard"`
  - all standard LÖVE keyboard constants
- `"mouse"`
  - `"x"`, `"y"`, `"wheelx"`, `"wheely"`, `"left"`, `"right"`, `"middle"`
  - 4, 5, 6, etc.
- `"gamepad"`
  - `"default"` or any existing mapping for a Joystick
    - `"axis"`
      - `"leftx"`, `"rightx"`, `"lefty"`, `"righty"`, `"triggerleft"`, `"triggerright"`
    - `"button"`
      - `"a"`, `"b"`, `"x"`, `"y"`, `"back"`, `"guide"`, `"start"`, `"leftstick"`, `"rightstick"`, `"leftshoulder"`, `"rightshoulder"`
    - `"hat"`
      - `"up"`, `"down"`, `"left"`, `"right"`
- `"joystick"`
  - `"default"` or any existing device mapping for a Joystick
    - `"axis"`
      - 1, 2, 3, etc.
    - `"button"`
      - 1, 2, 3, etc.
    - `"hat"`
      - 1, 2, 3, etc.
        - `"up"`, `"down"`, `"left"`, `"right"`

By default all joysticks and gamepads map to `"default"`, this is what you want most of the time.

Advanced

CTRL includes substantial assortment of features (hence its hefty size) on top of its basic functionality.

Automatic Input Grab

Most useful for input settings screens. Using `grab` function CTRL can be set to automatically bind whichever input user had pressed to a select action, and it can be set to filter out only desired inputs out of all incoming signals. All three arguments are optional, but either callback or autobind name should be present (otherwise it will not do anything useful).

```
local bindname = "foo"
callback = function ( input )
    --cancel input if user have pressed "escape", otherwise bind
    if input[ 1 ] == "keyboard" and input[ 2 ] == "escape" then ctrl:grab ( )
    else ctrl:bind ( bindname, input ) end
end
ctrl:grab ( callback )
```

```
--set CTRL to autobind any next input to "foo" action, and only allow input from keyboard and gamepads buttons
ctrl:grab ( "foo", { { "keyboard" }, { "gamepad", nil, "button" } } )
```

It would make sense to save acquired input bindings. CTRL has facilities for that, the `saveData` and `loadData` functions. You can pass a filename in it and it will load/save data to it. If you pass nothing to `saveData`, it returns saved data string. You can pass such string into `loadData` instead of filename.

Custom Bindings

One of the most important bits of functionality is binding customization. This is achieved by passing third argument into `bind` function, the bind options table. It can contain `mapper` table that contains information about raw value re-mapping function attached to that binding, `filter` table that contains similar information about real-time final value mapping from raw values, and `events` table that has a list of events that this binding shall generate.

Mapper table contains two named values:

- `func`: function that will do the mapping
- `args` (optional): table of additional arguments that will be passed into that function

The mapper function should be either string name of a registered **mapper** function (see below) or a function value. When it's called, it's passed `raw, last mapped, args, ...` arguments. First argument is a raw value from the input device. Second is the value that it had output the last time it was called. Third is the `args` optional table that can contain anything or not exist at all. Finally, a `vararg` is passed. It contains all, if any, additional raw event parameters passed into general event handler function. For standard LÖVE events it's empty except for mouse events, then it's `dx dy` and `isTouch` from the mouse callbacks. You can define custom raw input events that pass variety of raw input information through this.

```
local fooMapper = function ( raw, last, args, ... )
    return raw * args.foo
end
ctrl:bind ( "foo", { "keyboard", "f" }, { mapper = { func = fooMapper, args = { foo = 1 } } } )
```

Filter table contains two named values:

- `func`: function that will do the mapping
- `args` (optional): table of additional arguments that will be passed into that function

As with mapper, filter function should be either string name of a registered **filter** function (see below) or a function value. When it's called, it's passed `dt, mapped or raw, last filtered, args` arguments. First argument is `dt` from the update function. Second is the current mapped (or raw, if no mapper set) value. Third is the value that it had output the last time it was called. Fourth is the `args` optional table that can contain anything or not exist at all.

```
local fooFilter = function ( dt, raw, last, args )
    return last + raw * dt
end
ctrl:bind ( "foo", { "keyboard", "f" }, { filter = { func = fooFilter } } )
```

Events table contains any list of event tables. Each of those tables contains three named values:

- `trigger`: function that will decide whether or not an input event shall be dispatched
- `handler`: callback function that will be called to handle the input event
- `args` (optional): table of additional arguments that will be passed into trigger function

The trigger function should be either string name of a registered **trigger** function (see below) or a function value. When it's called, it's passed `current, previous, args` arguments. First argument is the current value. Second is the previous value. Third is the `args` optional table that can contain anything or not exist at all. Handler function should be either string name of existing event handler function (simply defined as normal) or a function value. When it's called, it's passed `ctrl, name, value` arguments. First argument is the ctrl instance that triggered this event. Second is the bound action name that triggered this event. Third is the current value.

```
local fooTrigger = function ( curr, last, args )
    return curr > last
end
local inputFooBar = function ( ctrl, name, value )
    print ( "foobar", name, value )
end
ctrl:bind ( "foo", { "keyboard", "f" }, { events = { foobar = { trigger = fooTrigger, handler = inputFooBar } } } )
```

Mapper, filter and trigger functions can be "registered" and be referenced by their string name in the binding code. While not immediately beneficial, it allows the data saver to store your bindings exactly, without losing any bits of data: your attached function code cannot be saved to a file, but a string name can be. The event handler callbacks do not need to be explicitly registered, you simply define them on a CTRL instance the same way you define LÖVE input callbacks. The reason to do it that way instead of passing function itself into the binding options is the same as with mappers, filters and triggers.

```
ctrl:addMapperFunction ( "fooMapper", fooMapper )
ctrl:addFilterFunction ( "fooFilter", fooFilter )
ctrl:addTriggerFunction ( "fooTrigger", fooTrigger )
function ctrl:inputFooBar ( name, value )
    print ( "foobar", name, value )
end
```

CTRL already includes by default a number of mappers, filters and triggers:

- `deadzone`: clamps lower region of analog input to 0 (`args = { deadzone = 0.1 }`)
- `remap`: maps entire analog input region to a different output region (`args = { rawmin = 0, rawmax = 1, mapmin = 0, mapmax = 1, min = 0, max = 1 }`)

- `smooth`: smoothly moves output value towards raw value (`args = { speed = 1 }`)
- `ramp`: moves output value up or down depending on raw value (`args = { speed = 1, min = 0, max = 1 }`)

- `pressed`: triggered when value drops under 0.25
- `released`: triggered when value raises over 0.75
- `moved`: triggered when value changes by any amount

Gamepad Mapping

It works the same way as LÖVE's gamepadMapping functionality, except it accepts CTRL-fashion input table, so you can set gamepad mapping using `grab` function.

```
local joystick = love.joystick.getJoysticks[ 1 ]
local inputtype = "start"
local callback = function ( input )
    ctrl:setGamepadMapping ( joystick, inputtype, input )
end
--start grab, filter in only joystick buttons
ctrl:grab ( callback, { "joystick", nil, "button" } )
```

Custom Input Method

This allows to use custom input devices as well as purely virtual devices such as virtual keyboard and virtual joystick commonly found on mobile. There are two ways of doing it. First is piping custom input device events into standard events. Then custom device will be indistinguishable from normal device which it shadows. Second is defining new input type and manually calling general input handler

function.

```
ctrl:addInputs ( { "virtualkeyboard", "A" }, { "w", "s", "a", "d" } )
ctrl:addInputs ( { "virtualkeyboard", "B" }, { "u", "j", "h", "k" } )

ctrl:bind ( "foo", { "virtualkeyboard", "A", "w" } )
ctrl:bind ( "bar", { "virtualkeyboard", "B", "u" } )
 . . .
function virtualkeyboard:pressed ( section, key )
    ctrl:handleUpdate ( { "virtualkeyboard", section, key }, 1 )
end
function virtualkeyboard:released ( section, key )
    ctrl:handleUpdate ( { "virtualkeyboard", section, key }, 0 )
end
```

Device Mapping

If input type cannot be uniquely and persistently identified (like keyboards and mice), then its ID needs to be produced in run-time. There are many ways to do it and yet none are silver bullet be all end all type. This is why CTRL opts out of automatically trying to identify devices, instead defaulting to mapping them all to `"default"`, so you will need to implement your own ID generator function. Simply using defaults is what you need most of the time, but some scenarios, such as local multiplayer, demand differentiation between similar or even identical devices. One common strategy is to simply assign each gamepad their index number, and refer to them by that number. Another strategy involves acquiring GUID or literal name of the gamepad and, if it's not unique, adding a differentiating digit to it. Once you're done producing ID, you can assign joystick to this ID using `mapDevice` function:

```
local id = produceId ( joystick )
ctrl:mapDevice ( joystick, id )
```

Reference Manual

```
ctrl ( ), ctrl.new ( )
        creates and returns new input handler instance
ctrl:hookup ( )
        hooks up input handlers to leech off of default löve input callbacks
ctrl:bind ( name, input, [options] )
        binds specified input to an input name
        * name - input name to bind to
        * input - ctrl input address to bind
        * options (optional) - options table, can contain following fields:
                * mapper (optional) - table describing a raw value mapper
                        * func - callback mapper function, returns mapped raw value
                        * args (optional) - callback function arguments table
                * filter (optional) - table describing a value filter
                        * func - callback filter function, returns filtered value
                        * args (optional) - callback function arguments table
                * events (optional) - table containing list of events description tables. Each table contains:
                        * trigger - callback trigger function, returns true if event should be triggered
                        * handler - callback handler function, called if event is triggered
                        * args (optional) - callback trigger arguments table
ctrl:unbind ( [name], [input] )
        unbinds specified inputs
        * name (optional) - input name to unbind
        * input (optional) - ctrl input address to unbind
        if called without arguments, unbinds everything
        if called without input address, unbinds everything bound to a specific name
        if called without name, unbinds everything bound to a specific address
        input address can be partial address, then it acts like a filter, unbinds everything that matches
ctrl:grab ( [callback], [autobind], [filters] )
        starts tracking input devices for any activity, grabs the first one that changes
        * callback (optional) - function that will be called when activity is detected
                shall return true if it should ignore current input and track the next one
                called with the following arguments:
                        * input - ctrl input address table
                        * name - the autobind argument unmodified
        * autobind (optional) - string input name to which it would automatically bind anything it finds
        * filters (optional) - list of ctrl input addresses that it will match against during tracking
ctrl:getBindings ( [name], [input] )
        finds and returns list of all input addresses bound to a specified input name
        * name (optional) - name to look bindings for
        * input (optional) - ctrl input path to look in
```

```
        if name is not passed, finds all bindings
        input can be partial path, then it acts as a filter
        list entries have the following format:
                * name - string name of the input
                * input - ctrl input address table
                * options - table of input options (mapper, filter, events)
ctrl:isUp ( name )
ctrl:wasUp ( name )
        returns true if value of an input is under 0.25
        * name - input name to look for
ctrl:isDown ( name )
ctrl:wasDown ( name )
        returns true if value of an input is above 0.75
        * name - input name to look for
ctrl:getValue ( name )
ctrl:getLastValue ( name )
        returns value of an input
        * name - input name to look for
ctrl:resetValues ( )
        resets all values to 0
        focusing out and in a window combined with some ways of input handling can cause input glitches,
        resetting all values solves that problem
ctrl:mapDevice ( device, value )
        maps userdata or table device to a string, number or boolean value
        * device - the input device userdata, table or any other value by which it's referenced
        * value - string, number or boolean value by which its input would be addressed
        should be used with joysticks and gamepads and other devices without persistent ID
ctrl:getMapping ( device )
        returns previously mapped value for this device, defaults to "default"
        * device - the input device to look for
ctrl:setGamepadMapping ( joy, control, input )
        mirros love.joystick.setGamepadMapping, but provides seamless integraiton with the library
        * joy - löve Joystick object to map
        * control - löve Gamepad control (button, hat, axis)
        * input - ctrl joystick input path table
ctrl:getGamepadMapping ( joy, control )
        mirros love.joystick.getGamepadMapping, but provides seamless integraiton with the library
        * joy - löve Joystick object to get mapping from
        * control - löve Gamepad control (button, hat, axis)
        returns ctrl joystick input address table for selected Gamepad control
ctrl:addInputs ( input, new )
        adds new values to the valid inputs list
        * input - ctrl input path to add to
        * new - list of additional valid inputs, can contain a metatable
ctrl:handleUpdate ( input, raw, ... )
        processes input event and dispatches callbacks
        * input - ctrl input address table
        * raw - raw value
        * vararg - any additional values, will be passed into raw mapper function
ctrl:addTriggerFunction ( name, func )
        adds new trigger function to the list
        * name - string name of new event trigger function
        * func - callback function, shall return true if event should be triggered
                called with the following arguments:
                        * current value
                        * previous value
                        * function arguments table
ctrl:addFilterFunction ( name, func )
        adds new filter function to the list
        * name - string name of new value filter function
        * func - callback function, shall return filtered value
                called with the following arguments:
                        * time elapsed since last call
                        * raw input value
                        * previous filtered value
                        * function arguments table
ctrl:addMapperFunction ( name, func )
        adds new raw value mapper function to the list
        * name - string name of new raw value mapper function
        * func - callback function, shall return mapped raw value
                called with the following arguments:
                        * raw value
                        * previous mapped value
```

```
                        * function arguments table
                        * vararg passed into input event handler function
ctrl:saveData ( [filename] )
        saves all internal data to a file, returns data string if file is not provided
        * filename (optional) - filename to store data in
ctrl:loadData ( filename )
        loads data from a file or a string
        * filename - filename to load data from, or data string
ctrl:handleUpdate ( dt )
ctrl:handleKeypressed ( key, code, repeated )
ctrl:handleKeyreleased ( key, code, repeated )
ctrl:handleMousepressed ( x, y, button, touch )
ctrl:handleMousereleased ( x, y, button, touch )
ctrl:handleMousemoved ( x, y, dx, dy, touch )
ctrl:handleWheelmoved ( x, y )
ctrl:handleGamepadaxis ( joy, axis, value )
ctrl:handleGamepadpressed ( joy, button )
ctrl:handleGamepadreleased ( joy, button )
ctrl:handleJoystickaxis ( joy, axis, value )
ctrl:handleJoystickhat ( joy, hat, dir )
ctrl:handleJoystickpressed ( joy, button )
ctrl:handleJoystickreleased ( joy, button )
ctrl:handleJoystickadded ( joy )
ctrl:handleJoystickremoved ( joy )
        LÖVE input event handlers. To be called manually as appropriate if CTRL instance is not "hooked" up.
```

Links

- forum thread: https://love2d.org/forums/viewtopic.php?f=5&t=84093
- bitbucket repo: https://bitbucket.org/rcoaxil/ctrl ↗
- demo code/app: https://bitbucket.org/rcoaxil/ctrl/downloads/example.love ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CullMode

CullMode · revision 22235 · last edited 2026-04-19

Available since LÖVE 11.0

This enum is not supported in earlier versions.

How Mesh geometry is culled when rendering.

Constants

back

Back-facing triangles in Meshes are culled (not rendered). The vertex order of a triangle determines whether it is back- or front-facing.

front

Front-facing triangles in Meshes are culled.

none

Both back- and front-facing triangles in Meshes are rendered.

See Also

- love.graphics
- love.graphics.setMeshCullMode
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Cupid

Cupid · revision 25571 · last edited 2023-07-05

Cupid is a debugging shim for Love2d.

Put cupid.lua in your project directory and make this line the first line of your conf.lua:

```
require("cupid");
```

Out of the box you will get the following features:

- A developer console (press ~)
- A crash debugging window will catch lua errors
- Your source directory will be monitored for changes, and game reloaded then changes are found. (Set enable_watcher to false to disable this).

Download from bitbucket (https://bitbucket.org/basicer/cupid-git/).

Cursor

Cursor · revision 11241 · last edited 2025-10-01

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

Represents a hardware cursor.

Constructors

love.mouse.getSystemCursor	Gets a Cursor object representing a system-native hardware cursor.	Added since 0.9.0
love.mouse.newCursor	Creates a new hardware Cursor object from an image.	Added since 0.9.0

Functions

Cursor:getType	Gets the type of the Cursor.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

CursorType	Types of hardware cursors.	Added since 0.9.0

Supertypes

- Object

See Also

- love.mouse
- love.mouse.setCursor
- love.mouse.getCursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Cursor:getType

Cursor:getType · revision 11238 · last edited 2025-10-01

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the type of the Cursor.

Function

Synopsis

```
ctype = Cursor:getType( )
```

Arguments

None.

Returns

`CursorType ctype`

The type of the Cursor.

See Also

- Cursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

CursorType

CursorType · revision 31635 · last edited 2026-05-23

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Types of hardware cursors.

Not all cursor types are available on all operating systems. If an unsupported type is used, it will default to something else.

Constants

Cursor name	Description
image	A custom image cursor.
arrow	The default arrow cursor.
ibeam	Indicates selectable or editable text.
wait	An hourglass or spinner, indicating something is loading.
waitarrow	Ditto, but with the arrow cursor, meaning loading is low-priority.
crosshair	A cross-shaped cursor for more precise actions.
hand	A pointer, indicating an active button or link.
sizewe	Arrows pointing from east to west.
sizens	Arrows pointing from north to south.
sizenesw	Arrows pointing from north-east to south-west.
sizenwse	Arrows pointing from north-west to south-east.
sizeall	Arrows pointing in all cardinal directions.
no	The game or an element thereof cannot be interacted with.

Reference

Cursors LOVE.png

See also

- love.mouse
- Cursor
- Cursor:getType
- love.mouse.newCursor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data

Data · revision 29855 · last edited 2025-09-30

The superclass of all data.

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object

Subtypes

ByteData	Data object containing arbitrary bytes in an contiguous memory.	Added since 11.0	
CompressedData	Byte data compressed using a specific algorithm.	Added since 0.10.0	
CompressedImageData	Compressed image data designed to stay compressed in RAM and on the GPU.	Added since 0.9.0	
FileData	Data representing the contents of a file.		
FontData	A FontData represents a font.	Added since 0.7.0	Removed in 0.8.0
GlyphData	A GlyphData represents a drawable symbol of a font.	Added since 0.7.0	
ImageData	Raw (decoded) image data.		
SoundData	Contains raw audio samples.		

See Also

- love
- love.data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data:clone

Data:clone · revision 22222 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Creates a new copy of the Data object.

Function

Synopsis

```
clone = Data:clone( )
```

Arguments

None.

Returns

`Data clone`

The new copy.

See Also

- Data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data:getFFIPointer

Data:getFFIPointer · revision 27315 · last edited 2026-04-10

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Example
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets an FFI (http://luajit.org/ext_ffi.html) pointer to the Data.

This function should be preferred instead of Data:getPointer because the latter uses light userdata which can't store more all possible memory addresses on some new ARM64 architectures, when LuaJIT is used.


Use at your own risk. Directly reading from and writing to the raw memory owned by the Data will bypass any safety checks and thread-safety the Data might normally have.

Function

Synopsis

```
pointer = Data:getFFIPointer( )
```

Arguments

None.

Returns

cdata pointer

A raw `void*` pointer to the Data, or `nil` if FFI is unavailable.

Example

```
local mem = love.data.newByteData(2 ^ 30)                    -- 1 GB
local ptr = ffi.cast('uint8_t*', mem:getFFIPointer())        -- grab byte pointer
local byte100 = ptr[99]                                      -- get the 100th byte out of the allocated data mem
local uint32array = ffi.cast('uint32_t*', mem:getFFIPointer()) -- grab 4-byte integer pointer
uint32array[0] = 0xdeadbeef                                  -- treat mem as integer array, assign to 1st position
```

As an example usage, allocate memory and grab an FFI pointer. The pointer can be cast to any FFI ctype one wishes in order to access the memory accordingly. A pointer to the data could be cast to type `uint8_t*`, allowing one to access that memory like a 0-based sequential Lua table that contains byte values. Or the same memory could be treated as an array of 32-bit integers by recasting the pointer to `uint32_t*` . Note that these pointers will be invalid if the underlying ByteData goes out of scope.

See Also

- Data
- Data:getPointer
- Data:getSize
- Data:getString

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data:getPointer

Data:getPointer · revision 25480 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets a pointer to the Data. Can be used with libraries such as LuaJIT's FFI (http://luajit.org/ext_ffi.html).


Use at your own risk. Directly reading from and writing to the raw memory owned by the Data will bypass any safety checks and thread-safety the Data might normally have.


Since LÖVE 11.3, Data:getFFIPointer is a preferred alternative because it can work with new 64-bit architectures.

Function

Synopsis

```
pointer = Data:getPointer( )
```

Arguments

None.

Returns

`light userdata pointer`

A raw pointer to the Data.

See Also

- Data
- Data:getSize
- Data:getString
- Data:getFFIPointer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data:getSize

Data:getSize · revision 16487 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the Data's size in bytes.

Function

Synopsis

```
size = Data:getSize( )
```

Arguments

None.

Returns

`number size`
: The size of the Data in bytes.

See Also

- Data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Data:getString

Data:getString · revision 29144 · last edited 2024-04-13

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the full Data as a string.

Function

Synopsis

```
data = Data:getString( offset, size )
```

Arguments

Available since LÖVE 12.0

`number offset (0)`
Optional byte offset into the Data's memory to copy from.

`number size (Data:getSize())`
Optionally limit the copied string to the specified number of bytes.

Returns

`string data`
The raw data.

See Also

- Data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Debug

Debug · revision 19056 · last edited 2021-03-14

From the Lua Reference Manual (http://w w w.lua.org/manual/5.1/manual.html#pdf-debug.debug):

> Enters an interactive mode with the user, running each string that the user enters. Using simple commands and other debug facilities, the user can inspect global and local variables, change their values, evaluate expressions, and so on. A line containing only the word `cont` finishes this function, so that the caller continues its execution. Note that commands for `debug.debug` are not lexically nested within any function, and so have no direct access to local variables.

```
function love.keypressed(key, u)
   --Debug
   if key == "rctrl" then --set to whatever key you want to use
      debug.debug()
   end
end
```

Caveat: The debug mode can only be activated if there is a terminal bound to LÖVE. On Windows, launching LÖVE from the command prompt is not sufficient. Instead, run LÖVE in console mode with the command line argument "`--console`" or the configuration file option "`t.console = true`".

Decoder

Decoder · revision 29467 · last edited 2025-09-29

On this page

- Constructors
- Functions
- Supertypes
- Examples
  - Streaming music to a queueable Source
- See Also
- Other Languages

An object which can gradually decode a sound file.

Constructors

love.sound.newDecoder	Attempts to find a decoder for the encoded sound data in the specified file.

Functions

Decoder:clone	Create new copy of existing decoder.	Added since 11.3	
Decoder:decode	Decodes a chunk of audio data to a SoundData.	Added since 11.0	
Decoder:getBitDepth	Returns the number of bits per sample.	Added since 0.9.0	
Decoder:getBits	Returns the number of bits per sample.		Removed in 0.9.0
Decoder:getChannelCount	Returns the number of channels in the stream.	Added since 11.0	
Decoder:getChannels	Returns the number of channels in the stream.		Deprecated in 11.0
Decoder:getDuration	Gets the duration of the sound file.	Added since 0.10.0	
Decoder:getSampleRate	Returns the sample rate of the Decoder.		
Decoder:seek	Sets the currently playing position of the Decoder.	Added since 11.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		

Supertypes

- Object

Examples

Streaming music to a queueable Source

```
local decoder = love.sound.newDecoder("music.ogg")
local queueableSource = love.audio.newQueueableSource(decoder:getSampleRate(), decoder:getBitDepth(), decoder:getChannelCount())

local function buffer(loop)
  local freeBufferCount = queueableSource:getFreeBufferCount()
  for i = 1, freeBufferCount do
    local soundData = decoder:decode()
    if not soundData then
      -- the decoder reached the end of the file!
      if not loop then
        return
      end

      -- Reset the decoder to the start and decode from the start
      decoder:seek(0)
      soundData = decoder:decode()
    end
    queueableSource:queue(soundData)
  end
end

function love.load()
  -- when the queueable Source has no buffers left to process it stops playing
  -- buffer some data before starting to play!
  buffer(true)
```

```
  queueableSource:play()
end

function love.update(dt)
  -- keep buffering so it doesn't stop
  buffer(true)
end
```

See Also

- love.sound

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:clone

Decoder:clone · revision 23487 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Creates a new copy of current decoder.

The new decoder will start decoding from the beginning of the audio stream.

Function

Synopsis

```
decoder = Decoder:clone( )
```

Arguments

None.

Returns

`Decoder decoder`

New copy of the decoder.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:decode

Decoder:decode · revision 29465 · last edited 2025-02-21

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Decodes the audio and returns a SoundData object containing the decoded audio data.

Function

Synopsis

```
soundData = Decoder:decode( )
```

Arguments

None.

Returns

`SoundData soundData (nil)`

Decoded audio data or nil if the decoder reached the end of the file.

Notes

Tracker modules may internally use loop points and jump markers in a way that a decoder never reaches the end of the file, causing decode to never return nil.

See Also

- Decoder
- SoundData
- Source:queue

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:getBitDepth

Decoder:getBitDepth · revision 11039 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been renamed from Decoder:getBits.

Returns the number of bits per sample.

Function

Synopsis

```
bitDepth = Decoder:getBitDepth( )
```

Arguments

None.

Returns

`number bitDepth`

Either 8, or 16.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Decoder:getBits

Decoder:getBits · revision 11038 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0

It has been renamed to Decoder:getBitDepth.

Returns the number of bits per sample.

Function

Synopsis

```
bitSize = Decoder:getBits( )
```

Arguments

None.

Returns

`number bitSize`

Either 8, or 16.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:getChannelCount

Decoder:getChannelCount · revision 21388 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

It has been renamed from Decoder:getChannels.

Returns the number of channels in the stream.

Function

Synopsis

```
channels = Decoder:getChannelCount( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Decoder:getChannels

Decoder:getChannels · revision 22108 · last edited 2024-03-10

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Deprecated in LÖVE 11.0

This function has been renamed to Decoder:getChannelCount.

Returns the number of channels in the stream.

Function

Synopsis

```
channels = Decoder:getChannels( )
```

Arguments

None.

Returns

`number channels`

1 for mono, 2 for stereo.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:getDuration

Decoder:getDuration · revision 15500 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Gets the duration of the sound file. It may not always be sample-accurate, and it may return -1 if the duration cannot be determined at all.

Function

Synopsis

```
duration = Decoder:getDuration( )
```

Arguments

None.

Returns

`number duration`

The duration of the sound file in seconds, or -1 if it cannot be determined.

See Also

- Decoder
- Source:getDuration
- SoundData:getDuration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:getSampleRate

Decoder:getSampleRate · revision 6182 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the sample rate of the Decoder.

Function

Synopsis

```
rate = Decoder:getSampleRate( )
```

Arguments

None.

Returns

`number rate`
: Number of samples per second.

See Also

- Decoder

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Decoder:seek

Decoder:seek · revision 23659 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Sets the currently playing position of the Decoder.

Function

Synopsis

```
Decoder:seek( offset )
```

Arguments

`number offset`
: The position to seek to, in seconds.

Returns

Nothing.

See Also

- Decoder
- Source:seek

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DeviceType

DeviceType · revision 30605 · last edited 2026-04-07

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Types of touch devices

Constants

touchscreen
: Touch screen with window-relative coordinates.

touchpad
: Touchpad or trackpad with absolute device coordinates.

touchpadrelative
: Touchpad or trackpad with screen cursor-relative coordinates.

See Also

- love.touch
- love.touch.getDeviceType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DisplayOrientation

DisplayOrientation · revision 22060 · last edited 2023-07-05

Available since LÖVE 11.3

This enum is not supported in earlier versions.

Types of device display orientation.

Constants

unknown
: Orientation cannot be determined.

landscape
: Landscape orientation.

landscapeflipped
: Landscape orientation (flipped).

portrait
: Portrait orientation.

portraitflipped
: Portrait orientation (flipped).

See Also

- love.window
- love.window.getDisplayOrientation
- love.displayrotated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceBasedCollision

DistanceBasedCollision · revision 29085 · last edited 2024-01-16

Collision between two circles

Another way of detecting collisions. This type works perfectly with circles. It checks if the distance of one object to the other is less than the sum of their radii (plural of radius.)

Variable Definitions : x1, y1, r1 = first circle's center coordinates and radius; x2, y2, r2 = second circle's center coordinates and radius.

```
function checkCircularCollision(x1, y1, r1, x2, y2, r2)
    local dx = x2 - x1
    local dy = y2 - y1
    local dist = math.sqrt(dx * dx + dy * dy)
    return dist < r1 + r2
end
```

Don't want an extra square root function? Try this other function

```
function checkCircularCollision(x1, y1, r1, x2, y2, r2)
    local dx, dy, sr = x2 - x1, y2 - y1, r1 + r2
    return dx*dx + dy*dy < sr*sr
end
```

Although I do keep the sqrt function to know that I am dealing with distances.

Do note that while the second may seem faster since it avoids a costly square root, that's still one function, instead of the second version's three squarings. That said, Löve uses LuaJIT, so if one really needs to optimize, check which is faster.

Collision between circle and rectangle

```
function checkCircleToRectangleCollision(cx, cy, cr, rectX, rectY, rectW, rectH)
    local nearestX = math.max(rectX, math.min(cx, rectX + rectW))
    local nearestY = math.max(rectY, math.min(cy, rectY + rectH))
    local dx, dy = math.abs(cx - nearestX), math.abs(cy - nearestY)
    if dx > cr or dy > cr then return false end
    return dx*dx + dy*dy < cr*cr
end
```

Collision between circle and polygon

```
function checkCircleToPolygonCollision (cx, cy, cr, poly)
    local function nearestPolygonPoint(cx, cy, x1, y1, dx, dy)
        local d = dx * dx + dy * dy
        if d == 0 then return x1, y1 end -- Length was 0
        local t = ((cx - x1) * dx + (cy - y1) * dy) / d
        t = math.max(0, math.min(1, t))
        return x1 + t * dx, y1 + t * dy
    end

    local x1, y1 = poly[#poly - 1], poly[#poly]
    local x2, y2 = poly[1], poly[2]
    for i = 1, #poly-1, 2 do
        local nearestX, nearestY = nearestPolygonPoint(cx, cy, x1, y1, x2-x1, y2-y1)
        local dx, dy = cx - nearestX, cy - nearestY
        if dx*dx + dy*dy < cr*cr then
            return true  -- Collision detected
        end
        x1, y1, x2, y2 = x2, y2, poly[i+2], poly[i+3]
    end
    return false
end
```

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint

DistanceJoint · revision 16449 · last edited 2023-07-05

Keeps two bodies at the same distance.



Constructors

love.physics.newDistanceJoint	Creates a DistanceJoint between two bodies.

Functions

DistanceJoint:getDamping	Gets the damping ratio.		Removed in 0.8.0
DistanceJoint:getDampingRatio	Gets the damping ratio.	Added since 0.8.0	
DistanceJoint:getFrequency	Gets the response speed.		
DistanceJoint:getLength	Gets the equilibrium distance between the two Bodies.		
DistanceJoint:setDamping	Sets the damping ratio.		Removed in 0.8.0
DistanceJoint:setDampingRatio	Sets the damping ratio.	Added since 0.8.0	
DistanceJoint:setFrequency	Sets the response speed.		
DistanceJoint:setLength	Sets the equilibrium distance between the two Bodies.		
Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:getDamping

DistanceJoint:getDamping · revision 10552 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
It has been replaced with DistanceJoint:getDampingRatio.

Gets the damping ratio.

Function

Synopsis

```
ratio = DistanceJoint:getDamping( )
```

Arguments

None.

Returns

`number ratio`
The damping ratio.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:getDampingRatio

DistanceJoint:getDampingRatio · revision 10551 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

It has been renamed from DistanceJoint:getDamping.

Gets the damping ratio.

Function

Synopsis

```
ratio = DistanceJoint:getDampingRatio( )
```

Arguments

None.

Returns

`number ratio`

The damping ratio.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

DistanceJoint:getFrequency

DistanceJoint:getFrequency · revision 6068 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the response speed.

Function

Synopsis

```
Hz = DistanceJoint:getFrequency( )
```

Arguments

None.

Returns

number Hz
: The response speed.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:getLength

DistanceJoint:getLength · revision 6069 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the equilibrium distance between the two Bodies.

Function

Synopsis

```
l = DistanceJoint:getLength( )
```

Arguments

None.

Returns

number l
: The length between the two Bodies.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:setDamping

DistanceJoint:setDamping · revision 10553 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
It has been replaced with DistanceJoint:setDampingRatio.

Sets the damping ratio.

Function

Synopsis

```
DistanceJoint:setDamping( ratio )
```

Arguments

`number ratio`
The damping ratio.

Returns

Nothing.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:setDampingRatio

DistanceJoint:setDampingRatio · revision 24818 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

It has been renamed from DistanceJoint:setDamping.

Sets the damping ratio.

Function

Synopsis

```
DistanceJoint:setDampingRatio( ratio )
```

Arguments

`number ratio`
: The damping ratio.

Returns

Nothing.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:setFrequency

DistanceJoint:setFrequency · revision 6071 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the response speed.

Function

Synopsis

```
DistanceJoint:setFrequency( Hz )
```

Arguments

`number Hz`
: The response speed.

Returns

Nothing.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceJoint:setLength

DistanceJoint:setLength · revision 6072 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the equilibrium distance between the two Bodies.

Function

Synopsis

```
DistanceJoint:setLength( l )
```

Arguments

number l
: The length between the two Bodies.

Returns

Nothing.

See Also

- DistanceJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DistanceModel

DistanceModel · revision 16476 · last edited 2026-05-17

Available since LÖVE 0.8.0
This enum is not supported in earlier versions.

The different distance models.

Extended information can be found in the chapter "3.4. Attenuation By Distance" of the OpenAL 1.1 specification (https://www.openal.org/documentation/openal-1.1-specification.pdf).

Constants

none
: Sources do not get attenuated.

inverse
: Inverse distance attenuation.

inverseclamped
: Inverse distance attenuation. Gain is clamped. In version 0.9.2 and older this is named **inverse clamped**.

linear
: Linear attenuation.

linearclamped
: Linear attenuation. Gain is clamped. In version 0.9.2 and older this is named **linear clamped**.

exponent
: Exponential attenuation.

exponentclamped
: Exponential attenuation. Gain is clamped. In version 0.9.2 and older this is named **exponent clamped**.

See Also

- love.audio
- love.audio.setDistanceModel
- love.audio.getDistanceModel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Donut

Donut · revision 19219 · last edited 2023-07-05

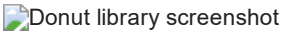
On this page

- Debug message control system? w th???
- Supprted languages
- Usage (Lua)
- Usage (MoonScript)
- Documentation
- License
- Dow nload
- Sourcecode
- See Also

Donut is a simplified debug message control system for LÖVE.

Debug message control system? wth???

You're able to add debug messages like this:


Donut library screenshot

Supprted languages

- Lua
- Moonscript

Usage (Lua)

```lua
require("donut")

function love.load()
    debug = Donut.init(10, 10)
    fps = debug.add("FPS")
    random = debug.add("Random")
end

function love.update(dt)
    debug.update(fps, love.timer.getFPS())
    debug.update(random, math.random(0, 100))
end

function love.keypressed(key, unicode)
    if key == "s" then -- show/hide with "s"
        debug.toggle()
    end
end

function love.draw()
    debug.draw()
    -- you could also use debug.draw(xoffset, yoffset) to move the message if you're using a cam
end
```

Usage (MoonScript)

```moonscript
require "donut"

love.load = ->
    export debug = Donut(10, 10)
    export fps = debug\add "FPS"
    export random = debug\add "Random"

love.update = (dt) ->
    debug\update fps, love.timer.getFPS()
    debug\update random, math.random(0, 100)
    ""

love.keypressed = (key, unicode) ->
    debug\toggle if key == "s"
```

```
    ""

love.draw = ->
    debug\draw 0, 0
```

Documentation

coming soon...

License

Donut is licensed under the terms of GNU Lesser General Public License v3 (http://w w w.gnu.org/copyleft/lgpl.html).

Download

- Download from my master-branch (https://github.com/Kasoki/Donut/zipball/master)
- Download v0.1.4 (https://github.com/Kasoki/Donut/zipball/v0.1.4)
- Download v0.1.3 (https://github.com/Kasoki/Donut/zipball/v0.1.3)
- Download v0.1.2 (https://github.com/Kasoki/Donut/zipball/v0.1.2)
- Download v0.1.1 (https://github.com/Kasoki/Donut/zipball/v0.1.1)
- Download v0.1 (https://github.com/Kasoki/Donut/zipball/v0.1)

Sourcecode

- Donut on Github (http://github.com/Kasoki/Donut)

See Also

- Donut post in LÖVE forum (https://love2d.org/forums/viewtopic.php?f=5&t=8103)

Dox

Dox · revision 16876 · last edited 2023-07-05

Dox

A Lua Documentation Generator Script

What it Does

Dox auto-generates HTML documentation of your code.

Features

- Multi-platform: dox works with all major versions of Windows and Linux.
- Dox is very easy to use.
- Dox is easy to modify (if desired).
- No external dependencies or installations.
- Executes in as little as one line of code.

Download Dox and view help documentation at Github

https://github.com/CentauriSoldier/Dox ↗

View the Discussion Thread

https://love2d.org/forums/viewtopic.php?f=5&t=82685

Drawable

Drawable · revision 10226 · last edited 2023-07-05

Superclass for all things that can be drawn on screen. This is an abstract type that can't be created directly.

Functions

Object:release	Immediately destroys the object's Lua reference. Added since 11.0
Object:type	Gets the type of the object as a string.
Object:typeOf	Checks whether an object is of a certain type.

Supertypes

- Object

Subtypes

Canvas	Off-screen render target.	Added since 0.8.0
Framebuffer	Off-screen render target.	Added since 0.7.0 Removed in 0.8.0
Image	Drawable image type.	
Mesh	A2D polygon mesh used for drawing arbitrary textured shapes.	Added since 0.9.0
ParticleSystem	Used to create cool effects, like fire.	
SpriteBatch	Store image positions in a buffer, and draw it in one call.	
Text	Drawable text.	Added since 0.10.0
Texture	Superclass for drawable objects which represent a texture.	Added since 0.9.1
Video	Adrawable video.	Added since 0.10.0

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DrawMode

DrawMode · revision 22384 · last edited 2023-07-05

Controls whether shapes are drawn as an outline, or filled.

Constants

fill
: Draw filled shape.

line
: Draw outlined shape.

See Also

- love.graphics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

DroppedFile

DroppedFile · revision 21988 · last edited 2026-04-06

Available since LÖVE 0.10.0

This type is not supported in earlier versions.

Represents a file dropped onto the window.

Note that the DroppedFile type can only be obtained from love.filedropped callback, and can't be constructed manually by the user.

Functions

These functions have parentheses in odd places. This is because the *File:* namespace is reserved in Mediawiki.

(File):close	Closes a File.	
(File):eof	If the end-of-file has been reached	Removed in 0.10.0
(File):flush	Flushes any buffered written data in the file to disk.	Added since 0.9.0
(File):getBuffer	Gets the buffer mode of a file.	Added since 0.9.0
(File):getFilename	Gets the filename that the File object was created with.	Added since 0.10.0
(File):getMode	Gets the FileMode the file has been opened with.	Added since 0.9.0
(File):getSize	Returns the file size.	
(File):isEOF	Gets whether end-of-file has been reached.	Added since 0.10.0
(File):isOpen	Gets whether the file is open.	Added since 0.9.0
(File):lines	Iterate over all the lines in a file.	
(File):open	Open the file for write, read or append.	
(File):read	Read a number of bytes from a file	
(File):seek	Seek to a position in a file	
(File):setBuffer	Sets the buffer mode for a file opened for writing or appending.	Added since 0.9.0
(File):tell	Returns the position in the file.	
(File):write	Write data to a file.	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- File
- Object

See Also

- love.filesystem
- love.filedropped

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape

EdgeShape · revision 10227 · last edited 2023-07-05

Available since LÖVE 0.8.0

This shape is not supported in earlier versions.

A EdgeShape is a line segment. They can be used to create the boundaries of your terrain. The shape does not have volume and can only collide with PolygonShape and CircleShape.

Constructors

love.physics.newEdgeShape	Creates a new EdgeShape.	Added since 0.8.0

Functions

EdgeShape:getNextVertex	Gets the vertex that establishes a connection to the next shape.	Added since 0.10.2	
EdgeShape:getPoints	Returns the local coordinates of the edge points.	Added since 0.8.0	
EdgeShape:getPreviousVertex	Gets the vertex that establishes a connection to the previous shape.	Added since 0.10.2	
EdgeShape:setNextVertex	Sets a vertex that establishes a connection to the next shape.	Added since 0.10.2	
EdgeShape:setPreviousVertex	Sets a vertex that establishes a connection to the previous shape.	Added since 0.10.2	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Shape:computeAABB	Returns the points of the bounding box for the transformed shape.	Added since 0.8.0	
Shape:computeMass	Computes the mass properties for the shape.	Added since 0.8.0	
Shape:destroy	Explicitly destroys the Shape.		Removed in 0.8.0
Shape:getBody	Get the body the shape is attached to.	Added since 0.7.0	Removed in 0.8.0
Shape:getBoundingBox	Gets the bounding box of the shape.		Removed in 0.8.0
Shape:getCategory	Gets the categories this shape is a member of.		Removed in 0.8.0
Shape:getCategoryBits	Gets the categories as a 16-bit integer.		Removed in 0.8.0
Shape:getChildCount	Returns the number of children the shape has.	Added since 0.8.0	
Shape:getData	Get the data set with setData.		Removed in 0.8.0
Shape:getDensity	Gets the density of the Shape.		Removed in 0.8.0
Shape:getFilterData	Gets the filter data of the Shape.		Removed in 0.8.0
Shape:getFriction	Gets the friction of this shape.		Removed in 0.8.0
Shape:getMask	Gets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:getRadius	Gets the radius of the shape.		
Shape:getRestitution	Gets the restitution of this shape.		Removed in 0.8.0
Shape:getType	Gets a string representing the Shape.		
Shape:isSensor	Checks whether a Shape is a sensor or not.		Removed in 0.8.0
Shape:rayCast	Casts a ray against the shape.	Added since 0.8.0	
Shape:setCategory	Sets the categories this shape is a member of.		Removed in 0.8.0
Shape:setData	Set data to be passed to the collision callback.		Removed in 0.8.0
Shape:setDensity	Sets the density of a Shape.		Removed in 0.8.0
Shape:setFilterData	Sets the filter data for a Shape.		Removed in 0.8.0
Shape:setFriction	Sets the friction of the shape.		Removed in 0.8.0
Shape:setMask	Sets which categories this shape should **NOT** collide with.		Removed in 0.8.0
Shape:setRestitution	Sets the restitution of the shape.		Removed in 0.8.0
Shape:setSensor	Sets whether this shape should act as a sensor.		Removed in 0.8.0
Shape:testPoint	Checks whether a point lies inside the shape.		
Shape:testSegment	Checks whether a line segment intersects a shape.		Removed in 0.8.0

Supertypes

- Shape

- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape:getNextVertex

EdgeShape:getNextVertex · revision 18111 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the vertex that establishes a connection to the next shape.

Setting next and previous EdgeShape vertices can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
x, y = EdgeShape:getNextVertex( )
```

Arguments

None.

Returns

`number x (nil)`
: The x-component of the vertex, or nil if EdgeShape:setNextVertex hasn't been called.

`number y (nil)`
: The y-component of the vertex, or nil if EdgeShape:setNextVertex hasn't been called.

See Also

- EdgeShape
- EdgeShape:setNextVertex
- EdgeShape:getPreviousVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape:getPoints

EdgeShape:getPoints · revision 8763 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the local coordinates of the edge points.

Function

Synopsis

```
x1, y1, x2, y2 = EdgeShape:getPoints( )
```

Arguments

None.

Returns

number x1
: The x-component of the first vertex.

number y1
: The y-component of the first vertex.

number x2
: The x-component of the second vertex.

number y2
: The y-component of the second vertex.

See Also

- EdgeShape

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape:getPreviousVertex

EdgeShape:getPreviousVertex · revision 18112 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Gets the vertex that establishes a connection to the previous shape.

Setting next and previous EdgeShape vertices can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
x, y = EdgeShape:getPreviousVertex( )
```

Arguments

None.

Returns

number x (nil)
: The x-component of the vertex, or nil if EdgeShape:setPreviousVertex hasn't been called.

number y (nil)
: The y-component of the vertex, or nil if EdgeShape:setPreviousVertex hasn't been called.

See Also

- EdgeShape
- EdgeShape:setPreviousVertex
- EdgeShape:getNextVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape:setNextVertex

EdgeShape:setNextVertex · revision 18106 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Sets a vertex that establishes a connection to the next shape.

This can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
EdgeShape:setNextVertex( x, y )
```

Arguments

number x
: The x-component of the vertex.

number y
: The y-component of the vertex.

Returns

Nothing.

See Also

- EdgeShape
- EdgeShape:getNextVertex
- EdgeShape:setPreviousVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EdgeShape:setPreviousVertex

EdgeShape:setPreviousVertex · revision 18107 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.10.2

This function is not supported in earlier versions.

Sets a vertex that establishes a connection to the previous shape.

This can help prevent unwanted collisions when a flat shape slides along the edge and moves over to the new shape.

Function

Synopsis

```
EdgeShape:setPreviousVertex( x, y )
```

Arguments

number `x`
: The x-component of the vertex.

number `y`
: The y-component of the vertex.

Returns

Nothing.

See Also

- EdgeShape
- EdgeShape:getPreviousVertex
- EdgeShape:setNextVertex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EffectType

EffectType · revision 30500 · last edited 2026-05-17

On this page

- Constants
- Parameters
  - chorus
  - compressor
  - distortion
  - echo
  - equalizer
  - flanger
  - reverb
  - ringmodulator
- See Also
- Other Languages

Available since LÖVE 11.0
This enum is not supported in earlier versions.

The different types of effects supported by love.audio.setEffect.

Constants

chorus
: Plays multiple copies of the sound with slight pitch and time variation. Used to make sounds sound "fuller" or "thicker".

compressor
: Decreases the dynamic range of the sound, making the loud and quiet parts closer in volume, producing a more uniform amplitude throughout time.

distortion
: Alters the sound by amplifying it until it clips, shearing off parts of the signal, leading to a compressed and distorted sound.

echo
: Decaying feedback based effect, on the order of seconds. Also known as delay; causes the sound to repeat at regular intervals at a decreasing volume.

equalizer
: Adjust the frequency components of the sound using a 4-band (low-shelf, two band-pass and a high-shelf) equalizer.

flanger
: Plays two copies of the sound; while varying the phase, or equivalently delaying one of them, by amounts on the order of milliseconds, resulting in phasing sounds.

reverb
: Decaying feedback based effect, on the order of milliseconds. Used to simulate the reflection off of the surroundings.

ringmodulator
: An implementation of amplitude modulation; multiplies the source signal with a simple waveform, to produce either volume changes, or inharmonic overtones.

Parameters

Full documentation for the underlying API (OpenAL) can be found in this PDF file (https://nrgcore.com/docs/manual/en-us/effects_extension_guide.pdf).

chorus

Parameter	Units	Range	Default
`EffectWaveform waveform`			"triangle"
`number phase`	Degrees	[-180, 180]	90
`number rate`	Hz	[0, 10]	1.1
`number depth`		[0, 1]	0.1
`number feedback`		[-1, 1]	0.25
`number delay`	Seconds	[0, 0.016]	0.016

compressor

Parameter	Units	Range	Default
`boolean enable`			true

distortion

Parameter	Units	Range	Default
`number gain`		[0.01, 1]	0.2
`number edge`		[0, 1]	0.2
`number lowcut`	Hz	[80, 24000]	8000

`number center`	Hz	[80, 24000]	3600
`number bandwidth`	Hz	[80, 24000]	3600

echo

Parameter	Units	Range	Default
`number delay`	Seconds	[0, 0.207]	0.1
`number tapdelay`	Seconds	[0, 0.404]	0.1
`number damping`		[0, 0.99]	0.5
`number feedback`		[0, 1]	0.5
`number spread`		[-1, 1]	-1

equalizer

Parameter	Units	Range	Default
`number lowgain`		[0.126, 7.943]	1
`number lowcut`	Hz	[50, 800]	200
`number lowmidgain`		[0.126, 7.943]	1
`number lowmidfrequency`	Hz	[200, 3000]	500
`number lowmidbandwidth`		[0.01, 1]	1
`number highmidgain`		[0.126, 7.943]	1
`number highmidfrequency`	Hz	[1000, 8000]	3000
`number highmidbandwidth`		[0.01, 1]	1
`number highgain`		[0.126, 7.943]	1
`number highcut`	Hz	[4000, 16000]	6000

flanger

Parameter	Units	Range	Default
`EffectWaveform waveform`			"triangle"
`number phase`	Degrees	[-180, 180]	0
`number rate`	Hz	[0, 10]	0.27
`number depth`		[0, 1]	1
`number feedback`		[-1, 1]	-.5
`number delay`	Seconds	[0, 0.004]	0.002

reverb

Parameter	Units	Range	Default
`number gain`		[0, 1]	0.32
`number highgain`		[0, 1]	0.89
`number density`		[0, 1]	1
`number diffusion`		[0, 1]	1
`number decaytime`	Seconds	[0.1, 20]	1.49

`number decayhighratio`		[0.1, 2]	0.83
`number earlygain`		[0, 3.16]	0.05
`number earlydelay`	Seconds	[0, 0.3]	0.05
`number lategain`		[0, 10]	1.26
`number latedelay`	Seconds	[0, 0.1]	0.011
`number roomrolloff`		[0, 10]	0
`number airabsorption`		[0.892, 1]	0.994
`boolean highlimit`			true

ringmodulator

Parameter	Units	Range	Default
`EffectWaveform waveform`			"sine"
`number frequency`	Hz	[0, 8000]	440
`number highcut`	Hz	[0, 24000]	800

See Also

- love.audio
- love.audio.setEffect

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

EffectWaveform

EffectWaveform · revision 21923 · last edited 2026-05-17

Available since LÖVE 11.0
This enum is not supported in earlier versions.

The different types of waveforms that can be used with the **ringmodulator** EffectType.

Constants

sawtooth
: A sawtooth wave, also known as a ramp wave. Named for its linear rise, and (near-)instantaneous fall along time.

sine
: A sine wave. Follows a trigonometric sine function.

square
: A square wave. Switches between high and low states (near-)instantaneously.

triangle
: A triangle wave. Follows a linear rise and fall that repeats periodically.

See Also

- love.audio
- EffectType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ELScheduler

ELScheduler · revision 25131 · last edited 2023-07-05

Embeddable Lua Scheduler is a pure Lua library to manage timers.

- Project and documentation: https://github.com/ImagicTheCat/ELScheduler ↗
- Forum topic (https://love2d.org/forums/viewtopic.php?f=5&t=88399)

EncodeFormat

EncodeFormat · revision 22618 · last edited 2023-07-05

Available since LÖVE 11.0

This enum is not supported in earlier versions.

Encoding format used to encode or decode data.

Constants

base64

Encode/decode data as base64 binary-to-text encoding.

hex

Encode/decode data as hexadecimal string.

See Also

- love.data
- love.data.encode
- love.data.decode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.event

enet.event · revision 27597 · last edited 2023-07-05

Description

An event is a table generated by host:service() or peer:receive() which will **always** contain a string named *type*, a enet.peer named *peer*, and a string or number named *data* depending on the kind of event. Receive has an additional field named *channel* which contains the channel the data was received on specified by enet.peer:send, by default this is 0.

Though be wary that host:service() and peer:receive() can return nil if no events are in the queue.

Structure

event.type	event.peer	event.data	event.channel
"receive"	peer	string	number
"disconnect"	peer	number	
"connect"	peer	number	

See Also

- lua-enet
- enet.host:service
- enet.peer:receive

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host

enet.host · revision 28451 · last edited 2023-07-05

Description

An ENet host for communicating with peers. On creation it will bind to a port on an address, unless otherwise specified, which will keep other applications from binding to the same port and address.

One can free the port by calling its destroy method; nil-ing the host object and calling collectgarbage() would work as well, since :destroy calls host:__gc internally, but this is cleaner:

```
local host = enet.host_create("*:6789")
host:destroy()
```

Functions

Function	Description
host:service	Wait for events, send and receive any ready packets.
host:check_events	Checks for any queued events and dispatches one if available.
host:compress_with_range_coder	Toggles an adaptive order-2 PPM range coder for the transmitted data of all peers.
host:connect	Connects a host to a remote host. Returns peer object associated with remote host.
host:flush	Sends any queued packets.
host:broadcast	Queues a packet to be sent to all connected peers.
host:channel_limit	Sets the maximum number of channels allowed.
host:bandwidth_limit	Sets the bandwidth limits of the host in bytes/sec.
host:get_socket_address	Returns a string that describes the socket address of the given host.
host:socket_get_address	Deprecated version of host:get_socket_address.
host:destroy	Destroys the host, freeing any bound ports and addresses. Alias for the host:__gc method.
host:total_sent_data	Returns the number of bytes that were sent through the given host.
host:total_received_data	Returns the number of bytes that were received by the given host.
host:service_time	Returns the timestamp of the last call to host:service() or host:flush().
host:peer_count	Returns the number of peers that are allocated for the given host.
host:get_peer	Returns the connected peer at the specified index (starting at 1).
host:__gc	Destroys the host, freeing any bound ports and addresses.

See Also

- lua-enet
- enet:host_create
- enet.event
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host create

enet.host create · revision 22695 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Returns a new host. All arguments are optional.

A bind_address of nil makes a host that can not be connected to (typically a client). Otherwise the address can either be of the form <ipaddress>:<port>, <hostname>:<port>, or *:<port>.

Example addresses include "127.0.0.1:8888", "localhost:2232", and "*:6767". If port is 0, the system automatically chooses an ephemeral port and you can get port number by host:get_socket_address().

Function

Synopsis

```
host = enet.host_create(bind_address, peer_count, channel_count, in_bandwidth, out_bandwidth)
```

Arguments

`string bind_address`
: The address to connect to in the format "ip:port".

`number peer_count`
: The max number of peers. Defaults to 64.

`number channel_count`
: The max number of channels. Defaults to 1.

`number in_bandwidth`
: Downstream bandwidth in bytes/sec. Defaults to 0 (unlimited).

`number out_bandwidth`
: Upstream bandwidth in bytes/sec. Defaults to 0 (unlimited).

Returns

`enet.host host`
: The requested host.

See Also

- lua-enet
- enet.host
- enet.host:channel_limit
- enet.host:bandwidth_limit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:bandwidth limit

enet.host:bandwidth limit · revision 18065 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the bandwidth limits of the host in bytes/sec. Set to 0 for unlimited.

Function

Synopsis

```
host:bandwidth_limit(incoming, outgoing)
```

Arguments

`number incoming`
: Download speed limit in bytes/sec.

`number outgoing`
: Upload speed limit in bytes/sec.

Returns

Nothing.

See Also

- lua-enet
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:broadcast

enet.host:broadcast · revision 28332 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Queues a packet to be sent to all connected peers.

Function

Synopsis

```
host:broadcast(data, channel, flag)
```

Arguments

`string data`
: The data to send.

`number channel`
: The channel to send the packet on. Defaults to 0.

`string flag`
: flag is one of "reliable", "unsequenced", or "unreliable". Reliable packets are guaranteed to arrive, and arrive in the order in which they are sent. Unreliable packets arrive in the order in which they are sent, but they aren't guaranteed to arrive. Unsequenced packets are neither guaranteed to arrive, nor do they have any guarantee on the order they arrive. Optional. Defaults to reliable.

Returns

Nothing.

Function

Available since LÖVE 12.0

This variant is not supported in earlier versions.

Synopsis

```
host:send(pointer, size, channel, flag)
```

Arguments

`light userdata pointer`
: The data of the packet to send to all clients.

`number size`
: The size of the memory starting at the pointer.

`number channel`
: The channel to send the packet on. Optional. Defaults to 0.

`string flag`

flag is one of "reliable", "unsequenced", or "unreliable". Reliable packets are guaranteed to arrive, and arrive in the order in which they are sent. Unreliable packets arrive in the order in which they are sent, but they aren't guaranteed to arrive. Unsequenced packets are neither guaranteed to arrive, nor do they have any guarantee on the order they arrive. Optional. Defaults to reliable.

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:channel limit

enet.host:channel limit · revision 18064 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the maximum number of channels allowed. If it is 0 then the system maximum allowable value is used.

Function

Synopsis

```
host:channel_limit(limit)
```

Arguments

`number limit`
: The maximum number of channels allowed.

Returns

Nothing.

See Also

- lua-enet
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:check events

enet.host:check events · revision 18059 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Checks for any queued events and dispatches one if available. Returns the associated event if something was dispatched, otherwise nil.

Function

Synopsis

```
event = host:check_events()
```

Arguments

None.

Returns

`table event`
: An event or nil if no events are available.

See Also

- lua-enet
- enet.event
- enet.host:service

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:compress with range coder

enet.host:compress with range coder · revision 18132 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Toggles an adaptive order-2 PPM (https://en.w ikipedia.org/w iki/Prediction_by_partial_matching) range coder (https://en.w ikipedia.org/w iki/Range_encoding) for the transmitted data of all peers.

Function

Synopsis

```
host:compress_with_range_coder()
```

Arguments

None.

Returns

`boolean state`
: True if the compression algorithm is active, false if not.

See Also

- lua-enet
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:connect

enet.host:connect · revision 14133 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Connects a host to a remote host. Returns peer object associated with the remote host. The actual connection will not take place until the next host:service() is called, in which a "connect" event will be generated.

Function

Synopsis

```
peer = host:connect(address, channel_count, data)
```

Arguments

`string address`
: The address to connect to in the format "ip:port".

`number channel_count`
: The number of channels to allocate. It should be the same as the channel count on the server. Defaults to 1.

`number data`
: An integer value that can be associated with the connect event. Defaults to 0.

Returns

`enet.peer peer`
: A peer.

See Also

- lua-enet
- enet.event
- enet.peer
- enet.host:channel_limit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:destroy

enet.host:destroy · revision 18068 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Destroys the host structure and closes all of its connections. This function is also ran automatically by lua's garbage collector, since it's an alias to *host:__gc*.

Function

Synopsis

```
host:destroy()
```

Arguments

None.

Returns

Nothing.

See Also

- lua-enet
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:flush

enet.host:flush · revision 18119 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Sends any queued packets. This is only required to send packets earlier than the next call to host:service, or if host:service will not be called again.

Function

Synopsis

```
host:flush()
```

Arguments

None.

Returns

Nothing.

See Also

- lua-enet
- enet.host:service
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:get peer

enet.host:get peer · revision 14098 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the connected peer at the specified index (starting at 1). ENet stores all peers in an array of the corresponding host and re-uses unused peers for new connections. You can query the state of a peer using peer:state().

Function

Synopsis

```
host:get_peer(index)
```

Arguments

`number index`
: The index of the desired peer.

Returns

`enet.peer peer`
: The desired peer structure.

See Also

- lua-enet
- enet.host
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:get socket address

enet.host:get socket address · revision 18067 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns a string that describes the socket address of the given host. The string is formatted as “a.b.c.d:port”, where “a.b.c.d” is the IP address of the used socket.

Function

Synopsis

```
host:get_socket_address()
```

Arguments

None.

Returns

`string address`
: A string that describes the socket address.

See Also

- lua-enet
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:peer count

enet.host:peer count · revision 18073 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the number of peers that are allocated for the given host. This represents the maximum number of possible connections.

Function

Synopsis

```
host:peer_count()
```

Arguments

None.

Returns

`number limit`
: The maximum number of peers allowed.

See Also

- lua-enet
- enet.host
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:service

enet.host:service · revision 13660 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Wait for events, send and receive any ready packets. If an event is in the queue it will be returned and dequeued. Generally you will want to dequeue all waiting events every frame.

Function

Synopsis

```
event = host:service( timeout )
```

Arguments

`number timeout`
: The max number of milliseconds to be waited for an event. Default is 0.

Returns

`table event`
: An event or nil if no events occured.

See Also

- lua-enet
- enet.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:service time

enet.host:service time · revision 18072 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns the time-stamp of the last call to host:service() or host:flush(). The time-stamp is in milliseconds of the current time of day.

Function

Synopsis

```
host:channel_limit(limit)
```

Arguments

None.

Returns

`number timestamp`
: A time-stamp in milliseconds.

See Also

- lua-enet
- enet.host
- enet.host:service
- enet.host:flush

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:total received data

enet.host:total received data · revision 18071 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the number of bytes that were received by the given host.

Function

Synopsis

```
host:total_received_data()
```

Arguments

None.

Returns

`number bytes`
: The total number of bytes received.

See Also

- lua-enet
- enet.host
- enet.host:total_sent_data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.host:total sent data

enet.host:total sent data · revision 18069 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the number of bytes that were sent through the given host.

Function

Synopsis

```
host:total_sent_data()
```

Arguments

None.

Returns

`number bytes`
: The total number of bytes sent.

See Also

- lua-enet
- enet.host
- enet.host:total_received_data

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.linked version

enet.linked version · revision 18083 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the included ENet's version as a string.

Function

Synopsis

```
version = enet.linked_version()
```

Arguments

None.

Returns

`string version`
: ENet's version.

See Also

- lua-enet

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer

enet.peer · revision 28452 · last edited 2023-07-05

Description

An ENet peer which data packets may be sent or received from. In most applications you'll manually keep track of connected peers.

Functions

Function	Description
peer:disconnect	Requests a disconnection from the peer.
peer:disconnect_now	Force immediate disconnection from peer.
peer:disconnect_later	Request a disconnection from peer, but only after all queued outgoing packets are sent.
peer:reset	Forcefully disconnects peer. The peer is not notified of the disconnection.
peer:ping	Send a ping request to peer, updates round_trip_time. This is called automatically at regular intervals.
peer:receive	Attempts to dequeue an incoming packet for this peer.
peer:send	Queues a packet to be sent to peer.
peer:throttle_configure	Changes the probability at which unreliable packets should not be dropped.
peer:ping_interval	Specifies the interval in milliseconds that pings are sent to the other end of the connection (defaults to 500).
peer:timeout	Returns or sets the parameters when a timeout is detected.
peer:index	Returns the index of the peer.
peer:state	Returns the state of the peer.
peer:connect_id	Returns the field ENetPeer::connectID that is assigned for each connection.
peer:round_trip_time	Returns or sets the current round trip time (i.e. ping).
peer:last_round_trip_time	Returns or sets the round trip time of the previous round trip time computation.

See Also

- lua-enet
- enet.event
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:connect id

enet.peer:connect id · revision 18058 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the field ENetPeer::connectID that is assigned for each connection.

Function

Synopsis

```
peer:connect_id()
```

Arguments

None.

Returns

`number id`
: Unique ID that was assigned to the connection.

See Also

- lua-enet
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:disconnect

enet.peer:disconnect · revision 18048 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Requests a disconnection from the peer. The message is sent on the next host:service() or host:flush().

Function

Synopsis

```
peer:disconnect(data)
```

Arguments

`number data`
: An optional integer value to be associated with the disconnect.

Returns

Nothing.

See Also

- lua-enet
- enet.host
- enet.host:flush
- enet.host:service
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:disconnect later

enet.peer:disconnect later · revision 18050 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Request a disconnection from peer, but only after all queued outgoing packets are sent.

Function

Synopsis

```
peer:disconnect_later(data)
```

Arguments

`number data`
: Optional integer value to be associated with the disconnect.

Returns

Nothing.

See Also

- lua-enet
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:disconnect now

enet.peer:disconnect now · revision 18049 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Force immediate disconnection from peer. Foreign peer not guaranteed to receive disconnect notification.

Function

Synopsis

```
peer:disconnect_now(data)
```

Arguments

`number data`
: Optional integer value to be associated with the disconnect.

Returns

Nothing.

See Also

- lua-enet
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:index

enet.peer:index · revision 18045 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the index of the peer. All peers of an ENet host are kept in an array. This function finds and returns the index of the peer of its host structure.

Function

Synopsis

```
peer:index()
```

Arguments

None.

Returns

`number index`
: The index of the peer.

See Also

- lua-enet
- enet.peer
- enet.host

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:last round trip time

enet.peer:last round trip time · revision 18047 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns or sets the round trip time of the previous round trip time computation. If value is nil the current value of the peer is returned. Otherwise the the previous round trip time computation is set to the specified value and returned.

ENet performs some filtering on the round trip times and it takes some time until the parameters are accurate. To speed it up you can set the value of the last round trip time to a more accurate guess.

Function

Synopsis

```
peer:last_round_trip_time()
```

Arguments

None.

Returns

`number lastroundtriptime`
: The peer's last round trip time computation in milliseconds.

Synopsis

```
peer:last_round_trip_time(value)
```

Arguments

`number value`
: Integer value to be used as the last round trip time.

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:round_trip_time

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:ping

enet.peer:ping · revision 18052 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Send a ping request to peer, updates round_trip_time. This is called automatically at regular intervals.

Function

Synopsis

```
peer:ping()
```

Arguments

None.

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:round_trip_time

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:ping interval

enet.peer:ping interval · revision 18056 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Specifies the interval in milliseconds that pings are sent to the other end of the connection (defaults to 500).

Function

Synopsis

```
peer:ping_interval(interval)
```

Arguments

`number interval`
: Time in milliseconds to wait before automatically calling peer:ping().

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:ping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:receive

enet.peer:receive · revision 26541 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Attempts to dequeue an incoming packet for this peer. Returns nil if there are no packets waiting. Otherwise returns two values: The string representing the packet data, and the channel the packet came from.

Function

Synopsis

```
peer:receive()
```

Arguments

None.

Returns

`string data`
: Data from the received packet in a string.

`number channel`
: Channel the packet was received from.

See Also

- lua-enet
- enet.peer
- enet.peer:send
- enet.host
- enet.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:reset

enet.peer:reset · revision 18051 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Forcefully disconnects peer. The peer is not notified of the disconnection.

Function

Synopsis

```
peer:reset()
```

Arguments

None.

Returns

Nothing.

See Also

- lua-enet
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:round trip time

enet.peer:round trip time · revision 18046 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns or sets the current round trip time (i.e. ping). If value is nil the current value of the peer is returned. Otherwise the round trip time is set to the specified value and returned.

ENet performs some filtering on the round trip times and it takes some time until the parameters are accurate. To speed it up, set the last round trip time to an accurate guess.

Function

Synopsis

```
peer:round_trip_time()
```

Arguments

None.

Returns

`number roundtriptime`
: The peer's current round trip time in milliseconds.

Synopsis

```
peer:round_trip_time(value)
```

Arguments

`number value`
: Integer value to be used as the new round trip time.

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:last_round_trip_time

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:send

enet.peer:send · revision 28333 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Send a packet using Data instead of a String
- See Also
- Other Languages

Queues a packet to be sent to the peer.

Function

Synopsis

```
peer:send(data, channel, flag)
```

Arguments

`string data`
: The contents of the packet, it must be a string.

`number channel`
: The channel to send the packet on. Optional. Defaults to 0.

`string flag`
: flag is one of "reliable", "unsequenced", or "unreliable". Reliable packets are guaranteed to arrive, and arrive in the order in which they are sent. Unreliable packets arrive in the order in which they are sent, but they aren't guaranteed to arrive. Unsequenced packets are neither guaranteed to arrive, nor do they have any guarantee on the order they arrive. Optional. Defaults to reliable.

Returns

Nothing.

Function

Available since LÖVE 12.0

This variant is not supported in earlier versions.

Synopsis

```
peer:send(pointer, size, channel, flag)
```

Arguments

`light userdata pointer`
: The contents of the packet.

`number size`
: The size of the memory starting at the pointer.

`number channel`
: The channel to send the packet on. Optional. Defaults to 0.

`string flag`

flag is one of "reliable", "unsequenced", or "unreliable". Reliable packets are guaranteed to arrive, and arrive in the order in which they are sent. Unreliable packets arrive in the order in which they are sent, but they aren't guaranteed to arrive. Unsequenced packets are neither guaranteed to arrive, nor do they have any guarantee on the order they arrive. Optional. Defaults to reliable.

Returns

Nothing.

Examples

Available since LÖVE 12.0
This variant is not supported in earlier versions.

Send a packet using Data instead of a String

```
-- client
local enet = require("enet")
local host = enet.host_create()
local server = host:connect("localhost:10001", 0)
host:service(100)

-- populate byteData
local byteData = love.data.newByteData(5)
local ffi = require("ffi")
local ptr = ffi.cast("void*", byteData:getFFIPointer())
ffi.copy(ptr, "hello")

server:send(byteData:getPointer(), byteData:getSize(), 11, "reliable") -- send data

love.update = function()
  host:service(10)
end
```

```
-- server
local enet = require("enet")
local host = enet.host_create("*:10001", 64, 0)

local text, channel
love.update = function()
  local event = host:service(10)
  if event and event.type == "receive" then
    text = event.data
    channel = event.channel
  end
end

love.draw = function()
-- prints 'hello' once client joins and the channel it was sent on
  love.graphics.print(tostring(text).."\nChannel: "..tostring(channel))
end
```

See Also

- lua-enet
- enet.peer
- enet.peer:receive
- enet.host
- enet.event

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:state

enet.peer:state · revision 18057 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Returns the state of the peer as a string.

Function

Synopsis

```
peer:state()
```

Arguments

None.

Returns

`string state`

The peer's current state. It can be any of the following:

- "disconnected"
- "connecting"
- "acknowledging_connect"
- "connection_pending"
- "connection_succeeded"
- "connected"
- "disconnect_later"
- "disconnecting"
- "acknowledging_disconnect"
- "zombie"
- "unknown"

See Also

- lua-enet
- enet.peer

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:throttle configure

enet.peer:throttle configure · revision 18055 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Changes the probability at which unreliable packets should not be dropped.

Function

Synopsis

```
peer:throttle_configure(interval, acceleration, deceleration)
```

Arguments

`number interval`
: Interval in milliseconds to measure lowest mean RTT.

`number acceleration`
: Rate at which to increase throttle probability as mean RTT declines.

`number deceleration`
: Rate at which to decrease throttle probability as mean RTT increases.

Returns

Nothing.

See Also

- lua-enet
- enet.peer
- enet.peer:round_trip_time

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

enet.peer:timeout

enet.peer:timeout · revision 18043 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Returns or sets the parameters when a timeout is detected. This is happens either after a fixed timeout or a variable timeout of time that takes the round trip time into account. The former is specified with the maximum parameter.

Function

Synopsis

```
peer:timeout(limit, minimum, maximum)
```

Arguments

`number limit`
: A factor that is multiplied with a value that based on the average round trip time to compute the timeout limit.

`number minimum`
: Timeout value, in milliseconds, that a reliable packet has to be acknowledged if the variable timeout limit was exceeded before dropping the peer.

`number maximum`
: Fixed timeout in milliseconds for which any packet has to be acknowledged before dropping the peer.

Returns

Nothing.

Synopsis

```
peer:timeout()
```

Arguments

None.

Returns

`number limit`
: A factor that is multiplied with a value that based on the average round trip time to compute the timeout limit.

`number minimum`
: Timeout value, in milliseconds, that a reliable packet has to be acknowledged if the variable timeout limit was exceeded before dropping the peer.

`number maximum`
: Fixed timeout in milliseconds for which any packet has to be acknowledged before dropping the peer.

See Also

- lua-enet
- enet.peer

- enet.peer:round_trip_time

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Event

Event · revision 22273 · last edited 2023-07-05

Available since LÖVE 0.6.0

This enum is not supported in earlier versions.

Arguments to `love.event.push()` and the like.

Constants

Since 0.8.0, event names are no longer abbreviated.

Available since LÖVE 0.8.0

focus
: Window focus gained or lost

joystickpressed
: Joystick pressed

joystickreleased
: Joystick released

keypressed
: Key pressed

keyreleased
: Key released

mousepressed
: Mouse pressed

mousereleased
: Mouse released

quit
: Quit

Available since LÖVE 0.9.0

resize
: Window size changed by the user

visible
: Window is minimized or un-minimized by the user

mousefocus
: Window mouse focus gained or lost

threaderror
: A Lua error has occurred in a thread

joystickadded
: Joystick connected

joystickremoved
: Joystick disconnected

joystickaxis
: Joystick axis motion

joystickhat
: Joystick hat pressed

gamepadpressed
: Joystick's virtual gamepad button pressed

gamepadreleased
: Joystick's virtual gamepad button released

gamepadaxis
: Joystick's virtual gamepad axis moved

textinput

User entered text

Available since LÖVE 0.9.2

mousemoved
Mouse position changed

Available since LÖVE 0.10.0

lowmemory
Running out of memory on mobile devices system
textedited
Candidate text for an IME changed
wheelmoved
Mouse wheel moved
touchpressed
Touch screen touched
touchreleased
Touch screen stop touching
touchmoved
Touch press moved inside touch screen
directorydropped
Directory is dragged and dropped onto the window
filedropped
File is dragged and dropped onto the window.

Removed in LÖVE 0.8.0

jp
Joystick pressed
jr
Joystick released
kp
Key pressed
kr
Key released
mp
Mouse pressed
mr
Mouse released
q
Quit

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

f

Window focus gained or lost

See Also

- love.event
- love.event.push

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

examples.love

examples.love · revision 27875 · last edited 2023-07-05

Example browser containing many helpful examples.

You can find the source for this Browser in the GitHub Repo (https://github.com/love2d-community/LOVE-Example-Brow ser/)



For LÖVE 11.0: Download (https://github.com/love2d-community/LOVE-Example-Brow ser)

For LÖVE 0.10.0: Download (https://github.com/love2d-community/LOVE-Example-Brow ser/releases/dow nload/1.0/examples_test.love)

For LÖVE 0.9.0: Download (https://github.com/Codingale/LOVE/raw /master/0.9.0%20examples/0.9.0%20examples.love)

For LÖVE 0.8.0: Download (https://github.com/dow nloads/bartoleo/Love-Examples/examples.love)

For LÖVE 0.7.0: Download (http://commondatastorage.googleapis.com/loveclub/examples.love)

The main menu

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FactoryFunctions

FactoryFunctions · revision 29086 · last edited 2024-01-17

Factory Functions

A factory function is a design pattern used in programming where a function is responsible for creating and returning another function or object. It acts as a "factory" that produces instances of a certain type, often with customizable behavior or configuration.

See example: Lua.org subchapter 16.4 (https://www.lua.org/pil/16.4.html#:~:text=following%20factory%20function%3A-,function%20newAccount,-(initialBalance)%0A%20%20%20%20%20%20local%20self)

Counter

The function, that returns the function with counter:

```lua
function createCounter(startValue)
    local count = startValue or 0  -- Initial count

    -- the returned function is the actual counter function
    local function counter()
        -- adding 1 to the counter
        count = count + 1

        -- return count value
        return count
    end

    -- return generated function
    return counter
end
```

Usage:

```lua
-- create first counter:
local counter1 = createCounter()

print(counter1())  -- 1
print(counter2())  -- 2

-- create second counter with initial value:
local counter2 = createCounter(12)

-- keep counting:
print(counter1())  -- 3
print(counter2())  -- 13

-- keep counting:
print(counter1())  -- 4
print(counter2())  -- 14
```

FAQ

FAQ · revision 17184 · last edited 2021-03-14

Common questions newbies might have about LÖVE

How did LÖVE come to exist?

rude wrote (http://blogs.love2d.org/content/l%C3%B6vely-interviews-rude):

> Actually, I don't remember how it started. Possibly it was just a challenge to myself. If someone forced me to make up a story, however, it could be something like:
>
> Well, I was looking for a free 2D-centric game framework that was easy to get up and running, and that would work on all platforms without too much hassle, but I couldn't find any. There were many engines and frameworks available, but they were too focused on 3D, which made the API unattractive for use in 2D games. So I began to work on my own API; a glorious world blissfully ignorant of the z-axis, where the children could finally laugh and play freely, both as in beer, and as in freedom. *single tear*
>
> And so forth.

Why doesn't LÖVE include a vector library?

slime wrote (https://love2d.org/forums/viewtopic.php?f=3&t=81457&start=50#p193807):

> The main problem with LÖVE's APIs using vector objects is that in Lua, vector objects tend to create a lot of garbage if used heavily. So having them required for using LÖVE functions prevents optimal performance. Some engines go to extreme lengths to try to work around that: https://www.youtube.com/watch?v=wTjyM7d7_YA#t=23m6s ↗
>
> I don't want to make vector-specific function variants either, as that would make the API a lot messier.

More discussion (https://love2d.org/forums/viewtopic.php?f=3&t=82815)

File

File · revision 16551 · last edited 2023-07-05

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Represents a file on the filesystem. A function that takes a file path can also take a File.

Constructors

love.filesystem.newFile	Creates a new File object.

Functions

These functions have parentheses in odd places. This is because the *File:* namespace is reserved in Mediawiki.

(File):close	Closes a File.	
(File):eof	If the end-of-file has been reached	Removed in 0.10.0
(File):flush	Flushes any buffered written data in the file to disk.	Added since 0.9.0
(File):getBuffer	Gets the buffer mode of a file.	Added since 0.9.0
(File):getFilename	Gets the filename that the File object was created with.	Added since 0.10.0
(File):getMode	Gets the FileMode the file has been opened with.	Added since 0.9.0
(File):getSize	Returns the file size.	
(File):isEOF	Gets whether end-of-file has been reached.	Added since 0.10.0
(File):isOpen	Gets whether the file is open.	Added since 0.9.0
(File):lines	Iterate over all the lines in a file.	
(File):open	Open the file for write, read or append.	
(File):read	Read a number of bytes from a file	
(File):seek	Seek to a position in a file	
(File):setBuffer	Sets the buffer mode for a file opened for writing or appending.	Added since 0.9.0
(File):tell	Returns the position in the file.	
(File):write	Write data to a file.	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

BufferMode	Buffer modes for File objects.	Added since 0.9.0
FileMode	The different modes you can open a File in.	

Supertypes

- Object

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileData

FileData · revision 10727 · last edited 2023-07-05

Available since LÖVE 0.7.0
This object is not supported in earlier versions.

Data representing the contents of a file.

Constructors

love.filesystem.newFileData	Creates a new FileData object from a file on disk, or from a string in memory.	Added since 0.7.0

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
FileData:getExtension	Gets the extension of the FileData.	Added since 0.7.0
FileData:getFilename	Gets the filename of the FileData.	Added since 0.7.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Data
- Object

See Also

- love.filesystem
- File

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileData:getExtension

FileData:getExtension · revision 5763 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets the extension of the FileData.

Function

Synopsis

```
ext = FileData:getExtension( )
```

Arguments

None.

Returns

`string ext`

The extension of the file the FileData represents.

See Also

- FileData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileData:getFilename

FileData:getFilename · revision 5764 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets the filename of the FileData.

Function

Synopsis

```
name = FileData:getFilename( )
```

Arguments

None.

Returns

`string name`

The name of the file the FileData represents.

See Also

- FileData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileDecoder

FileDecoder · revision 23722 · last edited 2026-04-06

Available since LÖVE 0.7.0 and removed in LÖVE 11.0
This enum is not supported in earlier or later versions.

How to decode a given FileData.

Constants

file
: The data is unencoded.

base64
: The data is base64-encoded.

See Also

- love.filesystem

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileDialogType

FileDialogType · revision 29436 · last edited 2025-01-16

Available since LÖVE 12.0

This enum is not supported in earlier versions.

Possible types of file dialogs.

Constants

openfile

Retrieve one or more files to be opened for reading.

openfolder

Retrieve a folder.

savefile

Retrieve a valid filename to be opened for writing.

See Also

- love.window
- love.window.showFileDialog

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileMode

FileMode · revision 11686 · last edited 2023-07-05

The different modes you can open a File in.

Constants

r
: Open a file for read.

w
: Open a file for write.

a
: Open a file for append.

c
: Do not open a file (represents a closed file.)

See Also

- File
- File:open
- love.filesystem
- love.filesystem.newFile

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FileType

FileType · revision 22643 · last edited 2026-04-06

Available since LÖVE 11.0

This enum is not supported in earlier versions.

The type of a file.

Constants

file
: Regular file.

directory
: Directory.

symlink
: Symbolic link.

other
: Something completely different like a device.

See Also

- love.filesystem
- love.filesystem.getInfo

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FilterMode

FilterMode · revision 20216 · last edited 2023-07-05

How the image is filtered when scaling.



Constants

linear
: Scale image with linear interpolation.

nearest
: Scale image with nearest neighbor interpolation.

See Also

- love.graphics
- love.graphics.setDefaultFilter
- Texture:getFilter
- Texture:setFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FilterType

FilterType · revision 22683 · last edited 2025-11-25

Available since LÖVE 11.0

This enum is not supported in earlier versions.

Types of filters for Sources.

Constants

lowpass

Low-pass filter. High frequency sounds are attenuated.

highpass

High-pass filter. Low frequency sounds are attenuated.

bandpass

Band-pass filter. Both high and low frequency sounds are attenuated based on the given parameters.

See Also

- Source
- Source:setFilter
- Source:getFilter

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture

Fixture · revision 14217 · last edited 2023-07-05

Available since LÖVE 0.8.0

This type is not supported in earlier versions.

Fixtures attach shapes to bodies.

Constructors

love.physics.newFixture	Creates and attaches a fixture.	Added since 0.8.0

Functions

Fixture:destroy	Destroys the fixture.	Added since 0.8.0
Fixture:getBody	Returns the body the fixture is attached to.	Added since 0.8.0
Fixture:getBoundingBox	Returns the points of the fixture bounding box.	Added since 0.8.0
Fixture:getCategory	Returns the categories the fixture belongs to.	Added since 0.8.0
Fixture:getDensity	Returns the density of the fixture.	Added since 0.8.0
Fixture:getFilterData	Returns the filter data of the fixture.	Added since 0.8.0
Fixture:getFriction	Returns the friction of the fixture.	Added since 0.8.0
Fixture:getGroupIndex	Returns the group the fixture belongs to.	Added since 0.8.0
Fixture:getMask	Returns which categories this fixture should **NOT** collide with.	Added since 0.8.0
Fixture:getMassData	Returns the mass, its center and the rotational inertia.	Added since 0.8.0
Fixture:getRestitution	Returns the restitution of the fixture.	Added since 0.8.0
Fixture:getShape	Returns the shape of the fixture.	Added since 0.8.0
Fixture:getUserData	Returns the Lua value associated with this fixture.	Added since 0.8.0
Fixture:isDestroyed	Gets whether the Fixture is destroyed.	Added since 0.9.2
Fixture:isSensor	Returns whether the fixture is a sensor.	Added since 0.8.0
Fixture:rayCast	Casts a ray against the shape of the fixture.	Added since 0.8.0
Fixture:setCategory	Sets the categories the fixture belongs to.	Added since 0.8.0
Fixture:setDensity	Sets the density of the fixture.	Added since 0.8.0
Fixture:setFilterData	Sets the filter data of the fixture.	Added since 0.8.0
Fixture:setFriction	Sets the friction of the fixture.	Added since 0.8.0
Fixture:setGroupIndex	Sets the group the fixture belongs to.	Added since 0.8.0
Fixture:setMask	Sets which categories this fixture should **NOT** collide with.	Added since 0.8.0
Fixture:setRestitution	Sets the restitution of the fixture.	Added since 0.8.0
Fixture:setSensor	Sets whether the fixture should act as a sensor.	Added since 0.8.0
Fixture:setUserData	Associates a Lua value with the fixture.	Added since 0.8.0
Fixture:testPoint	Checks if a point is inside the shape of the fixture.	Added since 0.8.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Fixture:destroy

Fixture:destroy · revision 27753 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Destroys the fixture.

Function

Synopsis

```
Fixture:destroy( )
```

Arguments

None.

Returns

Nothing.

Notes

Destroying a Fixture may change the mass of the Body it was attached to.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getBody

Fixture:getBody · revision 8752 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the body to which the fixture is attached.

Function

Synopsis

```
body = Fixture:getBody( )
```

Arguments

None.

Returns

Body **body**

The parent body.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getBoundingBox

Fixture:getBoundingBox · revision 15242 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the points of the fixture bounding box. In case the fixture has multiple children a 1-based index can be specified. For example, a fixture will have multiple children with a chain shape.

Function

Synopsis

```
topLeftX, topLeftY, bottomRightX, bottomRightY = Fixture:getBoundingBox( index )
```

Arguments

number index (1)
: A bounding box of the fixture.

Returns

number topLeftX
: The x position of the top-left point.

number topLeftY
: The y position of the top-left point.

number bottomRightX
: The x position of the bottom-right point.

number bottomRightY
: The y position of the bottom-right point.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getCategory

Fixture:getCategory · revision 8623 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the categories the fixture belongs to.

Function

Synopsis

```
category1, category2, ... = Fixture:getCategory( )
```

Arguments

None.

Returns

`number category1`
: The first category.

`number category2`
: The second category.

See Also

- Fixture
- Fixture:setCategory

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getDensity

Fixture:getDensity · revision 8624 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the density of the fixture.

Function

Synopsis

```
density = Fixture:getDensity( )
```

Arguments

None.

Returns

`number density`

The fixture density in kilograms per square meter.

See Also

- Fixture
- Fixture:setDensity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getFilterData

Fixture:getFilterData · revision 8625 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the filter data of the fixture. Categories and masks are encoded as the bits of a 16-bit integer.

Function

Synopsis

```
categories, mask, group = Fixture:getFilterData( )
```

Arguments

None.

Returns

number categories
: The categories as an integer from 0 to 65535.

number mask
: The mask as an integer from 0 to 65535.

number group
: The group as an integer from -32768 to 32767.

See Also

- Fixture
- Fixture:setFilterData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getFriction

Fixture:getFriction · revision 8626 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the friction of the fixture.

Function

Synopsis

```
friction = Fixture:getFriction( )
```

Arguments

None.

Returns

`number friction`

The fixture friction.

See Also

- Fixture
- Fixture:setFriction

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getGroupIndex

Fixture:getGroupIndex · revision 30272 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the group the fixture belongs to. Fixtures with the same group will always collide if the group is positive or never collide if it's negative. The group zero means no group.

The groups range from -32768 to 32767.

Function

Synopsis

```
group = Fixture:getGroupIndex( )
```

Arguments

None.

Returns

`number group`
: The group of the fixture.

See Also

- Fixture
- Fixture:setGroupIndex
- Fixture:getCategory

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getMask

Fixture:getMask · revision 9501 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns which categories this fixture should **NOT** collide with.

Function

Synopsis

```
mask1, mask2, ... = Fixture:getMask( )
```

Arguments

None.

Returns

number mask1
: The first category selected by the mask.

number mask2
: The second category selected by the mask.

See Also

- Fixture
- Fixture:setMask

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getMassData

Fixture:getMassData · revision 8630 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the mass, its center and the rotational inertia.

Function

Synopsis

```
x, y, mass, inertia = Fixture:getMassData( )
```

Arguments

None.

Returns

number x
: The x position of the center of mass.

number y
: The y position of the center of mass.

number mass
: The mass of the fixture.

number inertia
: The rotational inertia.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getRestitution

Fixture:getRestitution · revision 8629 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the restitution of the fixture.

Function

Synopsis

```
restitution = Fixture:getRestitution( )
```

Arguments

None.

Returns

`number restitution`

The fixture restitution.

See Also

- Fixture
- Fixture:setRestitution

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getShape

Fixture:getShape · revision 27940 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the Shape of the fixture. This shape is a reference to the actual data used in the simulation. It's possible to change its values between timesteps.


Do not call any functions on this shape after the parent fixture has been destroyed. This shape will point to an invalid memory address and likely cause crashes if you interact further with it.

Function

Synopsis

```
shape = Fixture:getShape( )
```

Arguments

None.

Returns

Shape **shape**

The fixture's shape.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:getUserData

Fixture:getUserData · revision 19349 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns the Lua value associated with this fixture.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
value = Fixture:getUserData( )
```

Arguments

None.

Returns

`any value`
: The Lua value associated with the fixture.

See Also

- Fixture
- Fixture:setUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:isDestroyed

Fixture:isDestroyed · revision 14642 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether the Fixture is destroyed. Destroyed fixtures cannot be used.

Function

Synopsis

```
destroyed = Fixture:isDestroyed( )
```

Arguments

None.

Returns

`boolean destroyed`

Whether the Fixture is destroyed.

See Also

- Fixture
- Fixture:destroy

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Fixture:isSensor

Fixture:isSensor · revision 8619 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Returns whether the fixture is a sensor.

Function

Synopsis

```
sensor = Fixture:isSensor( )
```

Arguments

None.

Returns

`boolean sensor`

If the fixture is a sensor.

See Also

- Fixture
- Fixture:setSensor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:rayCast

Fixture:rayCast · revision 24715 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Casting 2 different rays against a box.
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Casts a ray against the shape of the fixture and returns the surface normal vector and the line position where the ray hit. If the ray missed the shape, nil will be returned.

The ray starts on the first point of the input line and goes towards the second point of the line. The fifth argument is the maximum distance the ray is going to travel as a scale factor of the input line length.

The childIndex parameter is used to specify which child of a parent shape, such as a ChainShape, will be ray casted. For ChainShapes, the index of 1 is the first edge on the chain. Ray casting a parent shape will only test the child specified so if you want to test every shape of the parent, you must loop through all of its children.

The world position of the impact can be calculated by multiplying the line vector with the third return value and adding it to the line starting point.

```
hitx, hity = x1 + (x2 - x1) * fraction, y1 + (y2 - y1) * fraction
```



There is a bug in 0.8.0 where the normal vector returned by this function gets scaled by love.physics.getMeter.

Function

Synopsis

```
xn, yn, fraction = Fixture:rayCast( x1, y1, x2, y2, maxFraction, childIndex )
```

Arguments

number x1
: The x position of the input line starting point.

number y1
: The y position of the input line starting point.

number x2
: The x position of the input line end point.

number y2
: The y position of the input line end point.

number maxFraction
: Ray length parameter.

number childIndex (1)
: The index of the child the ray gets cast against.

Returns

number xn
: The x component of the normal vector of the edge where the ray hit the shape.

number yn
: The y component of the normal vector of the edge where the ray hit the shape.

number fraction

The position on the input line where the intersection happened as a factor of the line length.

Examples

Casting 2 different rays against a box.

```
function love.load()
    -- Setting this to 1 to avoid all current scaling bugs.
    love.physics.setMeter(1)

    World = love.physics.newWorld()

    Box = {}
    Box.Body = love.physics.newBody(World, 400, 300, "dynamic")
    Box.Shape = love.physics.newRectangleShape(150, 150)
    Box.Fixture = love.physics.newFixture(Box.Body, Box.Shape)

    -- Giving the box a gentle spin.
    Box.Body:setAngularVelocity(0.5)

    Ray1 = {
        point1 = {},
        point2 = {},
    }
    Ray1.point1.x, Ray1.point1.y = 50, 50
    Ray1.point2.x, Ray1.point2.y = 400, 300
    Ray1.scale = 1

    Ray2 = {
        point1 = {},
        point2 = {},
    }
    Ray2.point1.x, Ray2.point1.y = 500, 400
    Ray2.point2.x, Ray2.point2.y = Ray2.point1.x + math.cos(math.pi*1.45), Ray2.point1.y + math.sin(math.pi*1.45)
    Ray2.scale = 150
end

function love.update(dt)
    World:update(dt)
end

function love.draw()
    -- Drawing the box.
    love.graphics.setColor(1, 1, 1)
    love.graphics.polygon("line", Box.Body:getWorldPoints( Box.Shape:getPoints() ))

    -- Drawing the input lines of the rays and the reach of ray 2 in gray.
    love.graphics.setLineWidth(3)
    love.graphics.setColor(50/255, 50/255, 50/255)
    love.graphics.line(Ray2.point1.x, Ray2.point1.y, Ray2.point1.x + (Ray2.point2.x - Ray2.point1.x) * Ray2.scale, Ray2.point1.y +
(Ray2.point2.y - Ray2.point1.y) * Ray2.scale)
    love.graphics.setColor(1, 1, 1)
    love.graphics.line(Ray1.point1.x, Ray1.point1.y, Ray1.point2.x, Ray1.point2.y)
    love.graphics.line(Ray2.point1.x, Ray2.point1.y, Ray2.point2.x, Ray2.point2.y)
    love.graphics.setLineWidth(1)

    -- Remember that the ray cast can return nil if it hits nothing.
    local r1nx, r1ny, r1f = Box.Fixture:rayCast(Ray1.point1.x, Ray1.point1.y, Ray1.point2.x, Ray1.point2.y, Ray1.scale)
    local r2nx, r2ny, r2f = Box.Fixture:rayCast(Ray2.point1.x, Ray2.point1.y, Ray2.point2.x, Ray2.point2.y, Ray2.scale)

    if r1nx then
        -- Calculating the world position where the ray hit.
        local r1HitX = Ray1.point1.x + (Ray1.point2.x - Ray1.point1.x) * r1f
        local r1HitY = Ray1.point1.y + (Ray1.point2.y - Ray1.point1.y) * r1f

        -- Drawing the ray from the starting point to the position on the shape.
        love.graphics.setColor(1, 0, 0)
        love.graphics.line(Ray1.point1.x, Ray1.point1.y, r1HitX, r1HitY)

        -- We also get the surface normal of the edge the ray hit. Here drawn in green
        love.graphics.setColor(0, 1, 0)
```

```
        love.graphics.line(r1HitX, r1HitY, r1HitX + r1nx * 25, r1HitY + r1ny * 25)
    end

    if r2nx then
        -- Calculating the world position where the ray hit.
        local r2HitX = Ray2.point1.x + (Ray2.point2.x - Ray2.point1.x) * r2f
        local r2HitY = Ray2.point1.y + (Ray2.point2.y - Ray2.point1.y) * r2f

        -- Drawing the ray from the starting point to the position on the shape.
        love.graphics.setColor(1, 0, 0)
        love.graphics.line(Ray2.point1.x, Ray2.point1.y, r2HitX, r2HitY)

        -- We also get the surface normal of the edge the ray hit. Here drawn in green
        love.graphics.setColor(0, 1, 0)
        love.graphics.line(r2HitX, r2HitY, r2HitX + r2nx * 25, r2HitY + r2ny * 25)
    end
end
```



Screenshot of this example

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setCategory

Fixture:setCategory · revision 20253 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the categories the fixture belongs to. There can be up to 16 categories represented as a number from 1 to 16.

All fixture's default category is 1.

Function

Synopsis

```
Fixture:setCategory( category1, category2, ... )
```

Arguments

number category1
: The first category.

number category2
: The second category.

Returns

Nothing.

See Also

- Fixture
- Fixture:getCategory

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setDensity

Fixture:setDensity · revision 8635 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the density of the fixture. Call Body:resetMassData if this needs to take effect immediately.

Function

Synopsis

```
Fixture:setDensity( density )
```

Arguments

`number density`
: The fixture density in kilograms per square meter.

Returns

Nothing.

See Also

- Fixture
- Fixture:getDensity

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setFilterData

Fixture:setFilterData · revision 21571 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the filter data of the fixture.

Groups, categories, and mask can be used to define the collision behaviour of the fixture.

If two fixtures are in the same group they either always collide if the group is positive, or never collide if it's negative. If the group is zero or they do not match, then the contact filter checks if the fixtures select a category of the other fixture with their masks. The fixtures do not collide if that's not the case. If they do have each other's categories selected, the return value of the custom contact filter will be used. They always collide if none was set.

There can be up to 16 categories. Categories and masks are encoded as the bits of a 16-bit integer.

When created, prior to calling this function, all fixtures have category set to 1, mask set to 65535 (all categories) and group set to 0.

This function allows setting all filter data for a fixture at once. To set only the categories, the mask or the group, you can use Fixture:setCategory, Fixture:setMask or Fixture:setGroupIndex respectively.

Function

Synopsis

```
Fixture:setFilterData( categories, mask, group )
```

Arguments

`number categories`
: The categories as an integer from 0 to 65535.

`number mask`
: The mask as an integer from 0 to 65535.

`number group`
: The group as an integer from -32768 to 32767.

Returns

Nothing.

See Also

- Fixture
- Fixture:getFilterData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setFriction

Fixture:setFriction · revision 26756 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the friction of the fixture.

Friction determines how shapes react when they "slide" along other shapes. Low friction indicates a slippery surface, like ice, while high friction indicates a rough surface, like concrete. **Range**: [0.0, 1.0]

Function

Synopsis

```
Fixture:setFriction( friction )
```

Arguments

`number friction`
: The fixture friction.

Returns

Nothing.

See Also

- Fixture
- Fixture:getFriction

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setGroupIndex

Fixture:setGroupIndex · revision 8611 · last edited 2025-11-23

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the group the fixture belongs to. Fixtures with the same group will always collide if the group is positive or never collide if it's negative. The group zero means no group.

The groups range from -32768 to 32767.

Function

Synopsis

```
Fixture:setGroupIndex( group )
```

Arguments

`number group`
: The group as an integer from -32768 to 32767.

Returns

Nothing.

See Also

- Fixture
- Fixture:getGroupIndex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setMask

Fixture:setMask · revision 29012 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the category masks of the fixture. There can be up to 16 categories represented as a number from 1 to 16.

This fixture will **NOT** collide with the fixtures that are in the selected categories if the other fixture also has a category of this fixture selected.

Calling this function with no arguments will make the fixture have no masks.

Function

Synopsis

```
Fixture:setMask( mask1, mask2, ... )
```

Arguments

`number mask1`
: The first category.

`number mask2`
: The second category.

`number ...`
: Additional categories.

Returns

Nothing.

Examples

```
Body.fixture:setCategory(1)
Body.fixture:setMask(1)              -- this fixture is in category 1 and will not physically collide with other fixtures in category 1
```

See Also

- Fixture
- Fixture:getMask
- Fixture:setCategory
- Fixture:getCategory

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setRestitution

Fixture:setRestitution · revision 26755 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets the restitution of the fixture. **Range**: [0.0, 1.0]

Restitution determines how much a body bounces after a collision. The value should be greater than zero and is usually in [0, 1].

You can think of restitution as a scale on the reflected velocity after a collision. A bouncy ball would have a high restitution (an elastic collision keeps most of the velocity) and a brick a low restitution (inelastic collision loses most of the velocity). A value of one would be perfectly elastic and maintain all of its velocity.

For more detail, see how Box2D's ContactSolver uses restitution to scale the velocityBias (https://github.com/love2d/love/blob/977c690fa97760ed195d2e32570a7ac8eb19318b/src/libraries/Box2D/Dynamics/Contacts/b2ContactSolver.cpp#L208-L214) and how it preserves the higher restitution of two bodies (https://github.com/love2d/love/blob/977c690fa97760ed195d2e32570a7ac8eb19318b/src/libraries/Box2D/Dynamics/Contacts/b2Contact.h#L42-L47) to bias towards bounciness.

Function

Synopsis

```
Fixture:setRestitution( restitution )
```

Arguments

`number restitution`
: The fixture restitution.

Returns

Nothing.

See Also

- Fixture
- Fixture:getRestitution

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setSensor

Fixture:setSensor · revision 29496 · last edited 2025-11-22

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Sets whether the fixture should act as a sensor and trigger the beginContact callback.

Fixtures with sensors do not cause collision responses (they pass through other objects instead of bouncing off), but the beginContact and endContact World callbacks will still be called for this fixture.

When dealing with sensor fixtures, a Contact object behaves differently than with solid fixtures. Specifically:

- getPositions() returns nil instead of the world positions
- getNormal() returns 0,0 instead of the collision normal
- getFriction() returns 0 instead of the friction
- getRestitution() returns 0 instead of the restitution

Function

Synopsis

```
Fixture:setSensor( sensor )
```

Arguments

`boolean sensor`
: The sensor status.

Returns

Nothing.

See Also

- love.physics
- Fixture
- Fixture:isSensor

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:setUserData

Fixture:setUserData · revision 19350 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Associates a Lua value with the fixture.

To delete the reference, explicitly pass nil.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
Fixture:setUserData( value )
```

Arguments

`any value`
: The Lua value to associate with the fixture.

Returns

Nothing.

See Also

- Fixture
- Fixture:getUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Fixture:testPoint

Fixture:testPoint · revision 8616 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.8.0

This method is not supported in earlier versions.

Checks if a point is inside the shape of the fixture.

Function

Synopsis

```
isInside = Fixture:testPoint( x, y )
```

Arguments

`number x`
: The x position of the point.

`number y`
: The y position of the point.

Returns

`boolean isInside`
: True if the point is inside or false if it is outside.

See Also

- Fixture

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Flux

Flux · revision 19222 · last edited 2023-07-05

Flux is a fast, lightweight tweening library for lua.

Flux allows the user to easily interpolate a numerical value field of an object between its current value to a new value over a specified duration, using a specified easing type. If you want to do something like fade a sprite in or out, slide something onto the screen, gradually enlarge something or fade something from one color to another, Flux provides a means of doing this with a single function call.

```
-- Moves the ball object to the position 200, 300 over 4 seconds
flux.to(ball, 4, { x = 200, y = 300 })
```

Check out the README (https://github.com/rxi/flux/blob/master/README.md) for instructions on setting up and using Flux in your projects. The github page for Flux is over here (https://github.com/rxi/flux/).

Links

- Github (https://github.com/rxi/flux/)
- Readme (https://github.com/rxi/flux/blob/master/README.md)
- flux.lua (https://github.com/rxi/flux/blob/master/flux.lua?raw =1)

Font

Font · revision 27730 · last edited 2023-07-05

Defines the shape of characters that can be drawn onto the screen.

Constructors

love.graphics.newFont	Creates a new Font from a TrueType Font or BMFont file.
love.graphics.newImageFont	Creates a new Font by loading a specifically formatted image. Added since 0.2.0

Functions

Font:getAscent	Gets the ascent of the Font in pixels.	Added since 0.9.0
Font:getBaseline	Gets the baseline of the Font in pixels.	Added since 0.9.0
Font:getDPIScale	Gets the DPI scale factor of the Font.	Added since 11.0
Font:getDescent	Gets the descent of the Font in pixels.	Added since 0.9.0
Font:getFilter	Gets the filter mode for a font.	Added since 0.9.0
Font:getHeight	Gets the height of the Font in pixels.	
Font:getKerning	Gets the kerning between two characters in the Font.	Added since 11.4
Font:getLineHeight	Gets the line height.	
Font:getWidth	Determines the width of the given text.	
Font:getWrap	Gets formatting information for text, given a wrap limit.	Added since 0.7.0
Font:hasGlyphs	Gets whether the Font can render a character or string.	Added since 0.9.0
Font:setFallbacks	Sets other Fonts to use if this Font doesn't have a specific character.	Added since 0.10.0
Font:setFilter	Sets the filter mode for a font.	Added since 0.9.0
Font:setLineHeight	Sets the line height.	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Object

See Also

- love.graphics
- love.graphics.getFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getAscent

Font:getAscent · revision 10301 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the ascent of the Font. The ascent spans the distance between the baseline and the top of the glyph that reaches farthest from the baseline.

Function

Synopsis

```
ascent = Font:getAscent( )
```

Arguments

None.

Returns

`number ascent`

The ascent of the Font in pixels.

See Also

- Font
- Font:getDescent
- Font:getBaseline

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getBaseline

Font:getBaseline · revision 10302 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the baseline of the Font. Most scripts share the notion of a baseline: an imaginary horizontal line on which characters rest. In some scripts, parts of glyphs lie below the baseline.

Function

Synopsis

```
baseline = Font:getBaseline( )
```

Arguments

None.

Returns

`number baseline`

The baseline of the Font in pixels.

See Also

- Font
- Font:getAscent
- Font:getDescent

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getDescent

Font:getDescent · revision 10300 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the descent of the Font. The descent spans the distance between the baseline and the lowest descending glyph in a typeface.

Function

Synopsis

```
descent = Font:getDescent( )
```

Arguments

None.

Returns

`number descent`

The descent of the Font in pixels.

See Also

- Font
- Font:getAscent
- Font:getBaseline

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getDPIScale

Font:getDPIScale · revision 22679 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This method is not supported in earlier versions.

Gets the DPI scale factor of the Font.

The DPI scale factor represents relative pixel density. A DPI scale factor of 2 means the font's glyphs have twice the pixel density in each dimension (4 times as many pixels in the same area) compared to a font with a DPI scale factor of 1.

The font size of TrueType fonts is scaled internally by the font's specified DPI scale factor. By default, LÖVE uses the screen's DPI scale factor when creating TrueType fonts.

Function

Synopsis

```
dpiscale = Font:getDPIScale( )
```

Arguments

None.

Returns

`number dpiscale`

The DPI scale factor of the Font.

See Also

- Font
- love.graphics.newFont
- love.graphics.getDPIScale
- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getFilter

Font:getFilter · revision 27994 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the filter mode for a font.

Function

Synopsis

```
min, mag, anisotropy = Font:getFilter( )
```

Arguments

None.

Returns

`FilterMode min`
: Filter mode used when minifying the font.

`FilterMode mag`
: Filter mode used when magnifying the font.

`number anisotropy`
: Maximum amount of anisotropic filtering used.

See Also

- Font
- Font:setFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getHeight

Font:getHeight · revision 29497 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the height of the Font. The height of the font is the size including any spacing needed for a single line.

Function

Synopsis

```
height = font:getHeight( )
```

Arguments

None.

Returns

number height
: The height of the Font in pixels.

See Also

- Font
- Font:getWidth

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getKerning

Font:getKerning · revision 27336 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.4

This function is not supported in earlier versions.

Gets the kerning between two characters in the Font.

Kerning is normally handled automatically in love.graphics.print, Text objects, Font:getWidth, Font:getWrap, etc. This function is useful when stitching text together manually.

Function

Synopsis

```
kerning = Font:getKerning( leftchar, rightchar )
```

Arguments

`string leftchar`
: The left character.

`string rightchar`
: The right character.

Returns

`number kerning`
: The kerning amount to add to the spacing between the two characters. May be negative.

Function

Synopsis

```
kerning = Font:getKerning( leftglyph, rightglyph )
```

Arguments

`number leftglyph`
: The unicode number for the left glyph.

`number rightglyph`
: The unicode number for the right glyph.

Returns

`number kerning`

The kerning amount to add to the spacing between the two glyphs. May be negative.

See Also

- Font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getLineHeight

Font:getLineHeight · revision 23661 · last edited 2025-10-01

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Gets the line height.

This will be the value previously set by Font:setLineHeight, or `1.0` by default.

Function

Synopsis

```
height = Font:getLineHeight( )
```

Arguments

None.

Returns

`number height`
: The current line height.

See Also

- Font
- Font:setLineHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getWidth

Font:getWidth · revision 26370 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Determines the maximum width (accounting for newlines) taken by the given string.

Function

Synopsis

```
width = font:getWidth( text )
```

Arguments

`string text`
: A string.

Returns

`number width`
: The width of the text.

See Also

- Font
- Font:getHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:getWrap

Font:getWrap · revision 27818 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets formatting information for text, given a wrap limit.

This function accounts for newlines correctly (i.e. '\n').

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
width, wrappedtext = Font:getWrap( text, wraplimit )
```

Arguments

`string text`
: The text that will be wrapped.

`number wraplimit`
: The maximum width in pixels of each line that *text* is allowed before wrapping.

Returns

`number width`
: The maximum width of the wrapped text.

`table wrappedtext`
: A sequence containing each line of text that was wrapped.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
width, wrappedtext = Font:getWrap( coloredtext, wraplimit )
```

Arguments

`table coloredtext`
: A table containing colors and strings, in the form of `{color1, string1, color2, string2, ...}`.

  `table color1`
  : A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

  `string string1`
  : A string of text which has a color specified by the previous color.

  `table color2`

A table containing red, green, blue, and optional alpha components to use as a color for the next string in the table, in the form of `{red, green, blue, alpha}`.

`string string2`
A string of text which has a color specified by the previous color.

`tables and strings ...`
Additional colors and strings.

`number wraplimit`
The maximum width in pixels of each line that *text* is allowed before wrapping.

Returns

`number width`
The maximum width of the wrapped text.

`table wrappedtext`
A sequence of strings containing each line of text that was wrapped, with the color information stripped out.

Function

Removed in LÖVE 0.10.0
This variant is not supported in that and later versions.

Synopsis

```
width, lines = Font:getWrap( text, wraplimit )
```

Arguments

`string text`
The text that would be wrapped.

`number wraplimit`
The maximum width in pixels of each line that *text* is allowed before wrapping.

Returns

`number width`
The maximum width of the wrapped text.

`number lines`
The number of lines that the wrapped text will have.

See Also

- Font
- love.graphics.printf
- Text:setf

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:hasGlyphs

Font:hasGlyphs · revision 11865 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Font can render a character or string.

Function

Synopsis

```
hasglyphs = Font:hasGlyphs( text )
```

Arguments

`string text`
: A UTF-8 encoded unicode string.

Returns

`boolean hasglyph`
: Whether the font can render all the UTF-8 characters in the string.

Function

Synopsis

```
hasglyphs = Font:hasGlyphs( character1, character2, ... )
```

Arguments

`string character1`
: A unicode character.

`string character2`
: Another unicode character.

Returns

`boolean hasglyph`
: Whether the font can render all the glyphs represented by the characters.

Function

Synopsis

```
hasglyphs = Font:hasGlyphs( codepoint1, codepoint2, ... )
```

Arguments

`number codepoint1`

A unicode codepoint number.

number codepoint2

Another unicode codepoint number.

Returns

boolean hasglyph

Whether the font can render all the glyphs represented by the codepoint numbers.

See Also

- Font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:setFallbacks

Font:setFallbacks · revision 17578 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.10.0

This function is not supported in earlier versions.

Sets the fallback fonts. When the Font doesn't contain a glyph, it will substitute the glyph from the next subsequent fallback Fonts. This is akin to setting a "font stack" in Cascading Style Sheets (CSS).

Function

Synopsis

```
Font:setFallbacks( fallbackfont1, ... )
```

Arguments

`Font fallbackfont1`
: The first fallback Font to use.

`Font ...`
: Additional fallback Fonts.

Returns

Nothing.

Notes

If this is called it should be before love.graphics.print, Font:getWrap, and other Font methods which use glyph positioning information are called.

Every fallback Font must be created from the same file type as the primary Font. For example, a Font created from a .ttf file can only use fallback Fonts that were created from .ttf files.

See Also

- Font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:setFilter

Font:setFilter · revision 29455 · last edited 2026-04-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Pixalated font
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the filter mode for a font.

Function

Synopsis

```
Font:setFilter( min, mag, anisotropy )
```

Arguments

`FilterMode min`
: How to scale a font down.

`FilterMode mag (min)`
: How to scale a font up.

`number anisotropy (1)`
: Maximum amount of anisotropic filtering used.

Returns

Nothing.

Examples

```
local font = love.graphics.newFont(40)
font:setFilter("linear", "nearest")

function love.draw()
    love.graphics.setFont(font)
    love.graphics.print("Hello", 100,100, 0, .5) -- Scaled down.
    love.graphics.print("World", 100,116, 0, 2)  -- Scaled up.
end
```

Pixalated font

File `pixelfont-11p.png`:


pixelfont-11p.png

main.lua:

```
local filename =  'pixelfont-11p.png'
local glyphs = " abcdefghijklmnopqrstuvwxyz" ..
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ0" ..
    "123456789.,!?-+/():;%&`'*#=[]\""

local fontImageData = love.image.newImageData(filename)
local font11p = love.graphics.newImageFont(fontImageData, glyphs)
font11p:setFilter( 'nearest', 'nearest' )

function love.draw ()
    love.graphics.setFont(font11p)
    love.graphics.print (glyphs, 0, 12)
```

```
    love.graphics.scale (2)
    love.graphics.print (glyphs, 0, 12)
    love.graphics.scale (2)
    love.graphics.print (glyphs, 0, 12)
    love.graphics.scale (2)
    love.graphics.print (glyphs, 0, 12)
end
```

See Also

- Font
- Font:getFilter
- FilterMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Font:setLineHeight

Font:setLineHeight · revision 23660 · last edited 2025-10-01

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Sets the line height.

When rendering the font in lines the actual height will be determined by the line height multiplied by the height of the font. The default is `1.0`.

Function

Synopsis

```
Font:setLineHeight( height )
```

Arguments

`number height`
: The new line height.

Returns

Nothing.

See Also

- Font
- Font:getLineHeight

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FontData

FontData · revision 15946 · last edited 2023-07-05

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

This type is not supported in earlier or later versions.

A FontData represents a font, containing the font Rasterizer and its glyphs.

Constructors

love.font.newFontData	Creates a new FontData.	Added since 0.7.0 Removed in 0.8.0

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Data
- Object

See Also

- love.font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FontForge

FontForge · revision 6268 · last edited 2021-03-14

An outline font editor that lets you create your own postscript, truetype, opentype, cid-keyed, multi-master, cff, svg and bitmap (bdf, FON, NFNT) fonts, or edit existing ones. Also lets you convert one format to another.

The most obvious aspect is that it is a drawing program like FreeHand, Inkscape or Illustrator which lets you draw the outlines of your letters. Unlike other drawing programs it expects you to draw many pictures at once (one or more for each letter) and collects them into a database.

It allows you to describe the way these pictures interact with each other (if you put one picture after the other then they should normally be separated by a certain distance -- the font's metrics, or if these two pictures are placed adjacent to one another then they turn into a third -- the font's ligatures, and so on).

Finally as a font editor it will bundle up all the pictures you have drawn, and all the metadata about how those pictures fit together, and will turn that bundle of stuff into a font that your computer can use to display text.

FontForge website (http://fontforge.sourceforge.net/)

FPSGraph

FPSGraph · revision 13391 · last edited 2023-07-05

A simple lightweight graphing utility for LOVE.

Github Repository: https://github.com/icrawler/FPSGraph ↗

Forum Post: https://love2d.org/forums/viewtopic.php?f=5&t=77612

Contributors

- Substitute541

FractalNoise

FractalNoise · revision 19059 · last edited 2021-03-14

Generates a fractal noise using LOVE's noise functions

```
-- x, y, z, w are the positions to sample
-- iter is how many iterations (octaves) of the noise (default: 1)
-- amp is the amplitude factor (default: 0.5)
-- freq is the frequency factor (default: 2)

-- returns a fractal noise value in the range [-1.0, 1.0]
function genFractalNoise1(x, iter, amp, freq)
    val = love.math.noise(x)*2-1
    freq = freq or 2
    iter = iter or 1
    amp = amp or 0.5
    local n = 0
    while n < iter-1 do
        val = val + (love.math.noise(x*freq)*2-1)*amp
        freq = freq * freq
        amp = amp * amp
        n = n + 1
    end
    return math.max(math.min(val, 1.0), -1.0)
end

-- returns a fractal noise value in the range [-1.0, 1.0]
function genFractalNoise2(x, y, iter, amp, freq)
    val = love.math.noise(x, y)*2-1
    freq = freq or 2
    iter = iter or 1
    amp = amp or 0.5
    local n = 0
    while n < iter-1 do
        val = val + (love.math.noise(x*freq, y*freq)*2-1)*amp
        freq = freq * freq
        amp = amp * amp
        n = n + 1
    end
    return math.max(math.min(val, 1.0), -1.0)
end

-- returns a fractal noise value in the range [-1.0, 1.0]
function genFractalNoise3(x, y, z, iter, amp, freq)
    local val = love.math.noise(x, y, z)*2-1
    freq = freq or 2
    iter = iter or 1
    amp = amp or 0.5
    local n = 0
    while n < iter-1 do
        val = val + (love.math.noise(x*freq, y*freq, z*freq)*2-1)*amp
        freq = freq * freq
        amp = amp * amp
        n = n + 1
    end
    return math.max(math.min(val, 1.0), -1.0)
end

-- returns a fractal noise value in the range [-1.0, 1.0]
function genFractalNoise4(x, y, z, w, iter, amp, freq)
    local val = love.math.noise(x, y, z, w)*2-1
    freq = freq or 2
    iter = iter or 1
    amp = amp or 0.5
    local n = 0
    while n < iter-1 do
        val = val + (love.math.noise(x*freq, y*freq, z*freq, w*freq)*2-1)*amp
        freq = freq * freq
        amp = amp * amp
        n = n + 1
    end
```

```
    return math.max(math.min(val, 1.0), -1.0)
end
```

Framebuffer

Framebuffer · revision 15908 · last edited 2023-07-18

On this page

- Constructors
- Functions
- Supertypes
- Examples
  - sample from the forum
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to Canvas.

A Framebuffer is used for off-screen rendering. Think of it as an invisible screen that you can draw to, but that will not be visible until you draw it to the actual visible screen. It is also known as "render to texture".

By drawing things that do not change position often (such as background items) to the Framebuffer, and then drawing the entire Framebuffer instead of each item, you can reduce the number of draw operations performed each frame.

Constructors

love.graphics.newFramebuffer	Creates a new Framebuffer.	Added since 0.7.0	Removed in 0.8.0

Functions

Framebuffer:getImageData	Get stored ImageData.	Added since 0.7.0	Removed in 0.8.0
Framebuffer:getWrap	Gets the wrapping properties of a Framebuffer.	Added since 0.7.2	Removed in 0.8.0
Framebuffer:renderTo	Render to a framebuffer using a function.	Added since 0.7.0	Removed in 0.8.0
Framebuffer:setWrap	Sets the wrapping properties of a Framebuffer.	Added since 0.7.2	Removed in 0.8.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		

Supertypes

- Drawable
- Object

Examples

sample from the forum

http://love2d.org/forums/viewtopic.php?f=4&t=2136&hilit=Framebuffer&start=20

See Also

- love.graphics
- love.graphics.newFramebuffer
- love.graphics.setRenderTarget

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Framebuffer:getImageData

Framebuffer:getImageData · revision 28644 · last edited 2023-07-18

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to Canvas:getImageData.

Returns the image data stored in the Framebuffer. Think of it as a screenshot of the hidden screen that is the framebuffer.

Function

Synopsis

```
data = Framebuffer:getImageData( )
```

Arguments

None.

Returns

`ImageData data`

The image data stored in the framebuffer.

See Also

- Framebuffer
- love.graphics.newScreenshot

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Framebuffer:getWrap

Framebuffer:getWrap · revision 28645 · last edited 2023-07-18

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.2 and removed in LÖVE 0.8.0

It has been renamed to Canvas:getWrap.

Gets the wrapping properties of a Framebuffer.

This functions returns the currently set horizontal and vertical wrapping modes for the framebuffer.

Function

Synopsis

```
horiz, vert = Framebuffer:getWrap( )
```

Arguments

None

Returns

`WrapMode horiz`
: Horizontal wrapping mode of the Framebuffer.

`WrapMode vert`
: Vertical wrapping mode of the Framebuffer.

See Also

- Framebuffer
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Framebuffer:renderTo

Framebuffer:renderTo · revision 28646 · last edited 2023-07-18

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Examples
  - Using an anonymous function for drawing to a Framebuffer
- See Also
- Other Languages

Available since LÖVE 0.7.0 and removed in LÖVE 0.8.0

It has been renamed to Canvas:renderTo.

Render to the Framebuffer using a function.

Function

Synopsis

```
Framebuffer:renderTo( func )
```

Arguments

`function func`

A function performing drawing operations.

Returns

Nothing.

Notes

This function will clear the target framebuffer.

This is a shortcut to love.graphics.setRenderTarget:

```
framebuffer:renderTo( func )
```

is the same as

```
love.graphics.setRenderTarget( framebuffer )
func()
love.graphics.setRenderTarget()
```

Examples

Using an anonymous function for drawing to a Framebuffer

```
framebuffer:renderTo(function()
    love.graphics.draw(image1, 0,0)
    love.graphics.draw(image2, 100,100)
end)
```

See Also

- Framebuffer

- love.graphics.setRenderTarget

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Framebuffer:setWrap

Framebuffer:setWrap · revision 28647 · last edited 2023-07-18

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.2 and removed in LÖVE 0.8.0

It has been renamed to Canvas:setWrap.

Sets the wrapping properties of a Framebuffer.

This function sets the way the edges of a Framebuffer are treated if it is scaled or rotated. If the WrapMode is set to 'clamp', the edge will not be interpolated. If set to 'repeat', the edge will be interpolated with the pixels on the opposing side of the framebuffer.

Function

Synopsis

```
Framebuffer:setWrap( horiz, vert )
```

Arguments

`WrapMode horiz`
: Horizontal wrapping mode of the framebuffer.

`WrapMode vert`
: Vertical wrapping mode of the framebuffer.

Returns

Nothing.

See Also

- Framebuffer
- WrapMode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Framework

Framework · revision 25320 · last edited 2023-07-05

Framework Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
HUMP	Gamedev-Powertools: Gamestates, timed function calls, tweening, vectors, object orientation, cameras and signals		Any	
HooECS	It's a full-featured Entity-Component-System framework for making games with Lua		Any	
tiny-ecs	Entity Component System for lua.		Any	
Zoetrope	A starter kit for LÖVE, including sprites, collisions, maps, and debugging.		0.9.x	
Pölygamy	Game State, Keyboard and Timer helpers		0.6.x	
RotLove	The Roguelike Toolkit for Love2D. Based on rot.js and libtcod.		0.6.2+	

FrictionJoint

FrictionJoint · revision 10232 · last edited 2023-07-05

Available since LÖVE 0.8.0

This type is not supported in earlier versions.

A FrictionJoint applies friction to a body.

Constructors

love.physics.newFrictionJoint	A FrictionJoint applies friction to a body.	Added since 0.8.0

Functions

FrictionJoint:getMaxForce	Gets the maximum friction force in Newtons.	Added since 0.8.0	
FrictionJoint:getMaxTorque	Gets the maximum friction torque in Newton-meters.	Added since 0.8.0	
FrictionJoint:setMaxForce	Sets the maximum friction force in Newtons.	Added since 0.8.0	
FrictionJoint:setMaxTorque	Sets the maximum friction torque in Newton-meters.	Added since 0.8.0	
Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FrictionJoint:getMaxForce

FrictionJoint:getMaxForce · revision 7131 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the maximum friction force in Newtons.

Function

Synopsis

```
force = FrictionJoint:getMaxForce( )
```

Arguments

None.

Returns

`number force`

Maximum force in Newtons.

See Also

- FrictionJoint
- FrictionJoint:setMaxForce

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FrictionJoint:getMaxTorque

FrictionJoint:getMaxTorque · revision 30489 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Gets the maximum friction torque in Newton-meters.

Function

Synopsis

```
torque = FrictionJoint:getMaxTorque( )
```

Arguments

None.

Returns

`number torque`

Maximum torque in Newton-meters.

See Also

- FrictionJoint
- FrictionJoint:setMaxTorque

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FrictionJoint:setMaxForce

FrictionJoint:setMaxForce · revision 7130 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the maximum friction force in Newtons.

Function

Synopsis

```
FrictionJoint:setMaxForce( force )
```

Arguments

`number maxForce`
: Max force in Newtons.

Returns

Nothing.

See Also

- FrictionJoint
- FrictionJoint:getMaxForce

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FrictionJoint:setMaxTorque

FrictionJoint:setMaxTorque · revision 7129 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.8.0

This function is not supported in earlier versions.

Sets the maximum friction torque in Newton-meters.

Function

Synopsis

```
FrictionJoint:setMaxTorque( torque )
```

Arguments

`number torque`
: Maximum torque in Newton-meters.

Returns

Nothing.

See Also

- FrictionJoint
- FrictionJoint:getMaxTorque

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

FullscreenType

FullscreenType · revision 16415 · last edited 2025-05-13

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

Types of fullscreen modes.

Constants

desktop
: Sometimes known as borderless fullscreen windowed (http://pcgamingwiki.com/wiki/Borderless_fullscreen_windowed) mode. A borderless screen-sized window is created which sits on top of all desktop UI elements. The window is automatically resized to match the dimensions of the desktop, and its size cannot be changed.

Available since LÖVE 0.10.0

This constant has been renamed from **normal**

exclusive
: Standard exclusive-fullscreen mode. Changes the display mode (actual resolution) of the monitor.

Removed in LÖVE 0.10.0

This constant has been renamed to **exclusive**

normal
: Standard exclusive-fullscreen mode. Changes the display mode (actual resolution) of the monitor.

Notes

In exclusive fullscreen mode, if a window size is used which does not match one of the monitor's supported display modes, the window will be resized to the next largest display mode.

Exclusive fullscreen mode is sometimes avoided by users because it can cause issues in some window managers and with multi-monitor setups. In OS X it prevents switching to a different program until fullscreen mode is exited. The "desktop" fullscreen mode generally avoids these issues.

See Also

- love.window
- love.window.setMode
- love.window.setFullscreen
- love.resize
- Config Files

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GamepadAxis

GamepadAxis · revision 27214 · last edited 2023-07-05

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Virtual gamepad axes.

Constants

leftx
: The x-axis of the left thumbstick.

lefty
: The y-axis of the left thumbstick.

rightx
: The x-axis of the right thumbstick.

righty
: The y-axis of the right thumbstick.

triggerleft
: Left analog trigger.

triggerright
: Right analog trigger.

Notes

The physical locations of the virtual gamepad axes for a Joystick correspond as closely as possible to the layout of the axes on a standard Xbox 360 controller.



See Also

- GamepadButton

- love.joystick
- Joystick
- Joystick:getGamepadAxis

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GamepadButton

GamepadButton · revision 28105 · last edited 2023-07-05

Available since LÖVE 0.9.0

This enum is not supported in earlier versions.

Virtual gamepad buttons.

Constants

a
: Bottom face button (A).

b
: Right face button (B).

x
: Left face button (X).

y
: Top face button (Y).

back
: Back button.

guide
: Guide button.

start
: Start button.

leftstick
: Left stick click button.

rightstick
: Right stick click button.

leftshoulder
: Left bumper.

rightshoulder
: Right bumper.

dpup
: D-pad up.

dpdown
: D-pad down.

dpleft
: D-pad left.

dpright
: D-pad right.

Available since LÖVE 12.0

misc1
: Xbox Series X controller share button, PS5 controller mic button, and Switch Pro controller capture button.

paddle1
: First paddle button.

paddle2
: Second paddle button.

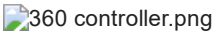
paddle3
: Third paddle button.

paddle4
: Fourth paddle button.

touchpad
: Controller touchpad press.

Notes

The physical locations of the virtual gamepad buttons for a Joystick correspond as closely as possible to the layout of the buttons on a standard Xbox 360 controller.



See Also

- GamepadAxis

- love.joystick
- Joystick
- Joystick:isGamepadDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GamepadType

GamepadType · revision 28107 · last edited 2023-07-21

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Types of gamepad controllers.

Constants

unknown
: An unrecognized gamepad type.

xbox360
: Xbox 360 controller.

xboxone
: Xbox One controller.

ps3
: PS3 (Dualshock 3) controller.

ps4
: PS4 (Dualshock 4) controller.

ps5
: PS5 (Dualsense) controller.

switchpro
: Switch Pro controller.

amazonluna
: Amazon Luna controller.

stadia
: Stadia controller.

virtual
: Software-based gamepad whose state is set programmatically.

shield
: nvidia Shield controller.

joyconleft
: Left-hand joycon.

joyconright
: Right-hand joycon.

joyconpair
: Both left- and right-hand joycons together.

See Also

- love.joystick
- Joystick
- Joystick:getGamepadType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Gangrene

Gangrene · revision 6741 · last edited 2021-03-14

Gangrene is just a prototype at the moment, even if its playable. So try to cut it some slack for not being finished, or not having polished and attractive art / UI.



There was a plague. We didn't know it at the time, but that was the calm before the storm. The people that died are still around, you see, and thats a problem since they really seem to want us dead. As for the rest of us, the survivors...would that we could all get alone, but there's not enough food and supplies for all of us...which means that some of us have to starve, and we're all fighting to make sure its not us.

At the moment there's just 22 cards, in no particular order: *Bandages, Bloodlust, Craft Shells, Demo Charge, Drug Cocktail, Diverted Package, Diversion, Full-auto, Grenade, Grave Goods, Iodine, Manufacture, Plagueflies, Rush Job, Requisition, Raid, Six-shot, Salvage, Scavenge, Secret Cache, Vaccine, Weapons Stockpile*

I welcome comments, thoughts, (constructive) criticism, and of cause art. :3

Planned Improvements:

- Better AI(s)
- More cards, in distinct sets
- Customisable decks
- Load / Save game
- Pre-designed Scenarios

  Forum Thread & Download (http://love2d.org/forums/viewtopic.php?f=5&t=3305)

Geany

Geany · revision 21823 · last edited 2021-03-14

Geany (http://geany.org/) is an open source Lua text editor for Linux, Mac OS X and Windows.

This article covers how to optimize Geany for working with LÖVE.

Auto completion

See this post (https://love2d.org/forums/viewtopic.php?f=4&t=84657&p=222789#p217559) for instructions on loading auto completion tags into Geany.

Run hotkey

1. Make sure you have a .lua file open.
2. Open **Build/Set Build Commands**
3. Create a new **Lua command** with the label "run with love"
4. The command should be:
    - Linux: `love %p`, assuming you installed LÖVE.
    - Mac OS X: `/Applications/love.app/Contents/MacOS/love %p`, assuming you installed LÖVE in the applications folder.
    - Windows: `C:\Program Files\Love\love.exe %p`, assuming that's where you installed LÖVE.

Build script

1. Make sure you have a .lua file open.
2. Open **Build/Set Build Commands**
3. Create a new **Lua command** with the label "build .love file" and working directory `%p`
4. The command should be:
    - Linux: `ts=$(date +%s); zip -r ../${PWD##*/}_${ts}.love *`, assuming you have `zip` installed.
    - Mac OS X: `ts=$(date +%s); zip -r ../${PWD##*/}_${ts}.love *`

Snippets

See Customizing Geany for LOVE thread (https://love2d.org/forums/viewtopic.php?f=3&t=7009).

Sources

- http://wiki.geany.org/howtos/configurebuildmenu ↗
- https://love2d.org/forums/viewtopic.php?f=3&t=7009

GearJoint

GearJoint · revision 16461 · last edited 2023-07-05

Keeps bodies together in such a way that they act like gears.



Constructors

love.physics.newGearJoint	Create a GearJoint connecting two Joints.

Functions

GearJoint:getJoints	Get the Joints connected by this GearJoint.	Added since 0.9.2	
GearJoint:getRatio	Get the ratio of a gear joint.		
GearJoint:setRatio	Set the ratio of a gear joint.		
Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	

Supertypes

- Joint
- Object

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GearJoint:getJoints

GearJoint:getJoints · revision 14656 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Get the Joints connected by this GearJoint.

Function

Synopsis

```
joint1, joint2 = GearJoint:getJoints( )
```

Arguments

None.

Returns

`Joint joint1`
: The first connected Joint.

`Joint joint2`
: The second connected Joint.

See Also

- GearJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GearJoint:getRatio

GearJoint:getRatio · revision 6076 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Get the ratio of a gear joint.

Function

Synopsis

```
ratio = GearJoint:getRatio( )
```

Arguments

None.

Returns

number ratio
: The ratio of the joint.

See Also

- GearJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GearJoint:setRatio

GearJoint:setRatio · revision 6077 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Set the ratio of a gear joint.

Function

Synopsis

```
GearJoint:setRatio( ratio )
```

Arguments

`number ratio`
: The new ratio of the joint.

Returns

Nothing.

See Also

- GearJoint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GIMP

GIMP · revision 17276 · last edited 2021-03-14

GIMP is an acronym for GNU Image Manipulation Program. It is a freely distributed program for such tasks as photo retouching, image composition and image authoring.

It has many capabilities. It can be used as a simple paint program, an expert quality photo retouching program, an online batch processing system, a mass production image renderer, an image format converter, etc.

GIMP is expandable and extensible. It is designed to be augmented with plug-ins and extensions to do just about anything. The advanced scripting interface allows everything from the simplest task to the most complex image manipulation procedures to be easily scripted.

GIMP is written and developed under X11 on UNIX platforms. But basically the same code also runs on MS Windows and Mac OS X.

GIMP Website (http://www.gimp.org/)

Git

Git · revision 26791 · last edited 2021-07-21

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

GlyphData

GlyphData · revision 10234 · last edited 2023-07-05

Available since LÖVE 0.7.0
This type is not supported in earlier versions.

A GlyphData represents a drawable symbol of a font Rasterizer.

Constructors

Rasterizer:getGlyphData	Gets glyph data of a specified glyph.	Added since 0.7.0
love.font.newGlyphData	Creates a new GlyphData.	Added since 0.7.0

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
GlyphData:getAdvance	Gets glyph advance.	Added since 0.7.0
GlyphData:getBearing	Gets glyph bearing.	Added since 0.7.0
GlyphData:getBoundingBox	Gets glyph bounding box.	Added since 0.7.0
GlyphData:getDimensions	Gets glyph dimensions.	Added since 0.7.0
GlyphData:getFormat	Gets glyph pixel format.	Added since 11.0
GlyphData:getGlyph	Gets glyph number.	Added since 0.7.0
GlyphData:getGlyphString	Gets glyph string.	Added since 0.7.0
GlyphData:getHeight	Gets glyph height.	Added since 0.7.0
GlyphData:getWidth	Gets glyph width.	Added since 0.7.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Supertypes

- Data
- Object

See Also

- love.font

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getAdvance

GlyphData:getAdvance · revision 25267 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph advance.

Function

Synopsis

```
advance = GlyphData:getAdvance()
```

Arguments

None.

Returns

`number advance`
Glyph advance.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getBearing

GlyphData:getBearing · revision 25268 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph bearing.

Function

Synopsis

```
bx, by = GlyphData:getBearing()
```

Arguments

None.

Returns

number bx
: Glyph bearing X.

number by
: Glyph bearing Y.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getBoundingBox

GlyphData:getBoundingBox · revision 25269 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph bounding box.

Function

Synopsis

```
x, y, width, height = GlyphData:getBoundingBox()
```

Arguments

None.

Returns

number x
: Glyph position x.

number y
: Glyph position y.

number width
: Glyph width.

number height
: Glyph height.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getDimensions

GlyphData:getDimensions · revision 25263 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph dimensions.

Function

Synopsis

```
width, height = GlyphData:getDimensions()
```

Arguments

None.

Returns

number width
: Glyph width.

number height
: Glyph height.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getFormat

GlyphData:getFormat · revision 25273 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets glyph pixel format.

Function

Synopsis

```
format = GlyphData:getFormat()
```

Arguments

None.

Returns

`PixelFormat format`

Glyph pixel format.

See Also

- GlyphData
- PixelFormat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getGlyph

GlyphData:getGlyph · revision 25265 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph number.

Function

Synopsis

```
glyph = GlyphData:getGlyph()
```

Arguments

None.

Returns

`number glyph`

Glyph number.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getGlyphString

GlyphData:getGlyphString · revision 25266 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph string.

Function

Synopsis

```
glyph = GlyphData:getGlyphString()
```

Arguments

None.

Returns

`string glyph`

Glyph string.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getHeight

GlyphData:getHeight · revision 25262 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph height.

Function

Synopsis

```
height = GlyphData:getHeight()
```

Arguments

None.

Returns

`number height`

Glyph height.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GlyphData:getWidth

GlyphData:getWidth · revision 25261 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.7.0

This function is not supported in earlier versions.

Gets glyph width.

Function

Synopsis

```
width = GlyphData:getWidth()
```

Arguments

None.

Returns

`number width`
: Glyph width.

See Also

- GlyphData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Goo

Goo · revision 16301 · last edited 2023-07-05


O.png

Please note that this lib...

- Has known issues, including stubbed functionality

Goo is a GUI lib licensed under public domain license. It includes an animation lib for tweening variables.

It is currently a work in progress, but should work for the most part.

Links

- Original Thread (http://love2d.org/forums/viewtopic.php?f=5&t=1523)
- Git Repository (http://github.com/perky/Goo)
- Documentation (http://locofilm.co.uk/love/goodocs/index.html)

Gradients

Gradients · revision 29163 · last edited 2024-04-24

On this page

- gradient (Mesh variant)
  - Synopsis
  - Arguments
  - Returns
  - Source
  - Notes
- gradient (Image variant)
  - Synopsis
  - Source
- drawinrect
  - Synopsis
  - Source
- Examples
  - drawing horizontal rainbow that fills the entire screen using Mesh variant
  - draw a 100x100 horizontal rainbow (that is, red on top, purple on bottom) in the center of the screen
  - draw a vertical greyscale spectrum (that is, black on left, white on right) filling the whole screen

gradient (Mesh variant)

Synopsis

```
mesh = gradientMesh( direction, color1, color2, ... )
```

Creates a gradient object (actually Mesh) from a table of colors to be evenly spaced throughout the gradient. This is better version of gradient function below.

Arguments

`string direction`
: Gradient direction (**horizontal** or **vertical**).

`table color1`
: First color table.

`table color2`
: Second color table.

`table ...`
: Additional color table.

Returns

`Mesh mesh`
: The gradient object.

Source

```lua
-- Color multipler
local COLOR_MUL = love._version >= "11.0" and 1 or 255

function gradientMesh(dir, ...)
    -- Check for direction
    local isHorizontal = true
    if dir == "vertical" then
        isHorizontal = false
    elseif dir ~= "horizontal" then
        error("bad argument #1 to 'gradient' (invalid value)", 2)
    end

    -- Check for colors
    local colorLen = select("#", ...)
    if colorLen < 2 then
        error("color list is less than two", 2)
    end

    -- Generate mesh
    local meshData = {}
    if isHorizontal then
        for i = 1, colorLen do
            local color = select(i, ...)
            local x = (i - 1) / (colorLen - 1)
```

```
            meshData[#meshData + 1] = {x, 1, x, 1, color[1], color[2], color[3], color[4] or (1 * COLOR_MUL)}
            meshData[#meshData + 1] = {x, 0, x, 0, color[1], color[2], color[3], color[4] or (1 * COLOR_MUL)}
        end
    else
        for i = 1, colorLen do
            local color = select(i, ...)
            local y = (i - 1) / (colorLen - 1)

            meshData[#meshData + 1] = {1, y, 1, y, color[1], color[2], color[3], color[4] or (1 * COLOR_MUL)}
            meshData[#meshData + 1] = {0, y, 0, y, color[1], color[2], color[3], color[4] or (1 * COLOR_MUL)}
        end
    end

    -- Resulting Mesh has 1x1 image size
    return love.graphics.newMesh(meshData, "strip", "static")
end
```

Notes

The resulting Mesh has dimensions of 1x1. Scaling factor in love.graphics.draw determines the size of the gradient. drawinrect function below is not needed (can't be used really) when using this variant.

gradient (Image variant)

Synopsis

```
gradient {color1, color2, color3, ..., direction = 'horizontal' or 'vertical'}
```

Creates a gradient object (really just an Image) from a table of colors to be evenly spaced throughout the gradient, in the direction specified by the "direction" key of the parameter table.

Source

```
function gradient(colors)
    local direction = colors.direction or "horizontal"
    if direction == "horizontal" then
        direction = true
    elseif direction == "vertical" then
        direction = false
    else
        error("Invalid direction '" .. tostring(direction) .. "' for gradient.  Horizontal or vertical expected.")
    end
    local result = love.image.newImageData(direction and 1 or #colors, direction and #colors or 1)
    for i, color in ipairs(colors) do
        local x, y
        if direction then
            x, y = 0, i - 1
        else
            x, y = i - 1, 0
        end
        result:setPixel(x, y, color[1], color[2], color[3], color[4] or 255)
    end
    result = love.graphics.newImage(result)
    result:setFilter('linear', 'linear')
    return result
end
```

drawinrect

Synopsis

```
drawinrect(img, x, y, w, h, r, ox, oy, kx, ky)
```

A convenience function to draw scaled images in a rectangle of absolute size (rather than with a size relative to the size of the image, which is what love.graphics.draw() does). Useful for gradients, because you will almost always want to draw them scaled, and you don't want their bounds to change if the number of colors does.

Source

```
function drawinrect(img, x, y, w, h, r, ox, oy, kx, ky)
    return -- tail call for a little extra bit of efficiency
    love.graphics.draw(img, x, y, r, w / img:getWidth(), h / img:getHeight(), ox, oy, kx, ky)
end
```

Examples

drawing horizontal rainbow that fills the entire screen using Mesh variant

```
-- assume you already have gradientMesh function
local rainbow

function love.load()
    rainbow = gradientMesh("horizontal",
        {255, 0, 0},
        {255, 255, 0},
        {0, 255, 0},
        {0, 255, 255},
        {0, 0, 255},
        {255, 0, 0}
    )
end

function love.draw()
    love.graphics.draw(rainbow, 0, 0, 0, love.graphics.getDimensions())
end
```

draw a 100x100 horizontal rainbow (that is, red on top, purple on bottom) in the center of the screen

```
require "gradient"

local rainbow = gradient {
    direction = 'horizontal',
    {255, 0, 0},
    {255, 255, 0},
    {0, 255, 0},
    {0, 255, 255},
    {0, 0, 255},
    {255, 0, 0}
}

function love.draw()
    drawinrect(rainbow, love.graphics.getWidth() / 2 - 50, love.graphics.getHeight() / 2 - 50, 100, 100)
end
```

draw a vertical greyscale spectrum (that is, black on left, white on right) filling the whole screen

```
require "gradient"

local greyscale = gradient {
    direction = 'vertical',
    {0, 0, 0},
    {255, 255, 255}
}

function love.draw()
    drawinrect(greyscale, 0, 0, love.graphics.getWidth(), love.graphics.getHeight())
end
```

GraphicsBuffer

GraphicsBuffer · revision 29104 · last edited 2026-01-07

On this page

- Constructors
- Functions
- Enums
- Supertypes
- See Also
- Other Languages

Available since LÖVE 12.0

This type is not supported in earlier versions.

Low-level data stored in graphics memory, including arrays of vertices, vertex indices, and custom collections of data accessible in Shaders.

Higher level graphics objects such as Meshes, SpriteBatches, TextBatches, and ParticleSystems make use of GraphicsBuffers internally.

Constructors

love.graphics.newBuffer	Creates a new GraphicsBuffer.	Added since 12.0

Functions

GraphicsBuffer:clear	Clears the entire GraphicsBuffer or a specified byte range within it to zero.	Added since 12.0
GraphicsBuffer:getElementCount	Gets the total number of array elements in this GraphicsBuffer.	Added since 12.0
GraphicsBuffer:getElementStride	Gets the size in bytes used by one array element of this GraphicsBuffer.	Added since 12.0
GraphicsBuffer:getSize	Gets the total size in bytes of the GraphicsBuffer's contents.	Added since 12.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

BufferDataFormat	Data formats of individual array elements GraphicsBuffers and Meshes.	Added since 12.0
BufferDataUsage	Usage hints for SpriteBatches, Meshes, and GraphicsBuffers to optimize data storage and access.	Added since 0.8.0

Supertypes

- Object

See Also

- love.graphics
- Mesh
- SpriteBatch
- Shader

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsBuffer:clear

GraphicsBuffer:clear · revision 28280 · last edited 2026-08-08

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Clears the entire GraphicsBuffer or a specified byte range within it to zero.

This can be more efficient than calling GraphicsBuffer:setArrayData when setting the buffer's contents to zero is desired.

Note that the array element count variants of love.graphics.newBuffer and friends will initialize the buffer's contents to zero when it's first created.

Function

Clears the entire GraphicsBuffer's contents to zero.

Synopsis

```
GraphicsBuffer:clear( )
```

Arguments

None.

Returns

Nothing.

Function

Clears a specified byte range within the GraphicsBuffer to zero.

Synopsis

```
GraphicsBuffer:clear( offset, size )
```

Arguments

`number offset`
: The offset in bytes within the Buffer.

`number size`
: The size in bytes to clear to zero.

Returns

Nothing.

Notes

The offset and size parameters must be multiples of 4.

See Also

- GraphicsBuffer
- GraphicsBuffer:getSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsBuffer:getElementCount

GraphicsBuffer:getElementCount · revision 28275 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the total number of array elements in this GraphicsBuffer.

Function

Synopsis

```
count = GraphicsBuffer:getElementCount( )
```

Arguments

None.

Returns

`number count`

The total number of array elements in this Buffer.

See Also

- GraphicsBuffer
- GraphicsBuffer:setArrayData
- GraphicsBuffer:getElementStride
- GraphicsBuffer:getSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsBuffer:getElementStride

GraphicsBuffer:getElementStride · revision 28281 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the size in bytes used by one array element of this GraphicsBuffer.

This may be larger than the sizes of individual array elements added together, due to internal alignment and padding requirements of different buffer usage modes.

Function

Synopsis

```
stride = GraphicsBuffer:getElementStride( )
```

Arguments

None.

Returns

`number stride`

The size in bytes used by one array element of this Buffer.

See Also

- GraphicsBuffer
- GraphicsBuffer:getFormat
- GraphicsBuffer:getElementCount
- GraphicsBuffer:getSize

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsBuffer:getSize

GraphicsBuffer:getSize · revision 28274 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the total size in bytes of the GraphicsBuffer's contents.

Function

Synopsis

```
size = GraphicsBuffer:getSize( )
```

Arguments

None.

Returns

`number size`

The total size in bytes of the Buffer's contents in VRAM.

See Also

- GraphicsBuffer
- GraphicsBuffer:getElementCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsFeature

GraphicsFeature · revision 23484 · last edited 2026-04-19

On this page

- Constants
  - Notes
  - Notes
- Notes
- See Also
- Other Languages

Available since LÖVE 0.8.0

This enum is not supported in earlier versions.

Graphics features that can be checked for with love.graphics.getSupported.

Constants

Available since LÖVE 0.10.0

clampzero
: Whether the "clampzero" WrapMode is supported.

lighten
: Whether the "lighten" and "darken" BlendModes are supported.

multicanvasformats
: Whether multiple Canvases with different formats can be used in the same love.graphics.setCanvas call.

Notes

`clampzero` is supported on all desktop systems, but only some mobile devices. If it's not supported and it's attempted to be set, the "clamp" wrap mode will automatically be used instead.

`lighten` is supported on all desktop systems, and OpenGL ES 3-capable mobile devices.

`multicanvasformats` is supported on OpenGL 3-capable desktop systems, and OpenGL ES 3-capable mobile devices.

Available since LÖVE 11.0

glsl3
: Whether GLSL 3 Shaders can be used.

instancing
: Whether mesh instancing is supported.

fullnpot
: Whether textures with non-power-of-two dimensions can use mipmapping and the 'repeat' WrapMode.

pixelshaderhighp
: Whether pixel shaders can use "highp" 32 bit floating point numbers (as opposed to just 16 bit or lower precision).

shaderderivatives
: Whether shaders can use the `dFdx`, `dFdy`, and `fwidth` functions for computing derivatives.

Notes

`glsl3` and `instancing` are supported on OpenGL 3 and OpenGL ES 3-capable systems.

`instancing` is also supported on some older systems that don't support GLSL 3, but vertex attribute-based instancing must be used in that case (instead of `love_InstanceID` in a GLSL 3 shader).

`fullnpot`, `pixelshaderhighp`, and `shaderderivatives` are supported on all desktop systems and most mobile systems, except for some older OpenGL ES 2 devices.

Removed in LÖVE 0.10.0

canvas
: Support for Canvas.

npot
: Support for textures with non-power-of-two sizes. See PO2 Syndrome.

subtractive
: Support for the subtractive blend mode.

Available since LÖVE 0.9.0 and removed in LÖVE 0.10.0

shader
: Support for Shaders.

hdrcanvas
: Support for HDR Canvases. Use love.graphics.getCanvasFormats instead.

multicanvas
: Support for simultaneous rendering to at least 4 canvases at once, with love.graphics.setCanvas. Use love.graphics.getSystemLimits instead.

mipmap
: Support for Mipmaps.

dxt
: Support for DXT compressed images (see CompressedFormat.) Use love.graphics.getCompressedImageFormats instead.

bc5
: Support for BC4 and BC5 compressed images. Use love.graphics.getCompressedImageFormats instead.

Available since LÖVE 0.9.1 and removed in LÖVE 0.10.0

Use love.graphics.isGammaCorrect or love.graphics.getCanvasFormats instead

srgb
: Support for gamma-correct rendering with the `srgb` window flag in love.window.setMode, and the "srgb" TextureFormat for Canvases and Images.

Removed in LÖVE 0.9.0

pixeleffect
: Support for PixelEffects.

Notes

Canvases, Shaders, mipmaps, npot textures, and the subtract BlendMode are always supported in version 0.10.0 and newer (due to its system requirements.)

For previous versions:

`subtractive`, `mipmap`, and `dxt` are supported on nearly every system.

`canvas`, `multicanvas`, `npot`, and `shader` have roughly the same minimum requirements for support: a DirectX 9.0c+ capable graphics card with drivers that support ~OpenGL 2.1+.

`hdrcanvas`, `bc5`, and `srgb` all share mostly the same minimum requirements for support as well: a DirectX 10+ capable graphics card with drivers that support ~OpenGL 3+.

DirectX 9.0c+ capable graphics cards include the nvidia GeForce 5000-series (2003) and newer, the ATI Radeon 9000-series and newer, and the Intel GMA x3100 GPU and newer.

DirectX 10+ capable graphics cards include the nvidia GeForce 8000-series (2006) and newer, the ATI/AMD HD 2000-series and newer, and the Intel HD 2000/3000 GPUs and newer.

See Also

- love.graphics
- love.graphics.getSupported

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

GraphicsGale

GraphicsGale · revision 30603 · last edited 2026-02-07

GraphicsGale is a simple-to-use yet powerful and feature-rich application that allows to edit and create animations and other types of graphics.

GraphicsGale Website (https://graphicsgale.com/us/)

GraphicsLimit

GraphicsLimit · revision 22295 · last edited 2026-04-09

Available since LÖVE 0.9.1

This enum is not supported in earlier versions.

Types of system-dependent graphics limits checked for using love.graphics.getSystemLimits.

Constants

pointsize
: The maximum size of points.

texturesize
: The maximum width or height of Images and Canvases.

multicanvas
: The maximum number of simultaneously active canvases (via love.graphics.setCanvas.)

Available since LÖVE 0.10.0

canvasmsaa
: The maximum number of antialiasing samples for a Canvas.

Available since LÖVE 11.0

texturelayers
: The maximum number of layers in an Array texture.

volumetexturesize
: The maximum width, height, or depth of a Volume texture.

cubetexturesize
: The maximum width or height of a Cubemap texture.

anisotropy
: The maximum amount of anisotropic filtering. Texture:setMipmapFilter internally clamps the given anisotropy value to the system's limit.

Removed in LÖVE 0.10.0

canvasfsaa
: The maximum number of antialiasing samples for a Canvas.

Notes

Attempting to create an Image with a width **or** height greater than the maximum supported will create a checkerboard-patterned image instead. Doing the same for a Canvas will result in an error.

It's safe to assume the maximum texture size will always be 2048 or greater. There is an online database (http://feedback.wildfiregames.com/report/opengl/feature/GL_MAX_TEXTURE_SIZE) which has collected info about the max texture size for various systems.

The value for the **multicanvas** system limit will generally be either 1, 4, or 8.

See Also

- love.graphics
- love.graphics.getSystemLimits

- Canvas:getMSAA

Other Languages

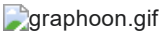
Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Graphoon

Graphoon · revision 16112 · last edited 2023-07-05

Graphoon

Graphoon is a force directed graph algorithm written in Lua.



It emerged from the graph calculation code used in both LoGiVi (https://github.com/rm-code/logivi) and LoFiVi (https://github.com/rm-code/lofivi). A force directed graph layout is achieved by simulating physical forces, which push and pull each node in the graph until a nice layout is found.

Grab it on github (https://github.com/rm-code/Graphoon).

Demo is available here (https://github.com/rm-code/Graphoon/releases/dow nload/1.0.1/demo.love) (The demo requires LÖVE 0.10.0).

Grease

Grease · revision 26554 · last edited 2023-07-05

Grease is a networking library for Lua, using LuaSocket, and optionally Lua-ENet. It was designed to operate within the LÖVE framework, but also works without it.

- Source on github (https://github.com/bartbes/love-misc-libs/tree/master/grease).
- Documentation here (https://github.com/bartbes/love-misc-libs/blob/master/grease/docs.md).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Gwee

Gwee · revision 9069 · last edited 2021-03-14

Gwee is no longer actively developed.

HashFunction

HashFunction · revision 21340 · last edited 2023-07-05

Available since LÖVE 11.0

This enum is not supported in earlier versions.

Hash algorithm of love.data.hash.

Constants

md5

MD5 hash algorithm (16 bytes).

sha1

SHA1 hash algorithm (20 bytes).

sha224

SHA2 hash algorithm with message digest size of 224 bits (28 bytes).

sha256

SHA2 hash algorithm with message digest size of 256 bits (32 bytes).

sha384

SHA2 hash algorithm with message digest size of 384 bits (48 bytes).

sha512

SHA2 hash algorithm with message digest size of 512 bits (64 bytes).

See Also

- love.data
- love.data.hash

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

HC

HC · revision 19223 · last edited 2023-07-05

Collision detection for LÖVE

HC is a pure Lua library to simplify collision detection.

It supports a variety of scale- and rotatable shapes: Rectangles, (non self-intersecting) polygons, circles and points.

In contrast to love.physics, HC will only detect collisions, but not resolve them - you can decide how you want to respond to a collision.

If you have trouble using love.physics for collision detection or if you feel the ten-ton hammer is too much, HC might be the right choice.

Documentation and sourcecode

The documentation can be found online here: http://hc.readthedocs.org/ ↗

You can view the sourcecode online here: https://github.com/vrld/HC ↗

HintingMode

HintingMode · revision 23981 · last edited 2025-10-14

Available since LÖVE 0.10.0

This enum is not supported in earlier versions.

True Type hinting mode.

Constants

normal

Default hinting. Should be preferred for typical antialiased fonts.

light

Results in fuzzier text but can sometimes preserve the original glyph shapes of the text better than normal hinting.

mono

Results in aliased / unsmoothed text with either full opacity or completely transparent pixels. Should be used when antialiasing is not desired for the font.

none

Disables hinting for the font. Results in fuzzier text.

See Also

- love.font
- love.font.newRasterizer
- love.font.newTrueTypeRasterizer
- love.graphics.newFont

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

HooECS

HooECS · revision 24549 · last edited 2023-07-05

HooECS is an Entity Component System framework for game development with Lua.

Originally written for the LÖVE 2D game engine, it is actually compatible with pretty much any game that uses lua! It is inspired by Richard Lords Introduction (http://w w w.richardlord.net/blog/w hat-is-an-entity-framew ork) to ECS.

If you don't have any idea what this entity component stuff is all about, click that link and give it a read! It's totally worth it!

This is a fork that's been improved upon.

The original software was written by Arne Beer and Rafael Epplee under the name lovetoys (https://love2d.org/wiki/Lovetoys).

For the foreseeable future. All lovetoys projects will stay 100% compatible with HooECS. The unit tests of lovetoys are merely expanded upon.

No core behavior is changed!

Links

- Github Repository (https://github.com/Hooodini/HooECS)
- Forum Thread (https://love2d.org/forums/viewtopic.php?f=5&t=84471)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

HS2

HS2 · revision 24559 · last edited 2023-07-05

HS2 is a simple and awesome high score table library. Features an iterator (just like the one from SICK) and support for multiple simultaneous high score tables.

Get it on GitHub: https://github.com/tesselode/HS2 ↗

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

HUDebug

HUDebug · revision 13353 · last edited 2023-07-05

lövely and quick onscreen debugging

Download and info: https://github.com/nunodonato/hudebug ↗

HUMP

HUMP · revision 25214 · last edited 2023-07-05

Helper Utilities for a Multitude of Problems is a set of lightweight yet mighty useful tools to build your game. It will help to get you over the hump.

Hump differs from other libraries in that every component is independent of the remaining ones. Hump's footprint is very small and it should fit nicely into your projects.

Features

- **gamestate** system for decoupling different states/phases of the game (e.g. intro, menu, level, ...),
- **timer** system for calling functions after or for some amount of time, and tweening,
- **vector** class for handling 2D math in an elegant way,
- simple but fast **class** system,
- simple yet powerful **signal/slot** implementation, and
- move-, zoom- and rotatable **camera** with locking and movement smoothing.

Links

- Documentation (http://hump.readthedocs.org)
- Source code (original GitHub repo. - seems to be abandoned) (https://github.com/vrld/hump)
- Source code (https://github.com/HDictus/hump/tree/temp-master)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=1795)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Image

Image · revision 28182 · last edited 2026-04-11

Drawable image type.

Constructors

love.graphics.newArrayImage	Creates a new array Image.	Added since 11.0
love.graphics.newCubeImage	Creates a new cubemap Image.	Added since 11.0
love.graphics.newImage	Creates a new Image.	
love.graphics.newVolumeImage	Creates a new volume Image.	Added since 11.0

Functions

These functions have parentheses in odd places. This is because the *Image:* namespace is reserved in Mediawiki.

(Image):getData	Gets the original ImageData or CompressedData used to create the Image.	Added since 0.9.0	Removed in 11.0
(Image):getFlags	Gets the flags used when the image was created.	Added since 0.10.0	Removed in 11.0
(Image):isCompressed	Gets whether the Image was created from CompressedData.	Added since 0.9.0	
(Image):isFormatLinear	Gets whether the Image was created with a the linear (non-gamma corrected) flag set to true.	Added since 11.0	
(Image):refresh	Reloads the Image's contents from the ImageData or CompressedData used to create the image.	Added since 0.9.0	Removed in 11.0
(Image):replacePixels	Replace the contents of an Image.	Added since 11.0	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		
Texture:getDPIScale	Gets the DPI scale factor of the Texture.	Added since 11.0	
Texture:getDepth	Gets the depth of a Volume Texture.	Added since 11.0	
Texture:getDepthSampleMode	Gets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0	
Texture:getDimensions	Gets the width and height of the Texture.	Added since 0.9.0	
Texture:getFilter	Gets the filter mode of the Texture.		
Texture:getFormat	Gets the pixel format of the Texture.	Added since 11.0	
Texture:getHeight	Gets the height of the Texture.		
Texture:getLayerCount	Gets the number of layers / slices in an Array Texture.	Added since 11.0	
Texture:getMipmapCount	Gets the number of mipmaps contained in the Texture.	Added since 11.0	
Texture:getMipmapFilter	Gets the mipmap filter mode for a Texture.	Added since 0.9.0	
Texture:getPixelDimensions	Gets the width and height in pixels of the Texture.	Added since 11.0	
Texture:getPixelHeight	Gets the height in pixels of the Texture.	Added since 11.0	
Texture:getPixelWidth	Gets the width in pixels of the Texture.	Added since 11.0	
Texture:getTextureType	Gets the type of the Texture.	Added since 11.0	
Texture:getWidth	Gets the width of the Texture.		
Texture:getWrap	Gets the wrapping properties of a Texture.		
Texture:isReadable	Gets whether the Texture can be drawn and sent to a Shader.	Added since 11.0	

Texture:setDepthSampleMode	Sets the comparison mode used when sampling from a depth texture in a shader.	Added since 11.0
Texture:setFilter	Sets the filter mode of the Texture.	
Texture:setMipmapFilter	Sets the mipmap filter mode for a Texture.	Added since 0.9.0
Texture:setWrap	Sets the wrapping properties of a Texture.	

Supertypes

- Texture
- Drawable
- Object

See Also

- ImageData
- love.graphics
- Canvas
- Image Formats
- GraphicsLimit

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData

ImageData · revision 29117 · last edited 2024-02-18

On this page

- Constructors
- Functions
- Enums
- Supertypes
- Examples
  - Load map from image
- See Also
- Other Languages

Raw (decoded) image data.

You can't draw ImageData directly to screen. See Image for that.

Constructors

Canvas:newImageData	Generates ImageData from the contents of the Canvas.	Added since 0.10.0
love.graphics.newScreenshot	Creates a screenshot and returns the ImageData.	Removed in 11.0
love.image.newImageData	Creates a new ImageData object.	

Functions

Data:clone	Creates a new copy of the Data object.	Added since 11.0
Data:getFFIPointer	Gets an FFI pointer to the Data.	Added since 11.3
Data:getPointer	Gets a pointer to the Data.	
Data:getSize	Gets the Data's size in bytes.	
Data:getString	Gets the full Data as a string.	Added since 0.9.0
ImageData:encode	Encodes the ImageData to a file format and optionally writes it to the save directory.	
ImageData:getDimensions	Gets the width and height of the ImageData in pixels.	Added since 0.9.0
ImageData:getFormat	Gets the pixel format of the ImageData.	Added since 11.0
ImageData:getHeight	Gets the height of the ImageData in pixels.	
ImageData:getPixel	Gets the color of a pixel.	
ImageData:getString	Gets the full ImageData as a string.	Removed in 0.9.0
ImageData:getWidth	Gets the width of the ImageData in pixels.	
ImageData:mapPixel	Transform an image by applying a function to every pixel.	
ImageData:paste	Paste into ImageData from another source ImageData.	
ImageData:setPixel	Sets the color of a pixel.	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	
Object:typeOf	Checks whether an object is of a certain type.	

Enums

ImageEncodeFormat	Image file formats supported by ImageData:encode.

Supertypes

- Data
- Object

Examples

Load map from image

(Note that this example creates a *lot* of tables.)

```
local filename      = "image.png"
local imageData     = love.image.newImageData(filename)
local width, height = imageData:getDimensions()
local map           = {}

for y = 1, height do
    map[y] = {}
    for x = 1, width do
        local r, g, b, a = imageData:getPixel(x-1, y-1)
        map[y][x]        = {r, g, b, a}
    end
end
```

See Also

- love.image
- Image

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:encode

ImageData:encode · revision 27876 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Draw a circle to a canvas and save it to a file
- User Notes
- See Also
- Other Languages

Encodes the ImageData to a file format and optionally writes it to the save directory.

Function

Available since LÖVE 0.10.0

This variant is not supported in earlier versions.

Synopsis

```
filedata = ImageData:encode( format, filename )
```

Arguments

`ImageEncodeFormat format`

The format to encode the image as.

`string filename (nil)`

The filename to write the file to. If nil, no file will be written but the FileData will still be returned.

Returns

`FileData filedata`

The encoded image as a new FileData object.

Function

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

Synopsis

```
ImageData:encode( outFile )
```

Arguments

`string outFile`

Name of a file to write encoded data to. The format will be automatically deduced from the file extension.

Returns

Nothing.

Function

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

This variant is not supported in earlier or later versions.

Synopsis

```
ImageData:encode( outFile, format )
```

Arguments

`string outFile`
: Name of a file to write encoded data to.

`ImageEncodeFormat format`
: The format to encode the image in.

Returns

Nothing.

Function

Removed in LÖVE 0.8.0
This variant is not supported in that and later versions.

Synopsis

```
data = ImageData:encode( format )
```

Arguments

`ImageEncodeFormat format`
: The format to encode the image in.

Returns

`Data data`
: The encoded image data.

Examples

Draw a circle to a canvas and save it to a file

```
function love.load()
    canvas = love.graphics.newCanvas(800, 600)

    love.graphics.setCanvas(canvas)
    love.graphics.circle("fill", 200,200, 100)
    love.graphics.setCanvas() -- The canvas must not be active when we call canvas:newImageData().

    local imagedata = canvas:newImageData()
    imagedata:encode("tga", "test.tga")
end

function love.draw()
    love.graphics.draw(canvas) -- Draw canvas on screen.
end
```

User Notes

- If you want the file to have an extension when saved add it in the file name. Example :

```
image:encode("png","aPngImage.png")
```

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getDimensions

ImageData:getDimensions · revision 21065 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the width and height of the ImageData in pixels.

Function

Synopsis

```
width, height = ImageData:getDimensions( )
```

Arguments

None.

Returns

`number width`
: The width of the ImageData in pixels.

`number height`
: The height of the ImageData in pixels.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getFormat

ImageData:getFormat · revision 22677 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.0

This function is not supported in earlier versions.

Gets the pixel format of the ImageData.

Function

Synopsis

```
format = ImageData:getFormat( )
```

Arguments

None.

Returns

`PixelFormat format`

The pixel format the ImageData was created with.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getHeight

ImageData:getHeight · revision 21064 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the height of the ImageData in pixels.

Function

Synopsis

```
height = ImageData:getHeight( )
```

Arguments

None.

Returns

number height
: The height of the ImageData in pixels.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getPixel

ImageData:getPixel · revision 22381 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Gets the color of a pixel at a specific position in the image.

Valid x and y values start at 0 and go up to image width and height minus 1. Non-integer values are floored.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.


Prior to 0.10.2, this function does not properly handle non-integer coordinates, and may produce an invalid result when non-integer values are passed.

Function

Synopsis

```
r, g, b, a = ImageData:getPixel( x, y )
```

Arguments

number x
: The position of the pixel on the x-axis.

number y
: The position of the pixel on the y-axis.

Returns

number r
: The red component (0-1).

number g
: The green component (0-1).

number b
: The blue component (0-1).

number a
: The alpha component (0-1).

Examples

When the mouse is clicked, reads the red, green, and blue value of the pixel under the mouse and uses it as the background color.

```
local imagedata = love.image.newImageData('path/to/Image.png')
local image     = love.graphics.newImage(imagedata)

function love.mousepressed(mx, my)
    if  0 <= mx and mx < image:getWidth()
    and 0 <= my and my < image:getHeight() then
        local r, g, b = imagedata:getPixel(mx, my)
        love.graphics.setBackgroundColor(r, g, b)
    end
end

function love.draw()
```

```
    love.graphics.draw(image, 0, 0)
end
```

See Also

- ImageData
- ImageData:setPixel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getString

ImageData:getString · revision 10684 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.9.0
It has been moved to Data:getString.

Gets the full ImageData as a string.

Function

Synopsis

```
pixels = ImageData:getString( )
```

Arguments

None.

Returns

`string pixels`
The raw data.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:getWidth

ImageData:getWidth · revision 21063 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets the width of the ImageData in pixels.

Function

Synopsis

```
width = ImageData:getWidth( )
```

Arguments

None.

Returns

number width
: The width of the ImageData in pixels.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:mapPixel

ImageData:mapPixel · revision 30475 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Brighten an image
  - Add colored stripes to an image
  - Tint grayscaled image
- See Also
- Other Languages

Transform an image by applying a function to every pixel.

This function is a higher-order function (https://en.wikipedia.org/wiki/Higher-order_function). It takes another function as a parameter, and calls it once for each pixel in the ImageData.

The passed function is called with six parameters for each pixel in turn. The parameters are numbers that represent the x and y coordinates of the pixel and its red, green, blue and alpha values. The function should return the new red, green, blue, and alpha values for that pixel.

```
function pixelFunction(x, y, r, g, b, a)
    -- template for defining your own pixel mapping function
    -- perform computations giving the new values for r, g, b and a
    -- ...
    return r, g, b, a
end
```

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

This function locks the ImageData until it is done, making it safe to use from multiple Threads, albeit without any performance gains.

Function

Synopsis

```
ImageData:mapPixel( pixelFunction, x, y, width, height )
```

Arguments

function pixelFunction
: Function to apply to every pixel.

Available since LÖVE 0.9.0

number x (0)
: The x-axis of the top-left corner of the area within the ImageData to apply the function to.

number y (0)
: The y-axis of the top-left corner of the area within the ImageData to apply the function to.

number width (ImageData:getWidth())
: The width of the area within the ImageData to apply the function to.

number height (ImageData:getHeight())
: The height of the area within the ImageData to apply the function to.

Returns

Nothing.

Examples

Brighten an image

```
local function brighten(x,y, r,g,b,a)
   r = math.min(r*3, 1)
```

```
   g = math.min(g*3, 1)
   b = math.min(b*3, 1)
   return r,g,b,a
end
imageData:mapPixel(brighten)
```

Add colored stripes to an image

```
local function stripey(x,y, r,g,b,a)
   r = math.min(r * math.sin(x*100)*2, 1)
   g = math.min(g * math.cos(x*150)*2, 1)
   b = math.min(b * math.sin(x* 50)*2, 1)
   return r,g,b,a
end
imageData:mapPixel(stripey)
```

(Source: https://web.archive.org/web/20150515111551/http://khason.net/blog/hlsl-pixel-shader-effects-tutorial/ ↗)

Tint grayscaled image

```
local tintColor = {r=0.6, g=0.5, b=0.3} -- Sepia tone.

local function tint(x,y, r,g,b,a)
    local gray = (r+g+b) / 3
    if gray < 0.5 then
        -- Between black and tintColor.
        r = 2 * gray*tintColor.r
        g = 2 * gray*tintColor.g
        b = 2 * gray*tintColor.b
    else
        -- Between tintColor and white.
        r = 2 * (gray + tintColor.r - gray*tintColor.r) - 1
        g = 2 * (gray + tintColor.g - gray*tintColor.g) - 1
        b = 2 * (gray + tintColor.b - gray*tintColor.b) - 1
    end
    return r,g,b,a
end

imageData:mapPixel(tint)
```

See Also

- ImageData
- ImageData:setPixel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:paste

ImageData:paste · revision 17809 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Paste into ImageData from another source ImageData.

Function

Synopsis

```
ImageData:paste( source, dx, dy, sx, sy, sw, sh )
```

Arguments

`ImageData source`
: Source ImageData from which to copy.

`number dx`
: Destination top-left position on x-axis.

`number dy`
: Destination top-left position on y-axis.

`number sx`
: Source top-left position on x-axis.

`number sy`
: Source top-left position on y-axis.

`number sw`
: Source width.

`number sh`
: Source height.

Returns

Nothing.

Notes

Note that this function just replaces the contents in the destination rectangle; it does not do any alpha blending.

See Also

- ImageData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageData:setPixel

ImageData:setPixel · revision 29008 · last edited 2026-04-24

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
- See Also
- Other Languages

Sets the color of a pixel at a specific position in the image.

Valid x and y values start at 0 and go up to image width and height minus 1.

In versions prior to 11.0, color component values were within the range of 0 to 255 instead of 0 to 1.

This function locks the ImageData until it is done, making it safe to use from multiple Threads, albeit without any performance gains.

Function

Synopsis

```
ImageData:setPixel( x, y, r, g, b, a )
```

Arguments

number x
: The position of the pixel on the x-axis.

number y
: The position of the pixel on the y-axis.

number r
: The red component (0-1).

number g
: The green component (0-1).

number b
: The blue component (0-1).

number a
: The alpha component (0-1).

Returns

Nothing.

Examples

Create a 32x1 pixel transparent-to-white gradient drawable image.

```
data = love.image.newImageData(32,1)
for i=0, 31 do   -- remember: start at 0
   data:setPixel(i, 0, 1, 1, 1, i / 31)
end
img = love.graphics.newImage(data)
```

See Also

- ImageData
- ImageData:getPixel
- ImageData:mapPixel

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageEncodeFormat

ImageEncodeFormat · revision 28282 · last edited 2026-04-24

Image file formats supported by ImageData:encode. See Image Formats for supported decode formats.

Constants

tga
: Targa image format.

Available since LÖVE 0.8.0

png
: PNG image format.

Available since LÖVE 12.0

exr
: EXR image format. Supports HDR data and other uses where the range of values can go outside of [0, 1].

Available since LÖVE 0.8.0 and removed in LÖVE 0.10.0

jpg
: JPG image format.

bmp
: BMP image format.

See Also

- love.image
- ImageData
- ImageData:encode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageFlag

ImageFlag · revision 25437 · last edited 2023-07-05

Image flags.

This needs more documentation!

Constants

linear
mipmaps

See Also

- love.graphics
- Image:getFlags

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ImageFontFormat

ImageFontFormat · revision 16056 · last edited 2023-07-05

The imagefont file is an image file in a format that LÖVE can load. It can contain transparent pixels, so a PNG file is preferable, and it also needs to contain spacer color that will separate the different font glyphs.

The *upper left pixel* of the image file is always taken to be the spacer color. All columns that have this color as their uppermost pixel are interpreted as separators of font glyphs. The areas between these separators are interpreted as the actual font glyphs.

It is possible to have more areas in the image than are required for the font in the love.graphics.newImageFont() call. The extra areas are ignored.

In versions prior to 0.10.0 the width of the separator area after a particular font glyph sets the amount of space that goes after the glyph. As of 0.10.0, it is possible to add negative spacing for all characters by using the 3-parameter version of love.graphics.newImageFont and giving a negative number as the third parameter.

See Also

- Tutorial:Fonts and Text

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

IndexDataType

IndexDataType · revision 22390 · last edited 2026-04-10

Available since LÖVE 11.0

This enum is not supported in earlier versions.

Vertex map datatype for Data variant of Mesh:setVertexMap.

Constants

uint16

The vertex map is array of unsigned word (16-bit).

uint32

The vertex map is array of unsigned dword (32-bit).

See Also

- love.graphics
- Mesh:setVertexMap
- Mesh

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Input

Input · revision 25312 · last edited 2023-07-05

Input Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
Wasx	A very versatile input manager for LÖVE		11.x	
Common Organization of Controls Kit	Comprehensive input handling library		0.8.0	
TLbind	A simple system for creating professional control schemes		0.7.0	
Tactile	A flexible and nice input library.		0.10.x	
InputField	Text input handling library for LÖVE		0.10.2+	
Lynput	Lynput is a library that makes input handling very easy and intuitive		0.10.0+	
CTRL	General Input Library		0.10.0	

InputField

InputField · revision 27966 · last edited 2023-07-05

InputField is a text input handling library for LÖVE. It handles single- and multi-line input, cursors, selections, copy/paste, undo/redo and more.

Links

- Forum thread: https://love2d.org/forums/viewtopic.php?f=5&t=92526
- Repository: https://github.com/ReFreezed/InputField ↗

inspect.lua

inspect.lua · revision 6363 · last edited 2023-07-05

Original forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2879).

Latest version:

https://github.com/kikito/inspect.lua ↗

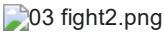
inspect.lua will make it easier to debug your tables by generating string representations of them, in a hierarchical way.

It comes with a default "max depth" of 4, in order to avoid "infinite recursion" as well as making its output more intelligible.

Do not attempt to use it for saving/restoring tables from a file. It generates invalid Lua (on purpose). It's a library for debugging and little else.

InvaderGame

InvaderGame · revision 6330 · last edited 2021-03-14



Almost everyone has played some kind of shoot-em-up game, but did you ever think about how the player’s plane could upgrade it’s weapon? This Invader game could tell you the secret. Weapon towers floating in the Universe, if a power source attached it, it would become to an active weapon, shooting at opponents. If it lost the power source, it will be floating again.

Enjoy it.

download here (http://love2d.org/forums/viewtopic.php?f=5&t=2825)

Isome

Isome · revision 7847 · last edited 2021-03-14





The game

Isome is an engine (not an actual game) which features logically 3D gameplay (things can move in 3 dimensions, and have 3D coordinates) and 2D, isometric, tile-based graphics. It includes a simple, mouse-driven map editor. If you have any questions about how it works or how to use it, please contact Taehl.

Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=2222)

Download

Direct from Dropbox (http://dl.dropbox.com/u/3713769/web/Love/isome/Isome.love)

An example level (the one in the "tower on the side of a mountain" screenshot) (http://dl.dropbox.com/u/3713769/web/Love/isome/isome.zil)

Media



A screenshot of the editor with the "layer" view mode active.

JesseH

JesseH · revision 10711 · last edited 2021-03-14

creator of the LöveOS link: https://love2d.org/forums/viewtopic.php?f=5&t=35315

he joined the community on: 19 Nov 2012, 08:04

Joint

Joint · revision 10237 · last edited 2023-07-05

On this page

- Constructors
- Functions
- Supertypes
- Subtypes
- Enums
- See Also
- Other Languages

Attach multiple bodies together to interact in unique ways.

Constructors

love.physics.newDistanceJoint	Creates a DistanceJoint between two bodies.	
love.physics.newFrictionJoint	A FrictionJoint applies friction to a body.	Added since 0.8.0
love.physics.newGearJoint	Create a GearJoint connecting two Joints.	
love.physics.newMotorJoint	Creates a joint between two bodies which controls the relative motion between them.	Added since 0.9.0
love.physics.newMouseJoint	Create a joint between a body and the mouse.	
love.physics.newPrismaticJoint	Creates a PrismaticJoint between two bodies.	
love.physics.newPulleyJoint	Creates a PulleyJoint to join two bodies to each other and the ground.	
love.physics.newRevoluteJoint	Creates a pivot joint between two bodies.	
love.physics.newRopeJoint	Creates a joint between two bodies that enforces a max distance between them.	Added since 0.8.0
love.physics.newWeldJoint	A WeldJoint essentially glues two bodies together.	Added since 0.8.0
love.physics.newWheelJoint	Creates a wheel joint.	Added since 0.8.0

Functions

Joint:destroy	Explicitly destroys the Joint.		
Joint:getAnchors	Get the anchor points of the joint.		
Joint:getBodies	Gets the bodies that the Joint is attached to.	Added since 0.9.2	
Joint:getCollideConnected	Gets whether the connected Bodies collide.		
Joint:getReactionForce	Returns the reaction force on the second body.		
Joint:getReactionTorque	Returns the reaction torque on the second body.		
Joint:getType	Gets a string representing the type.		
Joint:getUserData	Returns the Lua value associated with this Joint.	Added since 0.9.2	
Joint:isDestroyed	Gets whether the Joint is destroyed.	Added since 0.9.2	
Joint:setCollideConnected	Sets whether the connected Bodies should collide with each other.		Removed in 0.8.0
Joint:setUserData	Associates a Lua value with the Joint.	Added since 0.9.2	
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0	
Object:type	Gets the type of the object as a string.		
Object:typeOf	Checks whether an object is of a certain type.		

Supertypes

- Object

Subtypes

DistanceJoint	Keeps two bodies at the same distance.	
FrictionJoint	A FrictionJoint applies friction to a body.	Added since 0.8.0
GearJoint	Keeps bodies together in such a way that they act like gears.	
MotorJoint	Controls the relative motion between two Bodies	Added since 0.9.0
MouseJoint	For controlling objects with the mouse.	
PrismaticJoint	Restricts relative motion between Bodies to one shared axis.	
PulleyJoint	Allows you to simulate bodies connected through pulleys.	
RevoluteJoint	Allow two Bodies to revolve around a shared point.	
RopeJoint	Enforces a maximum distance between two points on two bodies.	Added since 0.8.0
WeldJoint	A WeldJoint essentially glues two bodies together.	Added since 0.8.0

WheelJoint	Restricts a point on the second body to a line on the first body.

Enums

JointType	Different types of joints.

See Also

- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:destroy

Joint:destroy · revision 8478 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Explicitly destroys the Joint. An error will occur if you attempt to use the object after calling this function.

In 0.7.2, when you don't have time to wait for garbage collection, this function may be used to free the object immediately.

Function

Synopsis

```
Joint:destroy( )
```

Arguments

None.

Returns

Nothing.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getAnchors

Joint:getAnchors · revision 6061 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Get the anchor points of the joint.

Function

Synopsis

```
x1, y1, x2, y2 = Joint:getAnchors( )
```

Arguments

None.

Returns

number x1
: The x-component of the anchor on Body 1.

number y1
: The y-component of the anchor on Body 1.

number x2
: The x-component of the anchor on Body 2.

number y2
: The y-component of the anchor on Body 2.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getBodies

Joint:getBodies · revision 14655 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets the bodies that the Joint is attached to.

Function

Synopsis

```
bodyA, bodyB = Joint:getBodies( )
```

Arguments

None.

Returns

`Body bodyA (nil)`
: The first Body.

`Body bodyB (nil)`
: The second Body.

See Also

- Joint
- Body

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getCollideConnected

Joint:getCollideConnected · revision 6062 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets whether the connected Bodies collide.

Function

Synopsis

```
c = Joint:getCollideConnected( )
```

Arguments

None.

Returns

boolean c

True if they collide, false otherwise.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getReactionForce

Joint:getReactionForce · revision 30491 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Returns the reaction force in newtons on the second body

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
x, y = Joint:getReactionForce( invdt )
```

Arguments

number invdt

How long the force applies. Usually the inverse time step or 1/dt.

Returns

number x

The x-component of the force.

number y

The y-component of the force.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
x, y = Joint:getReactionForce( )
```

Arguments

None.

Returns

number x

The x-component of the force.

number y

The y-component of the force.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getReactionTorque

Joint:getReactionTorque · revision 9059 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Returns the reaction torque on the second body.

Function

Available since LÖVE 0.8.0

This variant is not supported in earlier versions.

Synopsis

```
torque = Joint:getReactionTorque( invdt )
```

Arguments

`number invdt`

How long the force applies. Usually the inverse time step or 1/dt.

Returns

`number torque`

The reaction torque on the second body.

Function

Removed in LÖVE 0.8.0

This variant is not supported in that and later versions.

Synopsis

```
torque = Joint:getReactionTorque( )
```

Arguments

None.

Returns

`number torque`

The reaction torque on the second body.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getType

Joint:getType · revision 16353 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Gets a string representing the type.

Function

Synopsis

```
type = Joint:getType( )
```

Arguments

None.

Returns

`JointType type`
: A string with the name of the Joint type.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:getUserData

Joint:getUserData · revision 19355 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This method is not supported in earlier versions.

Returns the Lua value associated with this Joint.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
value = Joint:getUserData( )
```

Arguments

None.

Returns

`any value`
: The Lua value associated with the Joint.

See Also

- Joint
- Joint:setUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:isDestroyed

Joint:isDestroyed · revision 14643 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This function is not supported in earlier versions.

Gets whether the Joint is destroyed. Destroyed joints cannot be used.

Function

Synopsis

```
destroyed = Joint:isDestroyed( )
```

Arguments

None.

Returns

`boolean destroyed`

Whether the Joint is destroyed.

See Also

- Joint
- Joint:destroy

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:setCollideConnected

Joint:setCollideConnected · revision 16174 · last edited 2025-12-09

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Removed in LÖVE 0.8.0
This function is not supported in that and later versions.

Sets whether the connected Bodies should collide with each other.

Function

Synopsis

```
Joint:setCollideConnected( collide )
```

Arguments

`boolean collide`
: True for the Bodies to collide, false otherwise.

Returns

Nothing.

See Also

- Joint

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joint:setUserData

Joint:setUserData · revision 25350 · last edited 2025-11-25

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.2

This method is not supported in earlier versions.

Associates a Lua value with the Joint.

To delete the reference, explicitly pass nil.


Use this function in one thread and one thread only. Using it in more threads will make Lua cry and most likely crash.

Function

Synopsis

```
Joint:setUserData( value )
```

Arguments

`Variant value`
: The Lua value to associate with the Joint.

Returns

Nothing.

See Also

- Joint
- Joint:getUserData

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

JointType

JointType · revision 30303 · last edited 2025-11-25

Different types of joints.

Constants

distance
: A DistanceJoint. Keeps two bodies at the same distance.

friction
: A FrictionJoint. A FrictionJoint applies friction to a body.

gear
: A GearJoint. Keeps bodies together in such a way that they act like gears.

mouse
: A MouseJoint. For controlling objects with the mouse.

prismatic
: A PrismaticJoint. Restricts relative motion between Bodies to one shared axis.

pulley
: A PulleyJoint. Allows you to simulate bodies connected through pulleys.

revolute
: A RevoluteJoint. Allow two Bodies to revolve around a shared point.

rope
: A RopeJoint. The RopeJoint enforces a maximum distance between two points on two bodies.

weld
: A WeldJoint. A WeldJoint essentially glues two bodies together.

wheel
: A WheelJoint. A WheelJoint restricts a point on the second body to a line on the first body.

See Also

- Joint
- love.physics

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick

Joystick · revision 28997 · last edited 2023-11-13

On this page

- Constructors
- Functions
- Enums
- Supertypes
- Examples
- Display the last button pressed of a controller on-screen
- Move a circle with the d-pad of a controller
- Move circles and change sizes with sticks of gamepad
- See Also
- Other Languages

Available since LÖVE 0.9.0

This type is not supported in earlier versions.

Represents a physical joystick.

Constructors

love.joystick.getJoysticks	Gets a list of connected Joysticks.	Added since 0.9.0

Functions

Joystick:getAxes	Gets the direction of each axis.	Added since 0.9.0
Joystick:getAxis	Gets the direction of an axis.	Added since 0.9.0
Joystick:getAxisCount	Gets the number of axes on the joystick.	Added since 0.9.0
Joystick:getButtonCount	Gets the number of buttons on the joystick.	Added since 0.9.0
Joystick:getDeviceInfo	Gets the OS-independent device info of the joystick.	Added since 11.3
Joystick:getGUID	Gets a stable GUID unique to the type of the physical joystick.	Added since 0.9.0
Joystick:getGamepadAxis	Gets the direction of a virtual gamepad axis.	Added since 0.9.0
Joystick:getGamepadMapping	Gets the button, axis or hat that a virtual gamepad input is bound to.	Added since 0.9.0
Joystick:getGamepadMappingString	Gets the full gamepad mapping string of this Joystick, or nil if it's not recognized as a gamepad.	Added since 11.3
Joystick:getHat	Gets the direction of a hat.	Added since 0.9.0
Joystick:getHatCount	Gets the number of hats on the joystick.	Added since 0.9.0
Joystick:getID	Gets the joystick's unique identifier.	Added since 0.9.0
Joystick:getName	Gets the name of the joystick.	Added since 0.9.0
Joystick:getVibration	Gets the current vibration motor strengths on a Joystick with rumble support.	Added since 0.9.0
Joystick:isConnected	Gets whether the Joystick is connected.	Added since 0.9.0
Joystick:isDown	Checks if a button on the Joystick is pressed.	Added since 0.9.0
Joystick:isGamepad	Gets whether the Joystick is recognized as a gamepad.	Added since 0.9.0
Joystick:isGamepadDown	Checks if a virtual gamepad button on the Joystick is pressed.	Added since 0.9.0
Joystick:isVibrationSupported	Gets whether the Joystick supports vibration.	Added since 0.9.0
Joystick:setVibration	Sets the vibration motor speeds on a Joystick with rumble support.	Added since 0.9.0
Object:release	Immediately destroys the object's Lua reference.	Added since 11.0
Object:type	Gets the type of the object as a string.	

Object:typeOf	Checks whether an object is of a certain type.

Enums

GamepadAxis	Virtual gamepad axes.	Added since 0.9.0
GamepadButton	Virtual gamepad buttons.	Added since 0.9.0
JoystickHat	Joystick hat positions.	Added since 0.5.0
JoystickInputType	Types of Joystick inputs.	Added since 0.9.0

Supertypes

- Object

Examples

Display the last button pressed of a controller on-screen

```
local lastbutton = "none"

function love.gamepadpressed(joystick, button)
    lastbutton = button
end

function love.draw()
    love.graphics.print("Last gamepad button pressed: "..lastbutton, 10, 10)
end
```

Move a circle with the d-pad of a controller

```
function love.load()
    local joysticks = love.joystick.getJoysticks()
    joystick = joysticks[1]

    position = {x = 400, y = 300}
    speed = 300
end

function love.update(dt)
    if not joystick then return end

    if joystick:isGamepadDown("dpleft") then
        position.x = position.x - speed * dt
    elseif joystick:isGamepadDown("dpright") then
        position.x = position.x + speed * dt
    end

    if joystick:isGamepadDown("dpup") then
        position.y = position.y - speed * dt
    elseif joystick:isGamepadDown("dpdown") then
        position.y = position.y + speed * dt
    end
end

function love.draw()
    love.graphics.circle("fill", position.x, position.y, 50)
end
```

Move circles and change sizes with sticks of gamepad

```
function love.load()
    local joysticks = love.joystick.getJoysticks()
    joystick = joysticks[1]

    leftCircle = {x = 200, y = 280, size = 50}
    rightCircle = {x = 600, y = 280, size = 50}
    speed = 300
```

```
end

function love.update(dt)
    if not joystick then return end

    leftCircle.x = leftCircle.x + dt*speed*joystick:getGamepadAxis("leftx")
    leftCircle.y = leftCircle.y + dt*speed*joystick:getGamepadAxis("lefty")

    rightCircle.x = rightCircle.x + dt*speed*joystick:getGamepadAxis("rightx")
    rightCircle.y = rightCircle.y + dt*speed*joystick:getGamepadAxis("righty")
end

function love.draw()
    local leftSize = (1-joystick:getGamepadAxis("triggerleft"))*leftCircle.size
    local rightSize = (1-joystick:getGamepadAxis("triggerright"))*rightCircle.size
    love.graphics.circle("fill", leftCircle.x, leftCircle.y, leftSize)
    love.graphics.circle("fill", rightCircle.x, rightCircle.y, rightSize)
end
```

See Also

- love.joystick
- love.joystick.getJoystickCount
- love.joystickadded
- love.joystickremoved
- love.joystickpressed
- love.joystickreleased
- love.joystickaxis
- love.joystickhat
- love.gamepadpressed
- love.gamepadreleased
- love.gamepadaxis

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getAxes

Joystick:getAxes · revision 11293 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getAxes.

Gets the direction of each axis.

Function

Synopsis

```
axisDir1, axisDir2, ..., axisDirN = Joystick:getAxes( )
```

Arguments

None.

Returns

number axisDir1
: Direction of axis1.

number axisDir2
: Direction of axis2.

number axisDirN
: Direction of axisN.

See Also

- Joystick
- Joystick:getAxis
- Joystick:getAxisCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getAxis

Joystick:getAxis · revision 11294 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getAxis.

Gets the direction of an axis.

Function

Synopsis

```
direction = Joystick:getAxis( axis )
```

Arguments

`number axis`

The index of the axis to be checked.

Returns

`number direction`

Current value of the axis.

See Also

- Joystick
- Joystick:getAxes
- Joystick:getAxisCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getAxisCount

Joystick:getAxisCount · revision 11295 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getNumAxes.

Gets the number of axes on the joystick.

Function

Synopsis

```
axes = Joystick:getAxisCount( )
```

Arguments

None.

Returns

`number axes`

The number of axes available.

See Also

- Joystick
- Joystick:getAxis
- Joystick:getAxes

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getButtonCount

Joystick:getButtonCount · revision 11692 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getNumButtons.

Gets the number of buttons on the joystick.

Function

Synopsis

```
buttons = Joystick:getButtonCount( )
```

Arguments

None.

Returns

`number buttons`

The number of buttons available.

See Also

- Joystick
- Joystick:isDown

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Joystick:getDeviceInfo

Joystick:getDeviceInfo · revision 28156 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets the USB vendor ID, product ID, and product version numbers of joystick which consistent across operating systems.

Can be used to show different icons, etc. for different gamepads.

Function

Synopsis

```
vendorID, productID, productVersion = Joystick:getDeviceInfo( )
```

Arguments

None.

Returns

`number vendorID`
: The USB vendor ID of the joystick.

`number productID`
: The USB product ID of the joystick.

`number productVersion`
: The product version of the joystick.

Notes

Some Linux distribution may not ship with SDL 2.0.6 or later, in which case this function will returns 0 for all the three values.

See Also

- Joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getGamepadAxis

Joystick:getGamepadAxis · revision 27215 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Examples
  - Move x and y values based on a gamepad thumbstick
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the direction of a virtual gamepad axis. If the Joystick isn't recognized as a gamepad or isn't connected, this function will always return 0.

Function

Synopsis

```
direction = Joystick:getGamepadAxis( axis )
```

Arguments

`GamepadAxis axis`

The virtual axis to be checked.

Returns

`number direction`

Current value of the axis.

Examples

Move x and y values based on a gamepad thumbstick

```
function love.load()
    x = 0
    y = 0
    p1joystick = nil
end

function love.joystickadded(joystick)
    p1joystick = joystick
end

function love.update(dt)
    -- Check if joystick connected
    if p1joystick ~= nil then
        -- getGamepadAxis returns a value between -1 and 1.
        -- It returns 0 when it is at rest

        x = x + p1joystick:getGamepadAxis("leftx")
        y = y + p1joystick:getGamepadAxis("lefty")
    end
end
```

See Also

- Joystick
- Joystick:isGamepad
- Joystick:isGamepadDown
- Joystick:isConnected

- GamepadAxis
- love.gamepadaxis

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getGamepadMapping

Joystick:getGamepadMapping · revision 28189 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Function
  - Synopsis
  - Arguments
  - Returns
  - Notes
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the button, axis or hat that a virtual gamepad input is bound to.

Function

Synopsis

```
inputtype, inputindex, hatdirection = Joystick:getGamepadMapping( axis )
```

Arguments

`GamepadAxis axis`
: The virtual gamepad axis to get the binding for.

Returns

`JoystickInputType inputtype`
: The type of input the virtual gamepad axis is bound to.

`number inputindex`
: The 1-based index of the Joystick's button, axis or hat that the virtual gamepad axis is bound to.

`JoystickHat hatdirection (nil)`
: The direction of the hat, if the virtual gamepad axis is bound to a hat. nil otherwise.

Notes

Returns nil if the Joystick isn't recognized as a gamepad or the virtual gamepad axis is not bound to a Joystick input.

Function

Synopsis

```
inputtype, inputindex, hatdirection = Joystick:getGamepadMapping( button )
```

Arguments

`GamepadButton button`
: The virtual gamepad button to get the binding for.

Returns

`JoystickInputType inputtype`
: The type of input the virtual gamepad button is bound to.

`number inputindex`
: The 1-based index of the Joystick's button, axis or hat that the virtual gamepad button is bound to.

`JoystickHat hatdirection (nil)`

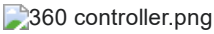
The direction of the hat, if the virtual gamepad button is bound to a hat. nil otherwise.

Notes

Returns nil if the Joystick isn't recognized as a gamepad or the virtual gamepad button is not bound to a Joystick input.

Notes

The physical locations for the virtual gamepad axes and buttons correspond as closely as possible to the layout of a standard Xbox 360 controller.



See Also

- Joystick
- Joystick:isGamepad
- Joystick:isGamepadDown
- Joystick:getGamepadAxis
- love.joystick.setGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getGamepadMappingString

Joystick:getGamepadMappingString · revision 23743 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 11.3

This function is not supported in earlier versions.

Gets the full gamepad mapping string of this Joystick, or nil if it's not recognized as a gamepad.

The mapping string contains binding information used to map the Joystick's buttons an axes to the standard gamepad layout, and can be used later with love.joystick.loadGamepadMappings.

Function

Synopsis

```
mappingstring = Joystick:getGamepadMappingString( )
```

Arguments

None.

Returns

`string mappingstring (nil)`

A string containing the Joystick's gamepad mappings, or nil if the Joystick is not recognized as a gamepad.

See Also

- Joystick
- Joystick:isGamepad
- love.joystick.loadGamepadMappings
- love.joystick.saveGamepadMappings
- love.joystick.setGamepadMapping
- Joystick:getGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getGamepadType

Joystick:getGamepadType · revision 28106 · last edited 2023-07-21

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the GamepadType of the Joystick, if it's recognized as a gamepad.

Function

Synopsis

```
gamepadtype = Joystick:getGamepadType( )
```

Arguments

None.

Returns

`GamepadType gamepadtype`

The type of the gamepad, or "unknown" if it can't be determined.

See Also

- Joystick
- Joystick:isGamepad

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getGUID

Joystick:getGUID · revision 11392 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets a stable GUID unique to the type of the physical joystick which does not change over time. For example, all Sony Dualshock 3 controllers in OS X have the same GUID. The value is platform-dependent.

Function

Synopsis

```
guid = Joystick:getGUID( )
```

Arguments

None.

Returns

`string guid`

The Joystick type's OS-dependent unique identifier.

See Also

- Joystick
- Joystick:getID
- love.joystick.setGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getHat

Joystick:getHat · revision 16737 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getHat.

Gets the direction of the Joystick's hat (https://en.wikipedia.org/wiki/Joystick#Hat_switch).

Function

Synopsis

```
direction = Joystick:getHat( hat )
```

Arguments

`number hat`
: The index of the hat to be checked.

Returns

`JoystickHat direction`
: The direction the hat is pushed.

See Also

- Joystick
- Joystick:getHatCount

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getHatCount

Joystick:getHatCount · revision 25635 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getNumHats.

Gets the number of hats (https://en.wikipedia.org/wiki/Joystick#Hat_switch) on the joystick.

Function

Synopsis

```
hats = Joystick:getHatCount( )
```

Arguments

None.

Returns

`number hats`

How many hats the joystick has.

See Also

- Joystick
- Joystick:getHat

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Joystick:getID

Joystick:getID · revision 11393 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the joystick's unique identifier. The identifier will remain the same for the life of the game, even when the Joystick is disconnected and reconnected, but it **will** change when the game is re-launched.

Function

Synopsis

```
id, instanceid = Joystick:getID( )
```

Arguments

None.

Returns

`number id`
: The Joystick's unique identifier. Remains the same as long as the game is running.

`number instanceid (nil)`
: Unique instance identifier. Changes every time the Joystick is reconnected. nil if the Joystick is not connected.

See Also

- Joystick
- Joystick:isConnected
- Joystick:getGUID

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getJoystickType

Joystick:getJoystickType · revision 28256 · last edited 2023-07-22

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the JoystickType of the Joystick.

JoystickTypes are broader categories than GamepadTypes.

Function

Synopsis

```
joysticktype = Joystick:getJoystickType( )
```

Arguments

None.

Returns

`JoystickType joysticktype`

The type of the joystick, or "unknown" if it can't be determined.

See Also

- Joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getName

Joystick:getName · revision 11299 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.getName.

Gets the name of the joystick.

Function

Synopsis

```
name = Joystick:getName( )
```

Arguments

None.

Returns

`string name`

The name of the joystick.

See Also

- Joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어

More info

Joystick:getPlayerIndex

Joystick:getPlayerIndex · revision 29131 · last edited 2024-03-31

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Gets the player index of this Joystick. This corresponds to an indicator light on many common gamepads.

Note that this is independent from the array index of this Joystick in the table returned by love.joystick.getJoysticks.

Function

Synopsis

```
index = Joystick:getPlayerIndex( )
```

Arguments

None.

Returns

`number index`

The 1-based player index being used for this Joystick, or -1 if the player index has not been set or cannot be determined.

See Also

- Joystick
- Joystick:setPlayerIndex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getSensorData

Joystick:getSensorData · revision 29274 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This callback is not supported in earlier versions.

Gets the latest data for the given sensor type for this Joystick.

The returned values have meaning based on the sensor type, for example an accelerometer will return acceleration values along each axis.

If the sensor was not enabled via Joystick:setSensorEnabled, this function may cause an error.

Function

Synopsis

```
x, y, z = Joystick:getSensorData( sensorType )
```

Arguments

`SensorType sensorType`
: The type of sensor.

Returns

`number x`
: The sensor's current 1st value.

`number y`
: The sensor's current 2nd value.

`number z`
: The sensor's current 3rd value.

See Also

- Joystick
- Joystick:setSensorEnabled
- love.joysticksensorupdated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:getVibration

Joystick:getVibration · revision 11814 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets the current vibration motor strengths on a Joystick with rumble support.

Function

Synopsis

```
left, right = Joystick:getVibration( )
```

Arguments

None.

Returns

`number left`
: Current strength of the left vibration motor on the Joystick.

`number right`
: Current strength of the right vibration motor on the Joystick.

See Also

- Joystick
- Joystick:setVibration
- Joystick:isVibrationSupported

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:hasSensor

Joystick:hasSensor · revision 29142 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets whether the specified sensor exists in the Joystick.

Function

Synopsis

```
available = Joystick:hasSensor( sensorType )
```

Arguments

`SensorType sensorType`

Type of sensor to check.

Returns

`boolean available`

Sensor availability status.

See Also

- Joystick
- Joystick:setSensorEnabled
- Joystick:getSensorData
- love.joysticksensorupdated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isConnected

Joystick:isConnected · revision 11292 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Joystick is connected.

Function

Synopsis

```
connected = Joystick:isConnected( )
```

Arguments

None.

Returns

`boolean connected`

True if the Joystick is currently connected, false otherwise.

See Also

- Joystick
- love.joystickadded
- love.joystickremoved

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isDown

Joystick:isDown · revision 28147 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

It has been moved from love.joystick.isDown.

Checks if a button on the Joystick is pressed.

LÖVE 0.9.0 had a bug which required the button indices passed to Joystick:isDown to be 0-based instead of 1-based, for example button 1 would be 0 for this function. It was fixed in 0.9.1.

Function

Synopsis

```
anyDown = Joystick:isDown( buttonN, ... )
```

Arguments

number buttonN
: The index of a button to check.

number ...
: Additional buttons to check.

Returns

boolean anyDown
: True if any supplied button is down, false if not.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
anyDown = Joystick:isDown({ buttonN, ... })
```

Arguments

table buttons
: Table of button indexes to check.

  number buttonN
  : The index of a button to check.

  number ...
  : Additional buttons to check.

Returns

boolean anyDown

True if any supplied button is down, false if not.

See Also

- Joystick
- Joystick:getButtonCount
- love.joystickpressed
- love.joystickreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isGamepad

Joystick:isGamepad · revision 27341 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Joystick is recognized as a gamepad. In LÖVE, a "gamepad" or "virtual gamepad" use names defined in GamepadAxis and GamepadButton instead of indexes so they are standardized across different operating systems and joystick models. Use gamepad-specific functions to query with these input names: Joystick:getGamepadAxis, Joystick:isGamepadDown, love.gamepadpressed, etc.

LÖVE automatically recognizes most popular controllers with a similar layout to the Xbox 360 controller as gamepads, but you can add more with love.joystick.setGamepadMapping.

Function

Synopsis

```
isgamepad = Joystick:isGamepad( )
```

Arguments

None.

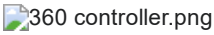
Returns

`boolean isgamepad`

True if the Joystick is recognized as a gamepad, false otherwise.

Notes

If the Joystick is recognized as a gamepad, the physical locations for the virtual gamepad axes and buttons correspond as closely as possible to the layout of a standard Xbox 360 controller.


360 controller.png

See Also

- Joystick
- Joystick:getGamepadAxis
- Joystick:isGamepadDown
- Joystick:getGamepadMapping

- love.gamepadaxis
- love.gamepadpressed
- love.gamepadreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isGamepadDown

Joystick:isGamepadDown · revision 28148 · last edited 2025-11-26

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
- Synopsis
- Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Checks if a virtual gamepad button on the Joystick is pressed. If the Joystick is not recognized as a Gamepad or isn't connected, then this function will always return false.

Function

Synopsis

```
anyDown = Joystick:isGamepadDown( button1, button2, button3, ... )
```

Arguments

`GamepadButton buttonN`

The gamepad button to check.

Returns

`boolean anyDown`

True if any supplied button is down, false if not.

Function

Available since LÖVE 0.10.2

This variant is not supported in earlier versions.

Synopsis

```
anyDown = Joystick:isGamepadDown({ button1, button2, button3, ... })
```

Arguments

`table buttons`

Table of gamepad buttons to check.

`GamepadButton buttonN`

The gamepad button to check.

Returns

`boolean anyDown`

True if any supplied button is down, false if not.

See Also

- Joystick
- Joystick:isGamepad
- Joystick:getGamepadAxis
- Joystick:isConnected

- love.gamepadpressed
- love.gamepadreleased

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isSensorEnabled

Joystick:isSensorEnabled · revision 29253 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Gets whether the specified sensor is currently enabled on this Joystick.

Function

Synopsis

```
enabled = Joystick:isSensorEnabled( sensorType )
```

Arguments

`SensorType sensorType`

Type of sensor to check.

Returns

`boolean enabled`

Whether the sensor is currently enabled.

See Also

- Joystick
- Joystick:setSensorEnabled
- Joystick:getSensorData
- love.joysticksensorupdated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:isVibrationSupported

Joystick:isVibrationSupported · revision 11816 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Gets whether the Joystick supports vibration.

Function

Synopsis

```
supported = Joystick:isVibrationSupported( )
```

Arguments

None.

Returns

`boolean supported`

True if rumble / force feedback vibration is supported on this Joystick, false if not.

Notes

The very first call to this function may take more time than expected because SDL's Haptic / Force Feedback subsystem needs to be initialized.

See Also

- Joystick
- Joystick:setVibration
- Joystick:getVibration

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:setPlayerIndex

Joystick:setPlayerIndex · revision 29300 · last edited 2024-08-24

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This function is not supported in earlier versions.

Sets the player index of this Joystick. This corresponds to an indicator light on many common gamepads.

Note that this is independent from the array index of this Joystick in the table returned by love.joystick.getJoysticks.

Function

Synopsis

```
Joystick:setPlayerIndex( index )
```

Arguments

`number index`
: The 1-based player index to use for this Joystick. `0` clears the set index.

Returns

Nothing.

See Also

- Joystick
- Joystick:getPlayerIndex

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:setSensorEnabled

Joystick:setSensorEnabled · revision 29273 · last edited 2024-06-15

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Available since LÖVE 12.0

This method is not supported in earlier versions.

Enables or disables the specified sensor on this Joystick.

The given sensor type must exist on this Joystick, otherwise this function may cause an error.

Function

Synopsis

```
Joystick:setSensorEnabled( sensorType, enable )
```

Arguments

`SensorType sensorType`
: Type of sensor to enable or disable.

`boolean enable`
: True to enable the given sensor, false to disable it.

Returns

Nothing.

See Also

- Joystick
- Joystick:hasSensor
- Joystick:isSensorEnabled
- Joystick:getSensorData
- love.joysticksensorupdated

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Joystick:setVibration

Joystick:setVibration · revision 27203 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Function
  - Synopsis
  - Arguments
  - Returns
- Notes
- Example
- See Also
- Other Languages

Available since LÖVE 0.9.0

This function is not supported in earlier versions.

Sets the vibration motor speeds on a Joystick with rumble support. Most common gamepads have this functionality, although not all drivers give proper support. Use Joystick:isVibrationSupported to check.

Function

Synopsis

```
success = Joystick:setVibration( left, right )
```

Arguments

`number left`
: Strength of the left vibration motor on the Joystick. Must be in the range of [0, 1].

`number right`
: Strength of the right vibration motor on the Joystick. Must be in the range of [0, 1].

Returns

`boolean success`
: True if the vibration was successfully applied, false if not.

Function

Disables vibration.

Synopsis

```
success = Joystick:setVibration( )
```

Arguments

None.

Returns

`boolean success`
: True if the vibration was successfully disabled, false if not.

Function

Available since LÖVE 0.9.2

This variant is not supported in earlier versions.

Synopsis

```
success = Joystick:setVibration( left, right, duration )
```

Arguments

number left
: Strength of the left vibration motor on the Joystick. Must be in the range of [0, 1].

number right
: Strength of the right vibration motor on the Joystick. Must be in the range of [0, 1].

number duration (-1)
: The duration of the vibration in seconds. A negative value means infinite duration.

Returns

boolean success
: True if the vibration was successfully applied, false if not.

Notes

If the Joystick only has a single vibration motor, it will still work but it will use the largest value of the left and right parameters.

The Xbox 360 controller on Mac OS X only has support for vibration if a modified version (https://github.com/d235j/360Controller/releases) of the Tattiebogle driver is used.

The very first call to this function may take more time than expected because SDL's Haptic / Force Feedback subsystem needs to be initialized.

Example

```
function love.gamepadaxis( joystick, axis, value )
    if axis == 'triggerleft' or axis == 'triggerright' then
        local left = joystick:getGamepadAxis("triggerleft")
        local right = joystick:getGamepadAxis("triggerright")
        success = joystick:setVibration(left, right)
    end
end
```

See Also

- Joystick
- Joystick:getVibration
- Joystick:isVibrationSupported

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

JoystickHat

JoystickHat · revision 25634 · last edited 2023-07-05

Available since LÖVE 0.5.0

This enum is not supported in earlier versions.

Joystick hat (https://en.wikipedia.org/wiki/Joystick#Hat_switch) positions.

Constants

c
: Centered

d
: Down

l
: Left

ld
: Left+Down

lu
: Left+Up

r
: Right

rd
: Right+Down

ru
: Right+Up

u
: Up

See Also

- love.joystick
- Joystick

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

JoystickInputType

JoystickInputType · revision 11331 · last edited 2023-07-05

Available since LÖVE 0.9.0
This enum is not supported in earlier versions.

Types of Joystick inputs.

Constants

axis
: Analog axis.

button
: Button.

hat
: 8-direction hat value.

See Also

- Joystick
- love.joystick
- love.joystick.setGamepadMapping
- Joystick:getGamepadMapping

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

JoystickType

JoystickType · revision 28257 · last edited 2023-07-20

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Types of Joysticks.

Constants

unknown
: An unrecognized joystick type.

gamepad
: A gamepad.

wheel
: Steering / racing wheel.

arcadestick
: Fighting game arcade stick.

flightstick
: HOTAS / flight stick.

dancepad
: Dance pad.

guitar
: Guitar.

drumkit
: Drum kit.

arcadepad
: Arcade pad.

throttle
: Standalone throttle.

See Also

- love.joystick
- Joystick
- Joystick:getJoystickType

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Jumper

Jumper · revision 30615 · last edited 2026-02-19

Jumper is a pathfinding library designed for uniform-cost 2D grid-based games. It aims to be lightning **fast** and **lightweight**. It also features a **clean public interface** with chaining features which makes it very friendly and easy to use.

Technical notes

Jumper implements a lots if search algorithms: A-star, Dijkstra, Breadth-First search, Depth First search and Jump Point search (the fastest amongst all).

A benchmark program running in console is available on Github (https://github.com/Yonaba/Jumper-Benchmark). It features a set of 132 maps from the 2012 Grid-based Path Planning Competition (http://movingai.com/GPPC/).

Usage

Jumper is **easy** to use. Here is a basic example using the latest version available.

```lua
-- Usage Example
-- First, set a collision map
local map = {
    {0,1,0,1,0 },
    {0,1,0,1,0 },
    {0,1,1,1,0 },
    {0,0,0,0,0 },
}
-- Value for walkable tiles
local walkable = 0

-- Library setup
local Grid = require ("jumper.grid") -- The grid class
local Pathfinder = require ("jumper.pathfinder") -- The pathfinder class

-- Creates a grid object
local grid = Grid(map)
-- Creates a pathfinder object using Jump Point Search
local myFinder = Pathfinder(grid, 'JPS', walkable)

-- Define start and goal locations coordinates
local startx, starty = 1,1
local endx, endy = 5,1

-- Calculates the path, and its length
local path, length = myFinder:getPath(startx, starty, endx, endy)
if path then
  print(('Path found! Length: %.2f'):format(length))
    for node, count in path:iter() do
      print(('Step: %d - x: %d - y: %d'):format(count, node.x, node.y))
    end
end

--> Output:
--> Path found! Length: 8.83
--> Step: 1 - x: 1 - y: 1
--> Step: 2 - x: 1 - y: 3
--> Step: 3 - x: 2 - y: 4
--> Step: 4 - x: 4 - y: 4
--> Step: 5 - x: 5 - y: 3
--> Step: 6 - x: 5 - y: 1
```

Jumper offers **a lot more options** to customize the pathfinding behavior. Please, kindly check out the Readme (https://github.com/Yonaba/Jumper/blob/master/README.md) to figure them out.

Feedback

If you have any question, proposal, idea to improve, or if you are just using Jumper in a project, I would love to hear your feedback. Issues/suggestions can be posted on the project issue tracker (https://github.com/Yonaba/Jumper/issues) on Github, or by e-mail (http://www.google.com/recaptcha/mailhide/d?k=01HyTvozzFr6QcQ0z5kWMOcg==&c=JKAjCplG02SOWgniiVzypsN1mkpjYjRXjt4Kfdf6g0Q=).

Links

- Source (https://github.com/Yonaba/Jumper)
- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=9322)
- README.md (https://github.com/Yonaba/Jumper/blob/master/README.md)
- Demos (https://github.com/Yonaba/Jumper-Examples)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Jupiter

Jupiter · revision 24502 · last edited 2023-07-05

Jupiter is a file I/O and table serialisation library written by Lafolie.

Usage

Designed to be easy to use, Jupiter takes a table structure and saves it to a file. You can edit these files in your favourite text editor, or even write them from scratch, and re-load them into your project at a later date.

A simple implementation looks like this:

```
jupiter = require "jupiter"

--setup a table
data = {_fileName = "example.txt", "Save this string!"}
--write the file (Jupiter handles filenames by referring to data._fileName)
success = jupiter.save(data)
--load another file into a table
newData = jupiter.load("filename.extension")
```

Get Jupiter

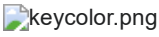
You can download the script from Jupiter on github (http://github.com/Lafolie/Jupiter)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

KeyColor

KeyColor · revision 12318 · last edited 2021-03-14



Associate keys with colors and press-hold them right in 30 seconds!

Made at a Berlin Mini Jam in 2013.

Download: http://g.qubodup.net/keycolor/ ↗

KeyConstant

KeyConstant · revision 28190 · last edited 2025-10-29

All the keys you can press. Note that some keys may not be available on your keyboard or system.

Constants

Key string	Description	Note(s)
Character keys		
a	The A key	
b	The B key	
c	The C key	
d	The D key	
e	The E key	
f	The F key	
g	The G key	
h	The H key	
i	The I key	
j	The J key	
k	The K key	
l	The L key	
m	The M key	
n	The N key	
o	The O key	
p	The P key	
q	The Q key	
r	The R key	
s	The S key	
t	The T key	
u	The U key	
v	The V key	
w	The W key	
x	The X key	
y	The Y key	
z	The Z key	
0	The zero key	
1	The one key	
2	The two key	
3	The three key	
4	The four key	
5	The five key	
6	The six key	
7	The seven key	

8	The eight key	
9	The nine key	
space	Space key	In version 0.9.2 and earlier this is represented by the actual space character ' '.
!	Exclamation mark key	
"	Double quote key	
#	Hash key	
$	Dollar key	
&	Ampersand key	
'	Single quote key	
(	Left parenthesis key	
)	Right parenthesis key	
*	Asterisk key	
+	Plus key	
,	Comma key	
-	Hyphen-minus key	
.	Full stop key	
/	Slash key	
:	Colon key	
;	Semicolon key	
<	Less-than key	
=	Equal key	
>	Greater-than key	
?	Question mark key	
@	At sign key	
[	Left square bracket key	
\	Backslash key	
]	Right square bracket key	
^	Caret key	
_	Underscore key	
`	Grave accent key	Also known as the "Back tick" key
Numpad keys		
kp0	The numpad zero key	
kp1	The numpad one key	
kp2	The numpad two key	
kp3	The numpad three key	
kp4	The numpad four key	
kp5	The numpad five key	
kp6	The numpad six key	
kp7	The numpad seven key	

kp8	The numpad eight key	
kp9	The numpad nine key	
kp.	The numpad decimal point key	
kp,	The numpad comma key	
kp/	The numpad division key	
kp*	The numpad multiplication key	
kp-	The numpad substraction key	
kp+	The numpad addition key	
kpenter	The numpad enter key	
kp=	The numpad equals key	
Navigation keys		
up	Up arrow key	
down	Down arrow key	
right	Right arrow key	
left	Left arrow key	
home	Home key	
end	End key	
pageup	Page up key	
pagedown	Page down key	
Editing keys		
insert	Insert key	
backspace	Backspace key	
tab	Tab key	
clear	Clear key	
return	Return key	Also known as the Enter key
delete	Delete key	
Function keys		
f1	The 1st function key	
f2	The 2nd function key	
f3	The 3rd function key	
f4	The 4th function key	
f5	The 5th function key	
f6	The 6th function key	
f7	The 7th function key	
f8	The 8th function key	
f9	The 9th function key	
f10	The 10th function key	
f11	The 11th function key	
f12	The 12th function key	

f13	The 13th function key	
f14	The 14th function key	
f15	The 15th function key	
f16	The 16th function key	
f17	The 17th function key	
f18	The 18th function key	
Modifier keys		
numlock	Num-lock key	Clear on Mac keyboards.
capslock	Caps-lock key	
scrolllock	Scroll-lock key	
rshift	Right shift key	
lshift	Left shift key	
rctrl	Right control key	
lctrl	Left control key	
ralt	Right alt key	
lalt	Left alt key	
rgui	Right gui key	Command key in OS X, Windows key in Windows.
lgui	Left gui key	Command key in OS X, Windows key in Windows.
mode	Mode key	
Application keys		
www	WWW key	
mail	Mail key	
calculator	Calculator key	
computer	Computer key	
appsearch	Application search key	
apphome	Application home key	
appback	Application back key	
appforward	Application forward key	
apprefresh	Application refresh key	
appbookmarks	Application bookmarks key	
Miscellaneous keys		
pause	Pause key	Sends a key release immediately on some platforms, even if held down.
escape	Escape key	On Android, this is the on screen back button.
help	Help key	
printscreen	Printscreen key	Sends a key release immediately on Windows, even if held down.
sysreq	System request key	
menu	Menu key	
application	Application key	Windows contextual menu, compose key.
power	Power key	

currencyunit	Currency unit key	e.g. the Euro (€) key.
undo	Undo key	

Examples

Note how love.keypressed gives direct key presses, whereas love.textinput gives the text that key combinations produce.

Let's take the "#" symbol as an example (assuming a U.S. keyboard):

```
function love.keypressed(key)
    print(key)
end
--> shift
--> 3

function love.textinput(text)
    print(text)
end
--> #
```

See Also

- love.keyboard
- love.keypressed
- love.keyreleased
- love.keyboard.isDown
- love.keyboard.getScancodeFromKey
- love.keyboard.getKeyFromScancode

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Knightvania

Knightvania · revision 20354 · last edited 2021-03-14

Description here...



Links

- GameJolt Page: http://gamejolt.com/games/knightvania/178413 ↗
- Itch.io Page : https://canoigomes.itch.io/knightvania ↗
- Forums :

Komoplode

Komoplode · revision 8028 · last edited 2021-03-14

Komoplode

Inspired by a game by Omega and Q entertainment, Komocode is a simple chain reaction game where players must explode objects in the longest chain as possible.



Download here: [1] (http://www.nicholaschan.net/2010/02/komoplode-3/)

Kuey

Kuey · revision 16094 · last edited 2023-07-05

That simple library encode and decode files with a key.

Example:

```
Kuey = require("Kuey")
local Encoded = Kuey.encode("Love is life!", "love2d") -- Encode the string with "love2d" as key
print(Encoded)                                         -- Show the encoded string
print(Kuey.decode(Encoded, "anykey"))                  -- Try to show a decoded string with any key
print(Kuey.decode(Encoded, "love2d"))                  -- Show the decoded string with the correct key
```

Output:

```
©Ïý╩R═■ÅÔ╠  ÿ╔ì
Wps_ýT~!ic3P,
Love is life!
```

Available functions:

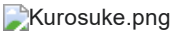
```
Kuey.encode(string, key) -- Return a string
Kuey.decode(string, key) -- Return a string
Kuey.encodeFile(filename, key[, outputname]) -- Return a string and write a file if outputname is defined.
Kuey.decodeFile(filename, key[, outputname]) -- Return a string and write a file if outputname is defined.
```

Forum post here! (https://love2d.org/forums/viewtopic.php?f=5&t=81643)

Kurosuke

Kurosuke · revision 5469 · last edited 2021-03-14

Kurosuke



A platformer set inside of the human eye, where floaters are cities and the local microscopic life is suffering from a terrible plague.

Multiplayer is possible with four players on one keyboard, or via key rebinding to joysticks (can be done ingame).

The game has a fully-functional ingame level editor, ingame option menus, saving of stats, and a host of other little features.

> Kurosuke (working title) (http://love2d.org/forums/viewtopic.php?f=5&t=2197) (download link in first post)

LalaLove

LalaLove · revision 7997 · last edited 2021-03-14

LalaLove

A game where you have to make planets happy by creating music via combining sound-sequences coming from stars and manipulating them with asteroids having different effects, such as faster/slower, higher/lower, or adding pauses.

HowTo: Connect Suns to Meteorids, fill the Meteorids-pattern and right-click everything to hear the sound!

Originally created in 20hours for the overnight-contest at Dusmania 2008 (Gamejam).



Downloads for Windows/Linux/MacOSX

http://gamejams.schattenkind.net/2011/10/lalalove.html (http://gamejams.schattenkind.net/2011/10/lalalove.html)

Pirate Bay

Lexicon

Lexicon · revision 2907 · last edited 2023-07-05

Bartbes

1. (n), a machine.
2. (n), a *sex* machine.

Box2D

1. (n), &^%&%&*^%!
2. (n), *!@$)&!

Circuloid

1. (n), a small green creature with a round body an two limbs. Typically smiles.
2. (n), have always been kind to us, fed us, marinated us.

Dolphin fetuses

1. (n), don't ask.

Knoll

1. (n), a grass hill with gleeful eyes.
2. (n), #love's resident bot.

Lover

1. (n), one who is skilled, or seeks to become skilled, with love.

Mike

1. (n), project co-founder, together with Rude.
2. (n), ???

Robin

1. (n), the Omniscient.
2. (n), not to be trusted.

Rude

1. (n), project co-founder, together with Mike.
2. (n), ???

Spork

1. (n), a piece of cutlery that combines features of both a spoon and a fork
2. (v), to punish or torture, usually directed at compilers or parsers.

Textmode

1. (n), meep!
2. (n), *huggles*

Thelinx

1. (n), the Strongest.
2. (n), the Supporter.

Virgin

1. (n), one who is new to love.

LibCompress

LibCompress · revision 6263 · last edited 2023-07-05

LibCompress is a compression and decompression library implemented entirely in Lua. It supports the LZW and Huffman algorithms, and can automatically choose the most efficient algorithm for your data.

- Original Wow-friendly LibCompress (http://w w w.w ow ace.com/addons/libcompress/)

See Also:

- TLTools

Library

Library · revision 16306 · last edited 2021-03-14

A library in context of game programming with LÖVE is a collection of useful functionality that make your life as game developer easier.

For example, a library may contain routines to simplify the handling of game objects such as monsters or powerups, or it could provide functions to handle sound more easily.

The scope of different libraries vary greatly. There are libraries that provide a single function to implement object orientation within Lua, to libraries that implement full blown game engines. Naturally, libraries with a narrower scope are more easily to integrate into your already existing project, while bigger ones impose a certain programming style. This is not necessarily bad, as setting a frame will often result in faster progress once you learned the concepts behind such a library.

Not every library is suited for your game and programming style. You have to pick and choose which one fits best.

All Libraries

Name	Description	LOVE Version	Keyword
30log	A small and effective object-orientation framework in 30 lines	Any	Class
Astray	Astray is a lua based maze, room and dungeon generation library for dungeon crawlers and rougelike video games.	Any	Procedural
Blob	A serialization and parser library for binary file formats	Any	Serializing
Cupid	A debugging shim providing a console, crash logging, and project reloading.	Any	Debug
Donut	Simplified debug message control system	Any	Debug
Dox	A Lua Documentation Generator Script	Any	Documentation
ELScheduler	Pure Lua library to manage timers	Any	Timer
Flux	Fast, lightweight tweening library	Any	Animation
Graphoon	A force directed graph algorithm written in Lua.	Any	Class
Grease	The one and (no longer) only networking library	Any	Networking
HC	A collision detection system	Any	Physics
HUMP	Gamedev-Powertools: Gamestates, timed function calls, tweening, vectors, object orientation, cameras and signals	Any	Framework
HooECS	It's a full-featured Entity-Component-System framework for making games with Lua	Any	Framework
Jumper	Fast pathfinding library for 2D grid-based maps	Any	AI
Kuey	Kuey is a library that encode/decode strings and files with a key.	Any	
LibCompress	Pure Lua Compression library	Any	
Lovebird	A browser-based debug console for LÖVE	Any	Debug
Lovetoys	It's a full-featured Entity-Component-System framework for making games with LUA. The original and only.	Any	ECS
Lua FOV	Field of View (FOV) algorithms implemented in Lua	Any	
LuaNameGen	Lua Name Generator for characters, objects, places, etc.	Any	Procedural
LuaTable	A library that converts tables to Lua code that can be saved.	Any	Serializing
Luaoop	OOP (Object Oriented Programming) class-based library	Any	Class
Lume	Utility functions geared towards game development	Any	
MGL	Mathematics for Graphics in pure Lua	Any	Math
MiddleClass	Object-orientation for Lua	Any	Class
Moses	A utility-belt library for fun and functional programming with Lua	Any	
Noobhub	Multiplayer and network messaging	Any	Networking
Popo	Character based programmable text library	Any	string
RL-Dice	A dice module aimed for roguelikes.	Any	Math
RandomLua	Random numbers generator for all platforms	Any	Math
SICK	Easy high-score keeping.	Any	
SQLite3	A simple, hassle-free, SQLite3 database solution for Lua.	Any	Database

Ser	A fast, robust, richly-featured table serialisation library	Any	
Serial	A serialization library optimized for very large tables.	Any	Serializing
Simple Educative Class Library	A class implementation	Any	Class
Stateswitcher	A state switcher library based on files (one file=one state).	Any	State
Transform (Library)	A library designed to ease the handling of coordinate/angle transformations in world	Any	Math
Tserial	Converts tables into strings and back. Good for saving games, multiplayer, etc.	Any	Serializing
Yonder	A ridiculously easy to use game state manager.	Any	State
array	A useful collection of methods to work with Lua tables in an array fashion.	Any	
beholder.lua	Event observation for Lua.	Any	
binser	Customizable Lua Serializer - Great for Saving Games	Any	Serializing
bitser	Serializes and deserializes Lua values with LuaJIT	Any	Serializing
bump.lua	Minimal collision detection lib.	Any	Physics
clasp	Tiny Lua class library	Any	Class
cron	A set of functions for executing actions at a certain time interval.	Any	Timer
cron.lua	Time-based functions for Lua.	Any	Timer
gamera	Camera library for LÖVE.	Any	
hxdx	Physics library (easier box2d)	Any	Physics
inspect.lua	Pretty-print for Lua.	Any	
libSaveTableToFile	Load and save tables as text files.	Any	Serializing
libxml	Lua XML Library	Any	Serializing
lol.lua	Object prototyping library for Lua.	Any	Class
love-loader	Load images and sounds in a separate thread.	Any	
love.bundle	Data saving lib for Love.	Any	Serializing
love.screen	Manage easily your screen resolution, caption, ...	Any	
luaFortune	Libraries for procedural generation in lua.	Any	
luasort	Array sorting algorithms	Any	Math
lure	HTML DOM Parser/Renderer.	Any	
memoize.lua	Generic memoization for Lua.	Any	
profile	A real-time profiler for finding bottlenecks in your game	Any	Debug
rstar	R*Tree implementation in Lua	Any	Data structure
sstrict.lua	Parses and detects mistakes in your Lua source code	Any	Debug
stateful.lua	Stateful Object Orientation for middleclass.	Any	State
strong	A library that provides many enhancements to strings.	Any	string
tile-collider	Module for resolving tile collisions (+slopes)	Any	Physics
tiny-ecs	Entity Component System for lua.	Any	Framework
tween	Small sets of functions for performing tweening in Lua	Any	Animation
tween.lua	Simple tweening lib for Lua.	Any	Animation
lua-https	A simple Lua HTTPS module using native platform backends where applicable.	12.0	Networking

Wasx	A very versatile input manager for LÖVE	11.x	Input
LOVELi	Retained mode layout and GUI library with StackLayout, AbsoluteLayout, Grid, Label, Button, Image, CheckBox, RadioButton, Switch, ProgressBar, Slider, TextBox and Border	11.5	Graphical User Interface
likeliHUD	A simple UI library with a QML-like declarative approach.	11.5	Graphical User Interface
subclass	A simple OOP library for Lua.	11.5	Class
layouter	Simple UI grid layout library.	11.4+	Graphical User Interface
cimgui-love	LÖVE module for Dear ImGui, obtained by wrapping cimgui with LuaJIT FFI.	11.4	Graphical User Interface
love.maker	A library that can minify, compile and compress your LÖVE2D project folder	11.3+	Distribution
TÖVE	Animated vector graphics and SVG rendering	11.2	SVG
breezefield	minimal love.physics wrapper	11.2	Physics
NoHighDPI	Disable High DPI scaling on Android devices. (useless since LÖVE 11.3)	11.1	Resolution
Live2LOVE	LÖVE library to show Live2D Cubism 2 models	11.0	Animation
Shädows	A dynamic shadows & lights library for löve	11.0	Lighting
lovesize	Library that makes it possible to work with a fixed game resolution, while scaling the screen and clipping objects outside of it, making a letterbox effect.	11.0	Resolution
FPSGraph	A simple lightweight graphing utility for LOVE	0.9.x	
Thranduil	UI module for LÖVE **sources, website : 404**	0.9.x	Graphical User Interface
Zoetrope	A starter kit for LÖVE, including sprites, collisions, maps, and debugging.	0.9.x	Framework
anim8	Animation Library for LÖVE.	0.9.x	Animation
sfxr.lua	Generate your sounds dynamically, at runtime	0.9.x	Sound
utf8	Provides basic support for manipulating UTF-8 strings.	0.9.2	string
utf8 (简体中文)	提供对 UTF-8 字符串操作的基本支持。	0.9.2	string
CameraMgr	A highly dynamic and robust camera library in around 200 lines of code.	0.9.1+	
StackingSceneMgr	A simple scene manager, scenes are lua files.	0.9.1+	
TransitionMgr	A simple transition manager, transitions are just lua files.	0.9.1+	
LuaPill	LuaPill (short for "Practical Isometric Layering Library for Lua") is an engine for generating isometric maps for Löve2D and Lua.	0.9.1	
Löve Bone	A robust object-oriented 2D Skeletal Animation framework for LÖVE.	0.9.1	Animation
Monocle	Debugging & Expression watching for Love2D	0.9.1	
lua-enet	Multiplayer networking module for games.	0.9.0	Networking
lua-enet (简体中文)	用于多人游戏的网络模块。	0.9.0	Networking
Quickie	Simple(r), but powerful GUI. **(renamed to SUIT)**	0.8.x	Graphical User Interface
Substitute541/Love3D	This allows you to simulate real 3D in a 2D engine. No Raycasting!	0.8.x	
fakecanvas	[WIP] Convenient canvas emulation for unsupported hardware	0.8.x	
Lope2D	A wrap over Love2D physics engine API. It makes dealing with the physics API easier and faster. It also has water pools simulation.	0.8.0+	Physics
newton	Export and load Box2D bodies made in PhysicsEditor	0.8.0+	Physics

Common Organization of Controls Kit	Comprehensive input handling library	0.8.0	Input
Jupiter	Table serialisation and file I/O.	0.8.0	Serializing
Lovely Tiles	Suite of tools for loading/rendering tilemaps	0.8.0	
LövelyMoon	Easily handle gamestates with this library!	0.8.0	State
text.lua	Pretty print texts	0.8.0	
babel	A simple library for internationalisation.	0.8	string
Goo	GUI library for Love	0.7.x	Graphical User Interface
LoveUI	GUI library for Love	0.7.x	Graphical User Interface
32 lines of goodness	32 lines of goodness is a small OO library that uses a domain specific language to make a neat syntax so OO is easy.	0.7.2	Class
LoveAStar	A* search, written in Lua, for use in LOVE.	0.7.2	AI
TLpath	Add A* pathfinding to games	0.7.1	AI
SLAM	A friendly sound manager.	0.7.0+	Sound
Message in a Bottle	A message library for LOVE.	0.7.0	Graphical User Interface
TEsound	A sound manager that makes it easy to use sound and music	0.7.0	Sound
TEsound (Español)	Un administrador de sonido que hace más fácil usar sonido y música	0.7.0	Sound
TLbind	A simple system for creating professional control schemes	0.7.0	Input
Pölygamy	Game State, Keyboard and Timer helpers	0.6.x	Framework
Swingers	A gesture library for LÖVE	0.6.x	
RotLove	The Roguelike Toolkit for Love2D. Based on rot.js and libtcod.	0.6.2+	Framework
Adult Lib	Run LÖVE 0.5.0 code with LÖVE 0.6.x.	0.6.2	Compatibility
socket	Module for HTTP, TCP, and UDP networking.	0.5.0	Networking
socket (简体中文)	HTTP、TCP和UDP网络模块。	0.5.0	Networking
Gspöt	GUI library for Love	0.11.x	Graphical User Interface
LUIGI	Lovely User Interfaces for Game Inventors	0.11.x	Graphical User Interface
Löve Frames	An advanced GUI library for LÖVE	0.11.x	Graphical User Interface
Talkies.lua	A messagebox system with multiple-choices, typing effect + sounds and more.	0.11.x	Graphical User Interface
Lovely Tactics Hex	Framework for creation of tactical RPGs.	0.11.4+	
love-nuklear	Lightweight immediate mode GUI for LÖVE games	0.11.2	Graphical User Interface
Easy GUI System	EGS is a simple GUI system with events and multiple controls, without over-complicating things.	0.10.x	Graphical User Interface
LoverNet	Module designed to make networking easier	0.10.x	Networking
Möan.lua	A messagebox system with multiple-choices and more **(renamed to Talkies)**	0.10.x	Graphical User Interface

SUIT	Simple User Interface Toolkit	0.10.x	Graphical User Interface
Tactile	A flexible and nice input library.	0.10.x	Input
fLUIds	love-imgui API wrap for simplicity	0.10.x	Graphical User Interface
love-imgui	IMGUI module for LOVE	0.10.x	Graphical User Interface
svglover	Load and display simple SVGs.	0.10.x	SVG
InputField	Text input handling library for LÖVE	0.10.2+	Input
ListBox	A dynamic ListBox for LÖVE 2D. Supports touches, mouses and keyboards	0.10.2	Graphical User Interface
LoveFS	LÖVE FileSystem Access	0.10.2	Filesystem
LÖVE PhysicsEditor Loader	PhysicsEditor Objects Models Loader	0.10.2	Physics
ONET	A Object Oriented Networking Library	0.10.2	Networking
Pseudo3D	Pseudo 3D figures for the löve framework **website: 404**	0.10.2	3D
TLfres	Lets games run at any resolution, easily and without stretching	0.10.2	Resolution
ziplib	A zipping library for löve	0.10.2	Compression
Lynput	Lynput is a library that makes input handling very easy and intuitive	0.10.0+	Input
vudu	Extensive in-engine debug interface with a console, variable browser, and more	0.10.0+	Debug
CTRL	General Input Library	0.10.0	Input
Lily	LÖVE Async Loading Library	0.10.0	Threaded Resource Loading
yaoui	UI Kit for LÖVE **(no longer maintained)**	0.10.0	Graphical User Interface
MiniFS	Minimal but powerful filesystem module.	0.10	Filesystem

LifeOnLove

LifeOnLove · revision 22311 · last edited 2021-03-14



I found it a little bit metaphoric and maybe it can be handy for someone...

main.lua

```lua
gameObjects = require("gameObjects")
grids = {}
step = 0
time = 0;

function love.load(arg)
  if arg[#arg] == "-debug" then require("mobdebug").start() end
  grids[0] = gameObjects.Grid:new(50,35)
  grids[1] = gameObjects.Grid:new(50,35)
  grids[0]:init("gun_s.txt")
  outGrid = 0
  inGrid = 0
  grid_switch()
end

function love.update(dt)
  time = time + dt
  if (time > 0.1) then
    time = 0
    grid_switch()
  end
end

function love.draw()
  love.graphics.setColor(0.085,1.0,0,0.1)
  love.graphics.rectangle("line", 265, 50, 250, 50)
  love.graphics.rectangle("line", 270, 55, 240, 40)
  love.graphics.setColor(0.085,1.0,0,1.0)
  love.graphics.print("LIFE", 375, 70);
  grids[outGrid]:draw(150, 200, 10)
end

function grid_switch()
  inGrid = step % 2
  outGrid = (step + 1) % 2
  step = step + 1
  grids[outGrid]:update_from(grids[inGrid])
end
```

---

gameObjects.lua

```lua
local gameObjects = {}

-- Meta class
gameObjects.Grid = {w = 0, h = 0}

-- Derived class method new
function gameObjects.Grid:new (w,h)
   local obj = {}
   setmetatable(obj, gameObjects.Grid)
   gameObjects.Grid.__index = gameObjects.Grid
   obj.w = w or 0
   obj.h = h or 0
   obj.cells = {}
   for i=1,w do
```

```
    obj.cells[i] = {}
   end
   for i = 1,w do
    for j = 1,h do
      obj.cells[i][j] = false
    end
   end
   return obj
end

function gameObjects.Grid:init(filename)
  local file = io.open(filename,"r")
  local line = file:read()
  local j = 1
  while line ~= nil and line ~= "" do
    for i=1,#line do
      local c = line:sub(i,i)
      if (c=='1') then
        self.cells[i][j] = true
      else
        self.cells[i][j] = false
      end
    end
    j = j + 1
    line = file:read()
  end
  io.close(file)
end

function gameObjects.Grid:update_from(grid)
  for i=1,self.w do
    for j=1,self.h do
      self.cells[i][j] = grid:check_cell(i,j)
    end
  end
end

function gameObjects.Grid:check_cell(x,y)
  local count = 0
  local positions = {{x-1,y},{x+1,y},{x,y-1},{x,y+1},{x-1,y+1},{x+1,y+1},{x-1, y-1},{x+1,y-1}}
  for i=1,8 do
    local ok = false
    local pos = positions[i]
    ok = self:is_legal(pos)
    if (ok) then
      ok = self.cells[pos[1]][pos[2]]
    end
    if (ok) then
      count = count + 1
    end
  end
  if (count > 3) then
    return false;
  end
  if (count < 2) then
    return false
  end
  if (count == 2) then
    return self.cells[x][y]
  end
  return true
end

function gameObjects.Grid:is_legal(pos)
  if (pos[1] <= 1 or pos[1] > self.w) then
    return false
  end
  if (pos[2] <= 1 or pos[2] > self.h) then
    return false
  end
  return true
end

```

```lua
function gameObjects.Grid:draw(x,y, cellsize)
  local fillType = "fill"
  for i=1,self.w do
    for j=1,self.h do
      if (self.cells[i][j] == true) then
        love.graphics.setColor(0.085,1.0,0,1.0)
        fillType = "fill"
      else
        love.graphics.setColor(0.085,1.0,0,0.1)
        fillType = "line"
      end

      love.graphics.rectangle(fillType, x+(i-1)*cellsize, y+(j-1)*cellsize, cellsize,cellsize)
    end
  end
  love.graphics.setColor(0.085,1.0,0,1.0)
  love.graphics.rectangle("line", x-10, y-10, self.w * cellsize + 20, self.h * cellsize + 20)
end

return gameObjects
```

---

gun_s.txt

```
00000000000000000000000000000000000000
00000000000000000000000001000000000000
00000000000000000000000101000000000000
00000000000001100000011000000000000110
00000000000010001000011000000000000110
01100000000100000100011000000000000000
01100000000100010110000101000000000000
00000000000100000100000001000000000000
00000000000010001000000000000000000000
00000000000011000000000000000000000000
00000000000000000000000000000000000000
```

Lily

Lily · revision 24633 · last edited 2023-07-05

Lily, LÖVE Async Loading Library.

Allows you to load resources in separate thread, like love-loader but uses more threads (which means faster) and more functions. It supports many resources, like Image, ImageData, Video, Source, SoundData, and more.

Example below correspond to Lily v3.0.6

Examples

Single loading

```
local lily = require("lily")
local myimage
local mysound

function love.load()
    lily.newImage("image.png"):onComplete(function(userdata, image)
        -- In v2.0, there's "userdata" before the return value
        myimage = image
    end)
    lily.newSource("song.wav"):onComplete(function(userdata, sound)
        -- In v2.0, there's "userdata" before the return value
        mysound = sound
        sound:play()
    end)
end

function love.draw()
    if myimage then love.graphics.draw(myimage, 0, 24, 0, 0.25, 0.25)
    else love.graphics.print("Loading image") end
    if not(mysound) then love.graphics.print("Loading song", 0, 12) end
end
```

Multi loading

```
local lily = require("lily")

function love.load()
    multilily = lily.loadMulti({
        {"newImage", "image1-0.png"},   -- You can use string
        {lily.newImage, love.filesystem.newFile("image1-1.png")},   -- or the function object
    })
    multilily:onComplete(function(_, lilies)
        image1 = lilies[1][1]
        image2 = lilies[2][1]
    end)
end

function love.update() end
function love.draw()
    if multilily:isComplete() then
        love.graphics.draw(image1, -1024, -1024)
        love.graphics.draw(image2)
    end
end
```

Links

- GitHub Repository (https://github.com/MikuAuahDark/lily) (including documentation)

- Forum Post (https://love2d.org/forums/viewtopic.php?f=5&t=84750)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LineJoin

LineJoin · revision 19181 · last edited 2026-04-19

Line join style.



Constants

miter
: The ends of the line segments beveled in an angle so that they join seamlessly.

none
: No cap applied to the ends of the line segments.

bevel
: Flattens the point where line segments join together.

See Also

- love.graphics
- love.graphics.setLineJoin

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LineStippleSnippet

LineStippleSnippet · revision 19065 · last edited 2021-03-14

With the removal of setLineStipple in LÖVE 0.8.0 here is an alternative, based on Bresenham's Line Drawing Algorithm.

```
function lineStipple( x1, y1, x2, y2, dash, gap )
    local dash = dash or 10
    local gap  = dash + (gap or 10)

    local steep = math.abs(y2-y1) > math.abs(x2-x1)
    if steep then
        x1, y1 = y1, x1
        x2, y2 = y2, x2
    end
    if x1 > x2 then
        x1, x2 = x2, x1
        y1, y2 = y2, y1
    end

    local dx = x2 - x1
    local dy = math.abs( y2 - y1 )
    local err = dx / 2
    local ystep = (y1 < y2) and 1 or -1
    local y = y1
    local maxX = x2
    local pixelCount = 0
    local isDash = true
    local lastA, lastB, a, b

    for x = x1, maxX do
        pixelCount = pixelCount + 1
        if (isDash and pixelCount == dash) or (not isDash and pixelCount == gap) then
            pixelCount = 0
            isDash = not isDash
            a = steep and y or x
            b = steep and x or y
            if lastA then
                love.graphics.line( lastA, lastB, a, b )
                lastA = nil
                lastB = nil
            else
                lastA = a
                lastB = b
            end
        end

        err = err - dy
        if err < 0 then
            y = y + ystep
            err = err + dx
        end
    end
end
```

Here's an alternative to the alternative above. This uses the function P(t) = A + t*(B-A), where A = (x1, y1) and B = (x2, y2) to find n = length/size points and draw n lines using love.graphics.line with distance = interval*size between each line.

```
function dottedLine(x1, y1, x2, y2, size, interval)
    local size = size or 5
    local interval = interval or 2

    local dx = (x1-x2)*(x1-x2)
    local dy = (y1-y2)*(y1-y2)
    local length = math.sqrt(dx+dy)
    local t = size/interval

    for i = 1, math.floor(length/size) do
        if i % interval == 0 then
            love.graphics.line(x1+t*(i-1)*(x2-x1), y1+t*(i-1)*(y2-y1),
                               x1+t*i*(x2-x1), y1*t*i*(y2-y1))
        end
```

```
    end
end
```

LineStyle

LineStyle · revision 28003 · last edited 2023-07-05

The styles in which lines are drawn.

Constants

rough
: Draw rough/aliased lines.

smooth
: Draw smooth/anti-aliased lines.

See Also

- love.graphics
- love.graphics.setLineStyle
- love.graphics.setLineStipple

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

ListBox

ListBox · revision 24514 · last edited 2023-07-05

A dynamic ListBox GUI library created by Luiz Renato for LÖVE 2D. Supports touches, mouses and keyboards

https://github.com/darkmetalic/ListBox ↗

DEMO:

```
function love.load()

font = love.graphics.newFont(15)

list = require "listbox"

local tlist={
x=200, y=100,
font=font,ismouse=true,
rounded=true,
w=200,h=300,showindex=true}

list:newprop(tlist)

list:additem("MySite","www.MySite.com")
list:additem("MyNumber",123456)
end

function love.keypressed(key)
list:key(key,true)
end

function love.wheelmoved(x,y)
list:mousew(x,y)
end

function love.update(dt)
list:update(dt)
end

function love.draw()
list:draw()
end
```

DEFAULT VALUES:

```
listbox={

x=100,y=100, -- listbox position x,y (number)

w=200,h=400, -- listbox size Width x Height (number)

hor=true,ver=true, -- Show scroller Horizontal and Vertical (boolean)

sort=false, -- sort mode: string "asc" or "desc" or boolean true(asc) or false(disabled)

enabled=true, -- enable or disable the listbox, it remains visible, but does not interact (boolean)

visible=true, -- visible or invisible, the listbox may interact invisibly if it is not disabled

adjust=true, -- adjuste (boolean)

rounded=false, -- rounded rectangle (boolean)

expand=false, -- expand the listbox (boolean)

expandmode="size", -- expand mode: size true (boolean|string) or position "pos" (string)

expx=12, -- expand pixels (number)

radius=15, -- scroll ball size/radius (number)
```

```lua
sep=";;", -- separator that limits where the text ends and the data starts in the index (string)

ismouse=true, -- if you will use mouse (boolean)

istouch=false, -- if you will use touches (boolean)

gw = love.graphics.getWidth(), -- first Width captured to resize, do not works if you confugured(conf.lua) in fullscreen (or set gw)

gh = love.graphics.getHeight(), -- first Height captured to resize, do not works if you confugured(conf.lua) in fullscreen (or set gh)

incenter=false, -- in center of screen (boolean)

asize=false, -- auto size the listbox (boolean)

font = love.graphics.newFont(12), -- listbox font (object)

fcolor={0,190,0}, -- font color RGB (table)

bordercolor={50,50,50}, -- border color RGB (table)

selectedcolor={50,50,50}, -- selected color RGB (table)

fselectedcolor={200,200,200}, -- font selected color RGB (table)

selectedtip="fill", -- selected mode "fill" or "line" (string)

bgcolor={20,20,20}, -- background selected color RGB (table)

showindex=false, -- show index (boolean)

indexzeros=1, -- index left zeros (number)

selected=false, -- first index selected (number) or (false) none

gpadding=5 -- global padding (number) }
```

FUNCTIONS:

```lua
--[[NOTE: all default values and functions are in lowercase

text = string or number
data = string or number
str = string or number
index = integer (number)
item = index (text and data)
tlist = table
mode = boolean or string
update = boolean (uplist)
font = object
[] = optional]]

listbox:additem(text,[data],[update]) or listbox:insert(text,[data],[update]) -- add new item, return true if successfully
ex: list:additem("MySite","www,mysite.com",true)
ex: list:insert("MySite","www,mysite.com",false)

listbox:delitem(index) or listbox:remove(index) -- delete a specific item, return true if successfully
ex: list:delitem(16)

listbox:delall() or listbox:empty() -- remove all items, no returns

listbox:getcount() or listbox:maxn() -- return max items (number) if successfully or false (boolean)

listbox:getselected() -- return selected index (number) if successfully or false (boolean)

listbox:getdata(index) -- return item data (string) if successfully or "" (empty string)

listbox:gettext(index) -- return item text (string) if successfully or "" (empty string)

listbox:getfusion(index) or listbox:concat(index) -- return item concatenated text..data (string) if successfully or (false)

listbox:getsize() -- two returns dimension Width Height of the listbox (number)
```

```
listbox:getpos() -- two returns position X Y of the listbox (number)

listbox:uplist() -- update/atualize the listbox, no returns

listbox:unselect() or listbox:deselect() -- deselect the listbox, no returns

listbox:setexpand(mode) -- set expand mode true enables or "size" and enables or "pos" and enables or false disable, no returns
ex: list:setexpand("size")
ex: list:setexpand("pos")
ex: list:setexpand(true)
ex: list:setexpand(false)

listbox:settext(index,text) -- replaces item text, return true if successfully
ex = list:settext(2,"Second")

listbox:setdata(index,data) -- replaces item data, return true if successfully

listbox:setitem(index,text,[data]) -- replaces item text and data, return true if successfully
ex = list:setitem(2,"Second","SecondData")

listbox:setfont(font) -- set the listbox font, return true if successfully

listbox:setselected(index) -- selects a item in the listbox, return true if successfully

listbox:newprop(tlist) -- creates a new property for the listbox

listbox:setrounded(boolean) -- listbox is rounded or rectangle

listbox:findlist(str,[mode],[sensible]) or listbox:search(str,[mode],[sensible]) -- return (table) index of all found str if successfully,
or false (boolean)
ex: result = list:findlist("love","text",true)
if result then print(result[1]) end

ex: result = list:search("love","textdata",false)
if result then list:setselected(table.maxn(result)) end

ex: result = list:findlist("love","data",false)
if result then list:setdata(result[1],"Love2D") end

listbox:resort(mode,update) -- sort the listbox in ascending or descending, no returns
ex: list:resort("asc",true)
ex: list:resort(true,true) -- ascending
ex: list:resort("desc",true)
ex: list:resort(false) -- disables, but not defaults sorted items

listbox:setsize(Width,Height) -- set dimension (number)

listbox:setpos(X,Y) -- set position (number)

listbox:setindex(boolean) -- show or hide index numbers counted, no returns

listbox:setvisible(boolean) -- visible or invisible, the listbox may interact invisibly if it is not disabled

listbox:setenabled(boolean) -- enable or disable the listbox, it remains visible, but does not interact (boolean)

listbox:isvisible() -- get if is visible, return boolean

listbox:isenabled() -- get if is enabled, return boolean

listbox:isover() -- get if is pointed(over) in the listbox, return boolean

listbox:isselectchange() -- get if item selected is changed, return boolean

listbox:isdoublec() -- get if is double click in the selected item, return boolean

listbox:autosize(boolean) -- auto size the listbox, no returns

listbox:center(boolean) -- auto center in the screen, no returns

listbox:export(file,[mode1],[mode2]) or listbox:save(file,[mode1],[mode2]) -- save all items in a file to save directory or on the side
ex: list:export("list.txt",true,"text") -- ("text" or true) exports only items texts in save directory (and index if enabled)
ex: list:export("list.txt",false,false) -- exports all items text..separator..data on the side (and index if enabled)
ex: list:export("list.txt") -- exports all items text..separator..data on the side (and index if enabled)
```

```
NOTE: on the side (mode1-false) not works in android or ios, automatically exports in your save directory (mode1-true)

listbox:import(file) or listbox:open(file) -- load all lines in the listbox, if have separator will add as data, return true if successfully
ex: list:import("newlist.list")
```

EXTRA TOOLS:

```
listbox:getfileext(filepath) -- return file extension

listbox:getfilename(filepath) -- return file name

listbox:getdirname(path) -- return directory name

listbox:onlynumbers(str) -- return only numbers of a given string

listbox:onlyletters(str) -- return only letters of a given string

listbox:insensible(string) -- return string insensible "aA"=="aa"

listbox:enudir(path,[extensions]) -- return (table) of all files enumerated with a specific extension (or no), or false
ex: result = list:enudir("music/rock",".wav .mp3 .ogg")
ex: result = list:enudir("images",".png .jpeg .bmp")
ex: result = list:enudir("contents")

listbox:sectotime(seconds,[milliseconds]) -- return a time of a given number, or "00:00:00"
ex: result = list:sectotime(1000) -- return 00:16:40
ex: result = list:sectotime(1e3,true) -- return 00:16:40.0

listbox:rgbtohex(tcolor) -- return hexadecimal (string) of a given RGB color (table)
ex: tcolor = {100,200,255}
result = list:rgbtohex(tcolor) -- return 64C8FF

ex: print("#"..list:rgbtohex({100,200,255})) -- print #64C8FF

listbox:mount(path) -- mounts a directory outside of its executable
```

MAIN FUNCTIONS:

```
listbox:key(key,shortcut) -- required if you want to interacts the listbox using keyboards, returns key pressed

ex: function love.keypressed(key)
list:key(key,true) -- if shortcut is enabled, select the first item character found while pressed a key
end

listbox:mousew(x,y) -- required if you want to interacts the listbox using mouse wheel

ex: function love.wheelmoved(x,y)
list:mousew(x,y)
end

listbox:update(dt) -- required if you want to interacts the listbox...

ex: function love.update(dt)
list:update(dt)
end

listbox:press(x,y) or list.ispress=true -- optional for mouses, required for touches

ex: function love.touchpressed(id, x, y, dx, dy, pressure)
list:press(x,y)
end

ex: function love.touchpressed(id, x, y, dx, dy, pressure)
list.ispress=true
end

listbox:moved(x,y) or list.ispress=true -- optional for mouses, required for touches

listbox:released(x,y) or list.ispress=false -- optional for mouses, required for touches

listbox:resize() -- required if you want to autosize the listbox on resize
```

```
ex: function love.resize(w,h)
list:resize()
end

listbox:draw() -- required if you want to use the listbox

ex: function love.draw()
list:draw()
end
```



Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Live2LOVE

Live2LOVE · revision 24640 · last edited 2023-07-05

LÖVE module to load and render Live2D Cubism 2 models.

Due to Live2D restrictions, this library must be compiled from source. Furthermore, source modification to LÖVE is needed for iOS and Android to `package.preload` this library.

Examples

If you have model definition file live2d/model.json

```lua
local Live2LOVE = require("Live2LOVE")
local model

function love.load()
    -- Live2LOVE fully respect love.filesystem
    -- so loading from fused game works out-of-the-box
    model = Live2LOVE.loadModel("live2d/model.json")
    model:setMotion("idle", "loop")
end

function love.draw()
    -- Model drawing also respects love.graphics
    -- state, so you can render model to Canvas (enable stencil bufffer!)
    -- or apply Shader to it. Furthermore, graphics transformation is also
    -- applied to the model.

    -- Push stack
    love.graphics.push("all")
    -- make the model cyan-masked, and 75% opaque
    love.graphics.setColor(0, 1, 1, 1, 0.75)
    -- Scale down the model by 50%
    love.graphics.scale(0.5, 0.5)
    -- Rotate it by 30 degrees
    love.graphics.rotate(math.pi/6)
    -- Move it to +300+200
    love.graphics.translate(300, 200)
    -- Draw the model
    model:draw()
    -- Pop stack
    love.graphics.pop()
end
```

Links

- GitHub repository (https://github.com/MikuAuahDark/Live2LOVE)
- Documentation (https://mikuauahdark.github.io/Live2LOVE) (incomplete)
- Forum post (https://love2d.org/forums/viewtopic.php?t=85306)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LoadMode

LoadMode · revision 28078 · last edited 2023-07-05

Available since LÖVE 12.0
This enum is not supported in earlier versions.

Possible load modes for love.filesystem.load. This is equivalent to 3rd parameter (mode) in Lua 5.3 load (https://www.lua.org/manual/5.3/manual.html#pdf-load)

Constants

b
: Only load the file if it's binary pre-compiled chunk.

t
: Only load the file if it's plain text file.

bt
: Always loads no matter if it's plain text or pre-compiled chunk.

Notes

For now, load modes other than "bt" is not supported in vanilla Lua 5.1.

See Also

- love.filesystem
- love.filesystem.load

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

lol.lua

lol.lua · revision 26719 · last edited 2023-07-05

lol.lua is an object prototyping implementation for Lua. It features **cloning**, **copying and deep copying**, and **dynamic prototyping**.

Example:

```lua
local lol = require('lol')

local point = lol:clone()
function point:init()
    self.x, self.y = 0, 0
end

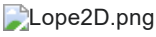
local p = point:clone()
print(p.x, p.y) -- 0    0
```

Grab it on Github (https://github.com/mpatraw/lol.lua).

- Source (https://github.com/mpatraw/lol.lua/blob/master/lol.lua)
- Examples (https://github.com/mpatraw/lol.lua/tree/master/examples)

Lope2D

Lope2D · revision 24450 · last edited 2023-07-05



Lope2D is a wrap over Love2D physics engine API. It makes dealing with the physics API easier and faster.

Most of beginners who wants to make his own game based on physics, usually have problems using its API. Bodies, shapes, fixtures. A chaos of methods, isn't it? Here the Lope comes.

Wiki + Downloads (https://bitbucket.org/erlimoen/lope2d-v2/)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

love.(underscore)os

love.(underscore)os · revision 29080 · last edited 2024-01-13

love._os

This is an platform dependent string that tells your game in what platform is running on.

This is useful when trying to make a multi-platform game where you can account on the capabilities of such device A good example of this is when a game detects that is running on mobile it'll show the on-screen gamepad if no physical input devices like a gamepad or mouse & keyboard are present.

Another good example is being a perfectionist and following theming rules of the device like accent color and dark mode, since there's no internal *generic* API call that love could provide us like lets ***make stuff up*** an so we call love.graphics.getSystemTheme() or something, we have to implement it ourselves and this depends of the platform: on windows is a registry key, on android we have to call a C++ function or something along those lines, on linux is even harder it depends on the window manager, desktop enviroment and what distro is the user running on, And for mac and iphone honestly it is not in my realm of knowledge.

Other good example is that if a device is known for having trouble running your game warn the user of such problems, a more shady use of this example is blocking/filtering/redirecting a device from an online service/feature that the game has, either because it gives too much advantage or makes problems on your server where you host it (a.k.a separating mobile/desktop/console users from each other unless they want to compete with other platforms or a platform does *not* meet your standards being blocked right away, either by processing power or unfair advantage)

This variable can *Officialy* have 6 outcomes that most of the time are self explanatory "Windows" "Android" "OS X" "Linux". Also take note that the ports can make use of this variable making them unreliable on non official supported platforms ports.

The official outcomes as shown in the source of v11.5 (/src/modules/system/Sytem.cpp:65) are:

MacOS: "OS X" IOS: "iOS" Windows universal windows package: "UWP" Windows binary executable: "Windows" Android: "Android" Linux based systems: "Linux"

In official builds of love when none of this are the platform being run on it returns "Unknown"

Love3D

Love3D · revision 28138 · last edited 2022-12-03

Love3D is the name of multiple 3d libraries and tech demos for LÖVE and is a term that can refer to the engine's 3d capabilities in general.

Active projects

Showing activity during the last 12 months:

- groverburger's 3D engine (https://github.com/groverburger/g3d) (2019-)
- Menori (https://github.com/rozenmad/Menori) simple 3D and 2D rendering library (2021-)
- 3DreamEngine (https://github.com/3dreamengine/3DreamEngine) An *awesome* 3d engine for Love

Inactive projects

- excessive/love3d (https://github.com/excessive/love3d) library (2015-2017)
  - Karai17/LÖVE3D (https://love2d.org/forums/viewtopic.php?f=5&t=78943) demo (2015)
- Substitute541/Love3D (2012)
- 10$man's L3D (https://love2d.org/forums/viewtopic.php?f=5&t=11307) (2012)
- ishkabible's 3D Engine (https://love2d.org/forums/viewtopic.php?f=5&t=2457) (2011)
- Jasoco's 3D Engine (https://love2d.org/forums/viewtopic.php?f=5&t=1399) (2010)

LOVE:About

LOVE:About · revision 22616 · last edited 2021-03-14

LÖVE is an **awesome** framework you can use to make 2D games in Lua. It’s free, open-source, and works on Windows, Mac OS X, Linux, iOS and Android. There's really nothing else you need to know, so get started!

LOVE:Autoconfirmed users

LOVE:Autoconfirmed users · revision 2315 · last edited 2021-03-14

Autoconfirmed users

This article has been copied from Wikipedia (http://w w w.w ikipedia.org/), the free encyclopedia. See Article (http://en.w ikipedia.org/w iki/Wikipedia:Autoconfirmed_users#Autoconfirmed_users)

A number of actions on the English Wikipedia are restricted to user accounts that pass certain thresholds of age (time passed since the first edit) and editcount: users who meet these requirements are considered part of the pseudo-group 'autoconfirmed'. Autoconfirmed status is checked every time a user performs a restricted action: consequently, it is granted automatically by the software. The precise requirements for autoconfirmed status vary according to circumstances: for most users on en.wiki, accounts that are more than four days old and have made at least 10 edits are considered autoconfirmed. However, users editing through the Tor network are subjected to stricter autoconfirmed thresholds: 90 days and 100 edits.

Autoconfirmed status is required to move pages, edit semi-protected pages, and upload files or upload a new version of an existing file. Autoconfirmed users are no longer required to enter a CAPTCHA for most events. Autoconfirmed users may mark pages created by others as patrolled in Special:NewPages and save books to the wiki. In addition, the Edit filter has a number of warning settings that only affect editors who are not autoconfirmed.

LOVE:Copyrights

LOVE:Copyrights · revision 12162 · last edited 2021-03-14

All content of the LÖVE Wiki is licensed under the GNU Free Documentation License (http://www.gnu.org/copyleft/fdl.html) 1.3.

LOVE:Users

LOVE:Users · revision 20303 · last edited 2025-05-21

Users

By registering at the forum (http://love2d.org/forums) you register here.

The follow information was copied from Wikipedia (http://w w w .w ikipedia.org/)'s page about registration benefits (http://en.w ikipedia.org/w iki/Wikipedia:W hy_create_an_account):

You do not have to log in to read Wikipedia, nor is a registered account required to edit Wikipedia articles—almost anyone can edit almost any article at any given time, even without logging in. When not logged in to an account, by default a person edits instead by their IP address, assigned by their internet service provider (ISP). Nevertheless, creating an account is quick, free, and non-intrusive, and it is generally considered a good idea to do so for a variety of reasons.

Summary of Benefits

(from Wikipedia (http://w w w .w ikipedia.org/), also)

Registered users can:

- Start new pages, including their own user page.
- Edit semi-protected pages once ten edits have been made, and their account is at least four days old.
- Rename pages, after awhile.
- Upload images, after awhile.
- Send, and optionally receive, e-mails to and from other users.
- Set personal preferences on their account.
- Keep a watchlist to track changes made on articles of interest.
- Choose a username. (Should be an appropriate name)
- More easily review their own contributions.
- Customize the appearance and behavior of the website to their own preference.
- Use more advanced editing tools.
- Vote in Arbitration Committee elections and Wikimedia Board elections.
- Edit without their IP address being visible, except to a few, highly trusted, users who have the Checkuser permission.
- Log into other Wikimedia projects.
- Customize their own signatures when they sign on talk pages by going to their preferences.

More Information

For more information please visit Wikipedia's article (http://en.w ikipedia.org/w iki/Wikipedia:Why_create_an_account).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LoveAStar

LoveAStar · revision 14687 · last edited 2023-07-05

A* search for pathfinding in games

LoveAStar in action [1] (http://imgur.com/w kVxm)

Benefits

- Runs optimally (for a static map, on my Dell XPS 8100, can build 62 consecutive paths on a 25 x 25 square grid while staying above 30 FPS)
- Extendable to any map type (square, rectangles, hexagons, world maps, possibly even 3D spaces)
- Easy to implement. Ships with instructions for "flattening" a map to be used in my version of A*.

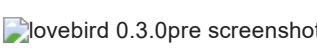
Download @ GitHub: [2] (https://github.com/mkosler/LoveAStar)

View the original thread @ Love forums: [3] (http://love2d.org/forums/viewtopic.php?f=5&t=7174)

Lovebird

Lovebird · revision 19225 · last edited 2023-07-05

Lovebird is a browser-based debug console for LÖVE.



Check out the README (https://github.com/rxi/lovebird/blob/master/README.md) for instructions on how to get it set up.

Once you have it running you can enter lua commands into the input box at the bottom, these will be run immediately. All print output will be printed to the output console.

At the right side of the page you can browse the globally accessible environment. This shows each variable name with its value next to it. If you click a variable's value its name will be inserted into the input box; this allows you to, for example, click a variable's value and type "= 10" to set that variable to 10.

Lovebird should work wherever LuaSocket is available -- this includes LÖVE 0.8+.

Links

- Readme (https://github.com/rxi/lovebird/blob/master/README.md)
- Github (https://github.com/rxi/lovebird)

LoveFS

LoveFS · revision 24437 · last edited 2023-07-05

Load and save files outside love.filesystem.

Latest version (https://github.com/linux-man/lovefs)

Original thread (https://love2d.org/forums/viewtopic.php?f=5&t=9479)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LOVELi

LOVELi · revision 29833 · last edited 2025-09-29

On this page

- LOVELi (LÖVE Layout & GUI)
- How to
- Layouts
  - StackLayout
  - AbsoluteLayout
  - Grid
  - FlexLayout
- Controls
  - Label
  - Button
  - ImageButton
  - Image
  - CheckBox
  - RadioButton
  - Switch
  - ProgressBar
  - Slider
  - TextBox
  - MaskedTextBox
  - Picker
  - Border
  - GraphicsView
  - ScrollView
- Keyboard Accessibility
- Joystick Accessibility

LOVELi (LÖVE Layout & GUI)

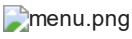
A LÖVE 2D retained mode layout and GUI library.

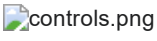
Includes StackLayout, AbsoluteLayout, Grid and FlexLayout layouts.

Includes Label, Button, ImageButton, Image, CheckBox, RadioButton, Switch, ProgressBar, Slider, TextBox, MaskedTextBox, Picker, Border, GraphicsView and ScrollView controls.

This library was inspired by .NET's MAUI controls.

Tested on version 11.5.





How to

1. Copy LOVELi folder to your project. Download it at GitHub (https://github.com/mtanksl/LOVELi)

2. Import the library and hook the following methods:

```
local loveli = require("LOVELi")
function love.load(arg)
  -- See below --
  layoutmanager = loveli.LayoutManager:new{}
    :with(rootcontrol)
end
function love.keypressed(key, scancode, isrepeat)
  layoutmanager:keypressed(key, scancode, isrepeat)
end
function love.textinput(text)
  layoutmanager:textinput(text)
end
function love.mousepressed(x, y, button, istouch, presses)
  layoutmanager:mousepressed(x, y, button, istouch, presses)
end
function love.mousereleased(x, y, button, istouch, presses)
  layoutmanager:mousereleased(x, y, button, istouch, presses)
end
function love.mousemoved(x, y, dx, dy, istouch)
  layoutmanager:mousemoved(x, y, dx, dy, istouch)
end
function love.wheelmoved(dx, dy)
  layoutmanager:wheelmoved(dx, dy)
end
function love.joystickhat(joystick, hat, direction)
  layoutmanager:joystickhat(joystick, hat, direction)
end
function love.joystickpressed(joystick, button)
```

```
  layoutmanager:joystickpressed(joystick, button)
end
function love.resize(w, h)
  layoutmanager:resize(w, h)
end
function love.update(dt)
  layoutmanager:update(dt)
end
function love.draw()
  layoutmanager:draw()
end
```

Layouts

3. Use one of the following layouts:

StackLayout

A StackLayout organizes elements in a one-dimensional stack, either horizontally or vertically.

```
local stacklayout = loveli.StackLayout:new{ orientation = "vertical", spacing = 5, width = "*", height = "*", margin = loveli.Thickness.parse(10) }
  :with(loveli.Button:new{ text = "Button 1", horizontaltextalignment = "start", verticaltextalignment = "center", width = 75, height = 23, horizontaloptions = "start" } )
  :with(loveli.Button:new{ text = "Button 2", horizontaltextalignment = "center", verticaltextalignment = "center", width = 75, height = "*", horizontaloptions = "center" } )
  :with(loveli.Button:new{ text = "Button 3", horizontaltextalignment = "end", verticaltextalignment = "center", width = 75, height = 23, horizontaloptions = "end" } )
local rootcontrol = stacklayout
```

AbsoluteLayout

An AbsoluteLayout is used to position and size elements using explicit values.

```
local absolutelayout = loveli.AbsoluteLayout:new{ width = 250, height = 250, margin = loveli.Thickness.parse(10) }
  :with(loveli.Button:new{ text = "Top Left", horizontaltextalignment = "center", verticaltextalignment = "center", x = 0, y = 0, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Top Center", horizontaltextalignment = "center", verticaltextalignment = "center", x = 125 -38.5, y = 0, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Top Right", horizontaltextalignment = "center", verticaltextalignment = "center", x = 250 -75, y = 0, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Center Left", horizontaltextalignment = "center", verticaltextalignment = "center", x = 0, y = 125 -11.5, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Center", horizontaltextalignment = "center", verticaltextalignment = "center", x = 125 -38.5, y = 125 -11.5, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Center Right", horizontaltextalignment = "center", verticaltextalignment = "center", x = 250 -75, y = 125 -11.5, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Bottom Left", horizontaltextalignment = "center", verticaltextalignment = "center", x = 0, y = 250 -23, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Bottom Center", horizontaltextalignment = "center", verticaltextalignment = "center", x = 125 -38.5, y = 250 -23, width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Bottom Right", horizontaltextalignment = "center", verticaltextalignment = "center", x = 250 -75, y = 250 -23, width = 75, height = 23 } )
local rootcontrol = absolutelayout
```

Grid

A Grid is used for displaying elements in rows and columns, which can have proportional or absolute sizes.

```
local grid = loveli.Grid:new{ rowdefinitions = { "auto", "1*", "1*" }, columndefinitions = { "auto", "1*", 150 }, width = "*", height = "*", margin = loveli.Thickness.parse(10) }
  :with(1, 1, absolutelayout)
```

```
  :with(2, 2, stacklayout)
local rootcontrol = grid
```

FlexLayout

FlexLayout is a layout that can arrange its children horizontally and vertically in a stack, and can also wrap its children if there are too many to fit in a single row or column. In addition, FlexLayout can control orientation and alignment, and adapt to different screen sizes.

```
local flexlayout = loveli.FlexLayout:new{ direction = "row", justifycontent = "spaceevenly", aligncontent = "center", width = 300, height = 100, verticaloptions = "center", horizontaloptions = "center", margin = loveli.Thickness.parse(10) }
  :with(loveli.Button:new{ text = "Button 1", width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Button 2", width = 75, height = 23, margin = loveli.Thickness.parse(10)  } )
  :with(loveli.Button:new{ text = "Button 3", width = 75, height = 23, verticaloptions = "start" } )
  :with(loveli.Button:new{ text = "Button 4", width = 75, height = 23 } )
  :with(loveli.Button:new{ text = "Button 5", width = 75, height = 23 } )
local rootcontrol = flexlayout
```

Controls

4. And the following controls:

Label

Label displays single, or multiple, lines of text.

```
loveli.Label:new{ text = "Label" }
```

Button

Button displays text and responds to a tap or click that directs an app to carry out a task.

```
loveli.Button:new{
  text = "Button",
  clicked = function(sender) end,
  -- Default values for text controls:
    ismultiline = false,
    font = love.graphics.getFont(),
    horizontaltextalignment = "center",
    verticaltextalignment = "center",
    textcolor = loveli.Color.parse(0x000000FF),
  -- Default values for style:
    backgroundcolor = loveli.Color.parse(0xC0C0C0FF),
    bordercolor = loveli.Color.parse(0xFFFFFFFF),
  -- Default values for size and position:
    x = 0,
    y = 0,
    width = "auto",
    height = "auto",
    minwidth = 0,
    minheight = math.huge,
    maxwidth = 0,
    maxheight = math.huge,
    margin = loveli.Thickness.parse(0),
    horizontaloptions = "start",
    verticaloptions = "start",
  -- Default values for all controls:
    name = nil,
    isvisible = true,
    isenabled = true
}
```

ImageButton

ImageButton view combines the Button view and Image view to create a button whose content is an image.

```
loveli.ImageButton:new{ source = "icon.png", aspect = "aspectfit", width = 64, height = 32, clicked = function(sender) end }
```

Image

Image displays an image that can be loaded from a local file.

```
loveli.Image:new{ source = "icon.png", aspect = "aspectfit", width = 64, height = 32 }
```

CheckBox

CheckBox enables you to select a boolean value using a type of button that can either be checked or empty.

```
loveli.CheckBox:new{ ischecked = false, checkedchanging = function(sender, oldvalue, newvalue) return true end, checkedchanged =
function(sender, oldvalue, newvalue) end }
```

RadioButton

RadioButton is a type of button that allows the selection of one option from a set.

```
loveli.RadioButton:new{ ischecked = false, groupname = "1", checkedchanging = function(sender, oldvalue, newvalue) return true end,
checkedchanged = function(sender, oldvalue, newvalue) end }
```

Switch

Switch enables you to select a boolean value using a type of button that can either be on or off.

```
loveli.Switch:new{ istoggled = false, toggling = function(sender, oldvalue, newvalue) return true end, toggled = function(sender, oldvalue,
newvalue) end }
```

ProgressBar

ProgressBar uses an animation to show that the app is progressing through a lengthy activity.

```
loveli.ProgressBar:new{ progress = 0.75 }
```

Slider

Slider enables you to select a value from a range.

```
loveli.Slider:new{ value = 5, minimum = 0, maximum = 10, valuechanging = function(sender, oldvalue, newvalue) return true end, valuechanged
= function(sender, oldvalue, newvalue) end }
```

TextBox

TextBox enables you to enter and edit a single, or multiple, lines of text.

```
local textbox = loveli.TextBox:new{ text = "TextBox", ispassword = false, maxlength = 255, textchanging = function(sender, oldvalue,
newvalue) return true end, textchanged = function(sender, oldtext, newtext) end }
```

MaskedTextBox

```
local maskedtextbox = loveli.MaskedTextBox:new{ mask = "(000) 000-0000", patterns = { ["0"] = { pattern = "%d" } }, textchanging =
function(sender, oldvalue, newvalue) return true end, textchanged = function(sender, oldtext, newtext) end }
```

Picker

Picker displays a short list of items, from which the user can select an item.

```
local picker = loveli.Picker:new{ selectedindexchanged = function(sender, oldindex, newindex) print(oldindex, newindex) end, itemssource = { "Option 1", "Option 2", "Option 3", "Option 4", { tostring = function(self) return "Option 5" end } }, text = "Select...", width = 100, height = "auto" } )
```

Border

Border is a container control that adds padding, draws a border, background, or both, around another control.

```
local border = loveli.Border:new{ padding = loveli.Thickness.parse(10), bordercolor = loveli.Color.parse(0x00000000), backgroundcolor = loveli.Color.parse(0x00000000) }
  :with(textbox)
```

GraphicsView

GraphicsView is a graphics canvas on which 2D graphics can be drawn.

```
local graphicsview = loveli.GraphicsView:new{ drawable = function(sender, x, y, width, height) end, width = "*", height = "*" }
```

ScrollView

ScrollView is a view that's capable of scrolling its content.

```
local scrollview = loveli.ScrollView:new{ scrolled = function(sender, dx, dy) end, orientation = "both", verticalscrollbarvisibility = "default", verticalscrollbarwidth = 5, verticalscrollbarincrement = 100, width = "*", height = "*" }
  :with(stacklayout)
```

Keyboard Accessibility

This library supports **tab**, **shift + tab**, **down** arrow and **up** arrow for navigation, **space** and **return** for action.

Joystick Accessibility

This library supports **down** hat and **up** hat for navigation, **any button** for action.

Lover

Lover · revision 3408 · last edited 2023-07-05

A **lover** is someone who uses LÖVE to create a game (/program/anything else), so, this is how it works:

Dev
: Spawns LÖVE.

Lover
: Develops games.

User
: Plays games.

The name lover has been acknowledged as the best name for this in the Tuesday Convention.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LoverNet

LoverNet · revision 16330 · last edited 2023-07-05

LoverNet is a LÖVE module designed to help make networking easier by leveraging JSON and enet written by josefnpat.

Links

- Source code (https://github.com/josefnpat/LoverNet)

LoveSpin

LoveSpin · revision 2786 · last edited 2021-03-14

LoveSpin is a game that steals graphics from this osu! beatmap (http://osu.ppy.sh/s/2463) and turns Loituma/Leekspin into a game where you show off your moving-mouse-in-circles skills!

It is developed and maintained by thelinx. Feedback goes here (http://love2d.org/forums/viewtopic.php?f=5&t=1621).



Download now! (http://love2d.org/forums/download/file.php?id=1704)

Changelog

1.2

- Now uses LoveUpdate.

1.1

- Now uses UaLove back-end.

Known issues

- If the music playback fails, you get about half a second to spin. That is a LÖVE issue, unfortunately.
- The programmer art is horrible.

LoveToAll

LoveToAll · revision 19698 · last edited 2021-03-14

This program (for Windows) uses functions from LoveToAndroid by startgamedev and Bat To Exe Converter by F2KO to the results for APK and EXE (Single EXE With all dlls embeded) in several ways possible by simply and easy methods (such as pressing a keyboard shortcut (custom), or by dragging on the shortcut icon, or by dragging into the program (DragAndDrop) or by pressing the button "BUILD"). That's enough to have the minimum requirements for an LOVE2D project only. You do not even have to create your .love zip file (the program is already going to all this for you) and compile the results in the same directory that your project is located. The .love files are inside "LOVE" folder of this program. APKS and EXES behind your project. Your job will be to locate the directories (this program tries to automatically find it too) and save it. Your last save is registered in the program and it will always open the settings that were previously saved.

ADDING AN ICON APK

You can add any image (png, jpg, ico, bmp ...) of any resolution, if it is not a png 144x144 image the program will automatically try to generate a new png 144px image and save it to the temporary folder "TMP" inside the program that Will be your new resized image to work. If it would try to fail, the entry will continue as it was. If no icon is inserted, the program will use its default icon.

ADDING AN ICON EXE

Only accepted icons (.ico) files. https://www.icoconverter.com ↗

ADDING A LOVE PROJECT

Only accept if inside the selected folder contain a main.lua file

ADDING A LOVE FOLDER

It will only accept if within the selected folder it contains a file love.exe

AUTO-DETECT "Settings"

Self-detecting searches on all your fixed drives in obvious places, it will not go into all your directories. In the case of projects, it will try to find some project if it is on your desktop.

BUILDING

The program will constantly use system functions like copy, paste, delete, read, create and execute. If your system blocks these features, several errors and failures will occur. Be aware of this.

BLUESTACKS

If you have Bluestacks installed on your computer, this program tries to locate it for you and add the installer.exe and run.exe, will be initiator after creating your APK automatically creates install runs.

JAVA JDK

To use the APK BUILD requires Java JDK "Java Development Kit", you can download it at http://www.oracle.com ↗

VARIABLES FIXES

%GAME% is your love.exe executable %APK% is your full path apk %APKTRUENAME% is the internal name of your apk on the android device %APKACT% is the second name of your apk's activity

NOTE: You can change the commands if you have other simulators and change the BATCH too (including more functions like arguments or scripts).

SHORTCUTS IN THE FIRST AREA

Ctrl + S or F10: Settings Ctrl + R or F11: Run Mode ON / OFF Ctrl + D or F12: Enables or disables auto-detection before entering settings

RUN MODE

Project (require Project and LOVE folder) APK Builder (require Project and JDK) EXE Builder (require Project and LOVE folder)

NOTE: For builders, the program uses a fast builder

NOTE: Dropping projects in shortcut icons (add all in settings) the program will use the fast builder and closes after.

TOOLS

SublimeText3 builders SublimeText3 autocomplete for Love2D and Lua LoveGame Demo Example Love 0.9.2 with touch and physic

NOTE: DON'T EXECUTES BUILDERS WITH PROJECTS INSIDE PROGRAM FILES

CMD ARGS toapk toexe

example: LoveToAll.exe "myprojectpath" toapk

For Windows XP/7/8 10?

Download (http://w w w.mediafire.com/file/9s8vek2feu2fm73/LoveToAll_Setup.exe)

Lovetoys

Lovetoys · revision 28135 · last edited 2023-07-05

Lovetoys is an Entity Component System framework for game development with lua. Originally written for the LÖVE 2D game engine it is now compatible with pretty much any game that uses lua! It is loosely based on Richard Lords Introduction (https://www.richardlord.net/blog/ecs/what-is-an-entity-framework.html) to ECS's. If you don't have any idea what this entity component stuff is all about, click that link and give it a read! It's totally worth it!

Lovetoys is a full-featured game development framework, not only providing the core parts like Entity, Component and System classes but also containing a Publish-Subscribe messaging system as well as a Scene Graph, enabling you to build even complex games easily and in a structured way.

Though we have not reached version 1.0 yet, the software is tested, used in multiple games and definitely stable. If you find any bugs please create an issue and report them. Or just create a pull request :).

Grab it on Github (https://github.com/Lovetoys/lovetoys) and give it a try!

If you want to see a proper way of implementation and some examples, have a look at our Example-project (https://github.com/Lovetoys/examples) or our Games (https://github.com/Lovetoys/Lovetoys-games).

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LoveUI

LoveUI · revision 16314 · last edited 2023-07-05

LoveUI is an imitation of a commercially available framework (for free). Adding to this framework is systematic, just subclass LoveUI.View and write your drawing and input handling code to create a new gui widget. Distributing input to various 'views' is systematic and formulaic. There is no overlapping, no mistakes; only one widget at a time can receive mouse/key inputs. And hopefully, it is simple to use, too.

Links

- LoveUI for 0.5.0 thread (http://love2d.org/forums/viewtopic.php?f=5&t=525)
- LoveUI for 0.6.0 thread (http://love2d.org/forums/viewtopic.php?f=5&t=978)
- LoveUI for 0.7.0 thread (https://love2d.org/forums/viewtopic.php?f=5&t=2574)
- Repository (https://github.com/meric/loveui.love)

LoveUpdate

LoveUpdate · revision 14760 · last edited 2021-03-14

LoveUpdate is a new way to release your game to make sure your users always get the up-to-date experience you intended. It is developed and maintained by thelinx.

It depends on UaLove, you need to use that framework in your game to use LoveUpdate.

It also depends on the fact that code needs to be available on GitHub.

However, you are free to customize LoveUpdate so it fits your own needs.

Download, fork and extend today! (http://github.com/TheLinx/LoveUpdate)

LuaC

LuaC · revision 17375 · last edited 2023-07-05



We're not kidding here; LuaC might have its uses, but it comes with a lot of strings attached.

- Don't use it.
- *(for experts)* Don't use it yet.



Although LÖVE accepts bytecode (because well, the VM does) we *don't* actually support it, if you have problems with bytecode, try plaintext before reporting bugs.

LuaC is tool which takes a lua source file and parses it into lua byte-code.

Sounds useful? Keep reading.

Lua byte-codes are not part of the Lua specification but rather a practical result of the LuaVM's design. The LuaVM is also not a part of any standardized specification, for it too is an implementation detail. **The only part of Lua that has a standardized specification is the language itself, and the CAPI. *Nothing else.***

The incompatibility of the byte-codes stems from this. Since there is no specification for Lua byte-codes, and **they are not meant to be ported between different builds of Lua**, there's no need to insulate the byte-codes from the vagaries and tribulations of the compiler. The upshot of this is that the compiler is free to rearrange the various `struct`s and `enum`s upon which the VM relies as deems fit (usually for efficiency), the down side is that **any byte-code is potentially idiosyncratic to the exact build of lua that created it.**

"But then, why create LuaC?", a question with a simple answer: One possible build configuration of Lua is parserless (an ultra-small size Lua build), which has the down side that it is unable to process source files on its own; it must be given pre-parsed source files...which is to say, byte-codes. The same limitations apply, though; **the build of LuaC used must match the VM.** A brittle solution, but tolerable given the circumstances.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LuaNameGen

LuaNameGen · revision 24512 · last edited 2023-07-05

LuaNameGen (Lua Name Generator) is a tool used to create random names (characters, objects, places, etc.) for games. It is based on libtcod's namegen (https://bitbucket.org/libtcod/libtcod/src/afba13253a79f16d10f596e2c9c99cf183f94b3c/src/namegen_c.c).

- Github page (https://github.com/LukeMS/lua-namegen): the library itself, documentation and examples;
- Love2d forum page (https://love2d.org/forums/viewtopic.php?f=5&t=83501): a place to discuss its current state, goals, ask questions and suggest things.

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Luaoop

Luaoop · revision 25011 · last edited 2023-07-05

Luaoop is a library to do OOP (Object Oriented Programming) which aims to be simple, powerful and optimized (especially for LuaJIT).

- Project and documentation: https://github.com/ImagicTheCat/Luaoop ↗
- Forum topic (https://love2d.org/forums/viewtopic.php?f=5&t=88233)

LuaPill

LuaPill · revision 14629 · last edited 2023-07-05

LuaPill (short for "Practical Isometric Layering Library for Lua") is an engine for generating isometric maps for Löve2D and Lua.

```
function love.load()
   map = require "luapill"
   local config = {
      tilewidth = 128,
      tileheight = 64,
      folder = "images"
   }
   map:setup(config)
end

function love.update(dt)
end

function love.draw()
   map:draw()
end
```

```
config = {
      tilewidth = number, -- required
      tilewidth = number, -- required
      folder = "path", -- required
      sortFolder = boolean, -- optional, default false
      defaultTile = number, -- optional, default 1
      tileIndex = number -- optional, default 1
   }
```

Grab it and give it a spin on [1] (https://github.com/Kyrremann/LuaPill). There is even a full working demo there.

Enjoy!

LuaTable

LuaTable · revision 24505 · last edited 2023-07-05

LuaTable.lua is table serialization library that converts tables to Lua code that can be saved. Please refer to the source code link for examples of use.

Links

- Forum thread (https://love2d.org/forums/viewtopic.php?f=5&t=82797)
- Source code (https://gist.github.com/RamiLego4Game/f656f5c1a118f77c3b7a08f4c65efaaf)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUBE.bin:pack

LUBE.bin:pack · revision 5509 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Packs a table into a single string in order to send it across a network. Used in conjunction with LUBE.bin:unpack once the data reaches the other side.

Function

Synopsis

```
string = lube.bin:pack( table )
```

Arguments

`table table`

A user defined table that you want to update.

Returns

`string string`

The table returned as a string, separated by predefined characters.

See Also

- LUBE
- LUBE.bin:setseperators
- LUBE.bin:unpack

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUBE.bin:setseperators

LUBE.bin:setseperators · revision 5511 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
  - See Also
  - Other Languages

If you want to share nested tables across a network, then using this function before packing each nest of tables will ensure that they get unpacked correctly.

Function

Synopsis

```
lube.bin:setseperators( null, one )
```

Arguments

`string null`
: A single character to separate each value.

`string one`
: A single character to separate the identifier, index and value.

Returns

Nothing.

See Also

- LUBE
- LUBE.bin:pack
- LUBE.bin:unpack

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUBE.bin:unpack

LUBE.bin:unpack · revision 5510 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
  - Returns
- See Also
- Other Languages

Unpacks a string into a table of values. Used in conjunction with LUBE.bin:pack once the string has been sent across the network.

Function

Synopsis

```
table = lube.bin:unpack( string )
```

Arguments

`string string`
: The string sent to a server or client that needs converting back into a table.

Returns

`table table`
: The string returned as a table. Note that if you used LUBE.bin:setseperators on the pack, you need to make sure to set them again on the other side.

See Also

- LUBE
- LUBE.bin:setseperators
- LUBE.bin:pack

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUBE.client

LUBE.client · revision 3702 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Creates a client object

Function

Synopsis

```
server = lube.client( sockettype )
```

Arguments

`string text`
: "tcp" , "udp" or user-defined

Returns

`ClientObject clientObject`
: The new client object.

See Also

- LUBE

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUBE.server

LUBE.server · revision 3654 · last edited 2023-07-05

On this page

- Function
  - Synopsis
  - Arguments
- Returns
- See Also
- Other Languages

Creates a server object

Function

Synopsis

```
server = lube.server( port, sockettype )
```

Arguments

Returns

`ServerObject serverObject`
: The new server object.

See Also

- LUBE

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

LUIGI

LUIGI · revision 25331 · last edited 2023-07-05

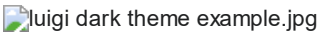
Lovely User Interfaces for Game Inventors is a GUI library for LÖVE and SDL.

Links

- Documentation (http://airstruck.github.io/luigi/doc/)
- Repository (https://github.com/airstruck/luigi)

Screenshots

Dark theme example:



Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

Lume

Lume · revision 13402 · last edited 2023-07-05

Lume is a library which provides a set of utility functions geared towards game development.

Links

- Github (https://github.com/rxi/lume/)
- Readme (https://github.com/rxi/lume/blob/master/README.md)

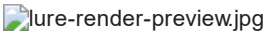
lure.article.renderpreview

lure.article.renderpreview · revision 9127 · last edited 2021-03-14

Markup

```
<html>
    <head>
        <title>Lure Test Page</title>
        <style type="text/css">
            html
            {
                width:500;
            }
            div.custom
            {
                height: 20px;
                width:50px;
            }
        </style>
    </head>
    <body>
        <div style="background-color:lightGrey;">
            <h1>Lure Test Page</h1>
            <p>Bacon ipsum dolor sit amet tenderloin shank sirloin short ribs ham hock bacon jowl sausage ground round andouille short loin chuck biltong frankfurter tongue. Tenderloin rump turducken, boudin kielbasa swine pig chicken. Ham hock ham spare ribs ribeye tenderloin. Salami bacon sausage, leberkas ham pork ribeye drumstick beef ribs turducken strip steak biltong tenderloin.</p>
            <p>Meatloaf spare ribs corned beef sausage, frankfurter shankle drumstick pork chop chicken short loin bresaola strip steak ribeye cow. Sausage pastrami tail cow, sirloin jerky spare ribs turducken. Beef salami biltong drumstick, meatball kielbasa frankfurter meatloaf pastrami ball tip flank. Shoulder sirloin jowl ham bacon tenderloin. Tri-tip pastrami jerky rump leberkas, salami sirloin drumstick short ribs fatback chuck. Turkey short ribs swine jowl beef cow leberkas shoulder sausage tri-tip. Leberkas pork chop ball tip meatloaf shankle jowl venison rump bacon t-bone tri-tip salami pastrami.</p>
            <div style="width:20px;background-color:blue;"      class="custom"></div>
            <div style="width:40px;background-color:red;"       class="custom"></div>
            <div style="width:60px;background-color:orange;"    class="custom"></div>
            <div style="width:80px;background-color:black;"     class="custom"></div>
        </div>
    </body>
    <script type="text/lua">
        print("successfully loaded test2.html")
    </script>
</html>
```

Render



lure.dom.attribute

lure.dom.attribute · revision 7484 · last edited 2021-03-14

Attribute Object

Desc

Attribute Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
value	Sets or returns the value of the attribute	Yes
name	Returns the name of the attribute	Yes
ownerElement	Returns the element node the attribute is attached to	No

Attribute Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

This Object does not declare additional Methods

Attribute Inherited Properties

This Object inherits the following properties from DOM Node

Property
nodeName
nodeType
nodeValue
textContent
text
xml

Attribute Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.cdata

lure.dom.cdata · revision 7486 · last edited 2021-03-14

CData Object

Desc

CData Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
data	Sets or returns the text of this node	Yes
length	Returns the length of the CDATAsection	Yes

CData Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
appendData()	Appends data to the node	Yes
deleteData()	Deletes data from the node	No
insertData()	Inserts data into the node	No
replaceData()	Replaces data in the node	No
splitText()	Splits the CDATAnode into two nodes	No
substringData()	Extracts data from the node	No

CData Inherited Properties

This Object inherits the following properties from DOM Node

Property
nodeName
nodeValue

CData Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.comment

lure.dom.comment · revision 7481 · last edited 2021-03-14

Comment Object

Desc

Comment Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

This Object does not declare additional Properties

Comment Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

This Object does not declare additional Methods

Comment Inherited Properties

This Object inherits the following properties from DOM CData

Property
Data
length
nodeName
nodeValue

Comment Inherited Methods

This Object inherits the following Methods from DOM CData

Method
appendData()
deleteData()
insertData()
replaceData()
splitText()
substringData()

lure.dom.document

lure.dom.document · revision 7610 · last edited 2021-03-14

Document Object

Desc

Document Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
documentElement	Returns the root node of the document	Yes
title	returns the title of the document	Yes
html	Returns the html root element if exists	Yes
body	Returns the html body element if exists	Yes
head	Returns the html head element if exists	Yes

Document Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
createAttribute(name)	Creates an attribute node with the specified name, and returns the new Attr object	Yes
createCDATASection()	Creates a CDATAsection node	Yes
createComment()	Creates a comment node	Yes
createElement()	Creates an element node	Yes
createTextNode()	Creates a text node	Yes
createStylesheet()	Creates a stylesheet object	Yes
renameNode()	Renames an element or attribute node	No

Document Inherited Properties

This Object inherits the following properties from DOM Node

Property
id
childNodes
firstChild
previousSibling
nextSibling
lastChild
nodeName
nodeType
nodeValue
test
xml

Document Inherited Methods

This Object inherits the following methods from DOM Node

Method
appendChild()
hasAttributes()
hasChildNodes()
insertBefore()
isEqualNode()
isSameNode()
normalize()

removeChild()
replaceChild()
getElementsByTagName()
getElementsByClassName()
getElementById()

lure.dom.DOMParser

lure.dom.DOMParser · revision 7608 · last edited 2021-03-14

DOMParser Object

Desc

DOMParser Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
parsedebug	Debug output of parser object	Yes

DOMParser Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
parseFromString()	Parses xml/html data from a string	Yes

DOMParser Inherited Properties

This Object does not inherit Properties from any base Object

DOMParser Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.element

lure.dom.element · revision 7478 · last edited 2021-03-14

Element Object

Desc

Element Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
tagName	Retruns the node's tagName	Yes

Element Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

This Object does not declare additional Methods

Element Inherited Properties

This Object inherits the following properties from DOM Node

Property
id
childNodes
firstChild
previousSibling
nextSibling
lastChild
nodeName
nodeType
nodeValue
parentNode
textContent
text
xml

Element Inherited Methods

This Object inherits the following methods from DOM Node

Method
appendChild()
cloneNode()
hasAttributes()
hasChildNodes()
insertBefore()
isEqualNode()
isSameNode()
normalize()
removeChild()
replaceChild()
getElementsByTagName()
getElementsByClassName()
getElementById()

lure.dom.HTMLCssRule

lure.dom.HTMLCssRule · revision 7615 · last edited 2021-03-14

Documentation does not yet exist.

lure.dom.HTMLScript

lure.dom.HTMLScript · revision 7617 · last edited 2021-03-14

Documentation does not yet exist.

lure.dom.HTMLStyle

lure.dom.HTMLStyle · revision 7616 · last edited 2021-03-14

Documentation does not yet exist.

lure.dom.HTMLStylesheet

lure.dom.HTMLStylesheet · revision 7612 · last edited 2021-03-14

HTMLStylesheet Object

Desc

HTMLStylesheet Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
title	Gets or Sets the stylesheet title property	No
cssRules	Collection of all CSSRules in the stylesheet	Yes
rules	Collection of all CSSRules in the stylesheet	Yes
type	Gets or Sets the type attribute of the stylesheet	No
disabled	Whether the stylesheet is disabled	Yes
href	href attribute specifying the URI of the css resource	No

HTMLStylesheet Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
addRule()	Adds a new CSSRule to the stylesheet	Yes
removeRule()	Removes a CSSRule at the specified index	Yes
deleteRule()	Removes a CSSRule at the specified index	Yes
insertRule()	Inserts a rule at the specified index	Yes

HTMLStylesheet Inherited Properties

This Object does not inherit Properties from any base Object

HTMLStylesheet Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.namednodemap

lure.dom.namednodemap · revision 7438 · last edited 2021-03-14

NamedNodeMap Object

Desc

NamedNodeMap Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
length	Returns the number of nodes in the list	Yes

NamedNodeMap Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
getNamedItem()	Returns the specified node (by name)	Yes
item()	Returns the node at the specified index	Yes
removeNamedItem()	Removes the specified node (by name)	Yes
setNamedItem()	Sets the specified node (by name)	Yes

NamedNodeMap Inherited Properties

This Object does not inherit Properties from any base Object

NamedNodeMap Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.node

lure.dom.node · revision 7727 · last edited 2021-03-14

Node Object

Desc

Node Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
id	gets or sets the nodes id attribute.	Yes
childNodes	Returns a NodeList of child nodes for a node	Yes
firstChild	Returns the first child of a node	Yes
previousSibling	Returns the node immediately preceding a node	Yes
nextSibling	Returns the node immediately following a node	Yes
lastChild	Returns the last child of a node	Yes
nodeName	Returns the name of a node, depending on its type	Yes
nodeType	Returns the type of a node	Yes
nodeValue	Sets or returns the value of a node, depending on its type	Yes
ownerDocument	Returns the root element (document object) for a node	Yes
parentNode	Returns the parent node of a node	Yes
textContent	Sets or returns the textual content of a node and its descendants	No
text	Returns the text of a node and its descendants.	No
xml	Returns the XML of a node and its descendants.	No

Node Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
appendChild()	Adds a new child node to the end of the list of children of a node	Yes
cloneNode()	Clones a node	No
hasAttributes()	Returns true if a node has attributes of any name, otherwise it returns false	Yes
hasAttribute()	Returns true if a node has the specified attribute, otherwise it returns false	Yes
hasChildNodes()	Returns true if a node has any child nodes, otherwise it returns false	Yes
insertBefore()	Inserts a new child node before an existing child node	No
isEqualNode()	Checks if two nodes are equal	No
isSameNode()	Checks if two nodes are the same node	Yes
normalize()	Puts all text nodes underneath a node (including attributes) into a "normal" form where only structure separates Text nodes.	No
removeChild()	Removes a child node	Yes
removeAttribute()	Removes attribute from the current node's attribute collection	Yes
replaceChild()	Replaces a child node	No
getAttribute()	Gets attribute value from the current node's attribute collection	Yes
setAttribute()	Sets attribute value from the current node's attribute collection. Creates the attribute if it doesn't exist	Yes
getElementsByTagName()	Returns a NodeList of all child elements with a specified name	Yes
getElementsByClassName()	Returns a NodeList of all child elements with a specified class under	Yes
getElementById()	Returns the child node with the specified id from the calling node	Yes

Node Inherited Properties

This Object does not inherit Properties from any base Object

Node Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.nodelist

lure.dom.nodelist · revision 7441 · last edited 2021-03-14

NodeList Object

Desc

NodeList Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
length	Returns the number of nodes in a node list	Yes

NodeList Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
item()	Returns the node at the specified index in a node list	Yes
addItem()	Adds a node to the nodeList object	Yes
removeItem()	Removes a node from the nodeList object	Yes

NodeList Inherited Properties

This Object does not inherit Properties from any base Object

NodeList Inherited Methods

This Object does not inherit Methods from any base Object

lure.dom.style

lure.dom.style · revision 7512 · last edited 2021-03-14

Documentation does not yet exist

lure.dom.text

lure.dom.text · revision 7480 · last edited 2021-03-14

Text Object

Desc

Text Properties

Properties present in this table are properties currently planned for implementation. Not all W3C DOM Properties will be implemented.

Property	Description	Implemented
isElementContentWhitespace	Returns true if the text node contains content whitespace, otherwise it returns false	No
wholeText	Returns all text of text nodes adjacent to this node, concatenated in document order	No

Text Methods

Methods present in this table are methods currently planned for implementation. Not all W3C DOM methods will be implemented.

Method	Description	Implemented
replaceWholeText()	Replaces the text of this node and all adjacent text nodes with the specified text	No

Text Inherited Properties

This Object inherits the following properties from DOM CData

Property
Data
length
nodeName
nodeValue

Text Inherited Methods

This Object inherits the following Methods from DOM CData

Method
appendData()
deleteData()
insertData()
replaceData()
splitText()
substringData()

lure.dom.window

lure.dom.window · revision 7424 · last edited 2021-03-14

Documentation does not yet exist

lure.dom.XMLHttpRequest

lure.dom.XMLHttpRequest · revision 7664 · last edited 2021-03-14

Documentation does not yet exist

lure.load

lure.load · revision 7605 · last edited 2021-03-14

Desc

Syntax/Usage

```
require("lure.lua")

htmlDoc = lure.load("test.html")

print(htmlDoc)
--OUTPUT: [object]:DOCUMENT_NODE
```

Arguments

Name	Type	Decription	Required
pObj	Any	The object to load into the lure.document scope. Can be a file/DOM Object/or xml/html text. If no arguments are specified, the function will return an empty DOM Object	No

Return value

Return Type	Description
DOM Object	On successful completion of the the load function, it will return a valid DOM Object, or nil on failure

lure.tuts.parsepreview

lure.tuts.parsepreview · revision 7468 · last edited 2021-03-14

Parser Preview

The parser in its current state implements many of the core DOM node objects. Although the library is not yet ready for prototype release, below is an example of the current xml/html parser and some basic use of the resulting DOM Object.

Below is very simple xml/html markup:

```
<html>
    <head>
        <title>Lure Test Page</title>
    </head>
    <body>
        <div id="div1">
            this is a sentence<br>separated by a break line.
            <div id="div2">Hello</div>
            <div id="div3">World</div>
        </div>
    </body>
</html>
```

And to parse and work with the DOM in LURE:

```
require("lure.lua")

htmlDoc = Lure.load("test.html")
print(htmlDoc.getElementById("div1").childNodes[1].data)
--OUTPUT: this is a sentence

print(htmlDoc.childNodes[1].childNodes[2].childNodes[1].childNodes[3].data)
--OUTPUT: separated by a break line.

newElement = htmlDoc.createElement("div")
newElement.setAttribute("id", "myNewElement")
htmlDoc.childNodes[1].childNodes[2].appendChild(newElement)
print(htmlDoc.getElementById("myNewElement").getAttribute("id"))
--OUTPUT: myNewElement
```

Since the parser is non-validating (no docTypes, namespaces or other shenanigans), we can parse and produce a DOM object using arbitrary xml structures. Lets represent a list of players using XML:

```
<players>
    <player id="p1">
                <desc>I am player 1</desc>
        <item type="weapon" />
    </player>
    <player id="p2">
        <item type="weapon" />
    </player>
    <player id="p3">
        <item type="weapon" />
    </player>
    <player id="p4">
        <item type="weapon" />
    </player>
    <player id="p5">
        <item type="weapon" />
    </player>
</players>
```

And to parse and work with the returned DOM in Lure:

```
require("lure.lua")

playersXML = lure.load("players.xml")

player1 = playersXML.documentElement.firstChild
```

```
print(player1.id)
--OUTPUT: p1

print(player1.childNodes[1].childNodes[1].data)
--OUTPUT: I am player 1

player1.childNodes[2].getAttribute("type")
--OUTPUT: weapon
```

lure.tuts.xmlhttprequest

lure.tuts.xmlhttprequest · revision 7694 · last edited 2021-03-14

Using XMLHttpRequest

Lure has the capability to make HTTP requests. This is accomplished by wrapping the already available socket.http library into a DOM XMLHttpRequest object. Lure's implimentation of XMLHttpRequest supports synchronous and asynchronous HTTP requests.

Shut up and show me how!!

GET Request

```
function love.load()

    --require lure library
    require('lure//lure.lua')

    --create new XMLHttpRequest Object
    http = XMLHttpRequest.new()

    --create a new request, set method, url, and sync option
    http.open("GET", "http://www.love2d.org/", true)

    --create callback function to trigger when request readyState changes
    http.onReadyStateChange = function()
        print(http.readyState)
        print(http.status)
        print(http.statusText)
        print(http.responseText)
    end

    --send your GET request!
    http.send()
end
```

POST Request

```
function love.load()

    --require lure library
    require("lure//lure.lua")

    --create new XMLHttpRequest Object
    http = XMLHttpRequest.new()

    --set url, and post params
    url     = "http://mydomain.com/"
    params  = "a=1&b=2"

    --create a new request, set method, url, and sync option
    http.open("POST", url, true)

    --set request headers
    http.setRequestHeader("Content-Type", "application/x-www-form-urlencoded")
    http.setRequestHeader("Content-Length", params:len())

    --create callback function to trigger when request readyState changes
    http.onReadyStateChange = function()
        print(http.readyState)
        print(http.status)
        print(http.statusText)
        print(http.responseText)
    end

    --send your POST request!
    http.send(params)

end
```

Lynput

Lynput · revision 24411 · last edited 2023-07-05

Lynput is a library that makes input handling very easy and intuitive.



How to use it

More information here (https://github.com/Lydzje/lynput/blob/master/README.md)

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

main.lua

main.lua · revision 19112 · last edited 2021-03-14

Mappy

Mappy · revision 6269 · last edited 2021-03-14

Mappy is a tile map editor which uses a fairly flexible file format called FMP. All the versions of Mappy and playback libraries use the same FMP files and are backwards compatible. Comprehensive help is included with the files and you should look at that for more information. Use of Mappy and it's libraries is free, I hope you find them useful.

- Web Site (http://tilemap.co.uk/mappy.php)

Math

Math · revision 25318 · last edited 2023-07-05

Math Libraries

Name	Description	Standalone Lua Module	LOVE Version	LOVE Min Version
MGL	Mathematics for Graphics in pure Lua		Any	
RL-Dice	A dice module aimed for roguelikes.		Any	
RandomLua	Random numbers generator for all platforms		Any	
Transform (Library)	A library designed to ease the handling of coordinate/angle transformations in world		Any	
luasort	Array sorting algorithms		Any	

MatrixLayout

MatrixLayout · revision 21356 · last edited 2026-04-20

Available since LÖVE 11.0
This enum is not supported in earlier versions.

The layout of matrix elements (row-major or column-major).

Constants

row
The matrix is row-major:

```
matrix = {
    {a, b, c, d}, -- row 1
    {e, f, g, h}, -- row 2
    -- etc.
}
-- or
matrix = {
    a, b, c, d, -- row 1
    e, f, g, h, -- row 2
    -- etc.
}
```

column
The matrix is column-major:

```
matrix = {
    {a, b, c, d}, -- column 1
    {e, f, g, h}, -- column 2
    -- etc.
}
-- or
matrix = {
    a, b, c, d, -- column 1
    e, f, g, h, -- column 2
    -- etc.
}
```

See Also

- Transform
- Shader
- Transform:setMatrix
- Shader:send

Other Languages

Dansk – Deutsch – English – Español – Français – Indonesia – Italiano – Lietuviškai – Magyar – Nederlands – Polski – Português – Română – Slovenský – Suomi – Svenska – Türkçe – Česky – Ελληνικά – Български – Русский – Српски – Українська – עברית – ไทย – 日本語 – 正體中文 – 简体中文 – Tiếng Việt – 한국어
More info

memoize.lua

memoize.lua · revision 6376 · last edited 2023-07-05

memoize.lua is a library that takes a function as parameter and returns a "cached" version of that function. The first time the function is called with some parameters, it will act normally.

The second time it's called, however, it will "remember" the data result from previous computations, and will recover it from the cache instead of recalculating it.

memoizing is useful for functions that take lots of time to compute, and always return the same output with the same input. It's also useful for functions that read data from the disk.

Latest version :

- https://github.com/kikito/memoize.lua ↗

See also

- Original forum thread (http://love2d.org/forums/viewtopic.php?f=5&t=2867).